AMARANTH · VISAYAS STATE UNIVERSITY

Amaranth v2

Software Engineering Documentation Package

Version v1.0 · 2026-10-19
Newsroom, public site and Salingkusog intramurals module
Prepared by the Amaranth development team [TO CONFIRM: names]

Contents

1. Amaranth v2 — Repository Audit Report
2. Project Overview and Vision
3. Software Requirements Specification (SRS)
4. Software Design Document (SDD) and Architecture
5. Database Design
6. UML and Process Diagrams
7. Security and Privacy Document
8. API and Integration Reference
9. Test Documentation
10. Deployment and Operations Runbook
11. Maintenance and Developer Guide
12. User Manual — Executive Editor, Managing Editor and Super Admin
13. User Manual — Section Editors, Staff Writers, Contributors and Business Manager
14. User Manual — Salingkusog Team
15. User Manual — Public Readers
16. Project Management Documents
17. Limitations, Known Issues and Future Work
18. Glossary and Appendices
19. Defense Demo Script
20. Defense Q&A Preparation
21. One-Page Cheat Sheets
22. Screenshot List

Amaranth v2 — Repository Audit Report

Phase 0: read-only audit of code, database and configuration · 1.0 · 2026-10-19

Document control

Field	Value
Title	Amaranth v2 — Repository Audit Report
Document ID	AMR-DOC-00
Version	1.0
Date	2026-10-19
Author	Amaranth development team [TO CONFIRM: names]
Status	Final (baseline for all other documents)
Source revision	`develop` branch at commit `d7f6f34` (2026-10-19)

How to read this report. Every statement points to a file, table or function in the repository. Facts that could not be checked in the code are marked **[TO CONFIRM]**. Live counts were read on 2026-10-19 from the Supabase project configured in the local `.env` with the public (anonymous) key; they count rows only and include no personal data.

Contents

1. Executive summary
2. Technology stack
3. Hosting, deployment and environment
4. Repository structure and architecture
5. Routes and pages
6. Server endpoints (Vercel functions and middleware)
7. Database inventory
8. Roles and permissions (role × feature matrix)
9. Features and modules
10. Salingkusog module
11. Statistics module
12. Tests, CI and scripts
13. Known issues, TODOs and technical debt
14. Gaps against a typical publication CMS and sports-results system
15. Live data snapshot
16. Items to confirm

1. Executive summary

Amaranth v2 is a single-page web application (React 18 + TypeScript, built with Vite 7) that serves both the **public news site** of Amaranth, the official student publication of Visayas State University (VSU), and the **staff newsroom** at `/admin`. All data lives in **Supabase** (PostgreSQL with Row Level Security, Auth, Storage and Realtime). The site is hosted on **Vercel**, which also runs six small serverless functions under `/api` and one routing middleware for link previews.

The database is defined in three re-runnable SQL files: `supabase/v2/schema.sql` (newsroom), `supabase/v2/salingkusog.sql` (intramurals add-on) and `supabase/v2/stats.sql` (first-party statistics). Together they define **50 tables, 19 enums, 6 views, 76 functions, 40 triggers, 60 indexes, 97 row-level-security policies and 3 storage buckets**. Every table has RLS enabled; the database, not the UI, is the source of truth for permissions.

There are **8 account roles** (including the anonymous reader there are 9 user classes) and one extra permission group, the **Salingkusog team**. The newsroom covers articles with an enforced editorial workflow, submissions and event reports from the public, a media library, staff and editorial-board management, site settings, statistics and the **Salingkusog** intramurals module (editions, factions, sports and events, venues, athletes, schedule import, automatic brackets and round robins, contests, live scores, results and medals, awards, medal table, points race).

Quality gates exist and pass: **255 unit and component tests in 48 files** (Vitest) and **278 database checks in 6 SQL test files** run against a throwaway local PostgreSQL. There is **no CI pipeline**: the checks run on developers' machines before each merge.

2. Technology stack

Layer	Technology (version from `package.json`)	Evidence
Language	TypeScript ^5.9.2 (strict)	`tsconfig.app.json`, `tsconfig.server.json`
UI library	React ^18.3.1, React DOM ^18.3.1	`src/main.tsx`
Routing	React Router DOM ^7.9.1 (data router, lazy routes)	`src/app/router.tsx`
Server state	TanStack Query ^5.90.2	`src/lib/queryClient.ts`
Styling	Tailwind CSS ^4.3.3 (`@theme` design tokens), clsx, tailwind-merge	`src/styles/tokens.css`, `src/lib/cn.ts`
Rich-text editor	TipTap ^3.32.0 (starter kit, image, link, YouTube)	`src/features/cms/articles/editor/`
HTML sanitising	DOMPurify ^3.4.17	`src/lib/sanitize.ts`
Icons	lucide-react ^0.487.0	components
Fonts	Montserrat Variable, Cinzel (self-hosted via Fontsource)	`@fontsource-variable/montserrat`, `@fontsource/cinzel`
Backend-as-a-service	Supabase JS ^2.58.0 (PostgreSQL, Auth, Storage, Realtime)	`src/lib/supabase.ts`

Layer	Technology (version from `package.json`)	Evidence
Hosting / functions	Vercel (static hosting, Node serverless functions, routing middleware)	`vercel.json`, `api/`, `middleware.ts`
Analytics	First-party statistics (`api/e.ts`, `stats.sql`) and Vercel Web Analytics ^1.5.0	`src/app/RootLayout.tsx`
CAPTCHA	Cloudflare Turnstile	`src/lib/turnstile.ts`, `api/_lib/forms.ts`
Build	Vite ^7.3.7 with `@vitejs/plugin-react-swc` ^4.3.3	`vite.config.ts`
Tests	Vitest ^5.0.5, Testing Library, jsdom ^27; PostgreSQL 16+ for SQL tests	`scripts/db-test.mjs`
Lint	ESLint ^9.37 with typescript-eslint, react-hooks, react-refresh	`eslint.config.js`
Runtime requirement	Node.js >= 20.19	`package.json` → `engines`

3. Hosting, deployment and environment

Hosting. Vercel builds with `npm run build` and serves `dist/` (`vercel.json`). Production URL in project notes: `amaranth-iota.vercel.app` (`docs/v2/PERF.md`) [TO CONFIRM: custom domain, if any]. Branch flow from project notes: feature branches → `develop` → pull request → `main` (`docs/v2/PROGRESS.md`).

Rewrites and headers (`vercel.json`):

- `/sitemap.xml` → `/api/sitemap`, `/rss.xml` → `/api/rss`, `/robots.txt` → `/api/robots`; every other non-asset path → `index.html` (single-page app).
- Long-lived cache for `/assets/*`; one-week cache for `/images`, `/fonts`, `/documents`.
- Security headers on all paths: `X-Content-Type-Options`, `Referrer-Policy`, `X-Frame-Options: SAMEORIGIN`, `Permissions-Policy`, `Strict-Transport-Security`, and a `Content-Security-Policy-Report-Only` header.

Database deployment. There is no migration tool. The SQL files are pasted into the Supabase SQL editor in the order `schema.sql` → `salingkusog.sql` → `stats.sql` → `seed.sql` (→ optional `seed_board.sql`). All files are idempotent (“re-runnable”); `scripts/db-test.mjs` proves this by applying them twice (`docs/v2/DB.md`).

Scheduled jobs. `stats.sql` schedules `amaranth-stats-nightly` with pg_cron (`15 16 * * *` UTC = 00:15 Manila). `docs/v2/DB.md` describes an optional `publish-due-articles` pg_cron job [TO CONFIRM: whether it is enabled in production].

Environment variables (names only; values are never stored in the repository):

Name	Used by	Purpose
`VITE_SUPABASE_URL`	browser, server fallback	Supabase project URL
`VITE_SUPABASE_ANON_KEY`	browser, server fallback	Public anonymous key (RLS applies)

Name	Used by	Purpose
`VITE_SITE_URL`	browser, server fallback	Canonical site URL; statistics only run on this host
`VITE_TURNSTILE_SITE_KEY`	browser	Public Turnstile site key
`VITE_STATS_DEBUG`	browser	Optional statistics debug logging
`VITE_ENABLE_FIXTURES`	build	Keeps `/dev` fixture screens in a build (never on Vercel)
`SUPABASE_URL`, `SUPABASE_ANON_KEY`, `SITE_URL`	`api/*`, `middleware.ts`	Server-side copies (fall back to the `VITE_` values)
`SUPABASE_SERVICE_ROLE_KEY`	`api/e.ts`, `api/submit.ts`	Server-only key for statistics ingest and public forms
`TURNSTILE_SECRET_KEY`	`api/submit.ts`	Server-only Turnstile verification secret
`PG_BIN`, `DB_TEST_PORT`	`scripts/db-test.mjs`	Local SQL test runner

Template: `.env.example` (placeholders only).

Third-party services: Supabase, Vercel (hosting, functions, Web Analytics), Cloudflare Turnstile. Fonts and icons are bundled; no other external scripts are loaded (`vercel.json` CSP lists only Supabase, Cloudflare challenges and video embed hosts).

4. Repository structure and architecture

```
/                    Vite + React app, Vercel config
├─ api/              Vercel serverless functions (Node ESM): r, e, submit, sitemap, rss, robots
│  └─ _lib/          shared server code: seo, forms, traffic, supabaseRest
├─ middleware.ts     Vercel routing middleware: Open Graph tags for link-preview crawlers
├─ src/
│  ├─ app/           router, root layout, error boundary
│  ├─ components/    shared UI kit (ui/) and story cards (story/)
│  ├─ features/
│  │  ├─ auth/       login, password reset, guards, role helpers (permissions.ts)
│  │  ├─ public/     public site: home, sections, article, search, archives, about, intake forms,
salingkusog, analytics
│  │  └─ cms/        newsroom: dashboard, articles, intake, media, people, settings, statistics,
salingkusog
│  ├─ lib/           Supabase client, sanitising, images, formatting, Salingkusog helpers,
Turnstile, edge cache
│  ├─ styles/        design tokens (tokens.css) and global CSS
│  └─ dev/           /dev fixture screens and checks (development builds only)
├─ supabase/
│  ├─ v2/            schema.sql, salingkusog.sql, stats.sql, seed.sql, seed_board.sql
│  └─ tests/         SQL test harness and *.test.sql
├─ scripts/          db-test.mjs, check-contrast.mjs, countdown-og.py
└─ docs/             v2 planning docs and this documentation package
```

Size: 206 source files (`.ts`/`.tsx`, excluding tests) and about 27,700 lines of TypeScript including tests; 4,082 lines of SQL in `supabase/v2`.

Architecture style. A thick client talks directly to Supabase through PostgREST with the user's JWT; Row Level Security and database triggers enforce every rule. A thin server layer on Vercel handles only what must not run in the browser: secret-key operations (`api/submit.ts`, `api/e.ts`), edge-cached public reads (`api/r.ts`), SEO documents (`api/sitemap.ts`, `api/rss.ts`, `api/robots.ts`) and crawler previews (`middleware.ts`). There are no server actions (this is not a Next.js project).

Layers:

Layer	Responsibility	Main files
Presentation	React pages, layouts, components	`src/features/**`, `src/components/**`
Client data access	TanStack Query hooks per feature	`*Api.ts`, `*Data.ts` files
Edge / server	Caching proxy, form gateway, stats ingest, SEO	`api/*.ts`, `middleware.ts`
Data and rules	Tables, RLS, triggers, RPCs, views	`supabase/v2/*.sql`
Platform	Auth, Storage, Realtime, pg_cron	Supabase

5. Routes and pages

From `src/app/router.tsx` and the nested routers.

Public site (layout `PublicLayout`):

Route	Page
`/`	Home (`HomePage`)
`/:section`	Section landing or list (12 sections, `SectionPage`)
`/latest`	Latest stories
`/article/:slug`	Article page with comments
`/search`	Full-text search and tag pages
`/archives`, `/archives/:month`	Print issues and monthly archive
`/about`, `/editorial-board`, `/privacy`	About, editorial board, privacy notice
`/submit`, `/submit/event`	Story submission, event report forms
`/contact`	Contact form and desk directory
`/salingkusog`	Medal tally (index)
`/salingkusog/schedule`	Schedule and results
`/salingkusog/winners`	Winners per event
`/salingkusog/factions`, `/salingkusog/factions/:slug`	Factions and faction profile
`/salingkusog/sports`, `/salingkusog/sports/:slug`	Sport page (bracket, standings or contest results)
`/salingkusog/match/:id`	Match recap
`/salingkusog/venues`	Venue guide
`/salingkusog-2026/countdown`	Full-screen countdown (outside the site layout)

Route	Page
`*`	Not found

Legacy redirects: `/home`, `/features`, `/blog`, `/s&t`, `/submit-article`, `/dashboard`, `/salingkusog-2026`.

Newsroom (guard `RequireStaff`, layout `AdminLayout`):

Route	Page
`/admin/login`, `/admin/forgot-password`, `/admin/set-password`	Authentication
`/admin`	Dashboard
`/admin/articles`, `/admin/articles/new`, `/admin/articles/:id`	Article list and editor
`/admin/submissions`, `/admin/submissions/:id`	Story submissions (intake)
`/admin/events`, `/admin/events/:id`	Event reports (intake)
`/admin/media`	Media library
`/admin/staff` (+ `/invites`, `/contributors`, `/board`)	Accounts, invites, contributors, editorial board
`/admin/settings` (+ `/home`, `/comments`, `/inbox`, `/sections`, `/contacts`, `/general`)	Front page, comments, inbox, sections, desks, general
`/admin/statistics`	Statistics
`/admin/salingkusog` (+ `scores`, `scores/:matchId`, `results`, `results/:eventId`, `awards`, `athletes`, `edition`, `factions`, `sports`, `venues`, `import`, `team`)	Salingkusog newsroom

Development only: `/dev/*` fixture screens (`src/dev/DevRoutes.tsx`), removed from production builds.

6. Server endpoints (Vercel functions and middleware)

Endpoint	Method	Who	Purpose	File
`/api/r`	GET	anyone	Edge-cached (60 s) proxy for a fixed list of public, slow-changing reads (`CACHEABLE` set); anon key, RLS applies	`api/r.ts`
`/api/e`	POST	site pages (same origin)	Statistics ingest: drops bots, classifies device/source, never forwards the IP; calls `stats_ingest` with the service key	`api/e.ts`, `api/_lib/traffic.ts`

Endpoint	Method	Who	Purpose	File
`/api/submit`	POST	anyone with a Turnstile token	The only entry for comments, story submissions, event reports and contact messages; issues one-time signed upload links for attachments; saves via service-role RPCs	`api/submit.ts`, `api/_lib/forms.ts`
`/api/sitemap` (`/sitemap.xml`)	GET	anyone	Sitemap of live articles and sections	`api/sitemap.ts`
`/api/rss` (`/rss.xml`)	GET	anyone	RSS feed	`api/rss.ts`
`/api/robots` (`/robots.txt`)	GET	anyone	robots.txt	`api/robots.ts`
middleware	GET	crawlers only	Injects Open Graph tags for articles and sections	`middleware.ts`, `api/_lib/seo.ts`

7. Database inventory

Counts were extracted with a parser and cross-checked with `grep` against the SQL files.

Object	schema.sql	salingkusog.sql	stats.sql	Total
Tables	29	12	9	**50**
Enums	13	6	0	**19**
Views	1	5	0	**6**
Functions	39	18	19	**76**
Triggers	19	21	0	**40**
Indexes	35	18	7	**60**
RLS policies (incl. storage)	64	25	8	**97**
Storage buckets	3	—	—	**3**

Tables.

- *Newsroom (`schema.sql`, 28 in `public` + 1 in `app`):* `profiles`, `staff_invites`, `sections`, `profile_sections`, `categories`, `tags`, `media_assets`, `contributors`, `publication_issues`, `articles`, `article_authors`, `article_tags`, `article_media`, `article_revisions`, `article_activity`, `article_daily_views`, `article_comments`, `article_comment_private`, `home_slots`, `article_submissions`, `event_reports`, `intake_files`, `contact_messages`, `team_terms`, `team_members`, `department_contacts`, `site_settings`, `audit_log`, `app.rate_events`.
- *Salingkusog (`salingkusog.sql`):* `sk_editions`, `sk_factions`, `sk_venues`, `sk_sports`, `sk_events`, `sk_athletes`, `sk_matches`, `sk_entries`, `sk_athlete_stats`, `sk_placements`, `sk_awards`, `sk_team`.
- *Statistics (`stats.sql`):* `app.stats_salts`, `stats_pageviews`, `stats_shares`, `staff_activity_daily`, `stats_daily_site`, `stats_daily_articles`, `stats_daily_dims`, `stats_daily_shares`, `stats_rollup_log`.

Enums (19): `app_role` (7 values), `article_status` (7), `nav_placement`, `contributor_type`, `asset_kind`, `article_media_usage`, `revision_kind`, `comment_status`, `intake_status`, `contact_category`, `contact_status`, `issue_type`, `team_role_group`, `sk_category` (10), `sk_division` (4), `sk_format` (5), `sk_match_status` (5), `sk_medal` (3), `sk_award_kind` (3).

Views (6, all `security_invoker`): `article_cards`, `sk_medal_table`, `sk_points_by_day`, `sk_head_to_head`, `sk_event_status`, `sk_event_leaders`.

Public RPCs (callable from the browser):

- anonymous and staff: `record_article_view`, `most_read_articles`, `search_articles`, `archive_months`, `sk_countdown`, `sk_points_timeline`;
- staff: `transition_article`, `add_article_note`, `convert_intake_to_article`, `dashboard_stats`, `media_usage`, `set_publish_access`, `sk_set_placements`, `sk_build_schedule`, `sk_create_contest`, `staff_heartbeat`, and the seven `stats_*` readers;
- service role only (through `/api`): `submit_comment`, `submit_article`, `submit_event_report`, `send_contact_message`, `stats_ingest`.

Storage buckets: `media` (public, 10 MB, images), `archives` (public, 60 MB, PDF and images, publishers write), `intake` (private, 20 MB, images, PDF, text and Word; written only through signed upload links from `/api/submit`).

Realtime publication: `sk_matches`, `sk_placements`, `sk_entries` (`salingkusog.sql`).

The full data dictionary is in `04-database-design.md`.

8. Roles and permissions

Roles (enum `public.app_role`): `super_admin`, `editor_in_chief` (shown as **Executive Editor**), `managing_editor`, `business_manager`, `section_editor`, `staff_writer`, `contributor`. A profile counts only while `profiles.is_active` is true (`app.current_role()`). Anonymous readers have no account. Accounts are created only for emails pre-authorised in `staff_invites`; anyone else who signs up gets an inactive profile with no access (`app.handle_new_user`).

Permission helpers (`schema.sql`, `salingkusog.sql`, `stats.sql`):

Helper	True for
`app.is_staff()`	any active role
`app.is_admin()`	super_admin, editor_in_chief
`app.can_publish()`	super_admin, editor_in_chief, managing_editor
`app.can_publish_articles()`	`can_publish()`, business_manager, or a profile with `can_publish = true`
`app.is_editor()`	super_admin, editor_in_chief, managing_editor, section_editor
`app.can_edit_section(s)`	`can_publish()`, or a section_editor assigned to section `s` (`profile_sections`)
`app.can_view_stats()`	super_admin, editor_in_chief, managing_editor
`app.can_manage_sk_team()`	super_admin, editor_in_chief, managing_editor

Helper	True for
`app.can_edit_sk()`	`can_manage_sk_team()`, or active staff listed in `sk_team`

Role × feature matrix (✓ = allowed, ○ = only own or assigned items, — = not allowed). SA = super_admin, EE = editor_in_chief (Executive Editor), ME = managing_editor, BM = business_manager, SE = section_editor, SW = staff_writer, CO = contributor, SKT = Salingkusog team member (any active role listed in `sk_team`), Pub = anonymous reader.

Feature (rule source)	SA	EE	ME	BM	SE	SW	CO	Pub
Read live articles, sections, published Salingkusog data (RLS `*_read`)	✓	✓	✓	✓	✓	✓	✓	✓
Create article drafts (`articles` insert: `is_staff`)	✓	✓	✓	✓	✓	✓	✓	—
Edit articles (`articles` update + `article_before_write`)	✓	✓	✓	○ own	○ own + assigned sections	○ own	○ own	—
Submit for review (draft → submitted)	✓	✓	✓	○	✓ section	○	○	—
Review, reject, return (in_review / rejected)	✓	✓	✓	—	✓ section	—	—	—
Publish, schedule, archive (`can_publish_articles`)	✓	✓	✓	✓	granted	granted	granted	—
Grant “Can publish” (`set_publish_access`)	✓	✓	✓	—	—	—	—	—

Feature (rule source)	SA	EE	ME	BM	SE	SW	CO	Pub
Feature / prioritise articles	✓	✓	✓	—	✓ section	—	—	—
Delete articles (`articles` delete)	✓	✓	○ drafts	○ drafts	○ drafts	○ drafts	○ drafts	—
Staff accounts, roles, invites (`is_admin`; super_admin only grants super_admin)	✓	✓ (not SA)	—	—	—	—	—	—
Sections, categories, desk contacts, general settings (`is_admin`)	✓	✓	—	—	—	—	—	—
Front page slots, ticker, print issues, editorial board (`can_publish`)	✓	✓	✓	—	—	—	—	—
Media library upload (`media` bucket: `is_staff`) / delete others' (`is_editor`)	✓	✓	✓	○	✓	○	○	—

Feature (rule source)	SA	EE	ME	BM	SE	SW	CO	Pub
Comment moderation, submissions, event reports, inbox (`is_editor`)	✓	✓	✓	—	✓	—	—	—
Contributors (bylines) manage (`is_editor`) / own byline	✓	✓	✓	○	✓	○	○	—
Audit log read (`is_admin`)	✓	✓	—	—	—	—	—	—
Statistics (`can_view_stats`)	✓	✓	✓	—	—	—	—	—
Salingkusog data entry (`can_edit_sk`)	✓	✓	✓	SKT	SKT	SKT	SKT	—
Salingkusog team membership and countdown settings	✓	✓	✓	—	—	—	—	—
Comment, submit stories and events, contact (via `/api/submit` + Turnstile)	✓	✓	✓	✓	✓	✓	✓	✓

Notes: “granted” means the profile’s `can_publish` flag, set in Staff → Accounts by SA, EE or ME. The newsroom UI mirrors these rules in `src/features/auth/permissions.ts`, but the database enforces them (`docs/v2/DB.md`).

9. Features and modules

Module	What it does	Routes	Key code	Key tables / RPCs
Authentication	Password sign-in, password reset, invite acceptance; `RequireStaff` guard shows an “access pending” screen to inactive or uninvited accounts; `RequireRole` limits screens by role	`/admin/login`, `/admin/forgot-password`, `/admin/set-password`	`src/features/auth/*`	`profiles`, `staff_invites`, `app.handle_new_user`
Dashboard	Counts and shortcuts per role	`/admin`	`cms/dashboard/*`	`dashboard_stats()`
Articles	List, TipTap editor, autosave, revisions, notes, workflow, scheduling, SEO fields, featured flag	`/admin/articles*`	`cms/articles/*`	`articles`, `article_*`, `transition_article`, `add_article_note`
Submissions and events (intake)	Review public story submissions and event reports; convert to a draft article	`/admin/submissions*`, `/admin/events*`	`cms/intake/*`	`article_submissions`, `event_reports`, `intake_files`, `convert_intake_to_article`
Media library	Upload (compressed WebP, 4 sizes), search, usage check, “Make web sizes” backfill	`/admin/media`	`cms/media/*`, `src/lib/images.ts`	`media_assets`, `media_usage()`
Staff	Accounts (roles, activation, publishing access), invites, contributors, editorial board	`/admin/staff*`	`cms/people/*`	`profiles`, `staff_invites`, `contributors`, `team_terms`, `team_members`, `set_publish_access`
Settings	Front page, comments moderation, inbox, sections and categories, desks, general (social links, ticker…)	`/admin/settings*`	`cms/settings/*`	`home_slots`, `article_comments`, `contact_messages`, `sections`, `categories`, `department_contacts`, `site_settings`
Statistics	Pageviews, readers, read time, shares, top articles, authors, sections, sources, devices, staff active time	`/admin/statistics`	`cms/statistics/*`, `api/e.ts`	`stats_*` tables and RPCs

Module	What it does	Routes	Key code	Key tables / RPCs
Salingkusog	See section 10	`/admin/salingkusog*`, `/salingkusog*`	`cms/salingkusog/*`, `public/salingkusog/*`	`sk_*`
Public site	Home, 12 sections with custom landings, article, search, archives, about, board, privacy, forms	public routes	`src/features/public/*`	`article_cards`, `search_articles`, `most_read_articles`
SEO and sharing	Open Graph for crawlers, sitemap, RSS, robots, canonical URLs	—	`middleware.ts`, `api/*`	`article_cards`
Form security	Turnstile gateway, signed uploads, rate limits, honeypot	—	`api/submit.ts`, `src/lib/sendForm.ts`	`app.rate_limit`, form RPCs

10. Salingkusog module

Salingkusog is VSU's intramural games. The module models one **edition** per year (`sk_editions`: dates, medal points 5/3/1 by default, `is_current`, `is_published`, countdown settings). Everything else belongs to an edition: **factions** (teams), **venues**, **sports** (category from `sk_category`), **events** (a sport + division + format), **athletes**, **matches** (games), **entries** (contest line-ups), **athlete stats**, **placements** (official places and medals) and **awards** (MVP of the day, athlete of the day, finals MVP).

Capability	How it works	Evidence
Team access	SA/EE/ME always; other active staff while listed in `sk_team`; changes audited	`app.can_edit_sk`, `sk_team_audit`
Schedule import	CSV of date, time, sport, event, round, sides, venue; validated before insert	`cms/salingkusog/scheduleCsv.ts`, `ImportTab.tsx`
Build schedule	Single elimination (bronze game, byes), round robin, round robin + final; games know where their sides come from (`source_a/_b`)	`sk_build_schedule`, `BuildScheduleModal.tsx`
Auto-advance	After each result, `app.sk_resolve` fills the next games; started games never change; knockout ties are refused	triggers `sk_matches_advance`, `sk_matches_knockout`
Contests	One session with every faction (dance, swimming, athletics…); results or judged places per entry; DNS/DNF/DQ	`sk_create_contest`, `sk_entries`, `ContestScorekeeper.tsx`, `src/lib/skContests.ts`
Live scores	Phone-friendly scorekeeper with periods, live note, box score; Realtime pushes to readers	`Scorekeeper.tsx`, publication `supabase_realtime`
Results and medals	Staff confirm places 1–3 (medals follow the place); suggestions come from the games	`sk_set_placements`, `ResultsTab.tsx`, `suggestedPodium`

Capability	How it works	Evidence
Medal table	Computed: gold, silver, bronze, total, points, rank	view `sk_medal_table`
Points race	Cumulative points per day, or in 1/3/6/12-hour steps	view `sk_points_by_day`, RPC `sk_points_timeline`
Head-to-head, event status, leaders	Computed views	`sk_head_to_head`, `sk_event_status`, `sk_event_leaders`
Awards and social cards	Daily awards; downloadable PNG cards	`cms/salingkusog/awards/*`
Public hub	Medal tally, schedule, winners, factions, sports (bracket / standings / contest results), match recaps, venues, countdown	`src/features/public/salingkusog/*`

There is **no field for birth dates, student numbers or eligibility documents**: `sk_athletes` stores name, slug, program (course/year) and an optional photo only.

11. Statistics module

First-party, cookie-less statistics (`docs/v2/STATS.md`). The browser tracker (`src/features/public/analytics/*`) batches events to `/api/e`, which strips bots and identifying data and calls `stats_ingest` with the service key. Visitor ids are hashed with a daily salt that is deleted after two days; Do Not Track and Global Privacy Control are honoured; raw events are kept 13 months, daily totals indefinitely, staff activity 24 months (`app.stats_nightly`). Only SA, EE and ME can read statistics (`app.can_view_stats`).

12. Tests, CI and scripts

Item	Status	Evidence
Unit and component tests	48 files, **255 tests**, all passing on 2026-10-19	`npm run test` (Vitest)
Database tests	6 files, **278 checks**, all passing on 2026-10-19	`npm run db:test` → `supabase/tests/*.test.sql`
Per-file database checks	public_access 39 · salingkusog 71 · sk_brackets 28 · sk_contests 17 · stats 48 · workflow 75	`supabase/tests/`
Lint, typecheck, build	Part of `npm run check`	`package.json`
Contrast check	All design-token colour pairs meet WCAG AA	`scripts/check-contrast.mjs`
Dev screen checks	`/dev` sweep: 53 screens, no overflow or naming issues at 375 px and 1024 px (2026-10-19)	`src/dev/domCheck.ts`
Dependency audit	`npm audit --omit=dev`: 0 vulnerabilities (2026-10-19)	npm

Item	Status	Evidence
Continuous integration	**None.** `.github/` holds prompts only; no workflow files	`.github/`
End-to-end browser tests	**None in the repository.** An old Katalon Studio project exists in the untracked, git-ignored `Amaranth/` folder	`.gitignore` line 25
Performance	Lighthouse results recorded in `docs/v2/PERF.md` (production, 2026-10-18: performance 67–72, accessibility 100, best practices 100, SEO 100 on mobile)	`docs/v2/PERF.md`

13. Known issues, TODOs and technical debt

No `TODO`, `FIXME` or `HACK` comments exist in `src`, `api`, `supabase`, `middleware.ts` or `scripts`. Issues found during this audit:

ID	Issue	Impact	Evidence
KI-01	No CI: tests only run locally	A regression could reach `main` if a step is skipped	`.github/`
KI-02	`docs/v2/DB.md` is out of date (says anyone may upload to `intake`; quotes 95 checks; Realtime only on `sk_matches`)	Misleading for maintainers	`docs/v2/DB.md` vs `schema.sql`
KI-03	Content-Security-Policy is in report-only mode	Browser does not yet block injected scripts	`vercel.json`
KI-04	`src/lib/database.types.ts` is hand-written, not generated	Types can drift from the schema	file header
KI-05	Games created before “Build schedule” (CSV or by hand) have no sources, so they do not advance automatically	Staff must fill later rounds by hand for those events	`sk_matches.source_a/_b`
KI-06	Opening `/salingkusog/match/:id` for a contest session shows a two-sided recap with empty sides	Cosmetic; contest cards do not link there	`RecapView.tsx`
KI-07	Faction profile lists only two-sided games, not contest results (medals are counted)	Incomplete history on faction pages	`public/salingkusog/factions.ts`
KI-08	Supabase Auth hardening (sign-ups off, leaked-password protection, MFA) depends on dashboard settings	Account takeover risk if left at defaults	`docs/v2/DB.md` §2.6 [TO CONFIRM current settings]

ID	Issue	Impact	Evidence
KI-09	Public reads through `/api/r` can be up to 60 s (stale up to 300 s) old	Expected trade-off for performance; live scores bypass it	`api/r.ts`
KI-10	No email notifications (for example “article submitted for review”)	Staff must check the newsroom	no mail library in `package.json`
KI-11	Old Katalon test project in `Amaranth/` (ignored)	Clutter; not run	`.gitignore`

14. Gaps against a typical publication CMS and sports-results system

Area	Typical expectation	Amaranth v2 status
Editorial workflow	Draft → review → publish with roles	Present and database-enforced
Revision history	Versions and restore	Present (newest 50 revisions per article)
Notifications	Email or in-app alerts on workflow changes	**Missing** (KI-10)
Two-factor authentication	MFA for admins	**Not configured in code** (Supabase supports it)
Automated testing in CI	Tests on every push	**Missing** (KI-01)
End-to-end tests	Browser tests of key flows	**Missing**
Backups	Scheduled, tested restores	Platform default [TO CONFIRM Supabase plan and backup policy]
Multilingual content	English and Filipino/Waray	English UI only [TO CONFIRM need]
Sports: heats and finals	Qualifying heats feeding a final	Not supported by design (straight finals chosen)
Sports: officials, protests, appeals	Officials list, protest workflow	**Missing**
Sports: athlete eligibility	Eligibility records and checks	**Not modelled**: no eligibility data in the system
Sports: audit of score edits	Who changed which score	Partial: `sk_matches.updated_by`; no full history
Offline scorekeeping	Works without signal, syncs later	**Missing** (needs connection)

15. Live data snapshot (2026-10-19)

Read with the anonymous key from the project configured in `.env` [TO CONFIRM it is the production project].

Item	Count
Sections / categories	12 / 35
Live articles (`article_cards`)	10

Item	Count
Editorial board members	19
Salingkusog editions	1 (2026, Oct 26–30; medal points 5 / 3 / 1; published)
Factions	4
Sports	27 (ball games 9, dance & cheer 5, racket 3, combat 2, esports 2, others 2, aquatics 1, athletics 1, mind games 1, pageant 1)
Events	74 (timed 33, round robin + final 25, judged 10, bracket 6)
Venues	21
Scheduled games and contest sessions	169
Contest entries	68
Placements (medals)	0 (the games have not started)

16. Items to confirm

1. Team member names, roles and student numbers to show on title pages (never personal data beyond names).
2. Institution unit (college/department), adviser name and title.
3. Audience and date of the defense presentation.
4. Production domain (custom domain or `amaranth-iota.vercel.app`).
5. Whether the `.env` Supabase project is production.
6. Supabase plan and backup policy; whether pg_cron `publish-due-articles` is enabled.
7. Current Supabase Auth settings (sign-ups disabled, leaked-password protection, MFA).
8. Whether athlete eligibility forms are collected outside the system, and by whom.
9. Project start date and milestone dates for the timeline (git history starts 2025-09-23; v2 rebuild started 2026-10-17).
10. Hosting and domain costs, if any.

Project Overview and Vision

Amaranth v2 — newsroom and publishing system of Amaranth, VSU · 1.0 · 2026-10-19

Document control

Field	Value
Title	Project Overview and Vision
Document ID	AMR-DOC-01
Version	1.0
Date	2026-10-19
Author	Amaranth development team [TO CONFIRM: names]
Adviser	[TO CONFIRM: name and title]
Status	Final draft for defense
Related documents	AMR-DOC-00 Audit Report, AMR-DOC-02 SRS, AMR-DOC-03 SDD

Contents

1. Background
2. Problem statement
3. Vision and product statement
4. Stakeholders and user classes
5. Objectives
6. Scope
7. Out of scope
8. Success criteria
9. Constraints and assumptions
10. Project at a glance

1. Background

Amaranth is the official student publication of Visayas State University (VSU) in Baybay City, Leyte. It publishes news, features, literary works, opinion and sports coverage, and covers **Salingkusog**, VSU's annual intramural games.

The first website (v1, git history from 2025-09-23) was a partial front end. The v2 rebuild started on 2026-10-17 (`docs/v2/PLAN.md`) and replaced it with a complete newsroom and publishing system. Phase 2 (`docs/v2/PLAN-PHASE2.md`) added the Salingkusog module and section-specific page layouts. A brand refresh ("Brand 2026") was approved by the Executive Editor on 2026-10-18 (`docs/v2/PROGRESS.md`).

2. Problem statement

The audit of the v1 site (`docs/v2/PLAN.md`, §1) found:

#	Problem in v1	Consequence
P1	The styling was 5,170 lines of pre-compiled CSS from a design export; the styling tool itself was not installed	New styles could not be added safely
P2	15 near-identical hand-written section pages (~210 lines each)	Every change had to be made 15 times
P3	A placeholder integration (`joomlaApi.ts`) that talked to nothing	Dead code and confusion
P4	No lint, type-check or tests	Regressions went unnoticed
P5	Database helper functions exposed in the public API; a storage policy let any writer delete any file; no safe way for readers to record article views	Security and data-integrity risks
P6	No editorial workflow, revisions, comment moderation, homepage curation, staff invites or audit log	The newsroom could not be run inside the system
P7	Slow pages: Lighthouse mobile performance 56, accessibility 91, about 1.6 MB transferred on the home page (`docs/v2/PERF.md`)	Poor experience on campus mobile data
P8	Intramural results were not managed in the site	No live scores, medal table or official standings online

3. Vision and product statement

> **For** the Amaranth editorial board, staff and the VSU community, **Amaranth v2** is a web newsroom and publishing system **that** lets staff write, review, schedule and publish stories under clear editorial roles, and lets the Salingkusog team run live scores, brackets, results and the official medal table. **Unlike** the v1 site, every permission is enforced by the database itself, the public site is fast and accessible on mobile, and intramural standings are computed automatically from recorded results.

4. Stakeholders and user classes

Stakeholder	Interest	Role in the system (`public.app_role`)
Executive Editor (Editor-in-Chief)	Editorial control, final say on publishing, staff management	`editor_in_chief`
Technical administrator	Accounts, configuration, recovery	`super_admin`
Managing Editor	Daily publishing, front page, statistics, Salingkusog team	`managing_editor`

Stakeholder	Interest	Role in the system (`public.app_role`)
Business Manager	Publishing of paid or institutional notices [TO CONFIRM purpose]	`business_manager`
Section editors	Review and edit stories in assigned sections	`section_editor`
Staff writers and contributors	Write drafts and submit for review	`staff_writer`, `contributor`
Salingkusog team	Enter schedules, scores, results and awards during the games	any active role listed in `sk_team`
Readers (students, faculty, alumni, public)	Read news, follow Salingkusog live, submit stories and events, comment	anonymous
Story and event contributors from the public	Send story submissions and event reports	anonymous (through the public forms)
Adviser and university administration	Oversight, accountability, data privacy	[TO CONFIRM whether they need accounts]
Future maintainers	Understand and extend the system	developers

5. Objectives

ID	Objective	Measured by
OBJ-1	Run the whole editorial process in the system: draft → submitted → in review → scheduled/published → archived, with notes and history	Workflow trigger `app.article_before_write`; 75 checks in `supabase/tests/workflow.test.sql`
OBJ-2	Enforce permissions in the database for every role	RLS on all 50 tables; 97 policies; role helpers in `schema.sql`
OBJ-3	Give readers a fast, accessible, mobile-first public site	Lighthouse accessibility 100, best practices 100, SEO 100 on production (`docs/v2/PERF.md`)
OBJ-4	Accept stories, event reports, comments and messages from the public safely	`/api/submit` with Turnstile, rate limits, honeypot, moderation
OBJ-5	Run Salingkusog in the system: setup, schedule, live scores, brackets, contests, results, medals and awards	`salingkusog.sql`; 116 checks in the three Salingkusog SQL test files
OBJ-6	Compute the medal table, points and rankings automatically from results	Views `sk_medal_table`, `sk_points_by_day`; RPC `sk_points_timeline`
OBJ-7	Measure readership without third-party trackers or cookies	`stats.sql`, `api/e.ts`, `/privacy` page
OBJ-8	Keep the code maintainable and tested	TypeScript strict, ESLint, 255 unit tests, 278 SQL checks

6. Scope

In scope (all implemented in the repository):

- **Public site:** home, 12 sections with their own landing layouts, article pages, search, archives, about, editorial board, privacy notice, story submission, event report and contact forms, comments.
- **Newsroom (`/admin`):** dashboard, articles and editor, editorial workflow, submissions and event reports, media library, staff and editorial board, settings, statistics.
- **Salingkusog:** public hub (medal tally, schedule and results, winners, factions, sports, match recaps, venues, countdown) and newsroom tools (edition, factions, sports and events, venues, athletes, schedule import, build schedule, contests, scorekeeper, results and medals, awards and social cards, team access).
- **Platform concerns:** link previews for social media, sitemap, RSS, security headers, form CAPTCHA, statistics with privacy safeguards.

7. Out of scope

Item	Reason
Reader accounts and paywalls	Readers stay anonymous (`docs/v2/PLAN.md`, decision 7)
Native mobile apps	The responsive web app serves phones
Email notifications	Not built (see AMR-DOC-13)
Print layout and press production	Print issues are uploaded as PDFs only
Advertising management	Not part of the brief
Athlete eligibility verification	Not modelled; the system stores only athlete name, program and optional photo
Qualifying heats and multiple entries per faction in contests	Straight finals and one entry per faction were chosen for Salingkusog 2026
Migration of v1 content	A fresh database was chosen (option B, `docs/v2/DB.md`)

8. Success criteria

ID	Criterion	Status (2026-10-19)	Evidence
SC-1	Every table protected by RLS	Met: 50 of 50	`schema.sql`, `salingkusog.sql`, `stats.sql`
SC-2	All automated tests pass before release	Met: 255 unit tests, 278 SQL checks	`npm run check`, `npm run db:test`
SC-3	Accessibility score 100 on key public pages	Met on `/`, `/news`, `/salingkusog`	`docs/v2/PERF.md` (2026-10-18)
SC-4	Colour contrast meets WCAG 2.1 AA for all design tokens	Met	`scripts/check-contrast.mjs`

ID	Criterion	Status (2026-10-19)	Evidence
SC-5	No secrets in the browser bundle	Met: service and Turnstile secrets used only in `api/`	`api/e.ts`, `api/submit.ts`
SC-6	Salingkusog standings need no manual tallying	Met: computed views and RPC	`sk_medal_table`, `sk_points_by_day`
SC-7	The newsroom is usable by all roles during the 2026 edition (Oct 26–30)	[TO CONFIRM after the event]	—
SC-8	Mobile Lighthouse performance ≥ 70 on the home page with real data	Partly met (68–72 on 2026-10-18; image-size fixes deployed after)	`docs/v2/PERF.md`

9. Constraints and assumptions

- **Platforms:** Supabase and Vercel managed services (no own servers).
- **Budget:** free or low-cost tiers [TO CONFIRM plans and costs].
- **Time zone:** Philippine time (UTC+8) for all Salingkusog days and schedules (`src/lib/manila.ts`, `app.manila_today()`).
- **Language:** English user interface.
- **Connectivity:** staff and scorekeepers need an internet connection.
- **Assumption:** staff accounts are created by invitation only and the Supabase “allow new users to sign up” setting is off (`docs/v2/DB.md`) [TO CONFIRM].

10. Project at a glance

Measure	Value	Source
Source files (TypeScript, excluding tests)	206	repository
Lines of TypeScript (incl. tests)	≈ 27,700	repository
Lines of SQL (`supabase/v2`)	4,082	repository
Database tables / views / functions	50 / 6 / 76	AMR-DOC-04
Roles	7 account roles + anonymous reader	`public.app_role`
Public routes / newsroom screens	24 public paths (+ 7 legacy redirects) / 35 newsroom paths and tabs	`src/app/router.tsx` and nested routers
Automated tests	255 unit + 278 SQL checks	AMR-DOC-08
Commits	183 (2025-09-23 → 2026-10-19)	git history
Salingkusog 2026 setup	4 factions, 27 sports, 74 events, 21 venues, 169 games and sessions	live snapshot, AMR-DOC-00 §15

Software Requirements Specification (SRS)

Amaranth v2 — newsroom, public site and Salingkusog module · 1.0 · 2026-10-19

Document control

Field	Value
Title	Software Requirements Specification
Document ID	AMR-DOC-02
Version	1.0
Date	2026-10-19
Author	Amaranth development team [TO CONFIRM: names]
Status	Final draft for defense
Basis	Implemented system at `develop` commit `d7f6f34`; structure adapted from IEEE 29148

Contents

1. Introduction
2. Overall description
3. User classes and characteristics
4. Functional requirements
5. Non-functional requirements
6. External interface requirements
7. Constraints and assumptions
8. User stories with acceptance criteria
9. Requirement-to-design index

1. Introduction

1.1 Purpose

This SRS states what Amaranth v2 must do and how well. It is written from the implemented system: every requirement names the code that satisfies it, so the document can be verified against the repository. Tests that cover each requirement are listed in AMR-DOC-08 (Test Documentation and Requirements Traceability Matrix).

1.2 Scope

Amaranth v2 consists of the public news site, the staff newsroom at `/admin`, and the Salingkusog intramurals module, backed by Supabase and hosted on Vercel (AMR-DOC-01 §6).

1.3 Definitions

Term	Meaning
RLS	Row Level Security: PostgreSQL rules that decide which rows a user may read or change
RPC	Remote procedure call: a database function called from the app through Supabase
Live article	Status `published` or `scheduled`, with `published_at` in the past (`app.is_live`)
Publisher	A user for whom `app.can_publish_articles()` is true
Editor	A user for whom `app.is_editor()` is true (SA, EE, ME, SE)
Salingkusog team	Active staff listed in `sk_team`, plus SA, EE and ME
Edition, faction, event, placement, contest	See AMR-DOC-14 Glossary

Requirement keywords: **shall** = mandatory and implemented; **should** = desirable.

1.4 Requirement ID scheme

Prefix	Area
FR-001 – FR-009	Authentication and accounts
FR-010 – FR-029	Articles and editorial workflow
FR-030 – FR-039	Public site and SEO
FR-040 – FR-049	Public intake: submissions, events, contact, comments
FR-050 – FR-059	Media library
FR-060 – FR-069	Staff, contributors, editorial board
FR-070 – FR-079	Settings and dashboard
FR-080 – FR-089	Statistics
FR-100 – FR-139	Salingkusog

Prefix	Area
NFR-xx	Non-functional requirements

2. Overall description

2.1 Product perspective

A single-page web application (React + TypeScript) that reads and writes Supabase directly with the signed-in user’s token. Database triggers and RLS enforce every rule. A thin Vercel layer handles secrets, caching and SEO (AMR-DOC-03).

2.2 Product functions (summary)

1. Staff sign in by invitation and get role-based access.
2. Staff write, review, schedule and publish articles under an enforced workflow.
3. Readers browse sections, read and search articles, comment, and submit stories, event reports and messages.
4. Editors moderate comments and review public submissions.
5. Staff manage media, contributors, the editorial board and site settings.
6. Managers view privacy-preserving statistics.
7. The Salingkusog team runs the intramural games: setup, schedules, live scores, brackets, contests, results, medals and awards; readers follow along live.

2.3 Operating environment

Modern evergreen browsers on desktop and mobile; Supabase (PostgreSQL 15+, Auth, Storage, Realtime, pg_cron); Vercel (static hosting, Node serverless functions, routing middleware).

3. User classes and characteristics

Class	Role value	Characteristics	Typical tasks
Super admin (SA)	`super_admin`	Technical administrator; at least one must stay active	Accounts, all settings, recovery
Executive Editor (EE)	`editor_in_chief`	Head of the publication	Final editorial control, staff, settings
Managing Editor (ME)	`managing_editor`	Runs daily publishing	Publish, front page, statistics, Salingkusog team
Business Manager (BM)	`business_manager`	Publishes without editorial admin rights	Publish and schedule own articles
Section Editor (SE)	`section_editor`	Assigned to one or more sections	Review and edit articles in those sections
Staff Writer (SW)	`staff_writer`	Regular staff	Draft and submit articles
Contributor (CO)	`contributor`	Occasional writer	Draft and submit articles
Salingkusog team member (SKT)	any role + `sk_team` row	Works on phones during the games	Scores, results, awards
Reader	anonymous	Students, faculty, alumni, public; mostly on mobile	Read, search, comment, submit, follow live scores

4. Functional requirements

Each requirement lists its priority (M = must, S = should) and the implementing evidence.

4.1 Authentication and accounts (FR-001 – FR-009)

ID	Requirement	Pri	Evidence
FR-001	The system shall let staff sign in with email and password.	M	`LoginPage.tsx` (`auth.signInWithPassword`)
FR-002	The system shall let staff reset a forgotten password by email and set a new password from the emailed link.	M	`ForgotPasswordPage.tsx`, `SetPasswordPage.tsx`
FR-003	The system shall grant a role only to emails pre-authorised in `staff_invites`; any other new account shall be created inactive with no access.	M	`app.handle_new_user`
FR-004	The system shall block inactive or uninvited users from the newsroom and show an “access pending” screen.	M	`RequireStaff` in `guards.tsx`, `app.current_role`
FR-005	Only SA and EE shall change roles and activation; only SA shall grant or remove the super_admin role.	M	`app.guard_profile_update`
FR-006	The system shall refuse to demote or deactivate the last active super admin.	M	`app.guard_profile_update`
FR-007	The system shall record role, activation and publishing-access changes in the audit log.	M	`app.audit`, `audit_log`
FR-008	The database shall let staff update their own display name, bio and avatar (but not role or access). A self-service profile screen is not built yet (AMR-DOC-13).	S	`profiles_update` policy, `profiles_guard`

4.2 Articles and editorial workflow (FR-010 – FR-029)

ID	Requirement	Pri	Evidence
FR-010	Any active staff member shall be able to create a draft article with title, category, kicker, dek, body (rich text), cover image and caption, authors, tags, byline override, SEO title and description, and social image.	M	`ArticleEditorPage.tsx`, `articles` table
FR-011	The section of an article shall always follow its category.	M	`app.article_before_write`
FR-012	The system shall generate a unique URL slug from the title when none is given, and compute reading time at 220 words per minute.	M	`app.article_before_write`
FR-013	The editor shall autosave changes 2.5 seconds after the last edit and keep the newest 50 revisions per article.	M	`AUTOSAVE_MS` in `ArticleEditorPage.tsx`; `app.prune_revisions`
FR-014	The system shall enforce these status transitions: draft → submitted (owner or editor); submitted → in review, rejected (editor) or draft (owner or editor); in review → draft or rejected (editor); any of draft, submitted, in review → scheduled or published (publisher); scheduled → published or draft (publisher); published → archived or draft (publisher); archived → published or draft (publisher); rejected → draft or submitted (owner or editor).	M	`app.article_before_write`, `transition_article`
FR-015	Owners who are not editors shall not edit an article while it is submitted, in review, scheduled, published or archived.	M	`app.article_before_write` (“locked while…”)
FR-016	Only publishers (SA, EE, ME, BM, or staff granted “Can publish”) shall publish, schedule, reschedule or archive.	M	`app.can_publish_articles`

ID	Requirement	Pri	Evidence
FR-017	Only SA, EE and ME shall grant or remove another person’s publishing access; changes shall be audited.	M	`set_publish_access`, `profiles_guard`
FR-018	Scheduling shall require a future time; a scheduled article shall go live automatically when the time passes, without a background job.	M	`app.is_live`; trigger check “Pick a future time”
FR-019	Every status change shall be logged with an optional note; staff shall be able to add notes without changing status.	M	`article_activity`, `transition_article`, `add_article_note`
FR-020	Only editors of the article’s section shall feature an article or change its editorial priority.	M	`app.article_before_write`
FR-021	Categories marked not assignable shall be reserved for editors.	M	`categories.is_assignable`; trigger check
FR-022	SA and EE shall delete any article; other staff shall delete only their own drafts.	M	`articles_delete` policy
FR-023	Staff shall see their own articles, articles they co-author, and articles in sections they edit; publishers shall see all.	M	`articles` select policies
FR-024	The article list shall filter by status, section, “mine” and “needs review”, with pagination.	M	`ArticlesListPage.tsx`, `cms/articles/api.ts`
FR-025	An article may be linked to a print issue and position.	S	`articles.issue_id`, `issue_position`
FR-026	Readers shall see article view counts only through aggregated “most read” lists.	S	`most_read_articles`

4.3 Public site and SEO (FR-030 – FR-039)

ID	Requirement	Pri	Evidence
FR-030	The home page shall show the lead story, latest ticker, most read, latest stories, literary folio, sports and views blocks, and a call to contribute.	M	`features/public/home/*`, `home_slots`

ID	Requirement	Pri	Evidence
FR-031	Each of the 12 sections shall have a landing page with its own layout and a paginated “all stories” view filterable by category.	M	`features/public/section/*`, `sectionLandings`
FR-032	Article pages shall show sanitised body HTML, authors, tags, related stories, share buttons and approved comments.	M	`ArticleView.tsx`, `src/lib/sanitize.ts`
FR-033	The system shall count at most one view per visitor, per article, per 30 minutes.	M	`record_article_view`; `docs/v2/STATS.md`
FR-034	Readers shall search live articles by text, section and date range, and browse by tag.	M	`search_articles`, `SearchPage.tsx`
FR-035	Readers shall browse archives by month and download published print issues.	M	`archive_months`, `ArchivesPage.tsx`, `archives` bucket
FR-036	The site shall provide About, Editorial Board and Privacy pages.	M	`features/public/about/*`
FR-037	Link previews on social media shall show the article’s or section’s title, description and image.	M	`middleware.ts`, `api/_lib/seo.ts`
FR-038	The site shall publish `sitemap.xml`, `rss.xml` and `robots.txt`.	M	`api/sitemap.ts`, `api/rss.ts`, `api/robots.ts`
FR-039	Old v1 addresses shall redirect to their v2 equivalents.	S	`redirects` in `router.tsx`

4.4 Public intake: submissions, events, contact, comments (FR-040 – FR-049)

ID	Requirement	Pri	Evidence
FR-040	Readers shall submit a story with name, email, affiliation, section, title, summary, body and up to 5 attachments of up to 20 MB each (JPG, PNG, WebP, PDF, Word, text).	M	`SubmitArticlePage.tsx`, `api/_lib/forms.ts`, `submit_article`
FR-041	Readers shall report an event with organisation, contact, title, date, venue, summary and attachments.	M	`SubmitEventPage.tsx`, `submit_event_report`

ID	Requirement	Pri	Evidence
FR-042	Readers shall send a contact message with a topic category.	M	`ContactPage.tsx`, `send_contact_message`
FR-043	Readers shall comment on live articles that allow comments; comments shall stay hidden until an editor approves them; replies shall be allowed only to approved comments; commenter emails shall be visible to editors only.	M	`submit_comment`, `article_comments`, `article_comment_private`
FR-044	Every public form shall pass a Cloudflare Turnstile check on the server before anything is saved.	M	`api/submit.ts`
FR-045	Public forms shall be rate-limited per email: comments 5 per 10 minutes, submissions 3 per hour, event reports 3 per hour, contact 3 per 10 minutes; a hidden honeypot field shall silently drop bots.	M	`app.rate_limit` calls in `schema.sql`
FR-046	Attachments shall upload only through one-time signed links issued after the CAPTCHA, and the submission shall be saved only with a valid signed ticket for the same content.	M	`api/submit.ts`, `makeTicket/readTicket`
FR-047	Editors shall review submissions and event reports (submitted, reviewing, accepted, declined) with notes, open attachments, and convert one into a draft article.	M	`IntakePage.tsx`, `convert_intake_to_article`
FR-048	Editors shall manage the contact inbox (unread, in progress, resolved, spam).	M	`ModerationTabs.tsx`, `contact_messages`
FR-049	Editors shall moderate comments (pending, approved, rejected, spam) and the site may switch comments off globally.	M	`site_settings` key `comments`, `ModerationTabs.tsx`

4.5 Media library (FR-050 – FR-059)

ID	Requirement	Pri	Evidence
FR-050	Staff shall upload images that are compressed in the browser to WebP and stored with sizes 320, 640, 960 and 1280 px.	M	`src/lib/images.ts`, `docs/v2/PERF.md`
FR-051	Each asset shall have alt text, caption and credit.	M	`media_assets`
FR-052	The library shall be searchable, filterable to “mine”, and paginated.	M	`MediaLibraryPage.tsx`, `mediaApi.ts`
FR-053	An asset in use (cover, social image, logo, issue, etc.) shall not be deletable; the library shall show where it is used.	M	`media_usage()`, foreign keys `on delete restrict`
FR-054	Editors shall generate missing web sizes for older images (“Make web sizes”).	S	`cms/media/*`
FR-055	Only publishers shall upload to the `archives` bucket (print-issue PDFs).	M	storage policy on `archives`

4.6 Staff, contributors and editorial board (FR-060 – FR-069)

ID	Requirement	Pri	Evidence
FR-060	SA and EE shall invite staff by email with a role and, for section editors, sections.	M	`InvitesTab.tsx`, `staff_invites`
FR-061	SA and EE shall change a member’s role, active status and assigned sections; SA, EE and ME shall see the Accounts tab to manage publishing access.	M	`StaffTab.tsx`, `PeoplePage.tsx`
FR-062	Editors shall manage contributor bylines (staff, student, guest, faculty, alumni); staff may edit their own byline.	M	`ContributorsTab.tsx`, `contributors` policies
FR-063	Publishers (SA, EE, ME) shall manage editorial board terms and members shown on the public Editorial Board page.	M	`BoardTab.tsx`, `team_terms`, `team_members`

4.7 Settings and dashboard (FR-070 – FR-079)

ID	Requirement	Pri	Evidence
FR-070	SA, EE and ME shall choose the home page lead story and literary folio pick (with pull quote).	M	`home_slots`, `HomeTab`
FR-071	SA and EE shall manage sections (name, description, navigation placement and order, accepts submissions) and categories.	M	`sections`, `categories`
FR-072	SA and EE shall manage desk contacts shown on the contact page.	M	`department_contacts`
FR-073	SA and EE shall edit general settings (site details, social links, comments switch); SA, EE and ME shall edit the latest-news ticker.	M	`site_settings` policy
FR-074	The dashboard shall show the counts the user may see: published, published this week, scheduled, drafts, in review, and article views this month versus last month (writers see their own drafts only).	M	`dashboard_stats()`
FR-075	SA and EE shall read the audit log.	S	`audit_log` select policy

4.8 Statistics (FR-080 – FR-089)

ID	Requirement	Pri	Evidence
FR-080	The public site shall record pageviews, reading time, scroll depth and share clicks without cookies, only on the production host and not for signed-in staff.	M	`features/public/analytics/*`
FR-081	The ingest endpoint shall drop bots and other origins, never store IP addresses, and hash visitor ids with a daily salt deleted after two days.	M	`api/e.ts`, `stats.sql`
FR-082	Do Not Track and Global Privacy Control shall limit tracking to an anonymous pageview.	M	`docs/v2/STATS.md`

ID	Requirement	Pri	Evidence
FR-083	SA, EE and ME shall view statistics by date range: summary with change versus the previous period, trend, top articles, authors, sections, sources, referrers, devices, browsers, share channels, and staff active time.	M	`StatisticsPage.tsx`, `stats_*` RPCs
FR-084	Raw events shall be kept 13 months, staff activity 24 months, daily totals indefinitely.	M	`app.stats_nightly`

4.9 Salingkusog (FR-100 – FR-139)

ID	Requirement	Pri	Evidence
FR-100	The team shall manage editions: year, title, dates, medal points (default 5/3/1), current and published flags, schedule PDF and venue map.	M	`sk_editions`, `EditionTab.tsx`
FR-101	Only one edition shall be current at a time; readers shall see an edition only once it is published.	M	`sk_single_current`, `app.sk_edition_visible`
FR-102	The team shall manage factions (name, short name, motto, colour, logo, order).	M	`sk_factions`, `FactionsTab.tsx`
FR-103	The team shall manage venues with a description and map position.	M	`sk_venues`, `VenuesTab.tsx`
FR-104	The team shall manage sports (category) and events (division, format, team or individual, result unit, venue). Formats: bracket, round robin, round robin + final, timed or measured, judged.	M	`sk_sports`, `sk_events`, `SportsTab.tsx`
FR-105	The team shall manage athletes (name, program, photo) per faction.	M	`sk_athletes`, `AthletesTab.tsx`
FR-106	Only SA, EE and ME shall add or remove Salingkusog team members; changes shall be audited.	M	`sk_team` policies, `sk_team_audit`

ID	Requirement	Pri	Evidence
FR-107	The team shall import the schedule from CSV (date, time, sport, event, round, side A, side B, venue) with validation and duplicate detection before saving.	M	`scheduleCsv.ts`, `ImportTab.tsx`
FR-108	“Build schedule” shall create an event’s games for single elimination (with bronze game and byes), round robin, or round robin + final, from factions in seed order, a first-game time and gaps between rounds and games.	M	`sk_build_schedule`, `BuildScheduleModal.tsx`
FR-109	Build schedule shall refuse to rebuild an event with games played and shall warn when the last round falls after the edition ends.	M	`sk_build_schedule`; `BuildScheduleModal.tsx`
FR-110	After each result, the system shall fill the next games automatically: winners and losers of knockout games, the next round-robin round once the previous one is finished, and the final from the round-robin standings.	M	`app.sk_resolve`, `app.sk_source_side`, trigger `sk_matches_advance`
FR-111	A game that has started shall never change teams automatically; an edit that would change it shall be refused with a message.	M	`app.sk_resolve`
FR-112	A knockout game shall not be finished as a tie.	M	`app.sk_knockout_needs_winner`; `Scorekeeper.tsx`
FR-113	Round-robin standings shall rank by wins, point difference, head-to-head among tied teams, points scored, then seed.	M	`app.sk_rr_table`, `roundRobinTable()`
FR-114	“Create contest” shall create one session with every chosen faction for timed or judged events, in faction order or a random draw.	M	`sk_create_contest`, `CreateContestModal.tsx`

ID	Requirement	Pri	Evidence
FR-115	The contest scorekeeper shall take each entry’s result (time, distance or points; unit “s” means lowest wins) or, for judged events, the announced place; entries may be marked did not start, did not finish or disqualified; equal results share a place; finishing shall require every competing entry to have a result.	M	`ContestScorekeeper.tsx`, `src/lib/skContests.ts`, `sk_entries`
FR-116	The scorekeeper shall update scores, periods, a live note, team statistics and athlete box scores, and move a game through not started, live, final, postponed and cancelled.	M	`Scorekeeper.tsx`, `BoxScore.tsx`, `statusActions()`
FR-117	Score changes, results and contest entries shall reach open public pages in real time.	M	Realtime publication; `useLiveGames()`
FR-118	The team shall enter results per event (places, faction, athlete, result, day awarded); medals follow the place (1 gold, 2 silver, 3 bronze).	M	`sk_set_placements`, `ResultsTab.tsx`
FR-119	When an event’s last game is final, the system shall suggest the podium from the games and staff shall confirm it before it counts.	M	`suggestedPodium()`, Results “Ready to confirm”
FR-120	The medal table shall be computed: gold, silver, bronze, total, points and rank (by points, then gold, then silver).	M	`sk_medal_table`
FR-121	The points race shall show cumulative points by day, or in 1, 3, 6 or 12-hour steps.	M	`sk_points_by_day`, `sk_points_timeline`
FR-122	The tally shall show “results in: X of Y matches” and, once every match is final, read “Official and final medal tally” and name the champion and runners-up.	M	`resultsProgress()`, `finalHeadline()`

ID	Requirement	Pri	Evidence
FR-123	The team shall record awards (MVP of the day, athlete of the day, finals MVP) with stat lines and quotes, and download social cards as PNG (1080×1350 and 1080×1080).	M	`sk_awards`, `AwardsTab.tsx`, `socialCard.ts`
FR-124	The public hub shall provide medal tally, schedule and results (day tabs, filters by sport, faction and venue), winners, factions and profiles, sport pages (bracket, standings or contest results), match recaps and a venue guide.	M	`features/public/salingkusog/*`
FR-125	Upcoming generated games shall say where each side comes from (“Winner of Semifinal 1”, “2nd in the standings”).	M	`pendingSide()`
FR-126	SA, EE and ME shall control a public countdown page that works before the edition is published.	M	`sk_countdown()`, `CountdownCard.tsx`, `sk_countdown_guard`
FR-127	The system shall compute head-to-head records, event completion and stat leaders.	S	`sk_head_to_head`, `sk_event_status`, `sk_event_leaders`

5. Non-functional requirements

ID	Category	Requirement	Evidence / verification
NFR-01	Performance	Public pages shall load routes lazily and split vendor code into chunks.	`router.tsx`, `vite.config.ts`
NFR-02	Performance	Public, slow-changing reads by signed-out readers shall be served from an edge cache (60 s, stale-while-revalidate 300 s); live scores shall bypass it.	`api/r.ts`, `src/lib/edgeCache.ts`
NFR-03	Performance	Images shall be served at the smallest suitable size (`srcset`, 320–1280 px) and lazy-loaded below the fold.	`Img.tsx`, `src/lib/media.ts`
NFR-04	Performance	Long lists shall be paginated (12–24 items per page).	`docs/v2/PERF.md`
NFR-05	Security	Every table shall have RLS enabled; no table shall be writable by anonymous users.	97 policies; `public_access.test.sql`
NFR-06	Security	Database helper functions shall live in the private `app` schema and use `set search_path = ''`.	`schema.sql`
NFR-07	Security	Secret keys (service role, Turnstile secret) shall be used only in server functions.	`api/e.ts`, `api/submit.ts`
NFR-08	Security	Article and recap HTML shall be sanitised; only YouTube, Vimeo, Facebook and Spotify iframes allowed.	`src/lib/sanitize.ts`
NFR-09	Security	Responses shall carry HSTS, X-Frame-Options, nosniff, Referrer-Policy and Permissions-Policy headers, and a Content-Security-Policy (currently report-only).	`vercel.json`
NFR-10	Privacy	No third-party trackers or cookies for statistics; IP addresses not stored; data retention limits enforced.	`stats.sql`, `/privacy`
NFR-11	Privacy	Commenter and submitter emails shall be visible to editors only.	`article_comment_private`; intake policies
NFR-12	Accessibility	Design-token colour pairs shall meet WCAG 2.1 AA contrast.	`scripts/check-contrast.mjs`

ID	Category	Requirement	Evidence / verification
NFR-13	Accessibility	Pages shall have one `h1`, ordered headings, named controls and alt text; motion shall respect reduced-motion settings.	`src/dev/domCheck.ts`; `useTween.ts`
NFR-14	Usability	Every screen shall work at 375 px width without horizontal scrolling.	`/dev` sweep (53 screens)
NFR-15	Usability	The scorekeeper shall be usable one-handed on a phone (large buttons, sticky actions).	`Scorekeeper.tsx`
NFR-16	Reliability	SQL files shall be idempotent and safe to re-run in any order after the first deploy.	`scripts/db-test.mjs` applies them twice
NFR-17	Reliability	Statistics failures shall never break the site; `/api/e` answers 501 when not configured.	`api/e.ts`
NFR-18	Maintainability	TypeScript strict mode, ESLint, and `npm run check` (lint, typecheck, tests, build) shall pass before every merge.	`package.json`, `docs/v2/PROGRESS.md`
NFR-19	Maintainability	Business rules shall live in the database so that the UI and direct API calls behave the same.	triggers and RLS
NFR-20	Compatibility	The system shall support current Chrome, Edge, Firefox and Safari on desktop and mobile.	[TO CONFIRM browser test matrix]
NFR-21	Time	All Salingkusog dates and times shall use Philippine time (UTC+8).	`src/lib/manila.ts`, `app.manila_today()`

6. External interface requirements

Interface	Description
User interface	Responsive web UI; brand tokens in `src/styles/tokens.css`; Montserrat and Cinzel fonts
Supabase REST (PostgREST)	Table reads/writes and RPCs with the user's JWT
Supabase Auth	Password sign-in, reset email, invitations
Supabase Storage	Buckets `media`, `archives`, `intake`
Supabase Realtime	Changes to `sk_matches`, `sk_placements`, `sk_entries`
Cloudflare Turnstile	Browser widget and server verification (`siteverify`)

Interface	Description
Vercel	Functions under `/api`, routing middleware, CDN cache, Web Analytics
Crawlers	Open Graph HTML via `middleware.ts`; `sitemap.xml`, `rss.xml`, `robots.txt`

Details of every endpoint and RPC: AMR-DOC-07.

7. Constraints and assumptions

- C1: Managed services only (Supabase, Vercel); no custom servers.
- C2: Database changes are delivered as re-runnable SQL pasted in the Supabase SQL editor (no migration tool).
- C3: English user interface.
- A1: Staff have internet access during work and during the games.
- A2: The Supabase “allow sign-ups” setting is disabled in production [TO CONFIRM].
- A3: Factions, sports and events for each edition are configured before the games start.

8. User stories with acceptance criteria

ID	Story	Acceptance criteria	Req.
US-01	As a staff writer, I want to save my draft automatically so that I never lose work.	Given an open draft, when I stop typing for 2.5 s, then the change is saved and appears in the revision history.	FR-013
US-02	As a staff writer, I want to submit my article for review.	Given my draft, when I choose “Submit for review”, then its status becomes submitted, I can no longer edit it, and the activity log shows the step.	FR-014, FR-015, FR-019
US-03	As a section editor, I want to return an article with a note.	Given a submitted article in my section, when I choose “Return to draft” with a note, then it becomes a draft and the writer sees the note.	FR-014, FR-019
US-04	As the Managing Editor, I want to schedule an article.	Given a reviewed article, when I pick a future time and schedule it, then it becomes public at that time without further action; a past time is refused.	FR-016, FR-018
US-05	As a staff writer without publishing access, I must not be able to publish.	When I try to publish (in the UI or by calling the API), then the database refuses with “You cannot move this article…”.	FR-016
US-06	As the Managing Editor, I want to give a trusted writer publishing access.	When I switch on “Can publish and schedule articles” for them, then they can publish their own articles and the change is in the audit log.	FR-017, FR-007
US-07	As a reader, I want to search for an article.	When I enter words and optionally a section and dates, then live articles matching them are listed, newest first, paginated.	FR-034
US-08	As a reader, I want to submit a story with a photo.	Given I pass the security check, when I attach up to 5 files of ≤ 20 MB and send, then the editors receive the submission with the files; a 6th file or a wrong type is refused.	FR-040, FR-044, FR-046

ID	Story	Acceptance criteria	Req.
US-09	As an editor, I want to approve a comment.	Given a pending comment, when I approve it, then it appears under the article; until then readers do not see it.	FR-043, FR-049
US-10	As an editor, I want to turn a good submission into a draft article.	When I choose “Convert to article” and a category, then a draft with the submission’s content exists and the submission is marked converted.	FR-047
US-11	As a Salingkusog team member, I want to build a bracket once.	Given 4 factions in seed order, when I build a single elimination with a bronze game, then 2 semifinals, a final and a bronze game exist; seeds 1 v 4 and 2 v 3 meet first.	FR-108
US-12	As a scorekeeper, I want the final to fill itself in.	When I finish both semifinals, then the winners appear in the final and the losers in the bronze game.	FR-110
US-13	As a scorekeeper, I must not finish a knockout game tied.	When the score is level and I press Finish, then the system refuses and asks for the tiebreak.	FR-112
US-14	As a scorekeeper, I want to record a swimming race.	Given a contest with 4 entries, when I enter times, then places update (fastest first, ties shared); DNF entries get no place; Finish is refused while a time is missing.	FR-114, FR-115
US-15	As the Managing Editor, I want to confirm medals.	When an event’s last game is final, then Results shows “Ready to confirm” with the suggested podium; nothing counts until I save.	FR-118, FR-119
US-16	As a reader, I want the medal tally to update live.	When a result is saved, then open tally pages update within seconds without reloading.	FR-117, FR-120
US-17	As a reader, I want to know whether the tally is final.	When every match is final, then the page reads “Official and final medal tally” and names the champion and runners-up.	FR-122
US-18	As the Executive Editor, I want readership numbers without tracking people.	When I open Statistics, then I see totals and trends; no cookies are set on readers and no IP address is stored.	FR-080 – FR-084

ID	Story	Acceptance criteria	Req.
US-19	As the Super admin, I want to invite a new staff member.	When I add their email and role to Invites and they accept the email invitation, then they get an active profile with that role.	FR-003, FR-060
US-20	As a section editor, I want to see only what I am responsible for.	When I open Articles, then I see my own articles and those in my sections, but not other sections' drafts.	FR-023

9. Requirement-to-design index

Requirement group	Design section (AMR-DOC-03)	Data (AMR-DOC-04)	Diagrams (AMR-DOC-05)
FR-001 – FR-008	§5.1 Authentication and authorisation	`profiles`, `staff_invites`, `audit_log`	SEQ-1 login and role check
FR-010 – FR-026	§5.2 Editorial workflow	`articles`, `article_*`	ACT-1, SEQ-2, STATE-1
FR-030 – FR-039	§5.3 Public site and SEO	`article_cards`, `sections`	DFD-1
FR-040 – FR-049	§5.4 Public intake gateway	intake tables, `article_comments`	SEQ-6 public form
FR-050 – FR-055	§5.5 Media pipeline	`media_assets`, buckets	—
FR-060 – FR-075	§5.6 Administration	people and settings tables	UC-1 … UC-3
FR-080 – FR-084	§5.7 Statistics	`stats_*`	DFD-1
FR-100 – FR-127	§5.8 Salingkusog engine	`sk_*` tables and views	ACT-2, SEQ-3 – SEQ-5, STATE-2

Software Design Document (SDD) and Architecture

Amaranth v2 — layers, components, decisions and trade-offs · 1.0 · 2026-10-19

Document control

Field	Value
Title	Software Design Document and Architecture
Document ID	AMR-DOC-03
Version	1.0
Date	2026-10-19
Author	Amaranth development team [TO CONFIRM: names]
Status	Final draft for defense
Basis	IEEE 1016 structure; implemented system at commit `d7f6f34`

Contents

1. Introduction
2. Architectural goals and drivers
3. System overview
4. Layered architecture
5. Subsystem designs
6. Cross-cutting concerns
7. Technology choices and justification
8. Architecture decision records (ADRs)
9. Trade-offs and their consequences

1. Introduction

This document explains **how** Amaranth v2 is built and **why**. It covers the architecture, the design of each subsystem, and the decisions behind them. Requirements are in AMR-DOC-02 and data structures in AMR-DOC-04; diagrams referenced here (ARCH-1, DEP-1, SEQ-n …) are in AMR-DOC-05.

2. Architectural goals and drivers

Driver	Consequence for the design
Small student team, no operations staff	Managed platforms only (Supabase, Vercel); no servers to patch
Permissions must hold even if the UI is bypassed	Rules live in PostgreSQL (RLS, triggers, security-definer helpers)
Readers mostly on mobile data	Code splitting, edge caching, responsive images, lazy loading
Live intramural scores	Supabase Realtime on the Salingkusog tables
No budget for third-party analytics, privacy concerns	First-party, cookie-less statistics
Public forms attract spam	Server-side CAPTCHA gateway, rate limits, honeypot
Maintainability by future student developers	TypeScript strict, small feature folders, re-runnable SQL, tests

3. System overview

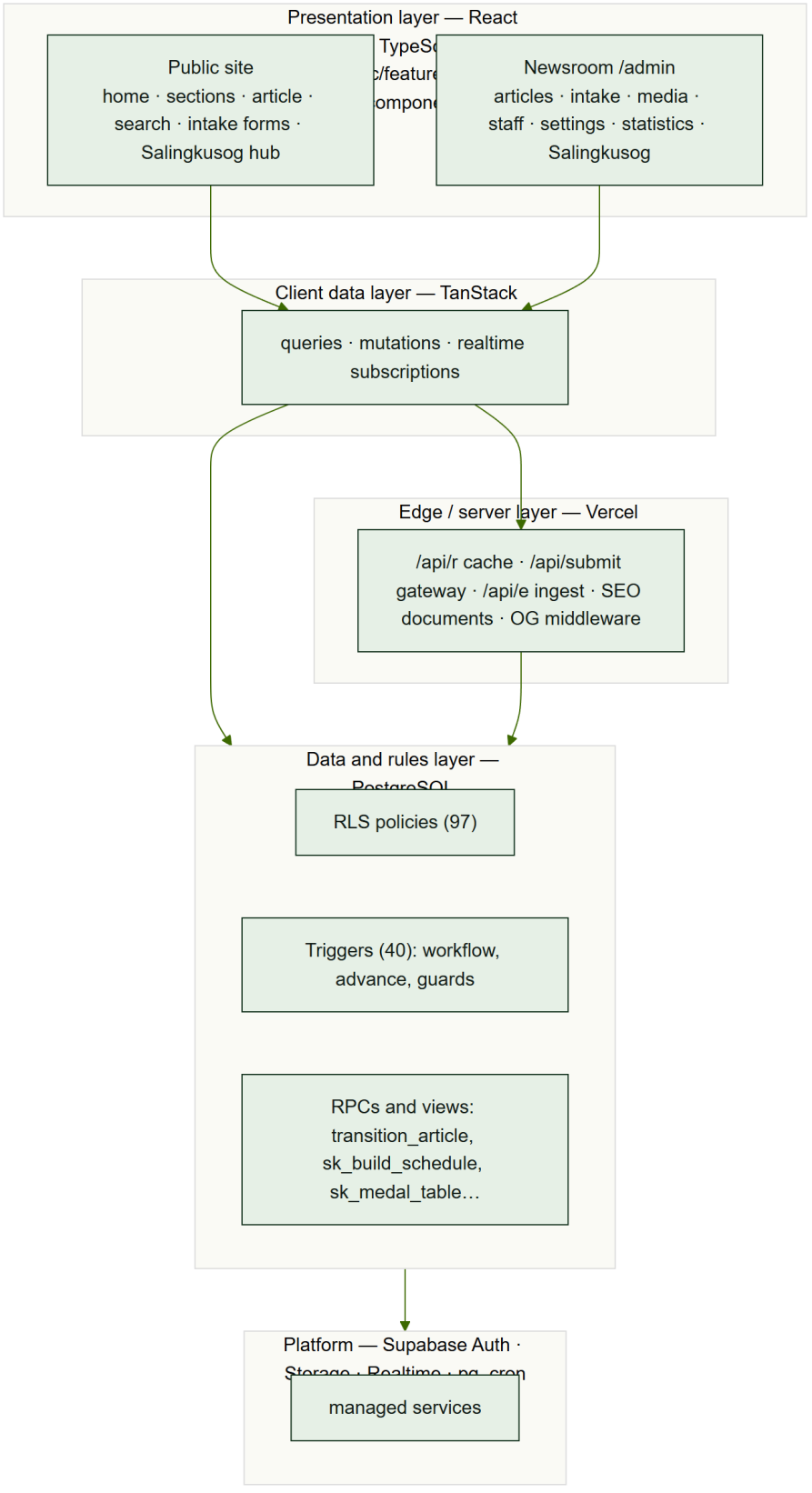
Amaranth v2 is a **single-page application** served from Vercel's CDN. The browser talks **directly to Supabase** (PostgREST, Auth, Storage, Realtime) using the signed-in user's JSON Web Token, or the anonymous key for readers. Because every table has RLS, the database decides what each request may see or change.

A thin **edge/server layer** on Vercel exists only where the browser cannot be trusted or where caching helps:

- `api/submit.ts` holds the Turnstile secret and the Supabase service key for public forms;
- `api/e.ts` holds the service key for statistics ingest;
- `api/r.ts` caches public reads at the edge;
- `api/sitemap.ts`, `api/rss.ts`, `api/robots.ts` and `middleware.ts` serve crawlers.

See CTX-1, DEP-1 and ARCH-1 (AMR-DOC-05).

4. Layered architecture



ARCH-1 — layered architecture.

Layer	Contents	Rules
L1 Presentation	React pages and components in `src/features/{public,cms,auth}` and `src/components`	No business rules beyond what the UI needs to show or hide; permissions mirrored from the DB in `permissions.ts` for display only
L2 Client data	TanStack Query hooks (`*Api.ts`, `*Data.ts`), `src/lib/supabase.ts`, `edgeCache.ts`, `sendForm.ts`	All reads and writes go through hooks; cache keys per feature; realtime invalidates keys
L3 Edge/server	`api/*.ts`, `middleware.ts`	Stateless; secrets only here; never trusts input; Node ESM with `.js` imports
L4 Data and rules	`supabase/v2/*.sql`: tables, RLS, triggers, RPCs, views	The source of truth for permissions and invariants
L5 Platform	Supabase Auth, Storage, Realtime, pg_cron	Managed services

Dependency rule: a layer may call only the layers below it. L1 never calls L3 directly except through L2 helpers (`sendForm`, `edgeCache`).

5. Subsystem designs

5.1 Authentication and authorisation (FR-001 – FR-008)

- **Sign-in:** `LoginPage.tsx` calls `auth.signInWithPassword`; `AuthProvider.tsx` listens to `onAuthStateChange`, loads the profile, and exposes `status` and `profile` (SEQ-1).
- **Account creation:** only through Supabase invitations. `app.handle_new_user` (trigger `on_auth_user_created`) creates a profile; if the email is in `staff_invites`, it receives that role, is activated and gets assigned sections; otherwise it is inactive.
- **Authorisation:** `app.current_role()` returns the role of the active profile. Helpers (`is_staff`, `is_admin`, `can_publish`, `can_publish_articles`, `is_editor`, `can_edit_section`, `can_view_stats`, `can_edit_sk`, `can_manage_sk_team`) are `security definer`, `stable`, with `search_path = ''`, in the private `app` schema so the API cannot call them directly.
- **Guards in the UI:** `RequireStaff` (active staff only) and `RequireRole` (per screen). These improve the experience; they are not the security boundary.
- **Profile protection:** trigger `profiles_guard` blocks email or id changes, limits role and activation changes to SA/EE, limits super_admin changes to SA, protects the last super admin, limits publishing-access changes to SA/EE/ME, and writes audit entries.

5.2 Editorial workflow (FR-010 – FR-026)

- **State machine** (STATE-1) enforced by `app.article_before_write` (BEFORE INSERT/UPDATE on `articles`). The trigger also derives the section from the category, builds the slug, computes reading time, sets timestamps, freezes owners' edits while an article is under review or live, and limits featuring to section editors.
- **Transitions** go through `public.transition_article(id, to, note, scheduled_for)`, which passes the note to the trigger via a transaction setting (`app.transition_note`). `app.article_after_write` logs `article_activity` and stores a `status` revision (SEQ-2).

- **Autosave and revisions:** the editor saves 2.5 s after the last change (`AUTOSAVE_MS`); revisions of kind `autosave`/`manual`/`status`; `app.prune_revisions` keeps the newest 50.
- **Scheduling without cron:** a scheduled article gets `published_at = scheduled_for`; all public reads filter with `app.is_live(status, published_at)`, so it appears on time. `app.publish_due_articles()` can flip the status for tidiness through pg_cron (optional).
- **Editor:** TipTap 3 with headings, lists, quotes, links, images from the media library and video embeds; it stores `body_json` (editor document), `body_html` (rendered) and `body_text` (search and reading time).

5.3 Public site and SEO (FR-030 – FR-039)

- **Routing:** `createBrowserRouter` with lazy routes; the public shell and home page are bundled eagerly because most visitors land there.
- **Sections:** one data-driven `SectionPage` plus a registry of custom landings (`sectionLandings`) for the 12 sections.
- **Article cards:** the `article_cards` view (security invoker) joins section, category, byline and cover so lists need one query.
- **Search:** `search_articles` uses the generated `tsvector` column `articles.search` with weights, plus section and date filters.
- **Edge cache:** for signed-out readers, `src/lib/edgeCache.ts` rewrites eligible GETs to `/api/r?p=…`; `api/r.ts` forwards them with the anon key and caches successful responses at the CDN for 60 s (`stale-while-revalidate=300`). Live Salingkusog data, comments and anything personal are excluded (`CACHEABLE` list).
- **Crawler previews:** `middleware.ts` detects crawler user agents and returns `index.html` with Open Graph tags for `/article/:slug` and section pages; regular browsers pass through untouched.

5.4 Public intake gateway (FR-040 – FR-049)

The four public forms (comment, story submission, event report, contact) call `sendForm()` in the browser, which obtains a Turnstile token and posts to `/api/submit` (SEQ-6).

1. `api/submit.ts` validates the form kind and whitelists its fields (`pickArgs`), checks attachments (≤ 5, ≤ 20 MB, allowed types — `pickFiles`), and verifies the token with Cloudflare `siteverify`.
2. With attachments, it creates **one-time signed upload URLs** in the private `intake` bucket and returns them with a **ticket** signed by HMAC-SHA-256 over the form kind, the field values, the file list and a 30-minute expiry.
3. The browser uploads the files, then posts again with the ticket; the server re-checks the ticket and calls the RPC with the service key.
4. The RPC (`submit_article`, `submit_event_report`, `submit_comment`, `send_contact_message`) validates text and email, applies the honeypot and per-email rate limit (`app.rate_limit`), checks that attached files exist, and inserts.

The RPCs are executable only by `service_role`, and the open anonymous upload policy was removed, so the gateway cannot be bypassed.

5.5 Media pipeline (FR-050 – FR-055)

Images are compressed in the browser (`src/lib/images.ts`) to WebP and uploaded in four widths (320, 640, 960, 1280). `media_assets` stores bucket, path, size, dimensions, alt text, caption and credit. `Img.tsx` builds `srcset`/`sizes`. Deletion is blocked while an asset is referenced (`on delete restrict` foreign keys) and `media_usage()` lists where it is used.

5.6 Administration (FR-060 – FR-075)

People (`cms/people`), settings (`cms/settings`) and dashboard (`cms/dashboard`) are standard list/detail screens over RLS-protected tables. `site_settings` is a key–value store (`jsonb`) with an `is_public` flag; the ticker key is editable by publishers, every other key only by admins.

5.7 Statistics (FR-080 – FR-084)

The tracker batches events and sends them with `sendBeacon` to `/api/e`. The function drops bots, rejects foreign origins and large bodies, classifies device/browser/OS and traffic source, and calls `stats_ingest` with the service key (no IP forwarded). The database hashes visitor ids with a daily salt, deduplicates article views, and `app.stats_nightly` (pg_cron, 00:15 Manila) rolls up daily tables and enforces retention. Read RPCs `stats_*` check `app.can_view_stats()` themselves, in addition to RLS.

5.8 Salingkusog engine (FR-100 – FR-127)

Concern	Design
Data ownership	Every `sk_*` row belongs to an edition; triggers copy `edition_id` from the parent (`app.sk_inherit_edition`) so data cannot mix editions; composite foreign keys `(edition_id, id)` enforce it
Visibility	Public reads only for published editions (`app.sk_edition_visible`); staff see drafts
Write access	`app.can_edit_sk()`: SA/EE/ME always, other staff while in `sk_team`
Game generation	`sk_build_schedule` creates games whose sides have **sources**: `seed:n`, `winner:<id>`, `loser:<id>`, `rank:n`
Advancement	Trigger `sk_matches_advance` → `app.sk_resolve(event)` recomputes every generated game's sides idempotently; it refuses to change a game that is not `upcoming`; recursion is stopped with `pg_trigger_depth()`
Round robin ranking	`app.sk_rr_table` (SQL) and `roundRobinTable()` (TypeScript) use the same order: wins, difference, head-to-head among the tied, points scored, seed
Contests	A contest session is an `sk_matches` row with `is_contest = true` and no sides; its line-up and results are `sk_entries` (unique per faction); ranking in `src/lib/skContests.ts` (unit "s" → lowest wins)
Results and medals	`sk_set_placements` replaces an event's placements in one transaction; medals derive from places; the UI suggests a podium (`suggestedPodium`) but staff must confirm

Concern	Design
Standings	Views compute medals, points and rank (`sk_medal_table`), cumulative points by day (`sk_points_by_day`), and an RPC computes hour steps (`sk_points_timeline`)
Live updates	Realtime publication on `sk_matches`, `sk_placements`, `sk_entries`; `useLiveGames()` invalidates the hub, standings, timeline and results queries; polling every 60 s remains as a fallback

See ACT-2, SEQ-3, SEQ-4, STATE-2 and ERD-3.

6. Cross-cutting concerns

Concern	Approach	Evidence
Error handling	Database errors are raised with readable messages and codes (`42501` permission, `22023` invalid input, `55000` state conflict) and shown by `skErrorText` / `errorText`	`src/lib/errors.ts`
Validation	Server-side in RPCs and triggers (length checks, email check, enums, check constraints); client-side for convenience	`app.check_text`, `app.check_email`
Time zone	Manila time for Salingkusog (`src/lib/manila.ts`, `app.manila_today`)	
Accessibility	Semantic HTML, labelled controls, reduced-motion support, contrast-checked tokens	`domCheck.ts`, `check-contrast.mjs`
Performance	Lazy routes, vendor chunks (react, supabase, query, editor), responsive images, edge cache, debounced inputs	`vite.config.ts`, `docs/v2/PERF.md`
Security headers	HSTS, frame, nosniff, referrer, permissions, CSP (report-only)	`vercel.json`
Logging and audit	`audit_log` for access changes, team changes and publishing access; `article_activity` for workflow; `sk_matches.updated_by`	`app.audit`
Configuration	Environment variables (AMR-DOC-09); `site_settings` for editorial configuration	

7. Technology choices and justification

Choice	Alternatives considered	Why this one
React 18 + TypeScript	Vue, Svelte, plain JS	Existing v1 code and team familiarity; strong typing catches errors early
Vite 7	Next.js, CRA	Fast builds; the app is a client-rendered SPA, so a server framework was unnecessary; SEO needs are covered by middleware
Supabase (PostgreSQL)	Firebase, custom Express + PostgreSQL	Relational data fits a newsroom and sports results; RLS gives database-enforced permissions; Auth, Storage and Realtime included; free tier
Vercel	Netlify, own VPS	Zero-ops hosting, CDN, serverless functions and middleware in one place

Choice	Alternatives considered	Why this one
TanStack Query	Redux, SWR	Caching, background refresh and invalidation for server state with little code
Tailwind CSS v4 with design tokens	CSS modules, styled-components	Fast, consistent styling from one token file; no runtime cost
TipTap 3	Quill (v1), Slate	Maintained, extensible, React-friendly; replaced the unmaintained Quill
DOMPurify	custom sanitiser	Proven HTML sanitiser for user-generated rich text
Cloudflare Turnstile	reCAPTCHA, hCaptcha	Free, privacy-friendly, invisible for most users
First-party statistics	Google Analytics	No cookies, no third-party data sharing, data stays in the project
Re-runnable SQL files	Migration tool (Supabase CLI)	Simple to apply from the dashboard by non-developers; proven idempotent by tests

8. Architecture decision records (ADRs)

Each record: context, decision, consequences. Dates come from `docs/v2/*.md` and git history.

ID	Title	Date	Status
ADR-001	The database is the security boundary (RLS + triggers)	2026-10-17	Accepted
ADR-002	Fresh v2 schema instead of migrating v1	2026-10-17	Accepted
ADR-003	Client-rendered SPA with crawler middleware for link previews	2026-10-17	Accepted
ADR-004	Private `app` schema for helper functions	2026-10-17	Accepted
ADR-005	Workflow enforced by a trigger, transitions through an RPC	2026-10-17	Accepted
ADR-006	Scheduling by `published_at` filter instead of a cron job	2026-10-17	Accepted
ADR-007	Readers have no accounts; public writes only through validated RPCs	2026-10-17	Accepted
ADR-008	Salingkusog standings computed by views, never typed	2026-10-17	Accepted
ADR-009	First-party, cookie-less statistics	2026-10-18	Accepted
ADR-010	Edge cache for public reads (`/api/r`)	2026-10-18	Accepted
ADR-011	Public forms through a CAPTCHA gateway with service-role RPCs	2026-10-19	Accepted
ADR-012	Generated games carry their sources; a trigger advances teams	2026-10-19	Accepted
ADR-013	Medals are suggested, staff confirm	2026-10-19	Accepted
ADR-014	Contests modelled as a session row plus entries	2026-10-19	Accepted
ADR-015	Hand-written database types	2026-10-17	Accepted, revisit

ADR-001 — The database is the security boundary. *Context:* the browser talks directly to Supabase; any check done only in React can be bypassed with the public key. *Decision:* enable RLS on every table, express permissions as `app.*` helpers, and enforce invariants with triggers. *Consequences:* the same

rules apply to the UI, the API and scripts; rules are testable in SQL (278 checks); UI permission code is a mirror that must be kept in sync.

ADR-002 — Fresh schema. *Context:* the v1 schema mixed legacy columns and exposed helpers. *Decision:* option B, a new `schema.sql` pasted into a new or emptied project (`docs/v2/DB.md`). *Consequences:* clean model; v1 content is not migrated (out of scope).

ADR-003 — SPA with crawler middleware. *Context:* social networks need Open Graph tags but do not run JavaScript. *Decision:* keep a Vite SPA and add Vercel routing middleware that injects tags for crawlers only. *Consequences:* simple hosting and fast navigation; search engines that run JavaScript index the SPA; crawler responses depend on Supabase availability.

ADR-004 — Private `app` schema. *Context:* functions in `public` are callable over the API. *Decision:* helpers live in `app`, which is not exposed; public RPCs are explicitly granted. *Consequences:* smaller attack surface; grants must be maintained per function.

ADR-005 — Workflow trigger plus RPC. *Context:* transitions must be enforced even on direct updates. *Decision:* `app.article_before_write` validates every status change; `transition_article` adds notes. *Consequences:* one place for the rules; error messages come from the database.

ADR-006 — Scheduling by filter. *Context:* pg_cron may not be available or may lag. *Decision:* public queries treat scheduled articles as live once `published_at` passes. *Consequences:* exact go-live time with no job; the CMS list shows “scheduled” until the optional job flips it.

ADR-007 — Anonymous readers. *Context:* reader accounts add privacy and support burden. *Decision:* no reader accounts; comments and submissions through validated, rate-limited RPCs. *Consequences:* simpler privacy story; moderation is needed for comments.

ADR-008 — Computed standings. *Context:* hand-typed tallies cause disputes. *Decision:* views compute medals, points and ranks from placements. *Consequences:* one source of truth; correcting a placement corrects every page.

ADR-009 — First-party statistics. *Context:* third-party analytics share reader data and need cookie consent. *Decision:* own ingest endpoint, hashed ids with rotating salts, retention limits. *Consequences:* privacy-friendly; fewer features than commercial tools; needs the service key on Vercel.

ADR-010 — Edge cache. *Context:* each Supabase read cost a CORS preflight plus the request; traffic spikes during events. *Decision:* signed-out reads of slow-changing public tables go through `/api/r`, cached 60 s at the CDN. *Consequences:* faster pages and fewer database queries; up to a minute of staleness for those tables (live scores excluded).

ADR-011 — CAPTCHA gateway. *Context:* per-email rate limits can be bypassed with new addresses; anonymous uploads could fill storage. *Decision:* forms go through `/api/submit` with Turnstile; uploads use signed URLs; form RPCs are service-role only. *Consequences:* bots are stopped before the database; forms depend on the Vercel function and three environment variables.

ADR-012 — Sources on generated games. *Context:* staff wanted to enter only the first round. *Decision:* games store where each side comes from; `app.sk_resolve` recomputes sides after every change and refuses to alter started games. *Consequences:* brackets and round robins advance automatically; games entered before this feature do not advance.

ADR-013 — Staff-confirmed medals. *Context:* medals affect the official tally. *Decision:* the system suggests the podium; a person saves it. *Consequences:* a human check on official results; one extra step per event.

ADR-014 — Contests as session + entries. *Context:* dance, swimming and athletics have every faction in one session. *Decision:* reuse `sk_matches` (flag `is_contest`, no sides) for scheduling and status, and add `sk_entries` for line-up and results. *Consequences:* contests appear in schedules and live states without a parallel model; two-sided views ignore them.

ADR-015 — Hand-written types. *Context:* no Supabase CLI or Docker locally when the project started. *Decision:* maintain `src/lib/database.types.ts` by hand. *Consequences:* types can drift; should be replaced by generated types (AMR-DOC-13).

9. Trade-offs and their consequences

Trade-off	Benefit	Cost / risk	Mitigation
Logic in the database	Single enforcement point, testable	SQL skills needed to change rules	SQL test suite, comments in SQL files
Thick client	Simple hosting, fast UI	Bundle size; logic duplicated for display	Code splitting; mirrors kept small
Managed services	No operations	Vendor lock-in, free-tier limits	Standard PostgreSQL; schema in plain SQL
No migration tool	Easy manual deploys	Manual steps can be missed	Idempotent files, `docs/v2/DB.md` paste order
Edge cache	Speed, resilience	Short staleness	Excludes live data and personal data
Report-only CSP	No risk of breaking pages	Not yet blocking	Switch to enforcing after a clean preview (AMR-DOC-13)

Database Design

Amaranth v2 — ERD, data dictionary, views, functions, triggers, indexes and RLS · 1.0 · 2026-10-19

Document control

Field	Value
Title	Database Design
Document ID	AMR-DOC-04
Version	1.0
Date	2026-10-19
Author	Amaranth development team [TO CONFIRM: names]
Status	Final draft for defense
Sources	`supabase/v2/schema.sql`, `salingkusog.sql`, `stats.sql`, `seed.sql`; tests in `supabase/tests/`

Contents

1. Overview and counts
2. Schemas, files and deployment order
3. Entity–relationship diagrams
4. Design principles and normalisation
5. Key tables explained
6. Views
7. Functions, RPCs and triggers (summary)
8. Row Level Security strategy
9. How Salingkusog results become the medal table, points and ranking
10. Indexing strategy
11. Storage
12. Testing the database

- Appendix A — Data dictionary (every table and column)
- Appendix B — Enumerated types
- Appendix C — Functions and RPCs
- Appendix D — Triggers
- Appendix E — Indexes
- Appendix F — RLS policies
- Appendix G — Storage buckets

1. Overview and counts

Object	Newsroom (`schema.sql`)	Salingkusog	Statistics	Total
Tables	29	12	9	50
Columns	—	—	—	448
Enums	13	6	0	19
Views	1	5	0	6
Functions	39	18	19	76
Triggers	19	21	0	40
Indexes	35	18	7	60
RLS policies (incl. storage)	64	25	8	97
Storage buckets	3	—	—	3

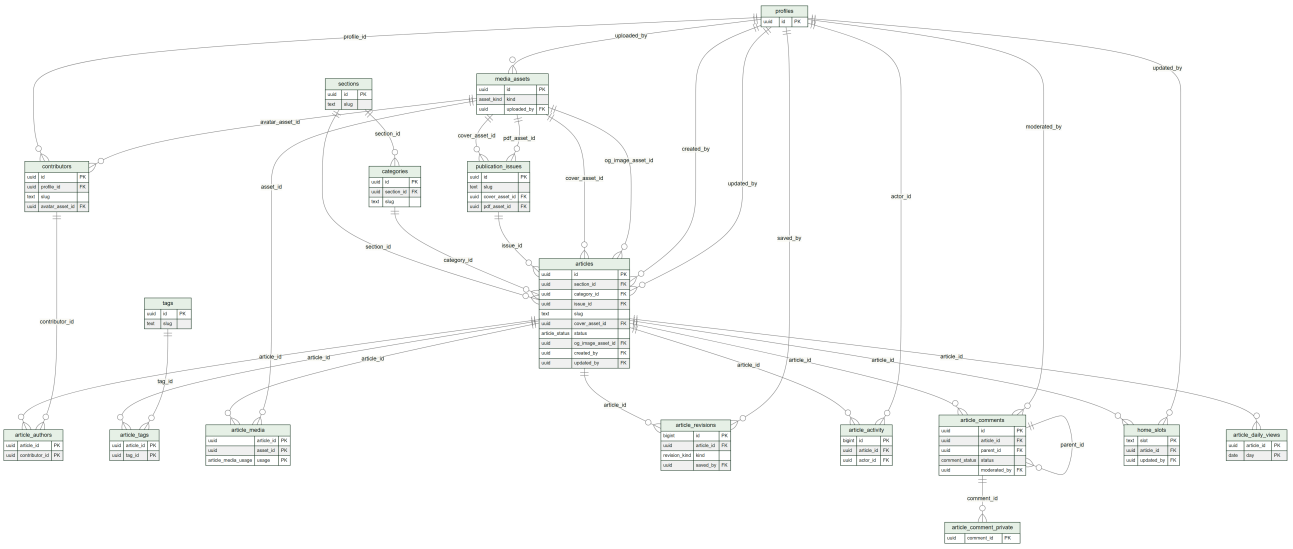
2. Schemas, files and deployment order

Schema	Purpose	Exposed through the API?
`public`	Application tables, views and the RPCs the app calls	Yes (subject to RLS and grants)
`app`	Private helpers (`app.current_role`, `app.can_publish`, …), trigger functions, `app.rate_events`, `app.stats_salts`	No
`auth`, `storage`	Supabase-managed	Through Supabase only

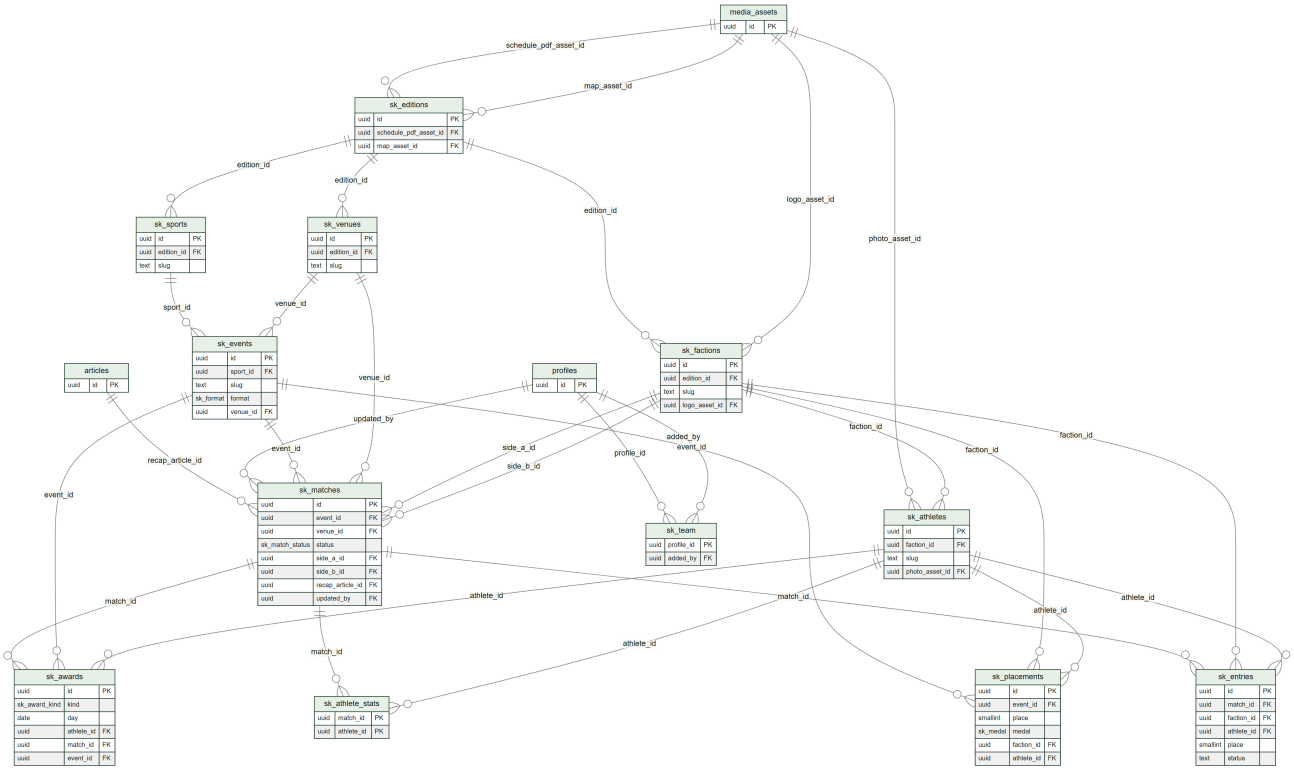
Deployment order (all files re-runnable): `schema.sql` → `salingkusog.sql` → `stats.sql` → `seed.sql` → optional `seed_board.sql` (AMR-DOC-09).

3. Entity–relationship diagrams

Four ERDs (AMR-DOC-05 §8) are generated from the SQL. The content core:



ERD-1 — content core.



ERD-3 — Salingkusog.

ERD-2 (people, intake, settings) and ERD-4 (statistics) are in AMR-DOC-05.

4. Design principles and normalisation

The schema is in **third normal form (3NF)** for the transactional tables: every non-key column depends on the key, the whole key and nothing but the key. Many-to-many relationships use link tables (`article_authors`, `article_tags`, `article_media`, `profile_sections`). Enumerated values use PostgreSQL enums, so invalid states cannot be stored.

Controlled denormalisations, each maintained by the database rather than the application:

Denormalisation	Why	How it stays correct
`articles.section_id` (derivable from the category)	Fast section filters and section-editor RLS checks	Set by `app.article_before_write` from the category on every write
`edition_id` on `sk_events`, `sk_athletes`, `sk_matches`, `sk_placements`, `sk_entries`	Simple RLS (`app.sk_edition_visible`) and edition-wide queries	Copied from the parent by `app.sk_inherit_edition`; composite foreign keys `(edition_id, id)` make a mismatch impossible
`articles.reading_minutes`, `articles.search`	Avoid recomputing on every read	Trigger (reading time) and a generated `tsvector` column
`articles.view_count`, `article_daily_views`	“Most read” without scanning events	Updated only by `record_article_view` (the trigger rejects other changes during a view count)
`sk_matches.winner_side`	Simple winner queries	Generated column from status and scores
`stats_daily_*` tables	Fast dashboards over long ranges	Rebuilt by the nightly rollup

Computed results are **views or RPCs**, never stored: `sk_medal_table`, `sk_points_by_day`, `sk_head_to_head`, `sk_event_status`, `sk_event_leaders`, `sk_points_timeline`, `app.sk_rr_table`.

5. Key tables explained

Table	Key points
`profiles`	Primary key = `auth.users.id`. `role` (`app_role`), `is_active`, `can_publish`. Only SA/EE change role and activation (trigger `profiles_guard`).
`staff_invites`	The only source of roles for new accounts (`app.handle_new_user`).
`articles`	33 columns. `status` (`article_status`), `published_at` (public timestamp), `scheduled_for`, `is_featured`, `editorial_priority`, SEO fields, `body_json`/`body_html`/`body_text`, generated `search`.
`article_activity`, `article_revisions`	Audit trail of the workflow and snapshots (pruned to 50).
`article_comments` + `article_comment_private`	Public comment text separated from the private email (editors only).

Table	Key points
`article_submissions`, `event_reports`, `intake_files`	Public intake with `intake_status`; files in the private `intake` bucket.
`sk_editions`	Dates, medal points (default 5 / 3 / 1), `is_current`, `is_published`, countdown fields.
`sk_events`	`format` (`bracket`, `round_robin`, `round_robin_final`, `timed`, `judged`), `is_team`, `result_unit`, `seeds`.
`sk_matches`	Games and contest sessions: `status`, scores, `periods` and `team_stats` (JSON arrays), `source_a`/`source_b`, `is_contest`, generated `winner_side`. Constraints: two different sides; final needs scores (except contests); contests have no sides; source format check.
`sk_entries`	Contest entries: unique `(match_id, faction_id)`; `result`, `place`, `status` (`ok`, `dns`, `dnf`, `dq`); athlete must belong to the faction (trigger).
`sk_placements`	Official places; check constraint ties the medal to the place (1 gold, 2 silver, 3 bronze); unique per event, place, faction and athlete.

Full columns: Appendix A.

6. Views

All six views use `security_invoker = true`, so the caller's RLS applies to the underlying tables.

View	Returns	Used by
`article_cards`	Article list fields joined with section, category, byline and cover	Public lists, sitemap, RSS, crawler previews
`sk_medal_table`	Per faction: gold, silver, bronze, total, points, rank	Medal tally, sports landing
`sk_points_by_day`	Cumulative points per faction for every day of the edition	Points race ("Day")
`sk_head_to_head`	Wins and losses between each pair of factions (final games)	Faction profiles
`sk_event_status`	Per event: completed (gold awarded), first and last game, live now	Header counters, progress
`sk_event_leaders`	Totals of each numeric stat per athlete per event	Top scorers

7. Functions, RPCs and triggers (summary)

- **76 functions:** 39 newsroom, 18 Salingkusog, 19 statistics. Helper functions are `security definer` with `search_path = ''` to prevent search-path attacks; RPCs that should respect RLS (`transition_article`, `sk_set_placements`, `sk_build_schedule`, `sk_create_contest`, `sk_points_timeline`) are `security invoker`.

- **40 triggers:** workflow (`articles_before_write`, `articles_after_write`), revisions pruning, profile guard, new-user handler, Salingkusog edition inheritance, match status timestamps, knockout winner check, automatic advancement, single current edition, countdown guard, team audit, and `updated_at` touch triggers.

Lists with arguments and purposes: Appendices C and D. Callable endpoints: AMR-DOC-07.

8. Row Level Security strategy

1. **Every table has RLS enabled** (50 of 50). Without a matching policy, a request sees and changes nothing.
2. **Policies call role helpers**, never repeat role lists. Changing who may publish means changing one function (`app.can_publish_articles`).
3. **Read and write are separate policies.** Public data has an `anon, authenticated` select policy; writes are `authenticated` only and further limited by helpers.
4. **Private data is split into separate tables** (`article_comment_private`) rather than hidden by column, because RLS works per row.
5. **Triggers complement RLS** for rules that depend on the *change* rather than the row (status transitions, role changes, locked articles).
6. **Statistics** are readable only by `app.can_view_stats()`; the read RPCs check it again.
7. **Storage** has its own policies on `storage.objects` per bucket (Appendix F).
8. **Policy reset on re-run:** `schema.sql` drops and recreates its policies but leaves `sk_*` and `stats_*` policies alone, so files can be re-run in any order.

Examples:

Table	Policy (simplified)	Meaning
`articles`	select for anon: `app.is_live(status, published_at)`	Readers see only live articles
`articles`	select/update for staff: own, co-authored, or `app.can_edit_section(section_id)`	Staff see their work and their sections
`articles`	delete: `app.is_admin()` or own draft	SA/EE delete anything; others only their drafts
`sk_matches`	select: `app.sk_edition_visible(edition_id)`; all: `app.can_edit_sk()`	Public sees published editions; the team edits
`site_settings`	all: `app.is_admin()` or (`app.can_publish()` and key = ‘ticker’)	Admins edit settings; publishers edit the ticker

9. How Salingkusog results become the medal table, points and ranking

Step 1 — games and contests produce outcomes. Two-sided games end with scores; `winner_side` is generated. Contest entries end with results or judged places. Generated games advance through `app.sk_resolve` (AMR-DOC-03 §5.8).

Step 2 — staff confirm placements. When an event's last game is final, the newsroom suggests places 1–3 (`suggestedPodium`): for brackets, from the final and bronze game; for round robins, from `app.sk_rr_table` / `roundRobinTable()`; for contests, from the ranked entries. Staff save them with `sk_set_placements(event_id, rows)`, which replaces the event's placements and sets `medal` from `place`. The check constraint `sk_placements_medal_matches_place` makes a wrong medal impossible.

Step 3 — the medal table is computed by `sk_medal_table`:

- `gold`, `silver`, `bronze` = counts of the faction's placements with that medal;
- `total` = gold + silver + bronze;
- `points` = gold × `points_gold` + silver × `points_silver` + bronze × `points_bronze` (edition settings, default 5 / 3 / 1);
- `rank` = `rank()` over the edition ordered by points, then gold, then silver (ties share a rank).

Worked example (anonymised factions, default points 5 / 3 / 1):

Faction	Gold	Silver	Bronze	Points	Rank
Faction A	2	0	1	2×5 + 0×3 + 1×1 = **11**	1 (more gold than B)
Faction B	1	2	0	1×5 + 2×3 + 0 = **11**	2
Faction C	0	1	2	0 + 3 + 2 = **5**	3
Faction D	0	0	0	**0**	4

Step 4 — the points race. `sk_points_by_day` builds every day of the edition (`generate_series(starts_on, ends_on)`), sums each faction's points by `awarded_on`, and returns the running total (0 before the first medal). `sk_points_timeline(edition, hours)` does the same in 1, 3 or 6-hour steps for the latest day with results, or 12-hour steps across the edition, using the placement's creation time when it falls on the award day.

Step 5 — progress and official result. The public tally counts finished matches over all non-cancelled matches; when all are final it shows "Official and final medal tally" with the champion (rank 1) and runners-up (`resultsProgress`, `finalHeadline`).

Round-robin standings (`app.sk_rr_table`): wins descending → point difference → wins against teams level on wins and difference (head-to-head) → points scored → seed order.

10. Indexing strategy

60 indexes support the main access paths: live-article lists by section, category and featured flag (partial indexes on live statuses); slug lookups (unique); full-text search (GIN on `articles.search`); foreign keys used in joins; Salingkusog schedule by edition and time, live games, events by round and slot; statistics by day. A performance check on 2026-10-18 confirmed matching indexes for the schema, Salingkusog and statistics queries (`docs/v2/PERF.md`). Full list: Appendix E.

11. Storage

Bucket	Public	Limit	Who writes
`media`	yes	10 MB, images	Active staff (own files); editors manage others'
`archives`	yes	60 MB, PDF and images	Publishers (SA, EE, ME)
`intake`	no	20 MB, images, PDF, text, Word	Only through signed upload links from `/api/submit`; editors read and delete

12. Testing the database

`npm run db:test` (`scripts/db-test.mjs`) starts a throwaway PostgreSQL cluster on `127.0.0.1`, loads a Supabase stub (`00_supabase_stub.sql`), applies every SQL file **twice**, loads fixtures (one user per role), and runs 6 test files with **278 checks** (all passing on 2026-10-19). Coverage per file: public access 39, workflow 75, Salingkusog 71, brackets 28, contests 17, statistics 48. See AMR-DOC-08.

Appendix A. Data dictionary

Generated from `supabase/v2/schema.sql`, `salingkusog.sql` and `stats.sql` on 2026-10-19. Types are PostgreSQL types; `timestamptz` is a timestamp with time zone. “Notes” come from comments in the SQL files.

A.1 Newsroom (`schema.sql`)

public.profiles

One row per staff account (linked to `auth.users`). Holds the role, activation and publishing-access flag.

Column	Type	Constraints / default	Notes
`id`	uuid	primary key references auth.users (id) on delete cascade	
`email`	text	not null	
`display_name`	text	not null	
`avatar_url`	text		
`bio`	text		
`role`	public.app_role	not null default ‘contributor’	
`is_active`	boolean	not null default false	
`last_seen_at`	timestamptz		
`created_at`	timestamptz	not null default now()	
`updated_at`	timestamptz	not null default now()	
`can_publish`	boolean	not null default false	added in schema.sql

public.staff_invites

Pre-authorised emails with the role (and sections) they receive when the invitation is accepted.

Column	Type	Constraints / default	Notes
`email`	text	primary key check (email = lower(email))	
`role`	public.app_role	not null default ‘staff_writer’	
`section_slugs`	text[]	not null default ‘{}’	
`invited_by`	uuid	references public.profiles (id) on delete set null	
`accepted_at`	timestamptz		
`created_at`	timestamptz	not null default now()	

public.sections

The 12 site sections (News, Sports, …) with navigation placement and order.

Column	Type	Constraints / default	Notes
`id`	uuid	primary key default gen_random_uuid()	
`slug`	text	not null unique check (slug ~ '^[a-z0-9]+(-[a-z0-9]+)*$')	
`name`	text	not null	
`description`	text		
`nav_placement`	public.nav_placement	not null default 'primary'	
`nav_order`	integer	not null default 0	
`accepts_submissions`	boolean	not null default true	
`created_at`	timestamptz	not null default now()	
`updated_at`	timestamptz	not null default now()	

public.profile_sections

Which sections a section editor may edit.

Column	Type	Constraints / default	Notes
`profile_id`	uuid	not null references public.profiles (id) on delete cascade	
`section_id`	uuid	not null references public.sections (id) on delete cascade	

Table constraints: `primary key (profile_id, section_id)`

public.categories

Categories inside a section; `is_assignable = false` reserves a category for editors.

Column	Type	Constraints / default	Notes
`id`	uuid	primary key default gen_random_uuid()	
`section_id`	uuid	not null references public.sections (id) on delete restrict	
`slug`	text	not null check (slug ~ '^[a-z0-9]+(-[a-z0-9]+)*$')	
`name`	text	not null	
`description`	text		
`sort_order`	integer	not null default 0	

Column	Type	Constraints / default	Notes
is_active	boolean	not null default true	
is_assignable	boolean	not null default true	writers may file into this category (editorials are editors-only, etc.)
created_at	timestamptz	not null default now()	
updated_at	timestamptz	not null default now()	

Table constraints: unique (section_id, slug)

public.tags

Free-form tags for articles.

Column	Type	Constraints / default	Notes
id	uuid	primary key default gen_random_uuid()	
slug	text	not null unique check (slug ~ '^[a-z0-9]+(-[a-z0-9]+)*$')	
name	text	not null	
created_at	timestamptz	not null default now()	

public.media_assets

Uploaded files in Storage with metadata (alt text, caption, credit, size).

Column	Type	Constraints / default	Notes
id	uuid	primary key default gen_random_uuid()	
bucket	text	not null default 'media' check (bucket in ('media', 'archives'))	
path	text	not null	
kind	public.asset_kind	not null default 'image'	
mime_type	text	not null	
size_bytes	integer	not null check (size_bytes > 0)	
width	integer	check (width is null or width > 0)	
height	integer	check (height is null or height > 0)	
alt_text	text	not null default ''	
caption	text		
credit	text		

Column	Type	Constraints / default	Notes
variants	jsonb	not null default '[]'::jsonb check (jsonb_typeof(variants) = 'array')	resized copies for srcset: [{"width": 640, "path": "…"}]
uploaded_by	uuid	references public.profiles (id) on delete set null	
created_at	timestamptz	not null default now()	
updated_at	timestamptz	not null default now()	

Table constraints: `unique (bucket, path)`

public.contributors

Public bylines (staff, students, guests, faculty, alumni); optionally linked to a profile.

Column	Type	Constraints / default	Notes
id	uuid	primary key default gen_random_uuid()	
profile_id	uuid	unique references public.profiles (id) on delete set null	
name	text	not null	
slug	text	not null unique check (slug ~ '^[a-z0-9]+(-[a-z0-9]+)*$')	
type	public.contributor_type	not null default 'staff'	
position	text		
program	text		
bio	text		
avatar_asset_id	uuid	references public.media_assets (id) on delete set null	
is_public	boolean	not null default true	
is_active	boolean	not null default true	
created_at	timestamptz	not null default now()	
updated_at	timestamptz	not null default now()	

public.publication_issues

Print issues with cover and PDF.

Column	Type	Constraints / default	Notes
id	uuid	primary key default gen_random_uuid()	
title	text	not null	

Column	Type	Constraints / default	Notes
slug	text	not null unique check (slug ~ '[5]+(-[a-z0-9]+)*$')	
volume	integer	check (volume is null or volume > 0)	
issue_number	integer	check (issue_number is null or issue_number > 0)	
issue_type	public.issue_type	not null default 'regular'	
issue_date	date	not null	
description	text		
cover_asset_id	uuid	references public.media_assets (id) on delete set null	
pdf_asset_id	uuid	references public.media_assets (id) on delete set null	
is_published	boolean	not null default false	
created_at	timestamptz	not null default now()	
updated_at	timestamptz	not null default now()	

public.articles

News stories and other pieces, with workflow status, schedule, SEO fields and full-text search vector.

Column	Type	Constraints / default	Notes
id	uuid	primary key default gen_random_uuid()	
section_id	uuid	not null references public.sections (id) on delete restrict	derived from category
category_id	uuid	not null references public.categories (id) on delete restrict	
issue_id	uuid	references public.publication_issues (id) on delete set null	
issue_position	integer	check (issue_position is null or issue_position > 0)	
title	text	not null check (length(btrim(title)) > 0)	
slug	text	not null unique check (slug ~ '[6]+(-[a-z0-9]+)*$')	
kicker	text		optional label next to the section, e.g. “Environment”
dek	text		subtitle / standfirst
excerpt	text	not null default ''	

Column	Type	Constraints / default	Notes
body_json	jsonb		editor document
body_html	text	not null default ''	
body_text	text	not null default ''	
reading_minutes	integer	not null default 1	
cover_asset_id	uuid	references public.media_assets (id) on delete restrict	
cover_caption	text		
byline_override	text		e.g. “By Sports Desk”; otherwise built from article_authors
status	public.article_status	not null default ‘draft’	
is_featured	boolean	not null default false	
editorial_priority	smallint	not null default 0 check (editorial_priority between 0 and 100)	
allow_comments	boolean	not null default true	
seo_title	text		
seo_description	text		
og_image_asset_id	uuid	references public.media_assets (id) on delete restrict	
submitted_at	timestamptz		
scheduled_for	timestamptz		
published_at	timestamptz		public timestamp; for scheduled articles it equals scheduled_for, so the article goes live by itself when that time passes (no cron required)
archived_at	timestamptz		
view_count	bigint	not null default 0	
created_by	uuid	references public.profiles (id) on delete set null	
updated_by	uuid	references public.profiles (id) on delete set null	
created_at	timestamptz	not null default now()	
updated_at	timestamptz	not null default now()	
search	tsvector	generated (computed)	

Table constraints: `constraint articles_scheduled_needs_time check (status <> 'scheduled' or scheduled_for is not null)`; `constraint articles_published_needs_time check (status not in ('published', 'scheduled') or published_at is not null)`

public.article_authors

Many-to-many link between articles and contributors (bylines).

Column	Type	Constraints / default	Notes
`article_id`	uuid	not null references public.articles (id) on delete cascade	
`contributor_id`	uuid	not null references public.contributors (id) on delete restrict	
`credit_label`	text	not null default ‘Author’	
`position`	smallint	not null default 1 check (position > 0)	

Table constraints: `primary key (article_id, contributor_id)`

public.article_tags

Many-to-many link between articles and tags.

Column	Type	Constraints / default	Notes
`article_id`	uuid	not null references public.articles (id) on delete cascade	
`tag_id`	uuid	not null references public.tags (id) on delete cascade	

Table constraints: `primary key (article_id, tag_id)`

public.article_media

Media used inside an article (inline, gallery, attachment).

Column	Type	Constraints / default	Notes
`article_id`	uuid	not null references public.articles (id) on delete cascade	
`asset_id`	uuid	not null references public.media_assets (id) on delete restrict	
`usage`	public.article_media_usage	not null default ‘inline’	
`position`	smallint	not null default 1	
`caption`	text		

Table constraints: `primary key (article_id, asset_id, usage)`

public.article_revisions

Snapshots of an article (autosave, manual, status); newest 50 kept.

Column	Type	Constraints / default	Notes
id	bigint	generated always as identity primary key	
article_id	uuid	not null references public.articles (id) on delete cascade	
kind	public.revision_kind	not null default ‘manual’	
title	text	not null	
body_json	jsonb		
body_html	text	not null default ’’	
saved_by	uuid	references public.profiles (id) on delete set null	audit only, unindexed
created_at	timestamptz	not null default now()	

public.article_activity

Workflow history: creation, status changes and notes.

Column	Type	Constraints / default	Notes
id	bigint	generated always as identity primary key	
article_id	uuid	not null references public.articles (id) on delete cascade	
actor_id	uuid	references public.profiles (id) on delete set null	audit only, unindexed
action	text	not null check (action in (‘created’, ‘status’, ‘note’))	
from_status	public.article_status		
to_status	public.article_status		
note	text		
created_at	timestamptz	not null default now()	

public.article_daily_views

View counts per article per day (feeds “most read”).

Column	Type	Constraints / default	Notes
article_id	uuid	not null references public.articles (id) on delete cascade	
day	date	not null	

Column	Type	Constraints / default	Notes
`views`	integer	not null default 0	

Table constraints: `primary key (article_id, day)`

public.article_comments

Reader comments with moderation status and optional parent (replies).

Column	Type	Constraints / default	Notes
`id`	uuid	primary key default gen_random_uuid()	
`article_id`	uuid	not null references public.articles (id) on delete cascade	
`parent_id`	uuid	references public.article_comments (id) on delete cascade	
`author_name`	text	not null check (length(btrim(author_name)) between 1 and 80)	
`body`	text	not null check (length(btrim(body)) between 2 and 3000)	
`status`	public.comment_status	not null default ‘pending’	
`moderated_by`	uuid	references public.profiles (id) on delete set null	audit only, unindexed
`moderated_at`	timestamptz		
`created_at`	timestamptz	not null default now()	

public.article_comment_private

Commenter email, separated so only editors can read it.

Column	Type	Constraints / default	Notes
`comment_id`	uuid	primary key references public.article_comments (id) on delete cascade	
`author_email`	text	not null	

public.home_slots

Front-page curation: lead story and literary folio pick.

Column	Type	Constraints / default	Notes
`slot`	text	primary key check (slot in (‘lead’, ‘folio’))	

Column	Type	Constraints / default	Notes
article_id	uuid	references public.articles (id) on delete set null	
pull_quote	text		
attribution	text		
updated_by	uuid	references public.profiles (id) on delete set null	audit only
updated_at	timestamptz	not null default now()	

public.article_submissions

Story submissions from the public, with review status.

Column	Type	Constraints / default	Notes
id	uuid	primary key default gen_random_uuid()	
submitter_name	text	not null	
submitter_email	text	not null	
submitter_affiliation	text		
section_id	uuid	references public.sections (id) on delete set null	
title	text	not null	
summary	text		
body	text		
status	public.intake_status	not null default ‘submitted’	
reviewer_id	uuid	references public.profiles (id) on delete set null	audit only
review_note	text		
reviewed_at	timestamptz		
converted_article_id	uuid	unique references public.articles (id) on delete set null	
created_at	timestamptz	not null default now()	
updated_at	timestamptz	not null default now()	

public.event_reports

Event reports from the public, with review status.

Column	Type	Constraints / default	Notes
id	uuid	primary key default gen_random_uuid()	
organization	text	not null	
contact_name	text	not null	

Column	Type	Constraints / default	Notes
`contact_email`	text	not null	
`event_title`	text	not null	
`event_date`	date		
`venue`	text		
`summary`	text	not null	
`status`	public.intake_status	not null default 'submitted'	
`reviewer_id`	uuid	references public.profiles (id) on delete set null	audit only
`review_note`	text		
`reviewed_at`	timestamptz		
`converted_article_id`	uuid	unique references public.articles (id) on delete set null	
`created_at`	timestamptz	not null default now()	
`updated_at`	timestamptz	not null default now()	

public.intake_files

Attachments of submissions and event reports in the private `intake` bucket.

Column	Type	Constraints / default	Notes
`id`	uuid	primary key default gen_random_uuid()	
`submission_id`	uuid	references public.article_submissions (id) on delete cascade	
`event_report_id`	uuid	references public.event_reports (id) on delete cascade	
`path`	text	not null unique	
`file_name`	text	not null	
`mime_type`	text		
`size_bytes`	integer		
`created_at`	timestamptz	not null default now()	

Table constraints: `constraint intake_files_one_parent check (num_nonnulls(submission_id, event_report_id) = 1)`

public.contact_messages

Messages from the contact form, with handling status.

Column	Type	Constraints / default	Notes
id	uuid	primary key default gen_random_uuid()	
name	text	not null	
email	text	not null	
subject	text	not null	
category	public.contact_category	not null default ‘general’	
message	text	not null	
status	public.contact_status	not null default ‘unread’	
handled_by	uuid	references public.profiles (id) on delete set null	audit only
handled_at	timestamptz		
created_at	timestamptz	not null default now()	
updated_at	timestamptz	not null default now()	

public.team_terms

Editorial board terms (for example a school year).

Column	Type	Constraints / default	Notes
id	uuid	primary key default gen_random_uuid()	
label	text	not null unique	e.g. “2026–2027”
starts_on	date	not null	
ends_on	date		
is_current	boolean	not null default false	
created_at	timestamptz	not null default now()	
updated_at	timestamptz	not null default now()	

public.team_members

Members of the editorial board per term.

Column	Type	Constraints / default	Notes
id	uuid	primary key default gen_random_uuid()	
term_id	uuid	not null references public.team_terms (id) on delete cascade	
role_group	public.team_role_group	not null	
position_title	text	not null	
contributor_id	uuid	references public.contributors (id) on delete set null	
display_name	text		

Column	Type	Constraints / default	Notes
photo_asset_id	uuid	references public.media_assets (id) on delete set null	
sort_order	integer	not null default 0	
created_at	timestamptz	not null default now()	
updated_at	timestamptz	not null default now()	

Table constraints: constraint team_members_named check (contributor_id is not null or display_name is not null)

public.department_contacts

Desk contacts shown on the contact page.

Column	Type	Constraints / default	Notes
id	uuid	primary key default gen_random_uuid()	
name	text	not null unique	
email	text		
phone	text		
description	text		
sort_order	integer	not null default 0	
created_at	timestamptz	not null default now()	
updated_at	timestamptz	not null default now()	

public.site_settings

Key–value site configuration (site details, social links, ticker, comments switch).

Column	Type	Constraints / default	Notes
key	text	primary key check (key ~ '[7]+$')	
value	jsonb	not null	
is_public	boolean	not null default false	
updated_by	uuid	references public.profiles (id) on delete set null	audit only
updated_at	timestamptz	not null default now()	

public.audit_log

Who changed access, publishing rights or the Salingkusog team, and when.

Column	Type	Constraints / default	Notes
id	bigint	generated always as identity primary key	

Column	Type	Constraints / default	Notes
actor_id	uuid	references public.profiles (id) on delete set null	audit only, unindexed
action	text	not null	
entity	text	not null	
entity_id	text		
details	jsonb	not null default '{}'::jsonb	
created_at	timestamptz	not null default now()	

app.rate_events

Internal log used by app.rate_limit (private app schema).

Column	Type	Constraints / default	Notes
key	text	not null	
created_at	timestamptz	not null default now()	

A.2 Salingkusog (salingkusog.sql)

public.sk_editions

One Salingkusog edition per year: dates, medal points, flags, schedule PDF, map, countdown.

Column	Type	Constraints / default	Notes
id	uuid	primary key default gen_random_uuid()	
year	integer	not null unique check (year between 2000 and 2100)	
title	text	not null	"Salingkusog 2026"
subtitle	text		"VSU Intramural Games"
starts_on	date	not null	
ends_on	date	not null	
points_gold	smallint	not null default 5 check (points_gold >= 0)	
points_silver	smallint	not null default 3 check (points_silver >= 0)	
points_bronze	smallint	not null default 1 check (points_bronze >= 0)	
is_current	boolean	not null default false	
is_published	boolean	not null default false	
schedule_pdf_asset_id	uuid	references public.media_assets (id) on delete set null	

Column	Type	Constraints / default	Notes
map_asset_id	uuid	references public.media_assets (id) on delete set null	
created_at	timestamptz	not null default now()	
updated_at	timestamptz	not null default now()	
countdown_enabled	boolean	not null default false	added in salingkusog.sql
countdown_at	timestamptz		added in salingkusog.sql

Table constraints: `constraint sk_editions_dates check (ends_on >= starts_on)`

public.sk_factions

Competing teams of an edition (name, short name, motto, colour, logo).

Column	Type	Constraints / default	Notes
id	uuid	primary key default gen_random_uuid()	
edition_id	uuid	not null references public.sk_editions (id) on delete cascade	
name	text	not null	
slug	text	not null check (slug ~ '[8]+(-[a-z0-9]+)*$')	
short_name	text		“PT”, used in compact score tables
motto	text		
color	text	not null default '#0e5a36' check (color ~ '^#[0-9a-fA-F]{6}$')	
logo_asset_id	uuid	references public.media_assets (id) on delete set null	
sort_order	integer	not null default 0	
created_at	timestamptz	not null default now()	
updated_at	timestamptz	not null default now()	

Table constraints: `unique (edition_id, slug)`; `unique (edition_id, id)`

public.sk_venues

Venues with description and position on the venue map.

Column	Type	Constraints / default	Notes
id	uuid	primary key default gen_random_uuid()	

Column	Type	Constraints / default	Notes
`edition_id`	uuid	not null references public.sk_editions (id) on delete cascade	
`name`	text	not null	
`slug`	text	not null check (slug ~ '[9]+(-[a-z0-9]+)*$')	
`description`	text		
`map_number`	smallint		the numbered pin on the venue map
`map_x`	numeric(5, 2)	check (map_x is null or map_x between 0 and 100)	% from left
`map_y`	numeric(5, 2)	check (map_y is null or map_y between 0 and 100)	% from top
`sort_order`	integer	not null default 0	
`created_at`	timestamptz	not null default now()	
`updated_at`	timestamptz	not null default now()	

Table constraints: `unique (edition_id, slug)`; `unique (edition_id, id)`

public.sk_sports

Sports of an edition with a category.

Column	Type	Constraints / default	Notes
`id`	uuid	primary key default gen_random_uuid()	
`edition_id`	uuid	not null references public.sk_editions (id) on delete cascade	
`name`	text	not null	“Basketball”
`slug`	text	not null check (slug ~ '[10]+(-[a-z0-9]+)*$')	
`category`	public.sk_category	not null	
`sort_order`	integer	not null default 0	
`created_at`	timestamptz	not null default now()	
`updated_at`	timestamptz	not null default now()	

Table constraints: `unique (edition_id, slug)`; `unique (edition_id, id)`

public.sk_events

An event inside a sport (division and format); `seeds` stores the seed order used by Build schedule.

Column	Type	Constraints / default	Notes
id	uuid	primary key default gen_random_uuid()	
edition_id	uuid	not null	
sport_id	uuid	not null	
name	text	not null	shown after the sport: “Men”, “100m Dash · Women”
slug	text	not null check (slug ~ '^[a-z0-9]+(-[a-z0-9]+)*$')	
division	public.sk_division	not null default ‘open’	
format	public.sk_format	not null default ‘bracket’	
is_team	boolean	not null default true	false = individual athletes win the medals
result_unit	text		“s”, “m”, “pts” for timed/measured events
venue_id	uuid		
sort_order	integer	not null default 0	
created_at	timestamptz	not null default now()	
updated_at	timestamptz	not null default now()	
seeds	uuid[]	not null default ‘{}’	added in salingkusog.sql

Table constraints: `unique (edition_id, slug)`; `unique (edition_id, id)`; `foreign key (edition_id, sport_id) references public.sk_sports (edition_id, id) on delete cascade`; `foreign key (edition_id, venue_id) references public.sk_venues (edition_id, id) on delete set null (venue_id)`

public.sk_athletes

Athletes per faction: name, program and optional photo only.

Column	Type	Constraints / default	Notes
id	uuid	primary key default gen_random_uuid()	
edition_id	uuid	not null	
faction_id	uuid	not null	
name	text	not null	
slug	text	not null check (slug ~ '^[a-z0-9]+(-[a-z0-9]+)*$')	
program	text		course / year
photo_asset_id	uuid	references public.media_assets (id) on delete set null	
created_at	timestamptz	not null default now()	
updated_at	timestamptz	not null default now()	

Table constraints: `unique (edition_id, slug)`; `unique (edition_id, id)`; `foreign key (edition_id, faction_id) references public.sk_factions (edition_id, id) on delete cascade`

public.sk_matches

Games and contest sessions: schedule, status, scores, periods, sources of generated sides, recap link.

Column	Type	Constraints / default	Notes
`id`	uuid	primary key default gen_random_uuid()	
`edition_id`	uuid	not null	
`event_id`	uuid	not null	
`round`	text		“Semifinal 1”, “Final”, “Bronze game”, “Group A”
`round_order`	smallint	not null default 0	column in the bracket (higher = later round)
`bracket_slot`	smallint		position within the round, top to bottom
`venue_id`	uuid		
`scheduled_at`	timestamptz	not null	
`status`	public.sk_match_status	not null default ‘upcoming’	
`side_a_id`	uuid		null = to be decided (TBD) in early brackets
`side_b_id`	uuid		
`score_a`	numeric(8, 2)		
`score_b`	numeric(8, 2)		
`periods`	jsonb	not null default ‘[]’::jsonb check (jsonb_typeof(periods) = ‘array’)	quarters / sets / games: [{“label”: “Q1”, “a”: 20, “b”: 16}, …]
`team_stats`	jsonb	not null default ‘[]’::jsonb check (jsonb_typeof(team_stats) = ‘array’)	team stats for the recap: [{“label”: “Field goal %”, “a”: “46%”, “b”: “41%”}, …]
`live_note`	text		“Set 4 · Now”, “Q3 · 4:12”
`started_at`	timestamptz		
`ended_at`	timestamptz		
`recap_article_id`	uuid	references public.articles (id) on delete set null	
`winner_side`	char(1)	generated (computed)	
`updated_by`	uuid	references public.profiles (id) on delete set null	audit only
`created_at`	timestamptz	not null default now()	
`updated_at`	timestamptz	not null default now()	
`source_a`	text		added in salingkusog.sql

Column	Type	Constraints / default	Notes
source_b	text		added in salingkusog.sql
is_contest	boolean	not null default false	added in salingkusog.sql

Table constraints: unique (edition_id, id); foreign key (edition_id, event_id) references public.sk_events (edition_id, id) on delete cascade; foreign key (edition_id, venue_id) references public.sk_venues (edition_id, id) on delete set null (venue_id); foreign key (edition_id, side_a_id) references public.sk_factions (edition_id, id) on delete set null (side_a_id); foreign key (edition_id, side_b_id) references public.sk_factions (edition_id, id) on delete set null (side_b_id); constraint sk_matches_two_sides check (side_a_id is null or side_b_id is null or side_a_id <> side_b_id); constraint sk_matches_final_needs_scores check (status <> 'final' or (score_a is not null and score_b is not null)); constraint sk_matches_sources check ((source_a is null or source_a ~ '^(seed:[0-9]+\|rank:[0-9]+\|(winner\|loser):[0-9a-f-]{36})$') and (source_b is null or source_b ~ '^(seed:[0-; constraint sk_matches_final_needs_scores check (status <> 'final' or is_contest or (score_a is not null and score_b is not null)); constraint sk_matches_contest_no_sides check (not is_contest or (side_a_id is null and side_b_id is null))

public.sk_entries

Contest line-up and results: one row per faction per session.

Column	Type	Constraints / default	Notes
id	uuid	primary key default gen_random_uuid()	
edition_id	uuid	not null	
match_id	uuid	not null	
faction_id	uuid	not null	
athlete_id	uuid		individual events: the faction's one entrant
lane	smallint	not null default 0	performance order / lane
result	numeric(10, 3)		seconds, metres or points (timed / measured)
place	smallint	check (place between 1 and 64)	judged: the place announced by the judges
status	text	not null default 'ok' check (status in ('ok', 'dns', 'dnf', 'dq'))	
created_at	timestamptz	not null default now()	
updated_at	timestamptz	not null default now()	

Table constraints: unique (match_id, faction_id); foreign key (edition_id, match_id) references public.sk_matches (edition_id, id) on delete cascade; foreign key (edition_id, faction_id) references public.sk_factions (edition_id, id) on delete cascade; foreign key (edition_id, athlete_id) references public.sk_athletes (edition_id, id) on delete set null (athlete_id)

public.sk_athlete_stats

Per-athlete box score values for a game (JSON of stat → value).

Column	Type	Constraints / default	Notes
match_id	uuid	not null references public.sk_matches (id) on delete cascade	
athlete_id	uuid	not null references public.sk_athletes (id) on delete cascade	
stats	jsonb	not null default '{}'::jsonb check (jsonb_typeof(stats) = 'object')	

Table constraints: primary key (match_id, athlete_id)

public.sk_placements

Official places of an event; medal follows the place; feeds the medal table.

Column	Type	Constraints / default	Notes
id	uuid	primary key default gen_random_uuid()	
edition_id	uuid	not null	
event_id	uuid	not null	
place	smallint	not null check (place between 1 and 64)	
medal	public.sk_medal		null for 4th place and below
faction_id	uuid	not null	
athlete_id	uuid		individual events
result	text		"12.84s", "78–71", "92.5 pts"
awarded_on	date	not null default ((now() at time zone 'Asia/Manila')::date)	
created_at	timestamptz	not null default now()	
updated_at	timestamptz	not null default now()	

Table constraints: foreign key (edition_id, event_id) references public.sk_events (edition_id, id) on delete cascade; foreign key (edition_id, faction_id) references public.sk_factions (edition_id, id) on delete cascade; foreign key (edition_id, athlete_id) references public.sk_athletes (edition_id, id) on delete set null (athlete_id); constraint sk_placements_medal_matches_place check (medal is null or (medal = 'gold' and place = 1) or (medal = 'silver' and place = 2) or (medal = 'bronze' and place = 3))

public.sk_awards

Daily and finals awards with stat line and quote.

Column	Type	Constraints / default	Notes
`id`	uuid	primary key default gen_random_uuid()	
`edition_id`	uuid	not null	
`kind`	public.sk_award_kind	not null	
`day`	date	not null	
`athlete_id`	uuid	not null	
`match_id`	uuid	references public.sk_matches (id) on delete set null	
`event_id`	uuid	references public.sk_events (id) on delete set null	
`headline`	text		“12.84s” or “Gold · 100m Dash, Women”
`stat_line`	jsonb	not null default ‘[]’::jsonb check (jsonb_typeof(stat_line) = ‘array’)	[{“label”:“PTS”,“value”:“24”}]
`quote`	text		
`created_at`	timestamptz	not null default now()	
`updated_at`	timestamptz	not null default now()	

Table constraints: `foreign key (edition_id, athlete_id) references public.sk_athletes (edition_id, id) on delete cascade`

public.sk_team

Staff allowed to enter Salingkusog data (besides SA, EE, ME).

Column	Type	Constraints / default	Notes
`profile_id`	uuid	primary key references public.profiles (id) on delete cascade	
`added_by`	uuid	references public.profiles (id) on delete set null	audit only
`created_at`	timestamptz	not null default now()	

A.3 Statistics (`stats.sql`)

app.stats_salts

Daily random salts for hashing visitor ids; deleted after two days.

Column	Type	Constraints / default	Notes
`day`	date	primary key	
`salt`	text	not null default gen_random_uuid()::text	

public.stats_pageviews

Raw pageview events (hashed ids, source, device, reading time, scroll).

Column	Type	Constraints / default	Notes
id	bigint	generated always as identity primary key	
client_id	uuid	not null unique	the browser’s id for this pageview (engagement attaches to it)
occurred_at	timestamptz	not null default now()	
day	date	not null default app.manila_today()	
path	text	not null check (length(path) between 1 and 300)	
article_id	uuid	references public.articles (id) on delete set null	
visitor_hash	text		null: Do Not Track
session_hash	text		
source	text	not null default ‘direct’ check (source in (‘direct’, ‘social’, ‘search’, ‘referral’, ‘internal’, ‘campaign’))	
referrer_host	text	check (referrer_host is null or length(referrer_host) <= 120)	
utm_source	text	check (utm_source is null or length(utm_source) <= 80)	
utm_medium	text	check (utm_medium is null or length(utm_medium) <= 80)	
utm_campaign	text	check (utm_campaign is null or length(utm_campaign) <= 80)	
device	text	not null default ‘desktop’ check (device in (‘mobile’, ‘tablet’, ‘desktop’))	
browser	text	not null default ‘Other’ check (length(browser) <= 40)	
os	text	not null default ‘Other’ check (length(os) <= 40)	
counted	boolean	not null default false	a deduplicated article view (one per visitor per article per 30 min)
active_seconds	integer	not null default 0 check (active_seconds between 0 and 14400)	
max_scroll	smallint	not null default 0 check (max_scroll between 0 and 100)	

public.stats_shares

Raw share-button events.

Column	Type	Constraints / default	Notes
id	bigint	generated always as identity primary key	
occurred_at	timestamptz	not null default now()	
day	date	not null default app.manila_today()	
article_id	uuid	not null references public.articles (id) on delete cascade	
channel	text	not null check (channel in ('facebook', 'messenger', 'x', 'copy', 'native', 'other'))	
visitor_hash	text		

public.staff_activity_daily

Newsroom active seconds per staff member per day.

Column	Type	Constraints / default	Notes
profile_id	uuid	not null references public.profiles (id) on delete cascade	
day	date	not null	
active_seconds	integer	not null default 0 check (active_seconds >= 0)	
last_beat_at	timestamptz		

Table constraints: `primary key (profile_id, day)`

public.stats_daily_site

Daily site totals.

Column	Type	Constraints / default	Notes
day	date	primary key	
pageviews	integer	not null default 0	
views	integer	not null default 0	
visitors	integer	not null default 0	
sessions	integer	not null default 0	
active_seconds	bigint	not null default 0	
read_samples	integer	not null default 0	
shares	integer	not null default 0	

public.stats_daily_articles

Daily totals per article.

Column	Type	Constraints / default	Notes
day	date	not null	
article_id	uuid	not null references public.articles (id) on delete cascade	
pageviews	integer	not null default 0	
views	integer	not null default 0	
visitors	integer	not null default 0	
active_seconds	bigint	not null default 0	
read_samples	integer	not null default 0	
scroll_sum	bigint	not null default 0	
scroll_samples	integer	not null default 0	
finished	integer	not null default 0	
shares	integer	not null default 0	

Table constraints: `primary key (day, article_id)`

public.stats_daily_dims

Daily totals per dimension (source, referrer, device, browser, OS).

Column	Type	Constraints / default	Notes
day	date	not null	
dim	text	not null check (dim in ('source', 'referrer', 'device', 'browser', 'os'))	
value	text	not null	
pageviews	integer	not null default 0	
visitors	integer	not null default 0	

Table constraints: `primary key (day, dim, value)`

public.stats_daily_shares

Daily share totals per channel.

Column	Type	Constraints / default	Notes
day	date	not null	
channel	text	not null	
shares	integer	not null default 0	

Table constraints: `primary key (day, channel)`

public.stats_rollup_log

Days already rolled up by the nightly job.

Column	Type	Constraints / default	Notes
`day`	date	primary key	
`rolled_at`	timestamptz	not null default now()	

Appendix B. Enumerated types

Enum	Values	File
`public.app_role`	`super_admin`, `editor_in_chief`, `managing_editor`, `section_editor`, `staff_writer`, `contributor`, `business_manager`	schema.sql
`public.article_status`	`draft`, `submitted`, `in_review`, `scheduled`, `published`, `archived`, `rejected`	schema.sql
`public.nav_placement`	`primary`, `more`, `hidden`	schema.sql
`public.contributor_type`	`staff`, `student`, `guest`, `faculty`, `alumni`	schema.sql
`public.asset_kind`	`image`, `document`, `audio`, `video`	schema.sql
`public.article_media_usage`	`inline`, `gallery`, `attachment`	schema.sql
`public.revision_kind`	`autosave`, `manual`, `status`	schema.sql
`public.comment_status`	`pending`, `approved`, `rejected`, `spam`	schema.sql
`public.intake_status`	`submitted`, `reviewing`, `accepted`, `declined`, `converted`	schema.sql
`public.contact_category`	`general`, `editorial`, `submission`, `technical`, `advertising`, `feedback`	schema.sql
`public.contact_status`	`unread`, `in_progress`, `resolved`, `spam`	schema.sql
`public.issue_type`	`regular`, `special`, `graduation`, `annual`, `commemorative`	schema.sql
`public.team_role_group`	`adviser`, `editorial_board`, `section_head`, `staff`	schema.sql
`public.sk_category`	`ball_games`, `athletics`, `racket_sports`, `aquatics`, `mind_games`, `esports`, `dance_cheer`, `combat_sports`, `pageant`, `others`	salingkusog.sql
`public.sk_division`	`men`, `women`, `mixed`, `open`	salingkusog.sql
`public.sk_format`	`bracket`, `round_robin`, `timed`, `judged`, `round_robin_final`	salingkusog.sql
`public.sk_match_status`	`upcoming`, `live`, `final`, `postponed`, `cancelled`	salingkusog.sql
`public.sk_medal`	`gold`, `silver`, `bronze`	salingkusog.sql
`public.sk_award_kind`	`mvp_of_day`, `athlete_of_day`, `finals_mvp`	salingkusog.sql

Appendix C. Functions and RPCs

Security: **definer** runs with the owner's rights (used for helpers that must read protected tables); **invoker** runs with the caller's rights, so RLS applies.

Function	Arguments	Returns	Security	File	Purpose (from SQL comments)
`app.slugify`	input text	text	invoker	schema.sql	Generic helpers
`app.touch_updated_at`		trigger	invoker	schema.sql	
`app.current_role`		public.app_role	definer	schema.sql	Permission helpers (security definer: they read profiles without RLS)
`app.is_staff`		boolean	definer	schema.sql	
`app.is_admin`		boolean	definer	schema.sql	
`app.can_publish`		boolean	definer	schema.sql	
`app.can_publish_articles`		boolean	definer	schema.sql	Publish / schedule articles: the three managing roles, the Business Manager, and anyone given publishing access in Staff. (Front page, settings and the like stay app.can_publish.)
`app.is_editor`		boolean	definer	schema.sql	
`app.can_edit_section`	p_section_id uuid	boolean	definer	schema.sql	
`app.is_live`	p_status public.article_status, p_published_at timestamptz	boolean	invoker	schema.sql	
`app.is_article_owner`	p_article_id uuid	boolean	definer	schema.sql	
`app.article_is_live`	p_article_id uuid	boolean	definer	schema.sql	
`app.can_view_article`	p_article_id uuid	boolean	definer	schema.sql	

Function	Arguments	Returns	Security	File	Purpose (from SQL comments)
`app.can_modify_article`	p_article_id uuid	boolean	definer	schema.sql	Owners may change an article's parts only while it is a draft or rejected; editors of its section may change it at any time.
`app.rate_limit`	p_key text, p_max integer, p_window interval	void	definer	schema.sql	
`app.audit`	p_action text, p_entity text, p_entity_id text, p_details jsonb default '{}'::jsonb	void	definer	schema.sql	
`app.handle_new_user`		trigger	definer	schema.sql	New auth user → profile (role only ever comes from staff_invites)
`app.guard_profile_update`		trigger	definer	schema.sql	Role / activation changes are admin-only; only super_admin grants super_admin; the last active super_admin can't be demoted or deactivated.
`app.article_before_write`		trigger	definer	schema.sql	submitted → draft (owner withdraws / editor returns) | in_review | rejected (editor) | scheduled, published (publisher) in_review → draft, rejected (editor) | scheduled, published (publisher) sc
`app.article_after_write`		trigger	definer	schema.sql	
`app.prune_revisions`		trigger	definer	schema.sql	keep the newest 50 revisions per article

Function	Arguments	Returns	Security	File	Purpose (from SQL comments)
`app.publish_due_articles`		integer	definer	schema.sql	flip due scheduled rows to published (optional: run from pg_cron; the public site already shows them once published_at has passed)
`public.record_article_view`	p_article_id uuid	void	definer	schema.sql	RPCs: public
`public.most_read_articles`	p_days integer default 7, p_limit integer default 5	table (id uuid, slug text, title text, section_slug text, section_name text, category_name text, published_at timestamptz, views bigint)	definer	schema.sql	
`public.search_articles`	p_query text, p_section text default null, p_from date default null, p_to date default null, p_limit integer default 20, p_offset integer default 0, p_live_only boolean default true	table (id uuid, slug text, title text, excerpt text, section_slug text, section_name text, category_name text, published_at timestamptz, status public.article_status, cover_bucket text, cover_path text, cover_alt text, rank real, total_count bigint)	invoker	schema.sql	Full-text search over live articles (RLS applies: staff also see their drafts unless p_live_only is true, which the public site always passes).
`public.archive_months`		table (month date, articles bigint)	definer	schema.sql	Months that have live articles, newest first (archive browsing).
`app.check_email`	p_email text	text	invoker	schema.sql	
`app.check_text`	p_value text, p_label text, p_min integer, p_max integer	text	invoker	schema.sql	

Function	Arguments	Returns	Security	File	Purpose (from SQL comments)
`public.submit_comment`	p_article_id uuid, p_name text, p_email text, p_body text, p_parent_id uuid default null, p_website text default null	uuid	definer	schema.sql	p_website is a honeypot field: real visitors never fill it in.
`app.attach_intake_files`	p_files jsonb, p_prefix text, p_submission uuid, p_event uuid	void	definer	schema.sql	p_files: [{“path”: “submissions//file.pdf”, “name”: “file.pdf”, “mime”: “…”, “size”: 123}] Files must already be uploaded to the private “intake” bucket.
`public.submit_article`	p_name text, p_email text, p_affiliation text, p_section_slug text, p_title text, p_summary text, p_body text, p_files jsonb default ‘[]’::jsonb, p_website text default null	uuid	definer	schema.sql	
`public.submit_event_report`	p_organization text, p_contact_name text, p_contact_email text, p_event_title text, p_event_date date, p_venue text, p_summary text, p_files jsonb default ‘[]’::jsonb, p_website text default null	uuid	definer	schema.sql	
`public.send_contact_message`	p_name text, p_email text, p_subject text, p_category public.contact_category, p_message text, p_website text default null	uuid	definer	schema.sql	

Function	Arguments	Returns	Security	File	Purpose (from SQL comments)
`public.transition_article`	p_article_id uuid, p_to public.article_status, p_note text default null, p_scheduled_for timestamptz default null	public.articles	invoker	schema.sql	RPCs: staff (run as the caller, so RLS and the workflow trigger apply)
`public.add_article_note`	p_article_id uuid, p_note text	void	definer	schema.sql	
`public.convert_intake_to_article`	p_kind text, p_intake_id uuid, p_category_id uuid	uuid	invoker	schema.sql	Turn a public submission or event report into a draft article.
`public.set_publish_access`	p_profile_id uuid, p_allowed boolean	void	definer	schema.sql	Staff → “Can publish”: the three managing roles turn publishing access on or off.
`public.dashboard_stats`		jsonb	definer	schema.sql	
`public.media_usage`	p_asset_id uuid	integer	definer	schema.sql	
`app.can_manage_sk_team`		boolean	definer	salingkusog.sql	Permission helpers
`app.can_edit_sk`		boolean	definer	salingkusog.sql	
`app.sk_edition_visible`	p_edition_id uuid	boolean	definer	salingkusog.sql	
`app.sk_inherit_edition`		trigger	definer	salingkusog.sql	Child rows always carry their parent’s edition (so a match can’t mix editions)
`app.sk_match_status`		trigger	definer	salingkusog.sql	Live / final timestamps and who touched the score last
`app.sk_single_current`		trigger	definer	salingkusog.sql	Only one current edition
`app.sk_countdown_guard`		trigger	definer	salingkusog.sql	Only the managing roles may switch the countdown page or move its time (team members edit everything else)

Function	Arguments	Returns	Security	File	Purpose (from SQL comments)
`app.sk_team_audit`		trigger	definer	salingkusog.sql	Team changes are audited
`public.sk_set_placements`	p_event_id uuid, p_rows jsonb	integer	invoker	salingkusog.sql	RPC: replace an event’s results in one step (newsroom “Results & medals”). Medals follow the place (1 gold, 2 silver, 3 bronze); RLS still applies. p_rows: [{“place”: 1, “faction_id”: “…”, “athlete_id
`app.sk_rr_table`	p_event_id uuid	table (faction_id uuid, rank integer, played integer, wins integer, losses integer, draws integer, pf numeric, pa numeric)	definer	salingkusog.sql	Round-robin standings: wins, point difference, head-to-head among the tied, points scored, seed. Group games = every game of the event that isn’t fed by a winner/loser/rank (so imported round robins w
`app.sk_resolve`	p_event_id uuid	void	definer	salingkusog.sql	Fills in the sides of the event’s generated games (idempotent).
`app.sk_source_side`	p_event_id uuid, p_format text, p_seeds uuid[], p_group_left integer, p_round integer, p_source text	uuid	definer	salingkusog.sql	One side of a generated game, or null while it isn’t known yet.
`app.sk_matches_advance`		trigger	definer	salingkusog.sql	After a game changes, move the teams along (one level deep: the updates it makes don’t recurse).
`app.sk_knockout_needs_winner`		trigger	definer	salingkusog.sql	Knockout games need a winner (overtime, penalties… go into the score)

Function	Arguments	Returns	Security	File	Purpose (from SQL comments)
`public.sk_build_schedule`	p_event_id uuid, p_format text, p_seeds uuid[], p_first_at timestamptz, p_round_gap_hours integer default 24, p_game_gap_minutes integer default 90, p_venue_id uuid default null, p_bronze boolean default true	integer	invoker	salingkusog.sql	RPC: build an event's games ("Build schedule" in the newsroom). p_format: bracket (single elimination), round_robin, or round_robin_final (top 2 play the final). p_seeds: factions in seed order. Repla
`public.sk_create_contest`	p_event_id uuid, p_at timestamptz, p_venue_id uuid default null, p_round text default null, p_factions uuid[] default null, p_shuffle boolean default false	uuid	invoker	salingkusog.sql	RPC: a contest session with every faction in it ("Create contest" in the newsroom). p_factions: who takes part, in performance order (default: all, by faction order); p_shuffle draws a random order in
`public.sk_countdown`		table (title text, subtitle text, starts_on date, countdown_at timestamptz)	definer	salingkusog.sql	RPC: the public countdown. Returns a row only while the current edition's countdown page is switched on (works before the edition itself is published).
`public.sk_points_timeline`	p_edition_id uuid, p_hours integer	table (faction_id uuid, bucket_end timestamptz, points integer)	invoker	salingkusog.sql	RPC: the points race in hour steps (1, 3, 6 or 12). Cumulative points per faction at the end of each step. 1/3/6 h cover the latest day with results; 12 h covers the whole edition. A result counts at
`app.can_view_stats`		boolean	definer	stats.sql	Helpers
`app.manila_today`		date	invoker	stats.sql	

Function	Arguments	Returns	Security	File	Purpose (from SQL comments)
`app.stats_hash`	p_raw text	text	definer	stats.sql	
`public.stats_ingest`	p_visitor text, p_session text, p_events jsonb, p_device text default ‘desktop’, p_browser text default ‘Other’, p_os text default ‘Other’	integer	definer	stats.sql	Write path: public.stats_ingest (service role only, called by /api/e) p_events: [{“t”:“pv”,“id”:“”,“path”:“/article/x”,“src”:“social”,“ref”:“facebook.com”,“us”:“…”,“um”:“…”,“uc”:“…”}, {“t”:“eng”
`public.staff_heartbeat`		integer	definer	stats.sql	Staff heartbeat: the newsroom calls this every ~30 s while the tab is visible and the person was active in the last minute. Credit is the time since the last beat, capped at 45 s, and only when that b
`app.stats_rollup_day`	p_day date	void	definer	stats.sql	Daily totals and retention
`app.stats_nightly`		void	definer	stats.sql	Nightly: total up every finished day not done yet, then apply retention.
`app.stats_live_from`		date	definer	stats.sql	First day still read from raw events (everything before it has daily totals)
`app.stats_site_days`	p_from date, p_to date	table (day date, pageviews bigint, views bigint, visitors bigint, sessions bigint, active_seconds bigint, read_samples bigint, shares bigint)	definer	stats.sql	Read path (internal): daily totals for finished days + raw events after them

Function	Arguments	Returns	Security	File	Purpose (from SQL comments)
`app.stats_article_totals`	p_from date, p_to date	table (article_id uuid, pageviews bigint, views bigint, visitors bigint, active_seconds bigint, read_samples bigint, scroll_sum bigint, scroll_samples bigint, finished bigint, shares bigint)	definer	stats.sql	
`app.stats_dim_totals`	p_from date, p_to date, p_dim text	table (value text, pageviews bigint, visitors bigint)	definer	stats.sql	
`app.stats_require`	p_from date, p_to date	void	definer	stats.sql	Read path (public RPCs): stats viewers only
`public.stats_overview`	p_from date, p_to date	jsonb	definer	stats.sql	Totals for the range and the same-length period before it, plus a daily series.
`public.stats_articles`	p_from date, p_to date, p_limit integer default 20	table (article_id uuid, slug text, title text, section_name text, category_name text, authors text, published_at timestamptz, views bigint, pageviews bigint, visitors bigint, avg_read_seconds integer, avg_scroll integer, finish_rate integer, shares bigint)	definer	stats.sql	
`public.stats_authors`	p_from date, p_to date	table (contributor_id uuid, name text, articles bigint, views bigint, avg_read_seconds integer, shares bigint)	definer	stats.sql	

Function	Arguments	Returns	Security	File	Purpose (from SQL comments)
`public.stats_sections`	p_from date, p_to date	table (section_name text, category_name text, articles bigint, views bigint, avg_read_seconds integer, shares bigint)	definer	stats.sql	
`public.stats_breakdown`	p_from date, p_to date, p_dim text	table (value text, pageviews bigint, visitors bigint)	definer	stats.sql	
`public.stats_share_channels`	p_from date, p_to date	table (channel text, shares bigint)	definer	stats.sql	
`public.stats_staff_usage`	p_from date, p_to date	table (profile_id uuid, display_name text, role public.app_role, is_active boolean, active_seconds bigint, days_active bigint, last_active_at timestamptz)	definer	stats.sql	

Appendix D. Triggers

Trigger	Table	Timing / events	Function
`on_auth_user_created`	`auth.users`	after insert	`app.handle_new_user`
`profiles_guard`	`public.profiles`	before update	`app.guard_profile_update`
`articles_before_write`	`public.articles`	before insert or update	`app.article_before_write`
`articles_after_write`	`public.articles`	after insert or update	`app.article_after_write`
`article_revisions_prune`	`public.article_revisions`	after insert	`app.prune_revisions`
`sk_matches_status`	`public.sk_matches`	before insert or update	`app.sk_match_status`
`sk_editions_single_current`	`public.sk_editions`	before insert or update of is_current	`app.sk_single_current`
`sk_editions_countdown_guard`	`public.sk_editions`	before insert or update	`app.sk_countdown_guard`
`sk_team_audit`	`public.sk_team`	after insert or delete	`app.sk_team_audit`

Trigger	Table	Timing / events	Function
sk_matches_advance	public.sk_matches	after insert or delete or update of status, score_a, score_b, side_a_id, side_b_id, source_a, source_b	app.sk_matches_advance
sk_matches_knockout	public.sk_matches	before insert or update	app.sk_knockout_needs_winner
sk_events_edition	public.sk_events	before insert or update	app.sk_inherit_edition
sk_matches_edition	public.sk_matches	before insert or update	app.sk_inherit_edition
sk_placements_edition	public.sk_placements	before insert or update	app.sk_inherit_edition
sk_athletes_edition	public.sk_athletes	before insert or update	app.sk_inherit_edition
sk_entries_edition	public.sk_entries	before insert or update	app.sk_inherit_edition
sk_editions_touch	public.sk_editions	before update	app.touch_updated_at
sk_factions_touch	public.sk_factions	before update	app.touch_updated_at
sk_venues_touch	public.sk_venues	before update	app.touch_updated_at
sk_sports_touch	public.sk_sports	before update	app.touch_updated_at
sk_events_touch	public.sk_events	before update	app.touch_updated_at
sk_athletes_touch	public.sk_athletes	before update	app.touch_updated_at
sk_matches_touch	public.sk_matches	before update	app.touch_updated_at
sk_placements_touch	public.sk_placements	before update	app.touch_updated_at
sk_awards_touch	public.sk_awards	before update	app.touch_updated_at
sk_entries_touch	public.sk_entries	before update	app.touch_updated_at
profiles_touch	public.profiles	before update	app.touch_updated_at
sections_touch	public.sections	before update	app.touch_updated_at
categories_touch	public.categories	before update	app.touch_updated_at
media_assets_touch	public.media_assets	before update	app.touch_updated_at
contributors_touch	public.contributors	before update	app.touch_updated_at
publication_issues_touch	public.publication_issues	before update	app.touch_updated_at
article_submissions_touch	public.article_submissions	before update	app.touch_updated_at
event_reports_touch	public.event_reports	before update	app.touch_updated_at
contact_messages_touch	public.contact_messages	before update	app.touch_updated_at
team_terms_touch	public.team_terms	before update	app.touch_updated_at
team_members_touch	public.team_members	before update	app.touch_updated_at
department_contacts_touch	public.department_contacts	before update	app.touch_updated_at
site_settings_touch	public.site_settings	before update	app.touch_updated_at
home_slots_touch	public.home_slots	before update	app.touch_updated_at

Appendix E. Indexes

Index	Table	Columns	Unique	Condition
profiles_email_key	public.profiles	lower(email)	yes	
profile_sections_section_idx	public.profile_sections	section_id		
media_assets_created_idx	public.media_assets	created_at desc		
media_assets_uploaded_by_idx	public.media_assets	uploaded_by		
publication_issues_date_idx	public.publication_issues	issue_date desc		
articles_live_idx	public.articles	published_at desc		where status in ('published', 'scheduled')
articles_live_section_idx	public.articles	section_id, published_at desc		where status in ('published', 'scheduled')
articles_live_category_idx	public.articles	category_id, published_at desc		where status in ('published', 'scheduled')
articles_featured_idx	public.articles	published_at desc		where is_featured
articles_issue_idx	public.articles	issue_id, issue_position		where issue_id is not null
articles_status_updated_idx	public.articles	status, updated_at desc		
articles_created_by_idx	public.articles	created_by, updated_at desc		
articles_cover_idx	public.articles	cover_asset_id		where cover_asset_id is not null
articles_og_image_idx	public.articles	og_image_asset_id		where og_image_asset_id is not null
articles_search_idx	public.articles	search		
article_authors_contributor_idx	public.article_authors	contributor_id		
article_tags_tag_idx	public.article_tags	tag_id		
article_media_asset_idx	public.article_media	asset_id		
article_revisions_article_idx	public.article_revisions	article_id, created_at desc		

Index	Table	Columns	Unique	Condition
article_activity_article_idx	public.article_activity	article_id, created_at desc		
article_activity_created_idx	public.article_activity	created_at desc		
article_daily_views_day_idx	public.article_daily_views	day, article_id		
article_comments_article_idx	public.article_comments	article_id, status, created_at		
article_comments_status_idx	public.article_comments	status, created_at desc		
article_comments_parent_idx	public.article_comments	parent_id		where parent_id is not null
article_submissions_status_idx	public.article_submissions	status, created_at desc		
event_reports_status_idx	public.event_reports	status, created_at desc		
intake_files_submission_idx	public.intake_files	submission_id		where submission_id is not null
intake_files_event_idx	public.intake_files	event_report_id		where event_report_id is not null
contact_messages_status_idx	public.contact_messages	status, created_at desc		
team_terms_one_current	public.team_terms	is_current	yes	where is_current
team_members_term_idx	public.team_members	term_id, role_group, sort_order		
team_members_contributor_idx	public.team_members	contributor_id		where contributor_id is not null
audit_log_created_idx	public.audit_log	created_at desc		
rate_events_key_idx	app.rate_events	key, created_at desc		
sk_editions_one_current	public.sk_editions	is_current	yes	where is_current
sk_events_sport_idx	public.sk_events	sport_id, sort_order		
sk_athletes_faction_idx	public.sk_athletes	faction_id		
sk_matches_schedule_idx	public.sk_matches	edition_id, scheduled_at		
sk_matches_live_idx	public.sk_matches	edition_id		where status = ‘live’
sk_matches_event_idx	public.sk_matches	event_id, round_order, bracket_slot		

Index	Table	Columns	Unique	Condition
sk_matches_side_a_idx	public.sk_matches	side_a_id		
sk_matches_side_b_idx	public.sk_matches	side_b_id		
sk_matches_venue_idx	public.sk_matches	venue_id, scheduled_at		
sk_entries_faction_idx	public.sk_entries	faction_id		
sk_entries_athlete_idx	public.sk_entries	athlete_id		where athlete_id is not null
sk_athlete_stats_athlete_idx	public.sk_athlete_stats	athlete_id		
sk_placements_unique	public.sk_placements	event_id, place, faction_id, coalesce(athlete_id, '00000000-0000-0000-0000-000000000000'::	yes	
sk_placements_edition_idx	public.sk_placements	edition_id, awarded_on		
sk_placements_faction_idx	public.sk_placements	faction_id		
sk_placements_athlete_idx	public.sk_placements	athlete_id		where athlete_id is not null
sk_awards_one_per_day	public.sk_awards	edition_id, kind, day	yes	where kind <> 'finals_mvp'
sk_awards_athlete_idx	public.sk_awards	athlete_id		
stats_pageviews_day_idx	public.stats_pageviews	day		
stats_pageviews_article_idx	public.stats_pageviews	article_id, day		where article_id is not null
stats_pageviews_dedupe_idx	public.stats_pageviews	visitor_hash, article_id, occurred_at desc		where article_id is not null
stats_shares_day_idx	public.stats_shares	day		
stats_shares_article_idx	public.stats_shares	article_id, day		
staff_activity_daily_day_idx	public.staff_activity_daily	day		
stats_daily_articles_article_idx	public.stats_daily_articles	article_id		

Appendix F. Row Level Security policies

`using` decides which existing rows a command can see or affect; `with check` decides which new or changed rows are accepted.

Table	Policy	Command	Roles	Using	With check
`profiles`	`profiles_select`	select	authenticated	id = auth.uid() or app.is_staff()	
`profiles`	`profiles_update`	update	authenticated	id = auth.uid() or app.is_admin()	id = auth.uid() or app.is_admin()
`staff_invites`	`staff_invites_admin`	all	authenticated	app.is_admin()	app.is_admin()
`sections`	`sections_read`	select	anon, authenticated	true	
`sections`	`sections_admin`	all	authenticated	app.is_admin()	app.is_admin()
`profile_sections`	`profile_sections_read`	select	authenticated	profile_id = auth.uid() or app.is_staff()	
`profile_sections`	`profile_sections_admin`	all	authenticated	app.is_admin()	app.is_admin()
`categories`	`categories_read`	select	anon, authenticated	is_active or app.is_staff()	
`categories`	`categories_admin`	all	authenticated	app.is_admin()	app.is_admin()
`tags`	`tags_read`	select	anon, authenticated	true	
`tags`	`tags_insert`	insert	authenticated		app.is_staff()
`tags`	`tags_editor_update`	update	authenticated	app.is_editor()	app.is_editor()
`tags`	`tags_editor_delete`	delete	authenticated	app.is_editor()	
`media_assets`	`media_read`	select	anon, authenticated	true	
`media_assets`	`media_insert`	insert	authenticated		app.is_staff() and uploaded_by = auth.uid() and (bucket = ‘media’ or app.can_publish())
`media_assets`	`media_update`	update	authenticated	uploaded_by = auth.uid() or app.is_editor()	app.is_staff()
`media_assets`	`media_delete`	delete	authenticated	uploaded_by = auth.uid() or app.is_editor()	

Table	Policy	Command	Roles	Using	With check
contributors	contributors_read	select	anon, authenticated	(is_public and is_active) or app.is_staff()	
contributors	contributors_editor	all	authenticated	app.is_editor()	app.is_editor()
contributors	contributors_self_update	update	authenticated	profile_id = auth.uid()	profile_id = auth.uid()
publication_issues	issues_read	select	anon, authenticated	is_published or app.is_staff()	
publication_issues	issues_publisher	all	authenticated	app.can_publish()	app.can_publish()
articles	articles_public_read	select	anon, authenticated	app.is_live(status, published_at)	
articles	articles_staff_read	select	authenticated	app.is_staff() and (created_by = auth.uid() or app.is_article_owner(id) or app.can_edit_section(section_id))	
articles	articles_insert	insert	authenticated		app.is_staff()
articles	articles_update	update	authenticated	app.is_staff() and (created_by = auth.uid() or app.is_article_owner(id) or app.can_edit_section(section_id))	app.is_staff()
articles	articles_delete	delete	authenticated	app.is_admin() or (status = ‘draft’ and created_by = auth.uid())	
article_authors	article_authors_read	select	anon, authenticated	app.article_is_live(article_id) or app.can_view_article(article_id)	
article_authors	article_authors_write	all	authenticated	app.can_modify_article(article_id)	app.can_modify_article(article_id)
article_tags	article_tags_read	select	anon, authenticated	app.article_is_live(article_id) or app.can_view_article(article_id)	
article_tags	article_tags_write	all	authenticated	app.can_modify_article(article_id)	app.can_modify_article(article_id)
article_media	article_media_read	select	anon, authenticated	app.article_is_live(article_id) or app.can_view_article(article_id)	

Table	Policy	Command	Roles	Using	With check
article_media	article_media_write	all	authenticated	app.can_modify_article(article_id)	app.can_modify_article(article_id)
article_revisions	article_revisions_read	select	authenticated	app.can_view_article(article_id)	
article_revisions	article_revisions_insert	insert	authenticated		app.can_modify_article(article_id) and saved_by = auth.uid() and kind in ('autosave', 'manual')
article_activity	article_activity_read	select	authenticated	app.can_view_article(article_id)	
article_daily_views	article_daily_views_staff	select	authenticated	app.is_staff()	
article_comments	comments_public_read	select	anon, authenticated	(status = 'approved' and app.article_is_live(article_id)) or app.is_editor()	
article_comments	comments_moderate	update	authenticated	app.is_editor()	app.is_editor()
article_comments	comments_delete	delete	authenticated	app.is_editor()	
article_comment_private	comment_private_editor	select	authenticated	app.is_editor()	
home_slots	home_slots_read	select	anon, authenticated	true	
home_slots	home_slots_write	all	authenticated	app.can_publish()	app.can_publish()
article_submissions	submissions_editor	all	authenticated	app.is_editor()	app.is_editor()
event_reports	event_reports_editor	all	authenticated	app.is_editor()	app.is_editor()
intake_files	intake_files_editor	select	authenticated	app.is_editor()	
intake_files	intake_files_editor_delete	delete	authenticated	app.is_editor()	
contact_messages	contact_editor	all	authenticated	app.is_editor()	app.is_editor()
team_terms	team_terms_read	select	anon, authenticated	true	
team_terms	team_terms_write	all	authenticated	app.can_publish()	app.can_publish()
team_members	team_members_read	select	anon, authenticated	true	

Table	Policy	Command	Roles	Using	With check
team_members	team_members_write	all	authenticated	app.can_publish()	app.can_publish()
department_contacts	department_contacts_read	select	anon, authenticated	true	
department_contacts	department_contacts_write	all	authenticated	app.is_admin()	app.is_admin()
site_settings	site_settings_read	select	anon, authenticated	is_public or app.is_staff()	
site_settings	site_settings_write	all	authenticated	app.is_admin() or (app.can_publish() and key = 'ticker')	app.is_admin() or (app.can_publish() and key = 'ticker')
audit_log	audit_log_admin	select	authenticated	app.is_admin()	
storage.objects	v2_public_read	select	anon, authenticated	bucket_id in ('media', 'archives')	
storage.objects	v2_media_insert	insert	authenticated		bucket_id = 'media' and app.is_staff()
storage.objects	v2_media_update	update	authenticated	bucket_id = 'media' and (owner_id = auth.uid()::text or app.is_editor())	bucket_id = 'media' and app.is_staff()
storage.objects	v2_media_delete	delete	authenticated	bucket_id = 'media' and (owner_id = auth.uid()::text or app.is_editor())	
storage.objects	v2_archives_write	all	authenticated	bucket_id = 'archives' and app.can_publish()	bucket_id = 'archives' and app.can_publish()
storage.objects	v2_intake_editor	select	authenticated	bucket_id = 'intake' and app.is_editor()	
storage.objects	v2_intake_editor_delete	delete	authenticated	bucket_id = 'intake' and app.is_editor()	
sk_editions	sk_editions_read	select	anon, authenticated	is_published or app.is_staff()	
sk_editions	sk_editions_write	all	authenticated	app.can_edit_sk()	app.can_edit_sk()
sk_athlete_stats	sk_athlete_stats_read	select	anon, authenticated	exists (select 1 from public.sk_matches m where m.id = match_id and app.sk_edition_visible(m.edition_id))	

Table	Policy	Command	Roles	Using	With check
sk_athlete_stats	sk_athlete_stats_write	all	authenticated	app.can_edit_sk()	app.can_edit_sk()
sk_team	sk_team_read	select	authenticated	app.is_staff()	
sk_team	sk_team_add	insert	authenticated		app.can_manage_sk_team()
sk_team	sk_team_remove	delete	authenticated	app.can_manage_sk_team()	
sk_factions	sk_factions_read	select	anon, authenticated	app.sk_edition_visible(edition_id)	
sk_venues	sk_venues_read	select	anon, authenticated	app.sk_edition_visible(edition_id)	
sk_sports	sk_sports_read	select	anon, authenticated	app.sk_edition_visible(edition_id)	
sk_events	sk_events_read	select	anon, authenticated	app.sk_edition_visible(edition_id)	
sk_athletes	sk_athletes_read	select	anon, authenticated	app.sk_edition_visible(edition_id)	
sk_matches	sk_matches_read	select	anon, authenticated	app.sk_edition_visible(edition_id)	
sk_placements	sk_placements_read	select	anon, authenticated	app.sk_edition_visible(edition_id)	
sk_awards	sk_awards_read	select	anon, authenticated	app.sk_edition_visible(edition_id)	
sk_entries	sk_entries_read	select	anon, authenticated	app.sk_edition_visible(edition_id)	
sk_factions	sk_factions_write	all	authenticated	app.can_edit_sk()	app.can_edit_sk()
sk_venues	sk_venues_write	all	authenticated	app.can_edit_sk()	app.can_edit_sk()
sk_sports	sk_sports_write	all	authenticated	app.can_edit_sk()	app.can_edit_sk()
sk_events	sk_events_write	all	authenticated	app.can_edit_sk()	app.can_edit_sk()
sk_athletes	sk_athletes_write	all	authenticated	app.can_edit_sk()	app.can_edit_sk()
sk_matches	sk_matches_write	all	authenticated	app.can_edit_sk()	app.can_edit_sk()
sk_placements	sk_placements_write	all	authenticated	app.can_edit_sk()	app.can_edit_sk()
sk_awards	sk_awards_write	all	authenticated	app.can_edit_sk()	app.can_edit_sk()
sk_entries	sk_entries_write	all	authenticated	app.can_edit_sk()	app.can_edit_sk()
stats_pageviews	stats_pageviews_read	select	authenticated	app.can_view_stats()	

Table	Policy	Command	Roles	Using	With check
`stats_shares`	`stats_shares_read`	select	authenticated	app.can_view_stats()	
`staff_activity_daily`	`staff_activity_daily_read`	select	authenticated	app.can_view_stats()	
`stats_daily_site`	`stats_daily_site_read`	select	authenticated	app.can_view_stats()	
`stats_daily_articles`	`stats_daily_articles_read`	select	authenticated	app.can_view_stats()	
`stats_daily_dims`	`stats_daily_dims_read`	select	authenticated	app.can_view_stats()	
`stats_daily_shares`	`stats_daily_shares_read`	select	authenticated	app.can_view_stats()	
`stats_rollup_log`	`stats_rollup_log_read`	select	authenticated	app.can_view_stats()	

Appendix G. Storage buckets

Bucket	Public	Size limit	Allowed types
`media`	yes	10 MB	image/jpeg, image/png, image/webp, image/avif, image/gif
`archives`	yes	60 MB	application/pdf, image/jpeg, image/png, image/webp
`intake`	no	20 MB	image/jpeg, image/png, image/webp, application/pdf, text/plain, application/msword, application/vnd.openxmlformats-officedocument.wordprocessingml.document

1. a-z0-9↩
2. a-z0-9↩
3. a-z0-9↩
4. a-z0-9↩
5. a-z0-9↩
6. a-z0-9↩
7. a-z0-9_↩
8. a-z0-9↩

9. a-z0-9↩
10. a-z0-9↩
11. a-z0-9↩
12. a-z0-9↩

UML and Process Diagrams

Amaranth v2 — context, data flow, use case, activity, sequence, component, deployment, state and ER diagrams · 1.0 · 2026-10-19

Document control

Field	Value
Title	UML and Process Diagrams
Document ID	AMR-DOC-05
Version	1.0
Date	2026-10-19
Author	Amaranth development team [TO CONFIRM: names]
Status	Final draft for defense
Sources	Mermaid files in `docs/diagrams/*.mmd`; PNG and SVG renders beside them

Contents

1. Diagram catalogue
2. Context and data flow
3. Use cases
4. Activities
5. Sequences
6. Structure: layers, components, classes, deployment
7. States
8. Entity–relationship diagrams

All diagrams are drawn from the code. Each caption names the files the diagram was checked against. Edit the `.mmd` source and re-render to update a diagram (AMR-DOC-10 §8).

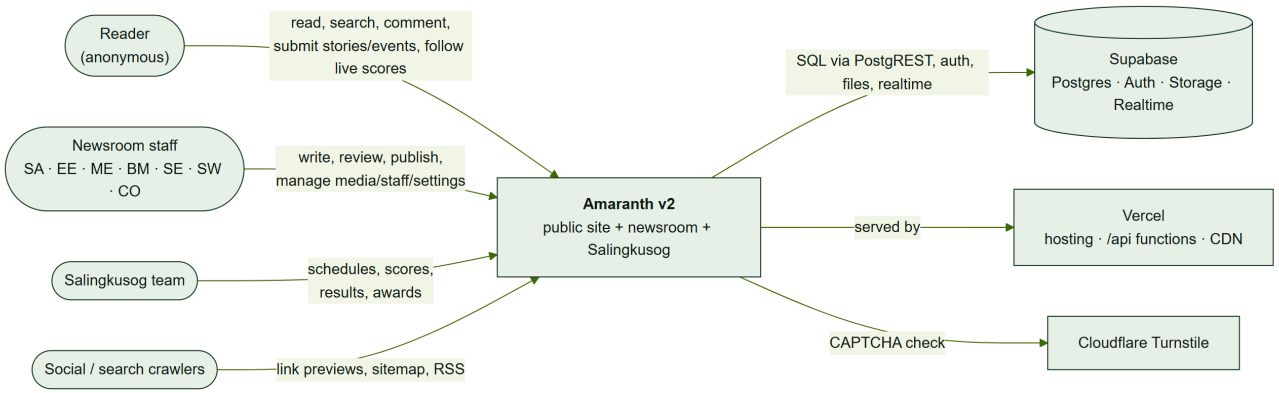
1. Diagram catalogue

ID	Diagram	File	Checked against
CTX-1	System context	`ctx-1-context`	`vercel.json`, `src/lib/supabase.ts`, `api/`
DFD-0	Data flow, level 0	`dfd-0-level0`	feature modules
DFD-1	Data flow, level 1	`dfd-1-level1`	tables per module
UC-1	Use cases: writers, editors, publishers	`uc-1-writers-editors`	`app.article_before_write`

ID	Diagram	File	Checked against
UC-2	Use cases: managers	`uc-2-managers`	role helpers, RLS
UC-3	Use cases: Salingkusog team	`uc-3-salingkusog`	`app.can_edit_sk`, `sk_team`
UC-4	Use cases: readers	`uc-4-readers`	public routes, `/api/submit`
ACT-1	Article submission to publication	`act-1-article-publication`	workflow trigger
ACT-2	Scheduling a game to an updated medal table	`act-2-game-to-medal-table`	`salingkusog.sql`
SEQ-1	Login and role check	`seq-1-login-role`	`LoginPage.tsx`, `AuthProvider.tsx`, `guards.tsx`
SEQ-2	Publish an article	`seq-2-publish-article`	`transition_article`, triggers
SEQ-3	Enter results and medals	`seq-3-enter-results`	`sk_set_placements`, `ResultsTab.tsx`
SEQ-4	Live score update	`seq-4-live-score`	`Scorekeeper.tsx`, `skData.ts`
SEQ-5	Schedule import	`seq-5-schedule-import`	`scheduleCsv.ts`, `ImportTab.tsx`
SEQ-6	Public form with CAPTCHA and attachments	`seq-6-public-form`	`api/submit.ts`, `src/lib/sendForm.ts`
ARCH-1	Layered architecture	`arch-1-layers`	repository structure
CMP-1	Front-end components	`cmp-1-components`	`src/` folders
CLS-1	Domain class model	`cls-1-domain`	core tables
DEP-1	Deployment	`dep-1-deployment`	`vercel.json`, Supabase services
STATE-1	Article lifecycle	`state-1-article`	`app.article_before_write`
STATE-2	Game status	`state-2-game`	`statusActions()` in `scoring.ts`, `sk_match_status`
ERD-1	Content ERD	`erd-1-content`	`schema.sql` (generated)
ERD-2	People, intake and settings ERD	`erd-2-people-intake`	`schema.sql` (generated)
ERD-3	Salingkusog ERD	`erd-3-salingkusog`	`salingkusog.sql` (generated)
ERD-4	Statistics ERD	`erd-4-statistics`	`stats.sql` (generated)

2. Context and data flow

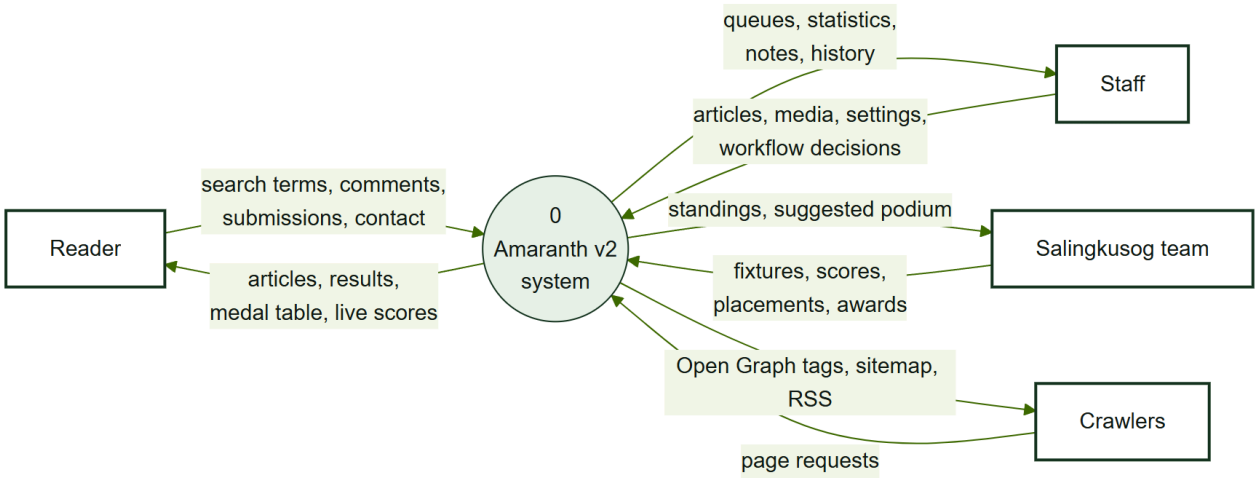
CTX-1 System context



CTX-1 — Amaranth v2 and the people and services around it.

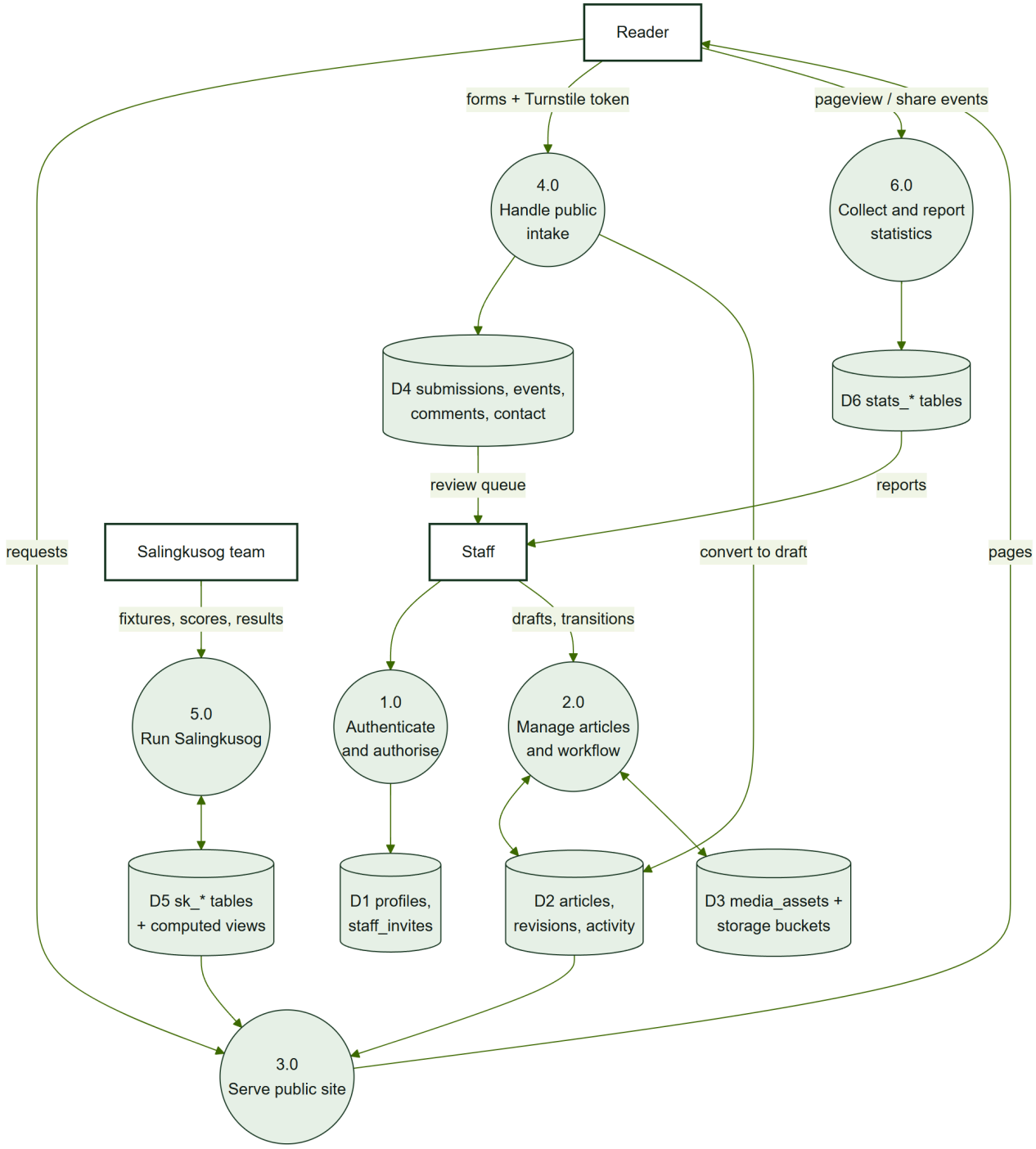
Amaranth v2 is one web application with three audiences. All persistent data and authentication are in Supabase; Vercel hosts the app and runs the small server functions; Cloudflare Turnstile checks public forms.

DFD-0 Level 0



DFD-0 — the system as a single process with its external entities.

DFD-1 Level 1



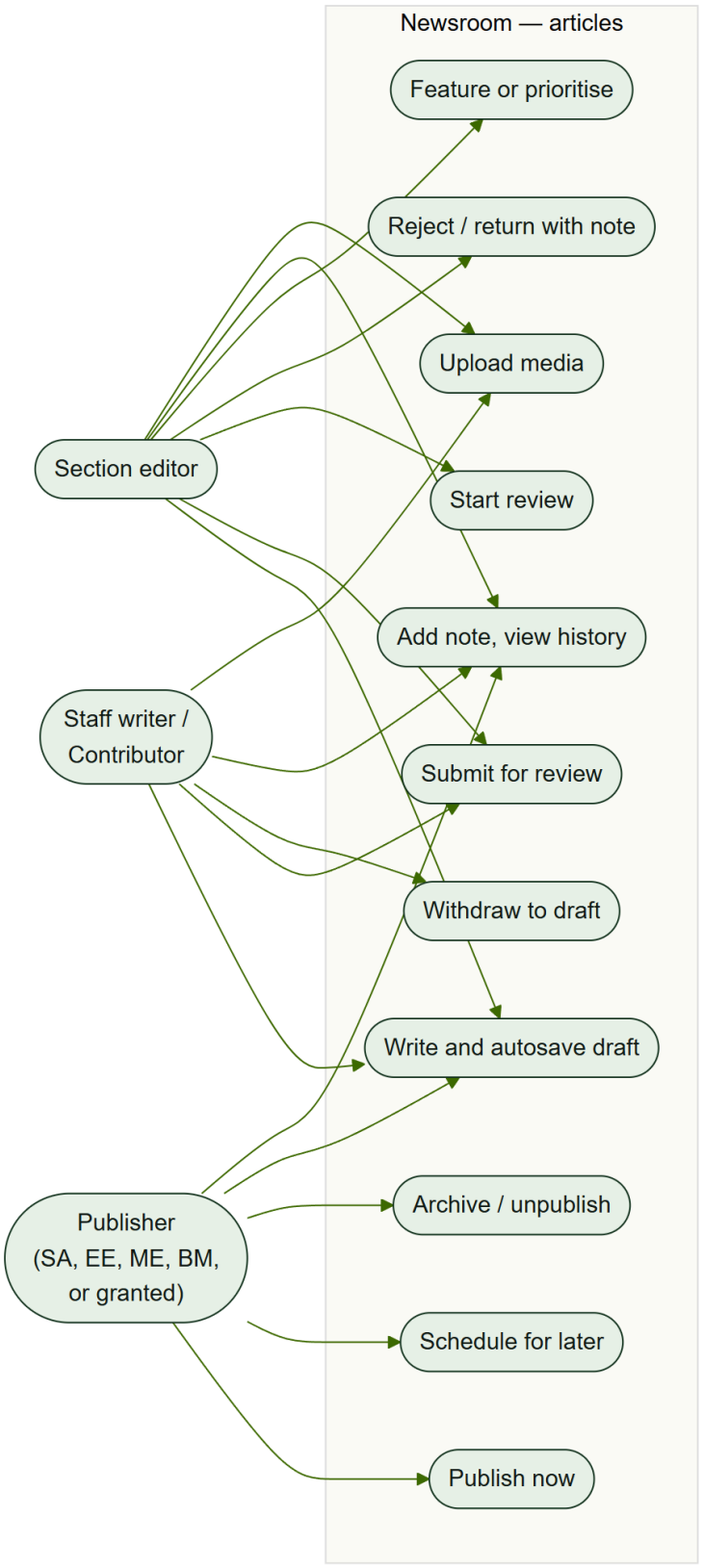
DFD-1 — six processes and their data stores.

Process	Main code	Data stores
1.0 Authenticate and authorise	`features/auth`, `app.current_role`	D1 `profiles`, `staff_invites`
2.0 Manage articles and workflow	`features/cms/articles`, `transition_article`	D2 articles, D3 media
3.0 Serve public site	`features/public`, `api/r.ts`, `middleware.ts`	D2, D5
4.0 Handle public intake	`api/submit.ts`, `features/cms/intake`	D4

Process	Main code	Data stores
5.0 Run Salingkusog	`features/cms/salingkusog`, `salingkusog.sql`	D5
6.0 Collect and report statistics	`api/e.ts`, `stats.sql`, `features/cms/statistics`	D6

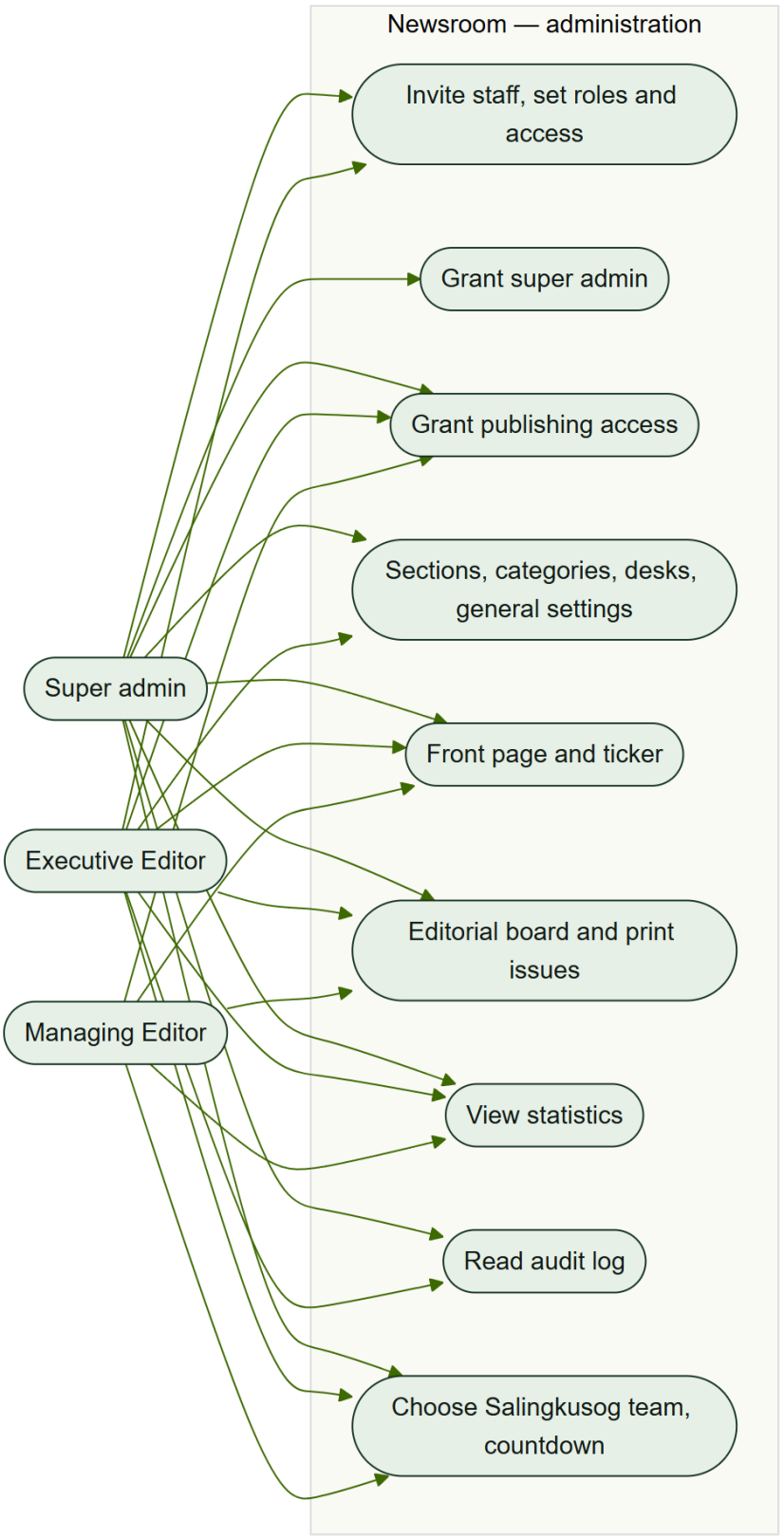
3. Use cases

UC-1 Writers, editors and publishers



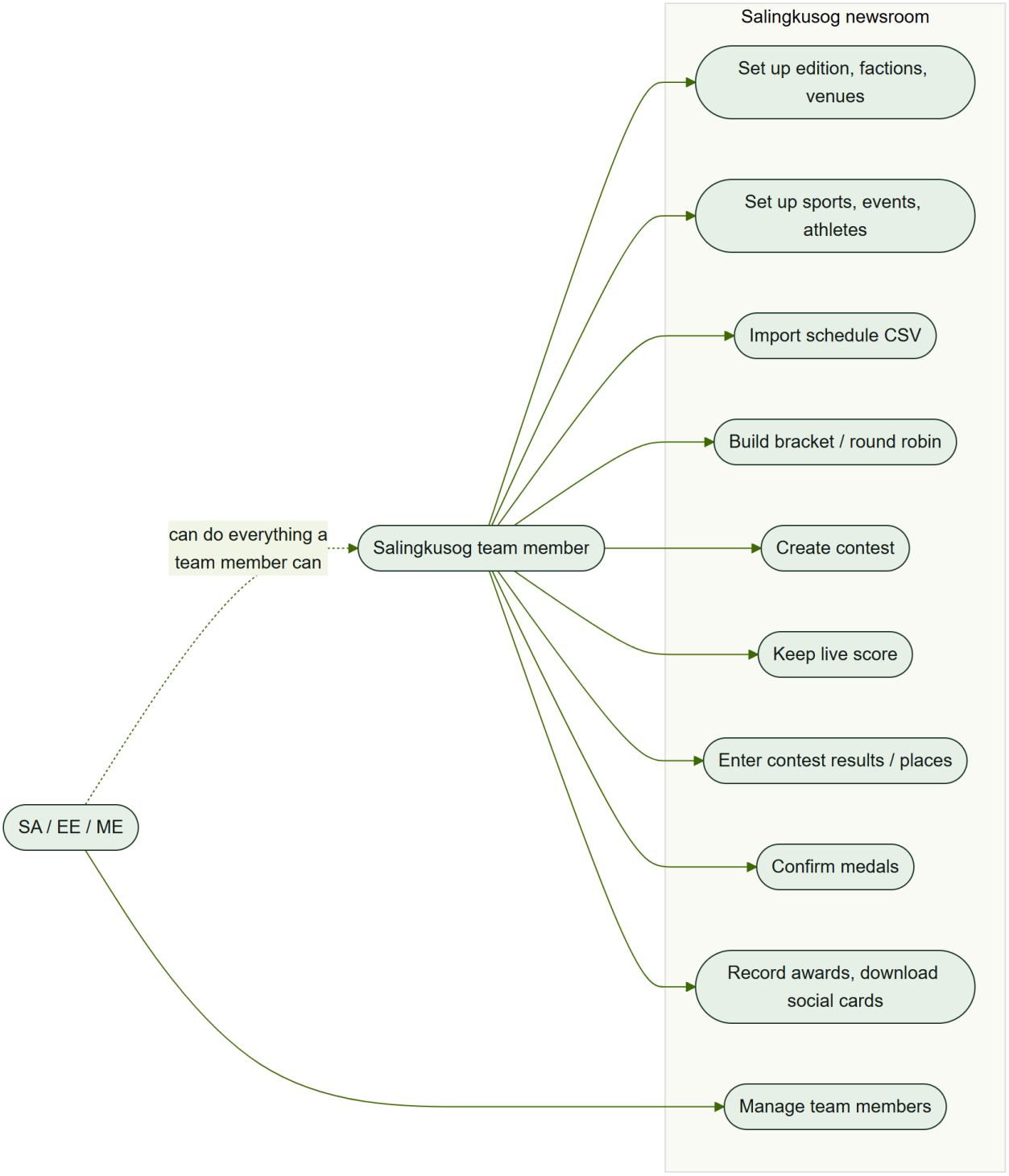
UC-1 — article use cases by role. “Publisher” means SA, EE, ME, BM or staff granted publishing access.

UC-2 Managers



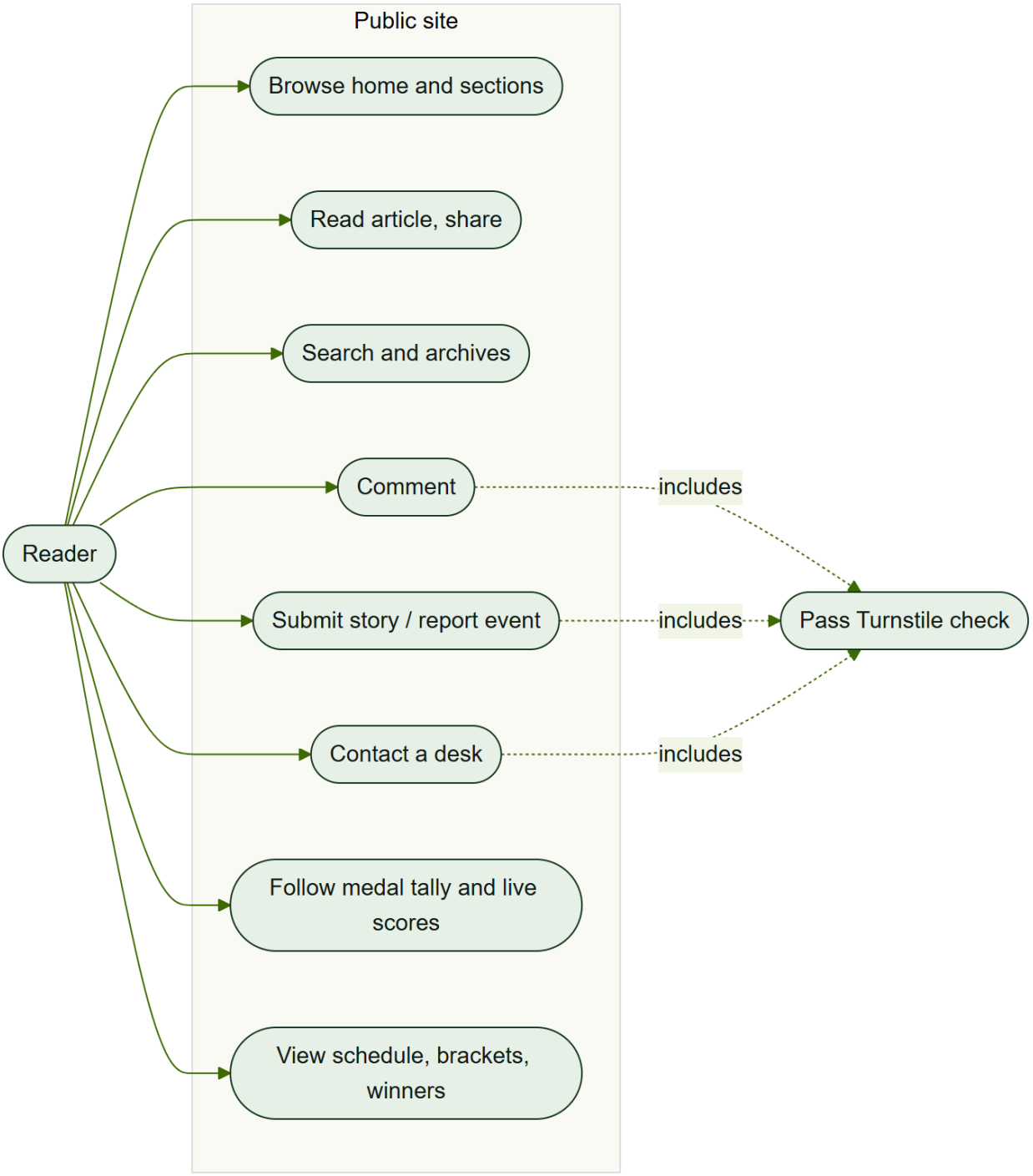
UC-2 — administration use cases for Super admin, Executive Editor and Managing Editor.

UC-3 Salingkusog team



UC-3 — Salingkusog newsroom use cases.

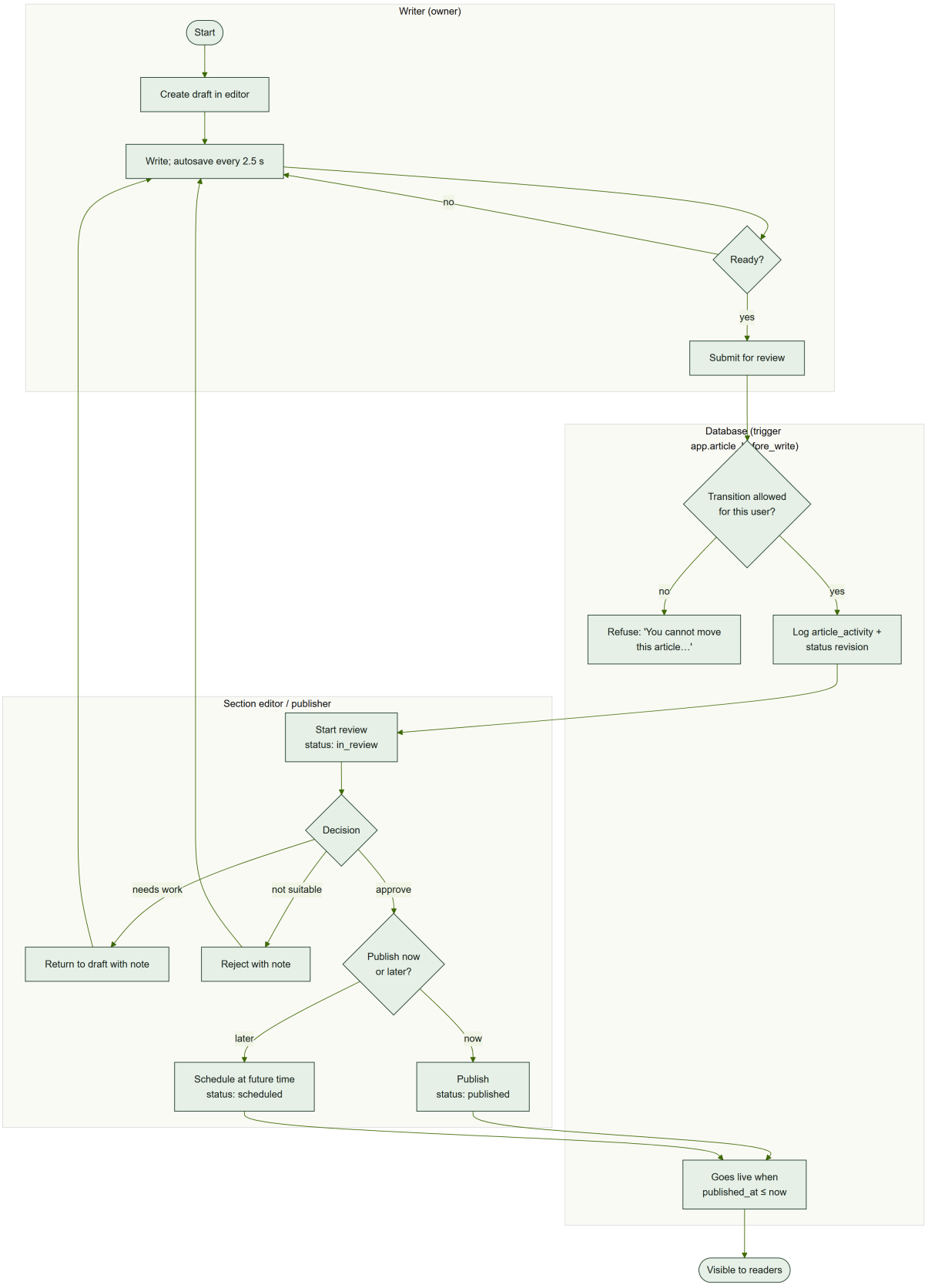
UC-4 Readers



UC-4 — public use cases; forms include the Turnstile check.

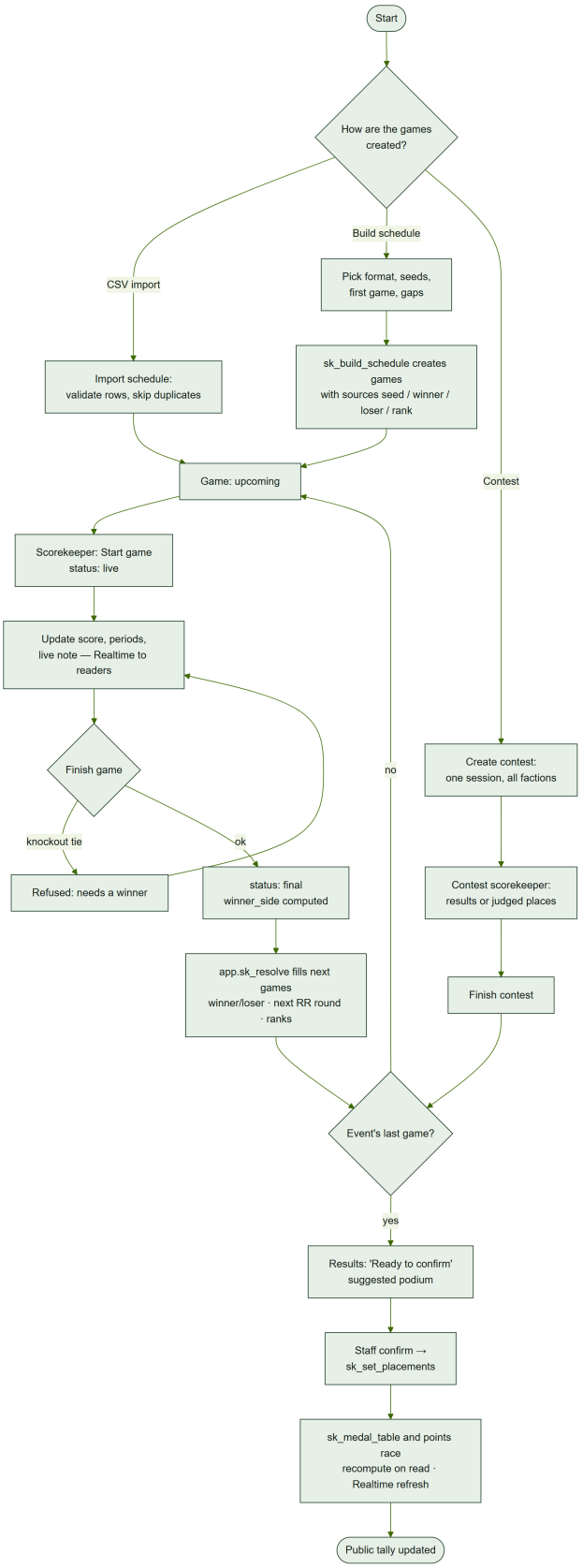
4. Activities

ACT-1 Article submission to publication



ACT-1 — from draft to a live article. Every transition is checked by the database trigger.

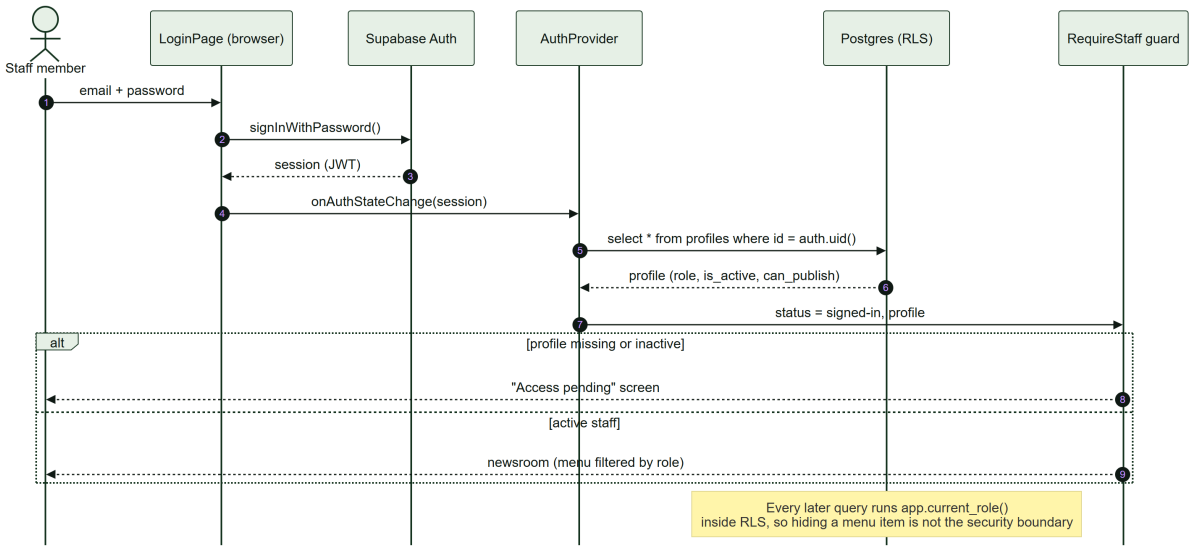
ACT-2 Scheduling a game to an updated medal table



ACT-2 — three ways to create games, live scoring, automatic advancement and staff-confirmed medals.

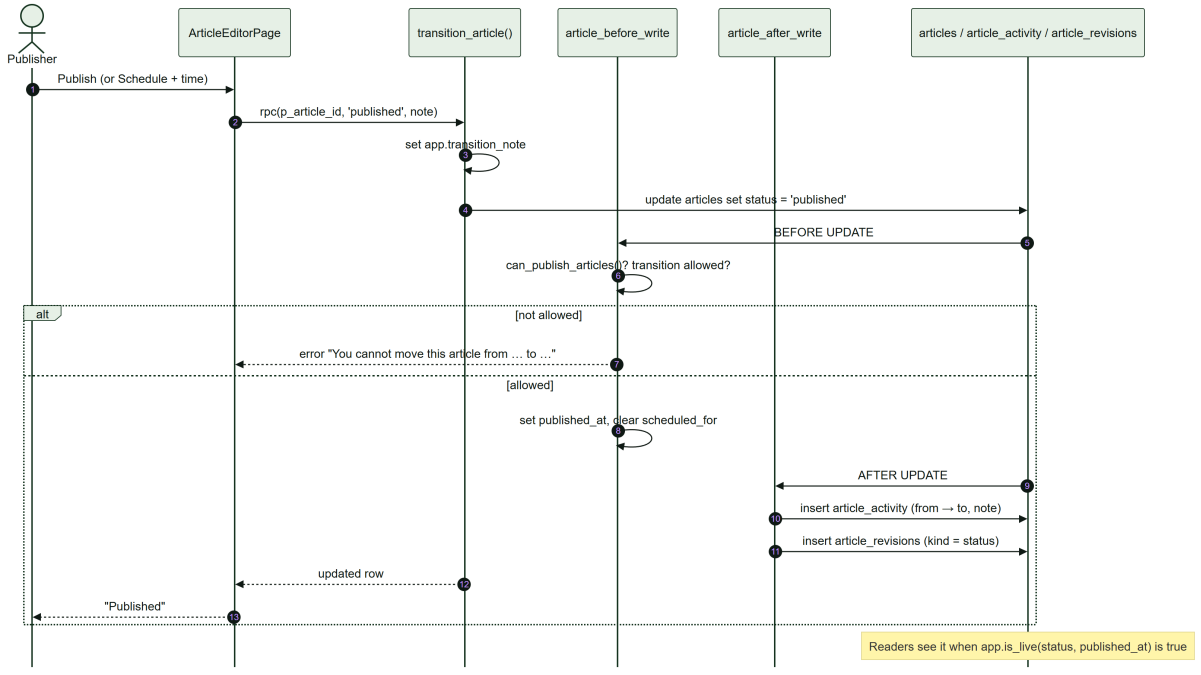
5. Sequences

SEQ-1 Login and role check



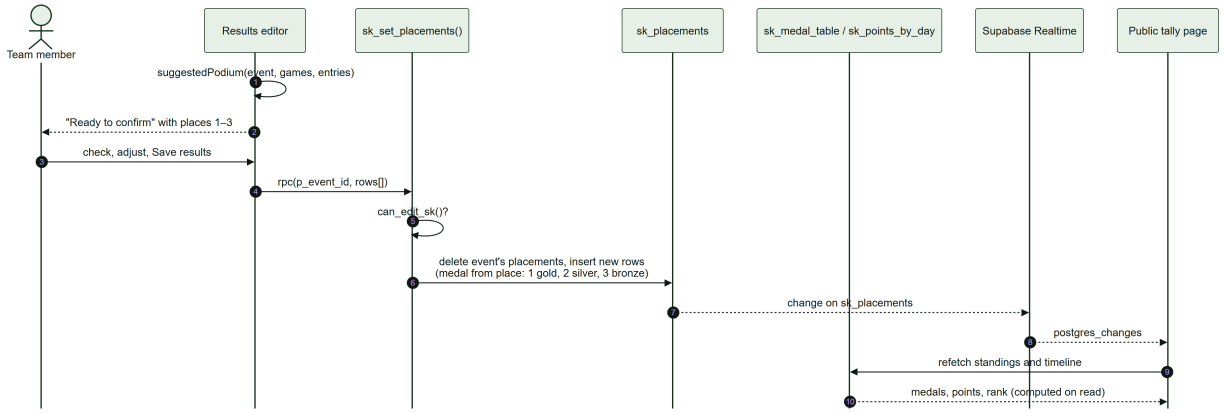
SEQ-1 — sign-in, profile load and the RequireStaff guard. The database re-checks the role on every query.

SEQ-2 Publish an article



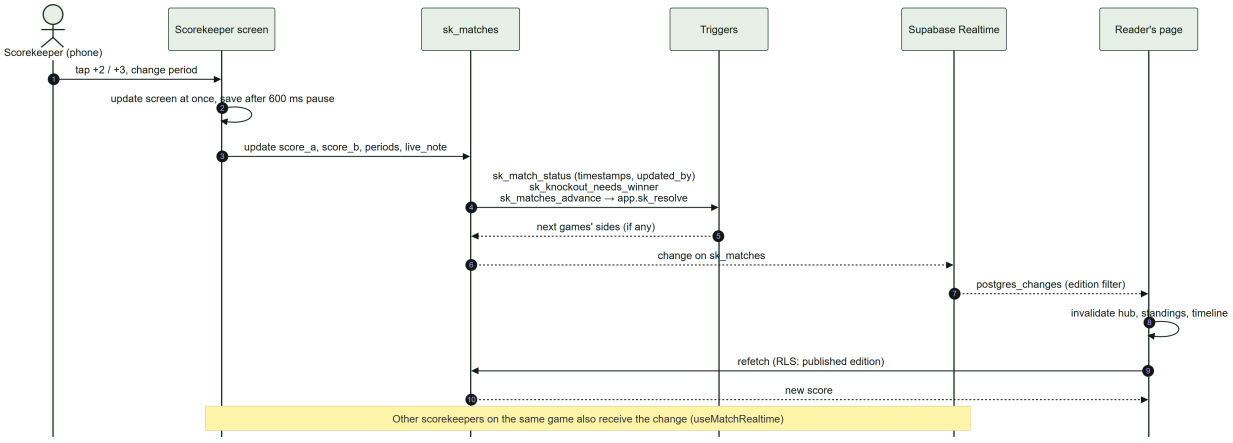
SEQ-2 — publishing through transition_article and the article triggers.

SEQ-3 Enter results and medals



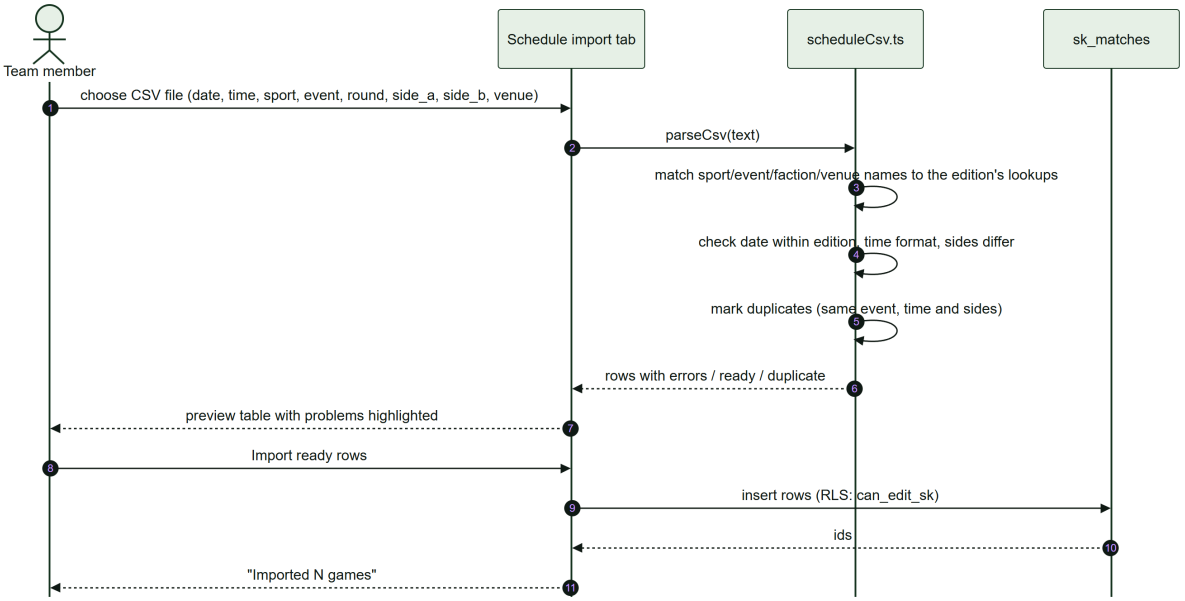
SEQ-3 — confirming medals; the medal table and points are computed when read.

SEQ-4 Live score update



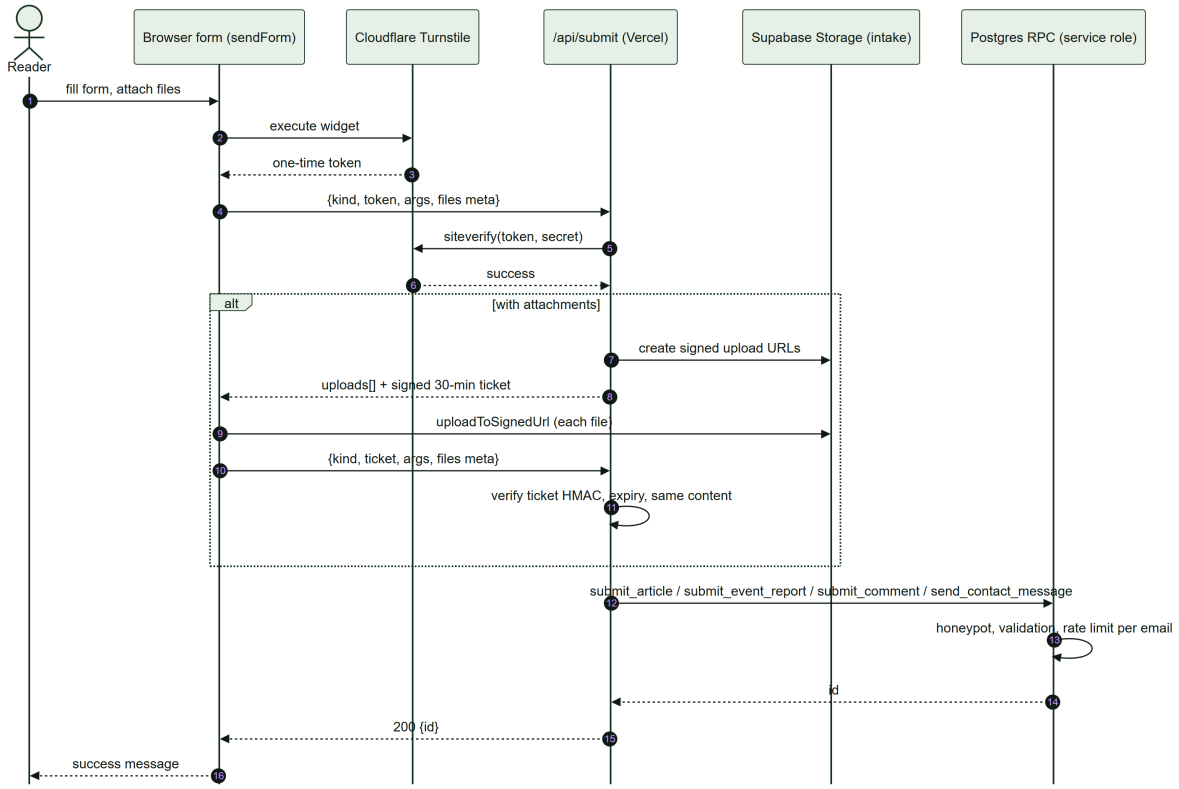
SEQ-4 — a score tap reaches every open page through Supabase Realtime.

SEQ-5 Schedule import



SEQ-5 — CSV import with validation and duplicate detection.

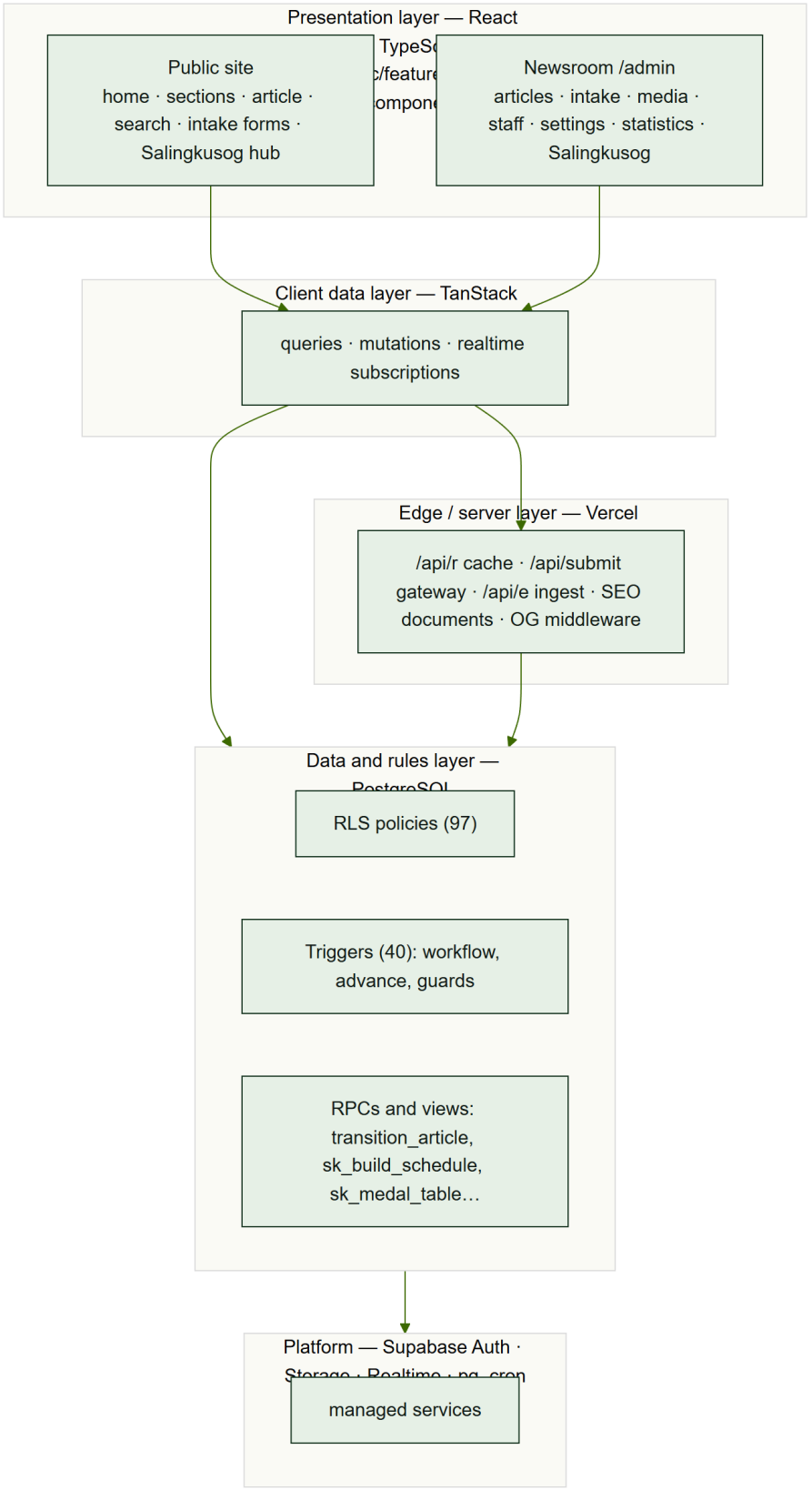
SEQ-6 Public form with CAPTCHA and attachments



SEQ-6 — the /api/submit gateway: Turnstile, signed uploads, signed ticket, service-role RPC.

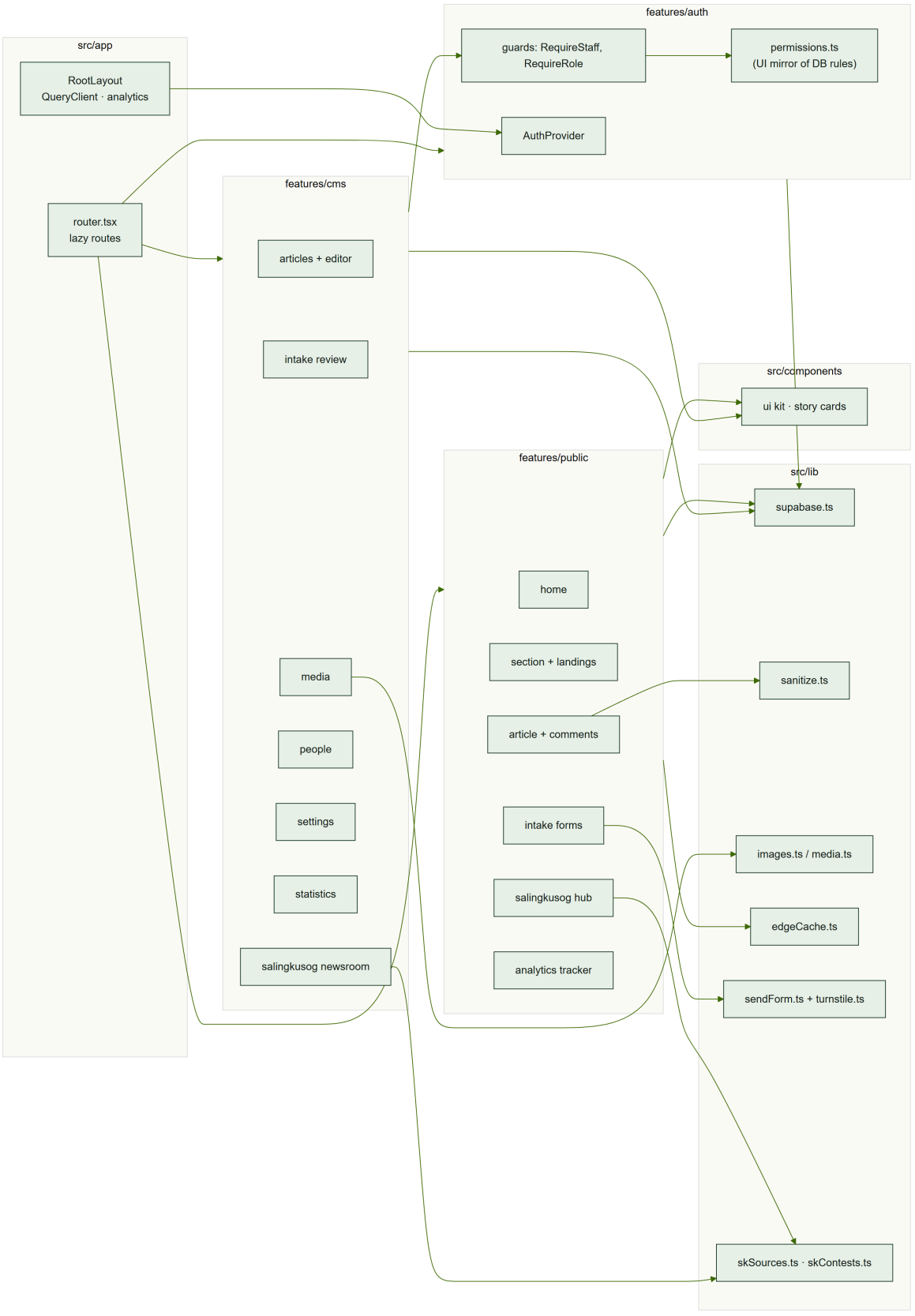
6. Structure

ARCH-1 Layered architecture



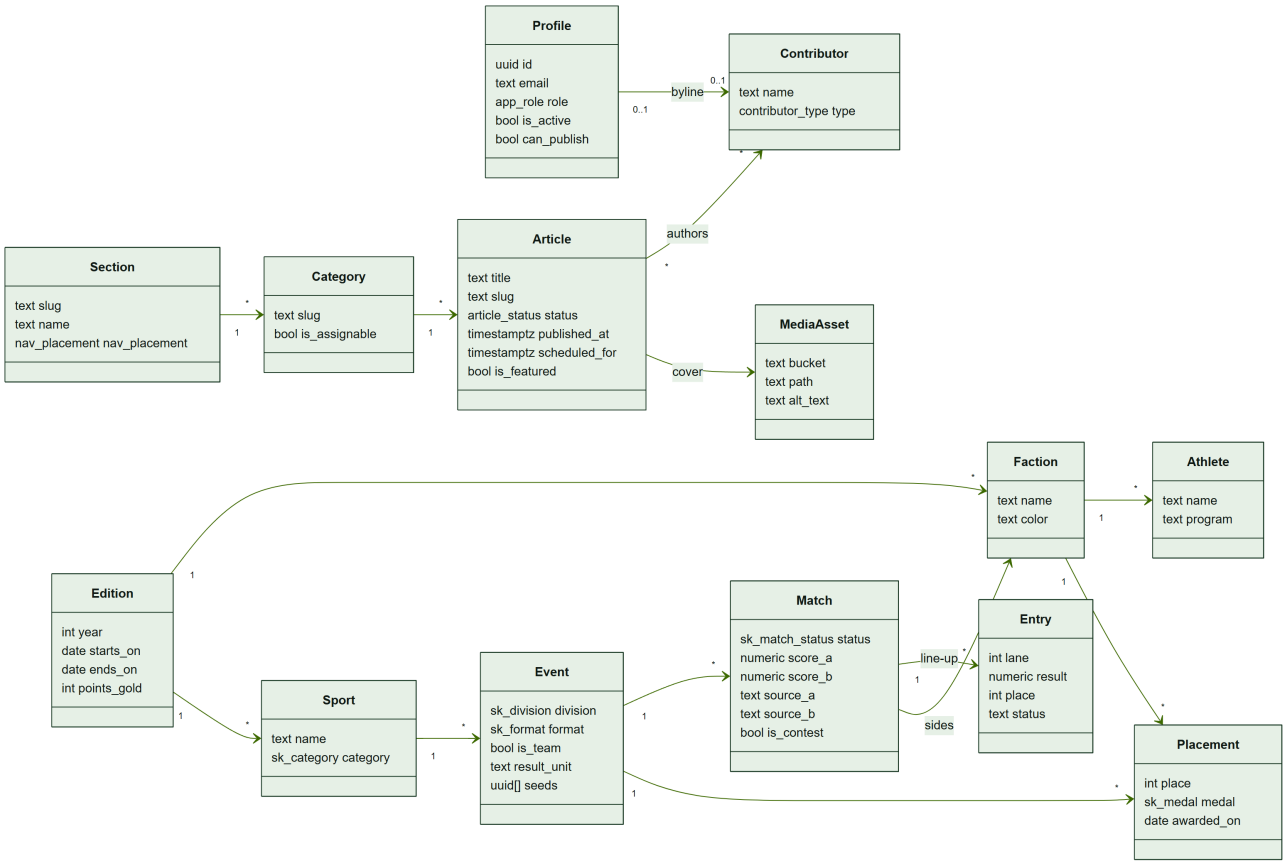
ARCH-1 — five layers from React pages to Supabase platform services.

CMP-1 Front-end components



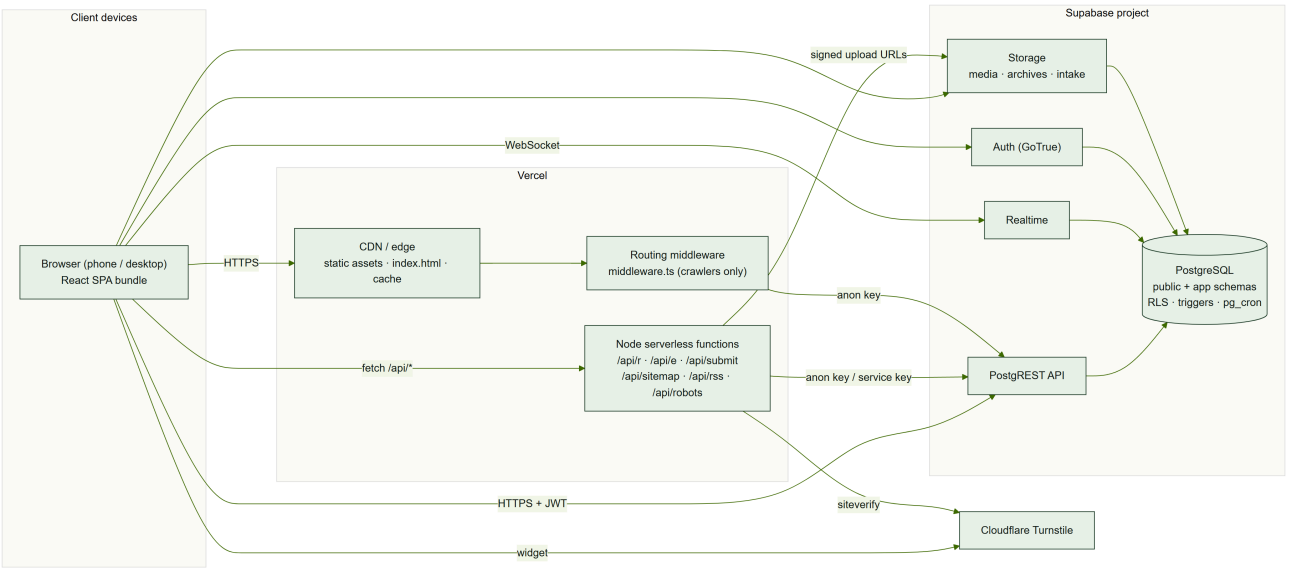
CMP-1 — main front-end modules and shared libraries.

CLS-1 Domain class model



CLS-1 — the main domain entities and their relationships (simplified; full columns in AMR-DOC-04).

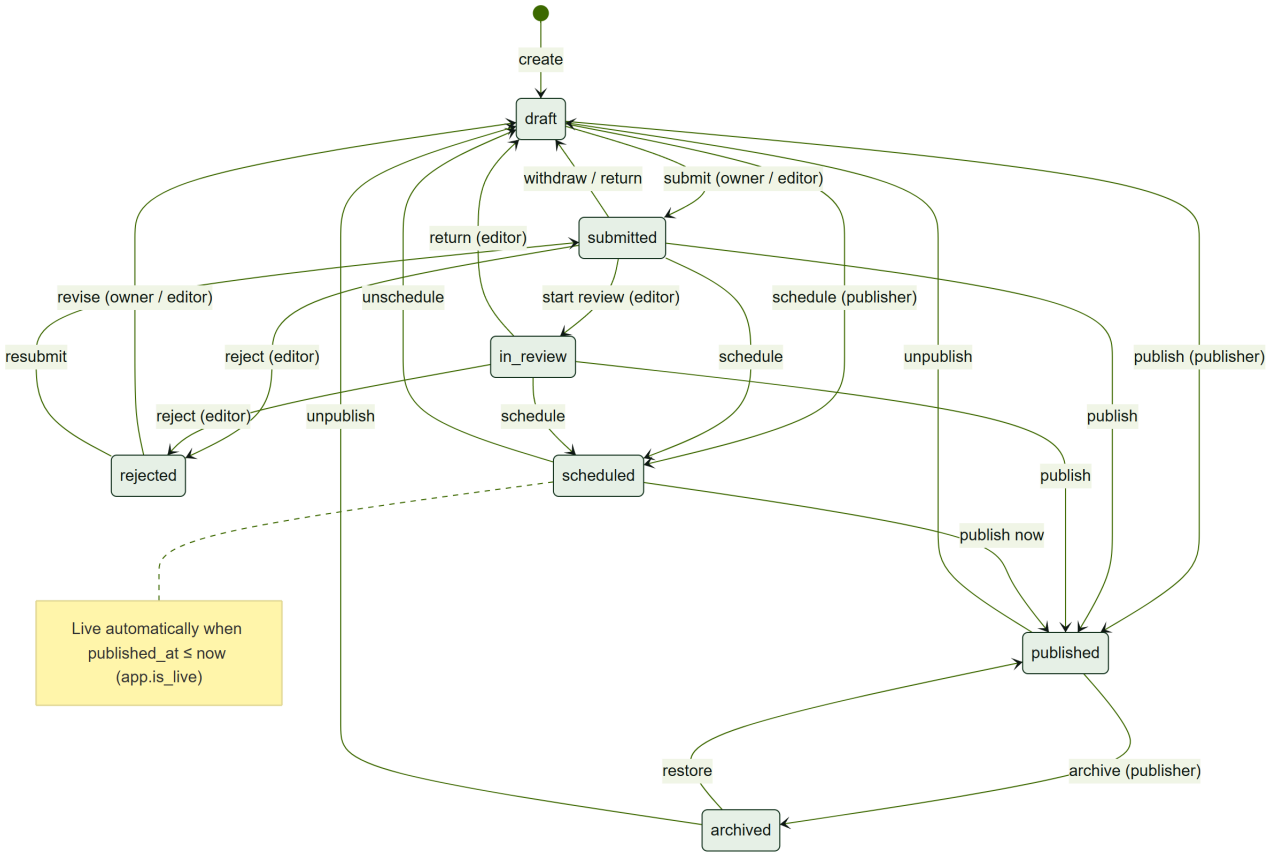
DEP-1 Deployment



DEP-1 — browsers, Vercel and Supabase services.

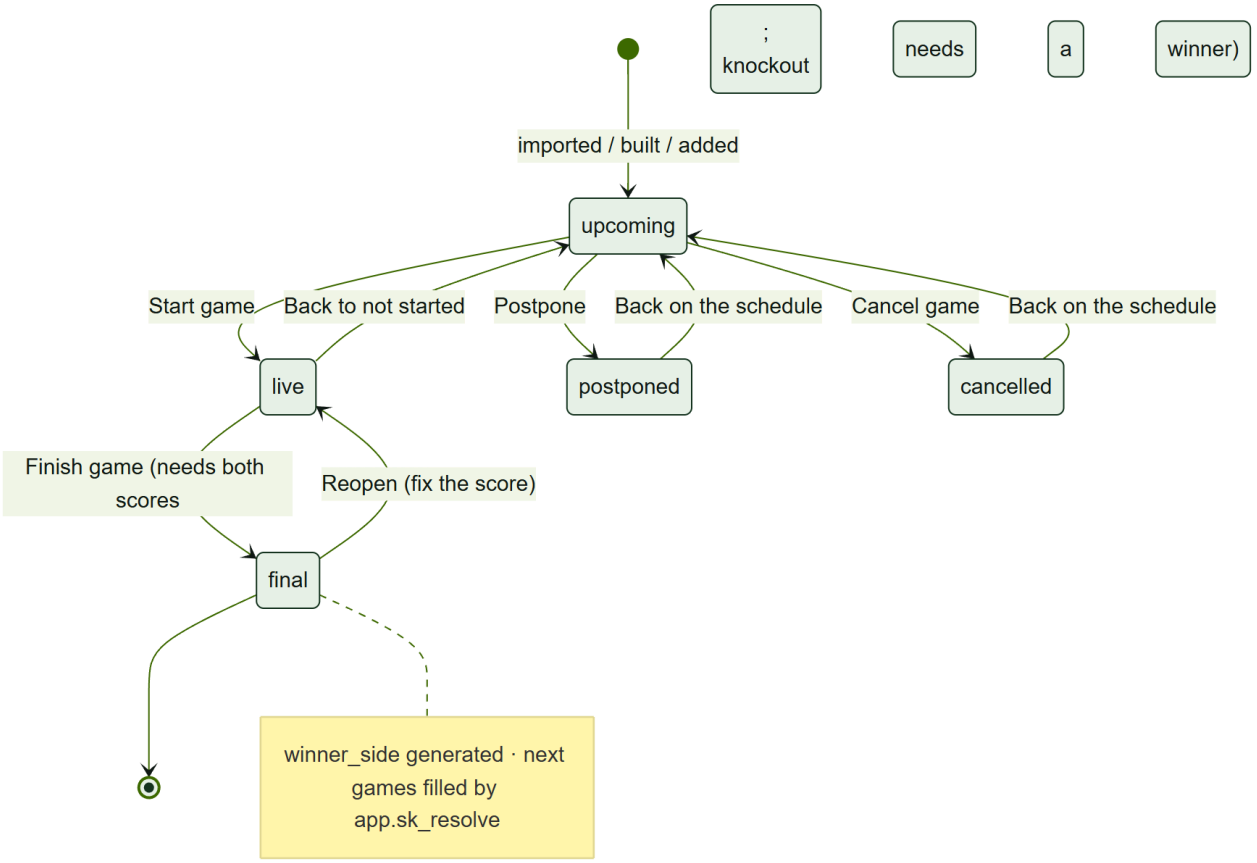
7. States

STATE-1 Article lifecycle



STATE-1 — article statuses and who may move between them.

STATE-2 Game status

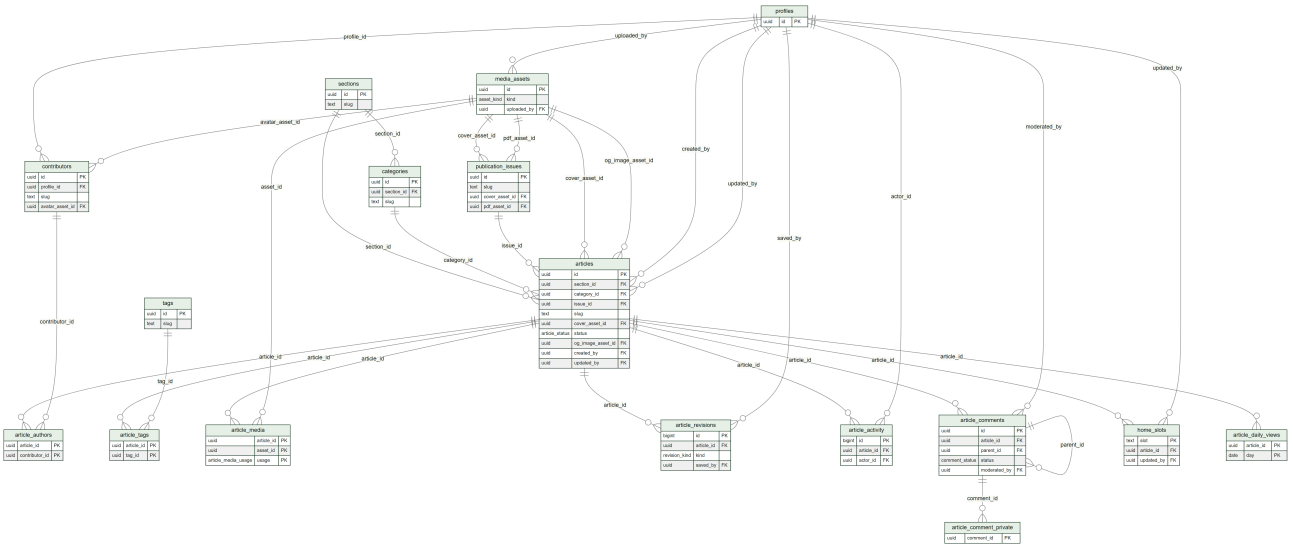


STATE-2 — statuses of a Salingkusog game (sk_match_status) and the scorekeeper actions.

8. Entity–relationship diagrams

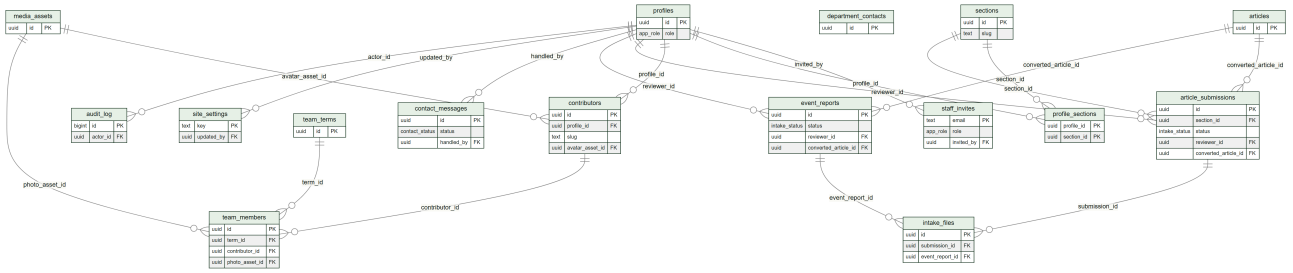
These are generated from the SQL files and show primary keys, foreign keys and a few key columns. Grey boxes without columns are tables from another group, shown for context. The complete data dictionary is in AMR-DOC-04.

ERD-1 Content



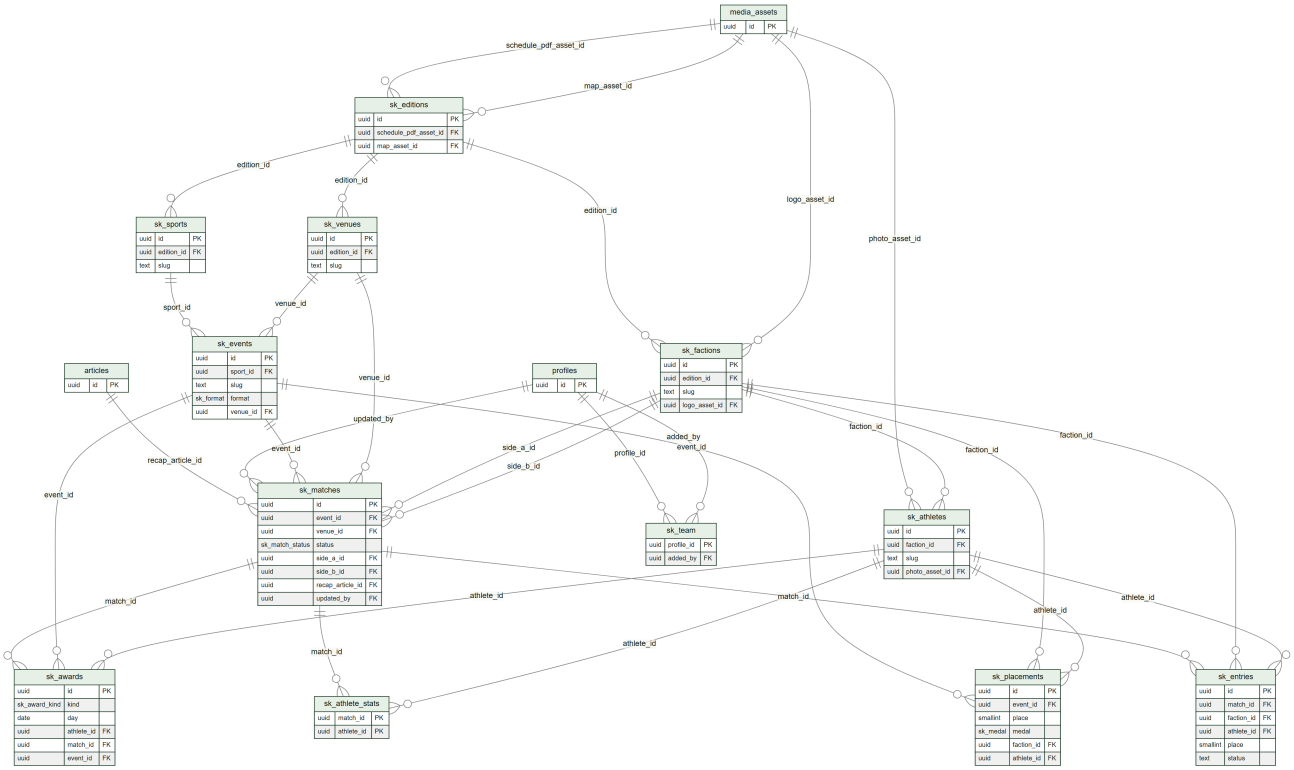
ERD-1 — sections, categories, articles and their related tables.

ERD-2 People, intake and settings



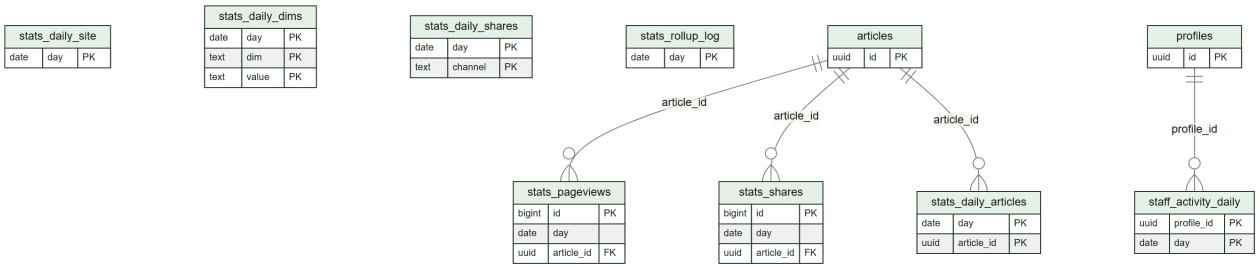
ERD-2 — staff, invitations, contributors, editorial board, public intake and settings.

ERD-3 Salingkusog



ERD-3 — editions, factions, sports, events, matches, entries, placements and awards.

ERD-4 Statistics



ERD-4 — raw events and daily rollups.

Security and Privacy Document

Amaranth v2 — authentication, authorisation, threat model and Data Privacy Act compliance · 1.0 · 2026-10-19

Document control

Field	Value
Title	Security and Privacy Document
Document ID	AMR-DOC-06
Version	1.0
Date	2026-10-19
Author	Amaranth development team [TO CONFIRM: names]
Status	Final draft for defense
Classification	Internal; contains no secrets or personal data

Contents

1. Security objectives
2. Authentication
3. Authorisation and RBAC
4. Row Level Security strategy
5. Threat model (STRIDE)
6. Input validation and output encoding
7. File upload safety
8. Secrets handling
9. Transport and browser security
10. Audit logging and accountability
11. Privacy and the Data Privacy Act of 2012 (RA 10173)
12. Security review findings and open items

1. Security objectives

ID	Objective
SO-1	Only invited, active staff can use the newsroom.
SO-2	Each role can do only what its job requires, even if the user bypasses the UI.
SO-3	Unpublished content, private contact details and statistics never reach readers.
SO-4	Public forms cannot be abused to spam, flood storage or inject scripts.
SO-5	Secrets never reach the browser.
SO-6	Changes to access and to official results are traceable.
SO-7	Personal data is collected minimally, protected and handled per RA 10173.

2. Authentication

Aspect	Implementation	Evidence
Method	Email and password through Supabase Auth (GoTrue); sessions as JWTs refreshed by supabase-js	`LoginPage.tsx`, `AuthProvider.tsx`
Account creation	By invitation only; `app.handle_new_user` gives a role only to emails in `staff_invites`; other sign-ups get an inactive profile with no access	`schema.sql`
Password reset	`resetPasswordForEmail` → emailed link → `/admin/set-password` (`updateUser`)	`ForgotPasswordPage.tsx`, `SetPasswordPage.tsx`
Inactive accounts	`app.current_role()` returns null when `is_active` is false, so every RLS check fails; the UI shows “access pending”	`guards.tsx`
Sign-ups switch	Supabase “allow new users to sign up” should be off (staff are invited)	`docs/v2/DB.md` §2.6 [TO CONFIRM current setting]
MFA	Not configured in the code; Supabase supports TOTP MFA	Recommendation R-03
Leaked-password protection	Supabase dashboard option	Recommendation R-04 [TO CONFIRM]

3. Authorisation and RBAC

Roles: `super_admin` (SA), `editor_in_chief` / Executive Editor (EE), `managing_editor` (ME), `business_manager` (BM), `section_editor` (SE), `staff_writer` (SW), `contributor` (CO); plus the **Salingkusog team** group (`sk_team`) and anonymous readers.

Permission helpers (private `app` schema, `security definer`, `search_path = ''`):

Helper	Grants
`is_staff()`	any active role
`is_admin()`	SA, EE
`can_publish()`	SA, EE, ME (front page, settings ticker, issues, editorial board, archives, granting publishing access)
`can_publish_articles()`	`can_publish()` + BM + profiles with `can_publish = true`
`is_editor()`	SA, EE, ME, SE
`can_edit_section(s)`	`can_publish()` or SE assigned to `s`
`can_view_stats()`	SA, EE, ME
`can_manage_sk_team()`	SA, EE, ME
`can_edit_sk()`	`can_manage_sk_team()` or active staff in `sk_team`

RBAC matrix (summary). ✓ allowed · ○ own/assigned only · — denied. The full matrix is in AMR-DOC-00 §8.

Capability	SA	EE	ME	BM	SE	SW	CO	SKT	Reader
Write drafts	✓	✓	✓	✓	✓	✓	✓	(own role)	—
Review / reject	✓	✓	✓	—	○	—	—	—	—
Publish / schedule	✓	✓	✓	✓	granted	granted	granted	—	—
Grant publishing access	✓	✓	✓	—	—	—	—	—	—
Roles, invites, activation	✓	✓ (no SA)	—	—	—	—	—	—	—
Settings (sections, desks, general)	✓	✓	—	—	—	—	—	—	—
Front page, ticker, board, issues	✓	✓	✓	—	—	—	—	—	—

Capability	SA	EE	ME	BM	SE	SW	CO	SKT	Reader
Moderate comments, intake, inbox	✓	✓	✓	—	✓	—	—	—	—
Statistics	✓	✓	✓	—	—	—	—	—	—
Audit log	✓	✓	—	—	—	—	—	—	—
Salingkusog data entry	✓	✓	✓	—	—	—	—	✓	—
Salingkusog team and countdown	✓	✓	✓	—	—	—	—	—	—
Read published content	✓	✓	✓	✓	✓	✓	✓	✓	✓
Comment, submit, contact	via `/api/submit` for everyone								✓

Separation of duties built in:

- Only SA grants or removes the super_admin role; at least one active SA must remain (`profiles_guard`).
- Writers cannot publish unless explicitly granted; the grant is audited.
- The Business Manager can publish but cannot grant access or change settings.
- Medals must be confirmed by a person (`sk_set_placements`), not written automatically.

4. Row Level Security strategy

1. RLS is enabled on **all 50 tables**; 97 policies (including storage) define access (AMR-DOC-04 Appendix F).
2. Policies call the helpers above, so a role change takes effect everywhere at once.
3. Triggers enforce rules that depend on the change: workflow transitions (`app.article_before_write`), role and access changes (`app.guard_profile_update`), Salingkusog consistency (`app.sk_inherit_edition`, `app.sk_knockout_needs_winner`, `app.sk_resolve`).
4. Views use `security_invoker`, so they never widen access.
5. RPCs that must respect RLS are `security invoker`; `security definer` functions validate their callers (`transition_article` invoker; `set_publish_access` checks `app.can_publish()`; `sk_set_placements` checks `app.can_edit_sk()`).
6. Grants are explicit: execution is revoked from `public` and `anon` and granted per function (`schema.sql` “Function privileges”); the four form RPCs are executable by `service_role` only.

7. 278 SQL checks verify the rules, including that anonymous users cannot read submissions, write any table, upload to `intake`, or call staff RPCs (`public_access.test.sql`, live probe on 2026-10-19 reported in `docs/v2/PROGRESS.md`).

5. Threat model (STRIDE)

Assets: unpublished articles, staff accounts, private emails, official Salingkusog results, the site's reputation, storage quota.

#	STRIDE	Threat	Mitigation in the code	Residual risk
T1	Spoofing	Stolen or guessed staff password	Supabase Auth; invite-only roles; inactive accounts powerless	No MFA yet (R-03)
T2	Spoofing	Self-registration to gain access	`handle_new_user` gives no role without an invite	Low
T3	Spoofing	Bots posing as readers on forms	Turnstile verified server-side; honeypot	Low
T4	Tampering	Writer publishes or edits a locked article by calling the API directly	Trigger enforces transitions and locks; RLS on update	Low
T5	Tampering	Member escalates own role	`profiles_guard`; only SA/EE change roles; only SA grants SA	Low
T6	Tampering	Altering Salingkusog results or started games	RLS `can_edit_sk`; started games never auto-change; medals need confirmation; `updated_by`	No full score history (R-07)
T7	Tampering	Forged form ticket to skip the CAPTCHA	HMAC-SHA-256 ticket with expiry bound to content; service key as HMAC key	Low
T8	Repudiation	Denying a role change or publishing-access grant	`audit_log` entries with actor and before/after	Medium for content edits (activity log covers status changes, not every edit)
T9	Information disclosure	Readers reading drafts or private emails	RLS `app.is_live`; emails in `article_comment_private` and intake tables (editors only)	Low
T10	Information disclosure	Reader tracking or IP collection	Hashed ids with 2-day salts; IP never forwarded; DNT/GPC honoured	Low
T11	Information disclosure	Secrets in the bundle	Secrets used only in `api/`	Low

#	STRIDE	Threat	Mitigation in the code	Residual risk
T12	Information disclosure	Stored XSS through article or comment HTML	DOMPurify with embed allow-list; comments rendered as text	Low
T13	Denial of service	Form flooding	Turnstile, per-email rate limits, body size limits	Platform DoS handled by Vercel/Supabase
T14	Denial of service	Storage exhaustion via uploads	Uploads only through signed links after CAPTCHA; 5 files × 20 MB; bucket limits	Orphan files if a user abandons after upload (R-08)
T15	Denial of service	Traffic spikes during events	Edge cache for public reads; CDN for assets	Live data hits Supabase directly
T16	Elevation of privilege	Calling private helpers through the API	Helpers in non-exposed `app` schema; `search_path = ''`	Low
T17	Elevation of privilege	Clickjacking the newsroom	`X-Frame-Options: SAMEORIGIN`, CSP `frame-ancestors 'self'` (report-only)	Enforce CSP (R-01)

6. Input validation and output encoding

Layer	Validation
Browser	Field validation for convenience (`validate.ts`), file type and size checks (`uploads.ts`)
`/api/submit`	Form kind allow-list; field allow-list per form; max value length 20,000; max body 200 KB; file count, type and size; Turnstile; ticket verification
Database	`app.check_email`, `app.check_text` length limits; check constraints (slugs, scores, places, statuses); enums; foreign keys; triggers
Output	Article and recap HTML sanitised with DOMPurify (iframes only from YouTube, Vimeo, Facebook, Spotify); external links get `rel="noopener noreferrer"`; comments and form text rendered as plain text by React; crawler meta tags escaped (`escapeHtml` in `api/_lib/seo.ts`)
URLs	Footer social links accept only `http(s)` (`settings.ts`); middleware slugs restricted to `[a-z0-9-]` before querying
Query injection	All database access through parameterised PostgREST/RPC calls; no string-built SQL from user input in the app

7. File upload safety

Bucket	Controls
`media` (public)	Staff only; images only (JPEG, PNG, WebP, AVIF, GIF); 10 MB; compressed to WebP in the browser; deletes limited to the owner or editors; in-use assets cannot be deleted
`archives` (public)	Publishers only; PDF and images; 60 MB
`intake` (private)	No direct uploads; one-time signed upload links from `/api/submit` after Turnstile; max 5 files × 20 MB; JPEG, PNG, WebP, PDF, text, Word; file names sanitised to `[\w-]` with a random folder; only editors can read or delete

The bucket MIME and size limits are enforced by Supabase Storage itself, in addition to the checks in code.

8. Secrets handling

Secret	Where it lives	Used by
`SUPABASE_SERVICE_ROLE_KEY`	Vercel environment (Production, Preview); `.env.local` for local dev	`api/e.ts`, `api/submit.ts`
`TURNSTILE_SECRET_KEY`	Vercel environment	`api/submit.ts`
Supabase anon key, Turnstile site key	Public by design (`VITE_` variables)	Browser

Rules: no secrets in the repository (`.env*` ignored except `.env.example` placeholders); `VITE_`-prefixed variables are the only ones bundled into the browser; the service key also serves as the HMAC key for form tickets; local development uses Cloudflare's published test keys when no Turnstile key is set.

9. Transport and browser security

`vercel.json` sets on every response: `Strict-Transport-Security: max-age=63072000; includeSubDomains`, `X-Content-Type-Options: nosniff`, `Referrer-Policy: strict-origin-when-cross-origin`, `X-Frame-Options: SAMEORIGIN`, `Permissions-Policy` (camera, microphone, geolocation, interest-cohort off), and `Content-Security-Policy-Report-Only` limiting scripts to the site and Cloudflare challenges, connections to Supabase and Cloudflare, and frames to Cloudflare and approved video hosts. All traffic is HTTPS (Vercel and Supabase).

10. Audit logging and accountability

Event	Where recorded
Role or activation change	`audit_log` action `profile.access` (before/after)
Publishing access change	`audit_log` action `profile.publish_access`
Salingkusog team add/remove	`audit_log` actions `salingkusog.team.add` / `.remove`
Article created, status changed, notes	`article_activity` (actor, from, to, note)
Article snapshots	`article_revisions` (newest 50)
Last change to a game	`sk_matches.updated_by`
Comment moderation	`article_comments.moderated_by`
Intake review	`reviewer_id`, `reviewed_at` on submissions and event reports

Only SA and EE can read `audit_log`.

11. Privacy and the Data Privacy Act of 2012 (RA 10173)

11.1 Personal data inventory

Data subject	Data collected	Purpose	Who can see it	Retention (as implemented)
Staff	Email, display name, role, optional bio and avatar; newsroom active time	Accounts, bylines, workload	Staff (names); SA/EE/ME (activity)	Account lifetime; activity 24 months
Commenters	Name (public once approved), email (private)	Moderation and replies	Editors	Not automatically deleted [gap R-09]
Story submitters	Name, email, affiliation, submission, attachments	Editorial review and credit	Editors	Not automatically deleted [gap R-09]
Event reporters	Organisation, contact name, email, event details	Coverage planning	Editors	Not automatically deleted [gap R-09]
Contact senders	Name, email, message	Replies	Editors	Not automatically deleted [gap R-09]
Readers	Hashed daily visitor id, page, source, device family, reading time, scroll	Readership statistics	SA/EE/ME (aggregates)	Raw 13 months, aggregates indefinitely; salts 2 days
Salingkusog athletes	Name, program (course/year), optional photo, results, stat lines, award quotes	Publishing intramural results	**Public** once the edition is published	Kept with the edition [TO CONFIRM policy]
Editorial board	Name, position, photo	About the publication	Public	Per term

Not collected by the system: birth dates, student numbers, addresses, phone numbers of athletes, medical or eligibility documents. `sk_athletes` has only name, slug, program and photo. If the intramurals organisers collect **eligibility forms**, they are outside this system [TO CONFIRM who holds them and where].

11.2 Data Privacy Act principles

RA 10173 principle	How the system supports it	Gap / action
Transparency	`/privacy` explains statistics, forms, staff activity and rights	Add a section on Salingkusog athletes and published results (R-10)
Legitimate purpose	Data used only for publishing, moderation, statistics	—
Proportionality (minimisation)	Athletes: name, program, photo only; stats: hashed, no IP, coarse device info	—

RA 10173 principle	How the system supports it	Gap / action
Security of personal data (Sec. 20)	RLS, private tables for emails, private bucket for intake, HTTPS, secrets server-side	MFA for admins (R-03)
Data subject rights (Sec. 16)	`/privacy` explains access, correction, deletion; requests via the contact page	Define a written procedure and a Data Protection Officer contact (R-11) [TO CONFIRM VSU DPO]
Retention	Stats retention enforced by `app.stats_nightly`	Retention for intake, comments and athlete data (R-09)
Consent for athletes' photos	—	Obtain consent through the intramurals organisers before publishing photos (R-12) [TO CONFIRM]
Breach notification	Supabase/Vercel logs	Incident procedure in AMR-DOC-09 §9; NPC notification within 72 hours where required [TO CONFIRM with VSU DPO]

12. Security review findings and open items

A security review was carried out on 2026-10-19 (`docs/v2/PROGRESS.md`, `fix/security-hardening`, `feature/turnstile-captcha`). Fixed: page-number range errors, open intake uploads, anonymous access to `set_publish_access`, unsafe social-link schemes, form spam (Turnstile). Open recommendations:

ID	Recommendation	Priority
R-01	Switch the Content-Security-Policy from report-only to enforcing after checking the browser console on a preview	High
R-02	Confirm "allow new users to sign up" is disabled in Supabase Auth	High
R-03	Require MFA for Super admin and Executive Editor accounts	High
R-04	Enable leaked-password protection in Supabase Auth	Medium
R-05	Add CI so security tests (SQL checks) run on every push	Medium
R-06	Replace hand-written database types with generated ones	Low
R-07	Keep a history of score and result changes (who changed what)	Medium
R-08	Clean up intake files never attached to a submission	Low
R-09	Define and automate retention for comments, submissions, event reports and contact messages	Medium
R-10	Extend `/privacy` to cover Salingkusog athletes and published results	High
R-11	Name a privacy contact or DPO and document how requests are handled	Medium

ID	Recommendation	Priority
R-12	Obtain consent for publishing athletes’ photos	High

API and Integration Reference

Amaranth v2 — Vercel endpoints, database RPCs and direct table access · 1.0 · 2026-10-19

Document control

Field	Value
Title	API and Integration Reference
Document ID	AMR-DOC-07
Version	1.0
Date	2026-10-19
Author	Amaranth development team [TO CONFIRM: names]
Status	Final draft for defense
Sources	`api/*.ts`, `middleware.ts`, `supabase/v2/*.sql` grants, `src/lib/database.types.ts`

Contents

1. Overview and conventions
2. Vercel endpoints
3. Routing middleware
4. Database RPCs (public)
5. Direct table access through PostgREST
6. Realtime channels
7. Error codes
8. External integrations

1. Overview and conventions

There are **no server actions** (this is a Vite single-page app, not Next.js). Clients use three kinds of interface:

Interface	Base	Auth
Vercel functions	`https://<site>/api/*`	None (public) or Turnstile token; secrets stay on the server
Supabase RPCs	`POST https://<project>.supabase.co/rest/v1/rpc/<name>` (called as `supabase.rpc(name, args)`)	Anon key for readers; user JWT for staff; service key from `/api` only
Supabase tables	`GET/POST/PATCH/DELETE /rest/v1/<table>` (called as `supabase.from(table)`)	Same as RPCs; RLS decides rows

“Who can call” below means the database grant **and** the checks inside the function. RLS still applies inside `security invoker` functions.

2. Vercel endpoints

2.1 `POST /api/submit` — public forms gateway

File: `api/submit.ts`, `api/_lib/forms.ts`. Caller: any browser with a valid Turnstile token. Uses `SUPABASE_SERVICE_ROLE_KEY` and `TURNSTILE_SECRET_KEY`.

Request (step 1, or the only step without files):

```
{
  "kind": "article",
  "token": "<turnstile token>",
  "args": {
    "p_name": "Sample Reader", "p_email": "reader@example.com", "p_affiliation": "BS Agriculture",
    "p_section_slug": "news", "p_title": "Sample title", "p_summary": null,
    "p_body": "Text …", "p_website": null
  },
  "files": [{ "name": "story.pdf", "mime": "application/pdf", "size": 120000 }]
}
```

`kind`	RPC	Allowed `args`
`comment`	`submit_comment`	`p_article_id, p_name, p_email, p_body, p_parent_id, p_website`
`article`	`submit_article`	`p_name, p_email, p_affiliation, p_section_slug, p_title, p_summary, p_body, p_website` (+ files)

kind	RPC	Allowed args
event	submit_event_report	p_organization, p_contact_name, p_contact_email, p_event_title, p_event_date, p_venue, p_summary, p_website (+ files)
contact	send_contact_message	p_name, p_email, p_subject, p_category, p_message, p_website

Responses:

Status	Body	When
200	{ "id": "<uuid>" }	Saved (no files, or step 2)
200	{ "uploads": [{ "path": "submissions/<uuid>/1-story.pdf", "token": "…" }], "ticket": "<signed>" }	Step 1 with files: upload each file with storage.from('intake').uploadToSignedUrl(path, token, file), then repeat the request with "ticket" instead of "token"
400	{ "message": "Unknown form." } / "Attach at most 5 files." / database validation message	Bad input
403	{ "message": "Please complete the security check and try again." }	Turnstile failed
403	{ "message": "This form expired. Please send it again." }	Ticket forged, expired (30 min) or for different content
413	{ "message": "That's too much text to send." }	Body over 200 KB
501	{ "message": "Forms aren't set up yet…" }	Missing server environment variables
502	{ "message": "…" }	Upstream (Supabase) error

Browser helper: sendForm(kind, args, files) in src/lib/sendForm.ts.

2.2 GET /api/r?p=<path>[&a=object] — edge-cached public reads

File: api/r.ts. Caller: browsers of signed-out readers (through src/lib/edgeCache.ts). Forwards GET /rest/v1/<path> with the anon key and caches successes for 60 s (s-maxage=60, stale-while-revalidate=300). Only resources in CACHEABLE are allowed: article_cards, articles, article_authors, article_tags, tags, sections, categories, site_settings, home_slots, publication_issues, team_terms, team_members, department_contacts, media_assets, sk_editions, sk_factions, sk_sports, sk_events, sk_venues, sk_athletes, rpc/most_read_articles, rpc/archive_months. a=object returns a single object (.single()). Errors: 400 “Not a cacheable read”; 501 if not configured.

Example: /api/r?p=sections%3Fselect%3D*%26order%3Dnav_order.asc

2.3 POST /api/e — statistics ingest

File: api/e.ts, api/_lib/traffic.ts. Caller: the site’s own pages (origin must be the site), sent with navigator.sendBeacon. Always answers 204 (or 501 when statistics are not configured, 405 for GET).

```
{ "v": "<visitor id>", "s": "<session id>",
  "e": [ { "t": "pv", "id": "<uuid>", "path": "/article/sample", "ref": "https://www.facebook.com/",
"us": "fb" },
         { "t": "eng", "id": "<uuid>", "secs": 74, "scroll": 82 },
         { "t": "share", "path": "/article/sample", "ch": "facebook" } ] }
```

Limits: body ≤ 16 KB, ≤ 25 events; bots dropped by user agent; device, browser and OS reduced to families; the IP address is never forwarded. Calls `stats_ingest` with the service key.

2.4 `GET /api/sitemap` (`/sitemap.xml`), `GET /api/rss` (`/rss.xml`), `GET /api/robots` (`/robots.txt`)

Files: `api/sitemap.ts`, `api/rss.ts`, `api/robots.ts`. Public, edge-cached. The sitemap lists sections and live articles; RSS lists the latest live articles; robots allows crawling and points to the sitemap.

3. Routing middleware

File: `middleware.ts` (matcher: `/article/:slug*` and top-level pages except `api`, `assets`, `admin`, `dev` and files). For crawler user agents only, it fetches the article (or section) with the anon key and returns `index.html` with `<title>`, description, canonical URL and Open Graph/Twitter tags (escaped by `escapeHtml`). Regular browsers pass through (`x-middleware-next`).

4. Database RPCs (public)

4.1 Reader RPCs (anon and authenticated)

RPC	Parameters	Returns	Notes
`record_article_view`	`p_article_id uuid`	void	Counts one view; dedupe handled with statistics (one per visitor per 30 min)
`most_read_articles`	`p_days int = 7`, `p_limit int = 5`	rows: id, slug, title, section_slug, section_name, category_name, published_at, views	“Most read” lists
`search_articles`	`p_query text`, `p_section text = null`, `p_from date = null`, `p_to date = null`, `p_limit int = 20`, `p_offset int = 0`, `p_live_only bool = true`	rows: id, slug, title, excerpt, section, category, published_at, status, cover, rank, total_count	Full-text search; invoker (RLS applies)
`archive_months`	—	rows: month, articles	Archive navigation
`sk_countdown`	—	title, subtitle, starts_on, countdown_at	Only while the countdown is switched on; works before the edition is published
`sk_points_timeline`	`p_edition_id uuid`, `p_hours int` (1, 3, 6 or 12)	rows: faction_id, bucket_end, points	Points race in hour steps; error 22023 for other hour values

Example (supabase-js):

```
const { data } = await supabase.rpc('search_articles', { p_query: 'intramurals', p_section:
'sports', p_limit: 12 });
```

4.2 Staff RPCs (authenticated; further checks inside)

RPC	Parameters	Returns	Who	Errors
`transition_article`	`p_article_id uuid`, `p_to article_status`, `p_note text = null`, `p_scheduled_for timestamptz = null`	the updated `articles` row	Per workflow rules (FR-014, FR-016)	42501 “You cannot move this article from X to Y”, 22023 “Pick a future time to schedule”, 42501 “Article not found or not editable”
`add_article_note`	`p_article_id uuid`, `p_note text`	void	Anyone who can view the article	42501

RPC	Parameters	Returns	Who	Errors
`convert_intake_to_article`	`p_kind text` (‘submission’ or ‘event’), `p_intake_id uuid`, `p_category_id uuid`	new article id	Editors	42501
`set_publish_access`	`p_profile_id uuid`, `p_allowed bool`	void	SA, EE, ME	42501 “Only the Super admin, Executive Editor or Managing editor…”, P0002 unknown staff member
`dashboard_stats`	—	jsonb: published, published_this_week, scheduled, drafts, in_review, views_this_month, views_last_month	Staff (values limited by role)	—
`media_usage`	`p_asset_id uuid`	integer (number of references)	Staff	—
`staff_heartbeat`	—	integer (seconds credited)	Staff	—
`stats_overview`	`p_from date`, `p_to date`	jsonb summary and trend	SA, EE, ME	42501
`stats_articles`	`p_from`, `p_to`, `p_limit int = 20`	top articles with views, visitors, read time, scroll, finish rate, shares	SA, EE, ME	42501
`stats_authors`	`p_from`, `p_to`	per contributor	SA, EE, ME	42501
`stats_sections`	`p_from`, `p_to`	per section and category	SA, EE, ME	42501
`stats_breakdown`	`p_from`, `p_to`, `p_dim text` (source, referrer, device, browser, os)	value, pageviews, visitors	SA, EE, ME	42501
`stats_share_channels`	`p_from`, `p_to`	channel, shares	SA, EE, ME	42501
`stats_staff_usage`	`p_from`, `p_to`	per staff: active seconds, days active, last active	SA, EE, ME	42501

4.3 Salingkusog RPCs (authenticated; Salingkusog team)

RPC	Parameters	Returns	Errors
`sk_set_placements`	`p_event_id uuid`, `p_rows jsonb` — `[{ "place": 1, "faction_id": "…", "athlete_id": null, "result": "12.84s", "awarded_on": "2026-10-27" }]`	number of rows saved	42501 “Only the Salingkusog team can enter results.”, 22023 “Results must be a list.”
`sk_build_schedule`	`p_event_id uuid`, `p_format text` (bracket, round_robin, round_robin_final), `p_seeds uuid[]` (2–16 factions in seed order), `p_first_at timestamptz`, `p_round_gap_hours int = 24`, `p_game_gap_minutes int = 90`, `p_venue_id uuid = null`, `p_bronze bool = true`	number of games created	42501 not on the team; 22023 invalid format, seeds, missing time; 55000 “This event already has games played…”
`sk_create_contest`	`p_event_id uuid`, `p_at timestamptz`, `p_venue_id uuid = null`, `p_round text = 'Final'`, `p_factions uuid[] = all`, `p_shuffle bool = false`	session id	42501; 22023 “Contests are for timed, measured or judged events.”, “Pick at least one faction.”

Example:

```
await supabase.rpc('sk_build_schedule', {
  p_event_id: eventId, p_format: 'bracket', p_seeds: [f1, f2, f3, f4],
  p_first_at: '2026-10-26T08:00:00+08:00', p_round_gap_hours: 24, p_game_gap_minutes: 90,
});
```

4.4 Service-role RPCs (called only by `/api`)

RPC	Parameters	Purpose	Limits
`submit_comment`	`p_article_id, p_name, p_email, p_body, p_parent_id, p_website`	New pending comment	5 per email per 10 min; article must be live and allow comments; replies only to approved comments
`submit_article`	`p_name, p_email, p_affiliation, p_section_slug, p_title, p_summary, p_body, p_files jsonb, p_website`	New story submission	3 per email per hour; files must exist in `intake`

RPC	Parameters	Purpose	Limits
`submit_event_report`	`p_organization, p_contact_name, p_contact_email, p_event_title, p_event_date, p_venue, p_summary, p_files, p_website`	New event report	3 per email per hour
`send_contact_message`	`p_name, p_email, p_subject, p_category, p_message, p_website`	New contact message	3 per email per 10 min
`stats_ingest`	`p_visitor, p_session, p_events jsonb, p_device, p_browser, p_os`	Statistics events	25 events per batch; rate-limited

In all form RPCs a non-empty `p_website` (honeypot) returns a fake success without saving.

5. Direct table access through PostgREST

Most screens read and write tables directly with supabase-js; RLS decides the rows. Summary by audience (full policies in AMR-DOC-04 Appendix F):

Audience	Read	Write
Anonymous	`sections`, `categories` (active), `tags`, `media_assets`, `contributors` (public), `publication_issues` (published), `articles` (live), `article_authors`/`tags`/`media` (live), `article_comments` (approved on live), `home_slots`, `team_terms`, `team_members`, `department_contacts`, `site_settings` (public), `article_cards`, Salingkusog tables and views of published editions	none
Staff	the above plus own and section articles, revisions, activity, profiles, staff-only settings	as allowed by role (AMR-DOC-06 §3)
Editors	plus comments (all), submissions, event reports, intake files, contact messages	moderation and review fields
SA, EE, ME	plus statistics tables	settings, board, issues, home slots (per helper)

Typical calls:

```
// live articles of a section, newest first
supabase.from('article_cards').select('*').eq('section_slug', 'news').order('published_at', {
ascending: false }).range(0, 11);
// update a live score (Salingkusog team)
supabase.from('sk_matches').update({ score_a: 54, score_b: 58, live_note: 'Q3' }).eq('id', matchId);
```

6. Realtime channels

Channel (client)	Table filter	Effect
`sk-public-<edition>` (`useLiveGames`)	`sk_matches`, `sk_placements`, `sk_entries` with `edition_id = <edition>`	Invalidates hub, standings, timeline and results queries on public pages
`sk-match-<id>` (`useMatchRealtime`)	`sk_matches` with `id = <id>`	Keeps several scorekeepers on the same game in sync

The publication `supabase_realtime` includes `sk_matches`, `sk_placements` and `sk_entries` (`salingkusog.sql`). Realtime respects RLS.

7. Error codes

SQLSTATE	Meaning in this system	Example message
42501	Not allowed	“You cannot move this article from draft to published.”
22023	Invalid input	“Pick a future time to schedule.”
23503	Unknown parent	“Unknown category.”
23514	Rule check failed	“A knockout game needs a winner…”
55000	State conflict	“Semifinal 2 has already started, so its teams can’t change.”
P0002	Not found	“Unknown event.”
PGRST103	Page past the end (PostgREST 416)	handled as an empty page (`pastLastPage`)

8. External integrations

Service	Use	Configuration
Supabase	Database, Auth, Storage, Realtime, pg_cron	`VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY`, `SUPABASE_SERVICE_ROLE_KEY`
Cloudflare Turnstile	CAPTCHA for public forms	`VITE_TURNSTILE_SITE_KEY`, `TURNSTILE_SECRET_KEY`
Vercel	Hosting, functions, middleware, Web Analytics	`vercel.json`; project settings
YouTube, Vimeo, Facebook, Spotify	Embeds allowed in article bodies	`src/lib/sanitize.ts` allow-list

Test Documentation

Amaranth v2 — test plan, strategy, cases, traceability and results · 1.0 · 2026-10-19

Document control

Field	Value
Title	Test Documentation (plan, cases, RTM, results)
Document ID	AMR-DOC-08
Version	1.0
Date	2026-10-19
Author	Amaranth development team [TO CONFIRM: names]
Status	Final draft for defense
Basis	IEEE 829 / ISO 29119-3 structure; results from runs on 2026-10-19

Contents

1. Test plan
2. Test strategy and levels
3. Test environment and tools
4. Entry and exit criteria
5. Sample test data
6. Results summary
7. Manual test cases (not yet run)
8. Defects found and fixed during development
9. Gaps and recommendations

- Appendix A — Automated test suites
- Appendix B — Requirements traceability matrix
- Appendix C — Unit and component test cases (255)
- Appendix D — Database test checks (278)

1. Test plan

Item	Plan
Objective	Show that Amaranth v2 meets the requirements in AMR-DOC-02, with emphasis on permissions, the editorial workflow and Salingkusog results
Scope	Front-end logic and components, server functions, database rules (RLS, triggers, RPCs, views), accessibility and responsive layout
Out of scope	Load testing; penetration testing by a third party; browser automation end-to-end (not yet set up)
Approach	Automated unit/component tests (Vitest), automated database tests (SQL harness), automated layout and accessibility checks (`/dev` sweep, contrast script), manual checks for the rest
Responsibilities	Developers write tests with each feature; the reviewer runs `npm run check` and `npm run db:test` before each merge (`docs/v2/PROGRESS.md`)
Schedule	Continuous during development (2026-10-17 → 2026-10-19); manual acceptance on the Vercel preview before merging to `main` [TO CONFIRM date]
Risks	No CI (tests depend on discipline); no browser end-to-end tests; live Supabase behaviour (Realtime, Auth emails) tested manually only

2. Test strategy and levels

Level	What is tested	Tool	Location
Unit	Pure helpers: formatting, slugs, permissions mirror, Salingkusog ranking, CSV parsing, contest ranking, sanitising, edge-cache routing, ticket signing	Vitest	`src/lib/*.test.ts`, `api/_lib/*.test.ts`, feature helpers
Component	React views rendered with Testing Library and fixtures: forms, comments, workflow panel, scorekeeper, results, build schedule, public hub	Vitest + Testing Library + jsdom	`src/features/**/*.test.tsx`
Database (integration)	RLS per role, workflow trigger, RPCs, views, Salingkusog advancement, contests, statistics privacy and retention	PostgreSQL 16+ via `scripts/db-test.mjs`	`supabase/tests/*.test.sql`
Static	Lint, strict type-check, production build	ESLint, TypeScript, Vite	`npm run check`

Level	What is tested	Tool	Location
Accessibility and layout	One `h1`, heading order, named controls, alt text, no overflow at 375/768/1024/1440 px; colour contrast	`src/dev/domCheck.ts` sweep; `scripts/check-contrast.mjs`; Lighthouse	`/dev` fixture screens
Manual / acceptance	Flows that need live services (Auth emails, Realtime, Turnstile, Storage) and role walkthroughs	Checklist	§7 and `docs/v2/PR.md`

Database harness. `npm run db:test` creates a temporary PostgreSQL cluster on `127.0.0.1`, loads `00_supabase_stub.sql` (stand-ins for Supabase's `auth`, `storage`, roles), applies `schema.sql`, `salingkusog.sql`, `stats.sql` and `seed.sql` **twice** (proving they are re-runnable), loads fixtures twice, runs each `*.test.sql` inside a transaction that is rolled back, and deletes the cluster. Helpers `tests.login(email)`, `tests.as_service()`, `tests.expect_ok`, `tests.expect_error`, `tests.expect_count`, `tests.expect_eq` (`01_helpers.sql`).

3. Test environment and tools

Item	Value
Developer machine	Windows 11, Node.js ≥ 20.19, PostgreSQL 18 binaries for the SQL harness
Unit test runner	Vitest 5.0.5, jsdom 27, Testing Library
Browser for layout checks	Chromium-based browser pane, Lighthouse 12.2.1 (`docs/v2/PERF.md`)
Staging	Vercel preview deployment of `develop` (behind Vercel sign-in)
Data	Fixtures in `supabase/tests/02_fixtures.sql` and `src/dev/*Fixtures.ts` (anonymised)

4. Entry and exit criteria

	Criteria
Entry	Feature code complete on its branch; fixtures updated
Exit (per merge)	`npm run check` passes (0 lint errors, 0 type errors, all unit tests, build) and, for database changes, `npm run db:test` passes
Exit (release to `main`)	All automated tests pass; manual cases MT-01 – MT-20 executed on the preview with no open critical defects [TO CONFIRM]

5. Sample test data

All test data is fictitious. Database fixtures (`02_fixtures.sql`) create one account per role with `@test.local` addresses:

Fixture account	Role
`admin@test.local`	super_admin
`eic@test.local`	editor_in_chief
`pub.me@test.local`	managing_editor
`bm@test.local`	business_manager
`news.editor@test.local`	section_editor (News)
`writer@test.local`, `writer2@test.local`	staff_writer
`stranger@test.local`	signed up, not invited (no access)

Plus five articles in different states (published, draft, submitted) across News and Sports, and an intake file. Salingkusog tests create their own editions with anonymised factions (“Tiamat”, “Vanguard”, “Dragons”, “Moirai”) and athletes (“Ana Swimmer”, “Ben Swimmer”).

Sample CSV for schedule import (FR-107):

```
date,time,sport,event,round,side_a,side_b,venue
2026-10-26,08:00,Basketball,Men,Semifinal 1,Faction A,Faction D,Main Gym
2026-10-26,9:30 AM,Basketball,Men,Semifinal 2,Faction B,Faction C,Main Gym
```

Sample `sk_set_placements` rows (FR-118):

```
[{ "place": 1, "faction_id": "<faction A>", "result": "78–71", "awarded_on": "2026-10-27" },
 { "place": 2, "faction_id": "<faction B>" },
 { "place": 3, "faction_id": "<faction C>" }]
```

6. Results summary

Runs executed on **2026-10-19** at commit `d7f6f34` (branch `develop`):

Run	Result
`npm run test` (Vitest)	**255 / 255 passed** in 48 suites
`npm run db:test`	**278 / 278 checks passed** in 6 suites (public access 39, Salingkusog 71, brackets 28, contests 17, statistics 48, workflow 75)
`npm run lint`, `npm run typecheck`, `npm run build`	Passed (part of `npm run check`)
`npm run check:contrast`	All design-token text pairs pass WCAG AA
`/dev` layout and accessibility sweep	53 screens, no issues at 375 px and 1024 px
`npm audit --omit=dev`	0 vulnerabilities
Lighthouse (production, mobile, 2026-10-18)	Performance 67–72, Accessibility 100, Best practices 100, SEO 100 on `/`, `/news`, `/salingkusog` (`docs/v2/PERF.md`)
Manual cases MT-01 – MT-20	**Not yet run** (scheduled for the preview acceptance)

Requirement coverage (Appendix B): **115 requirements; 98 have automated tests, 17 rely on manual cases; none are uncovered.**

7. Manual test cases (not yet run)

Status for all: **Not yet run**. Execute on the Vercel preview with test accounts (AMR-DOC-20 demo script) and record pass/fail with date and tester.

ID	Req.	Steps	Expected result
MT-01	FR-025	Edit an article; set print issue and position; save	Issue and position saved; article listed in the issue
MT-02	FR-036	Open `/about`, `/editorial-board`, `/privacy`	Pages load with current content; privacy sections for readers, forms, staff, rights
MT-03	FR-044	Submit the contact form normally; then send a request to `/api/submit` without a token	First succeeds; second answers 403 “Please complete the security check”
MT-04	FR-052	Media library: search by caption, toggle “mine”, page through	Results filter correctly; pagination works
MT-05	FR-062	As an editor, add a contributor byline; as a writer, edit own byline only	Editor can add; writer can edit only their own
MT-06	FR-070	As ME, set the lead story and folio pick with pull quote	Home page shows them

ID	Req.	Steps	Expected result
MT-07	FR-071	As EE, add a category and move a section to “More”	Navigation updates; category selectable in the editor
MT-08	FR-072	As EE, add a desk contact	Contact page lists it
MT-09	NFR-01	Open DevTools Network; navigate between pages	Route chunks load on demand; vendor chunks cached
MT-10	NFR-06	In the SQL editor, inspect `app` functions; try calling one via REST	Functions in `app` with `search_path=''`; REST call not found
MT-11	NFR-07	Search the production JS bundle for “service_role” and the Turnstile secret	Not found
MT-12	NFR-09	`curl -I` the production home page	HSTS, X-Frame-Options, nosniff, Referrer-Policy, Permissions-Policy, CSP-Report-Only present
MT-13	NFR-14	Open key screens at 375 px width on a phone	No horizontal scroll; controls usable
MT-14	NFR-15	Keep score for a full game on a phone with one hand	Buttons reachable; actions bar sticky
MT-15	NFR-17	Unset `SUPABASE_SERVICE_ROLE_KEY` on a preview; browse	`/api/e` answers 501; site works normally
MT-16	NFR-19	Try a forbidden change (e.g. writer publishes) through the REST API directly	Refused by the database, same message as in the UI
MT-17	NFR-20	Run the public site and newsroom smoke test on Chrome, Edge, Firefox, Safari (desktop and mobile)	Works on all [TO CONFIRM matrix]
MT-18	FR-117	Open `/salingkusog` on two devices; record a score and confirm a medal	Both update within seconds without reload
MT-19	FR-001	Sign in with a valid and an invalid password; reset password by email	Valid opens newsroom; invalid shows a friendly error; reset email arrives and works
MT-20	FR-122	Finish all games of a test edition	Tally reads “Official and final medal tally” and names champion and runners-up

8. Defects found and fixed during development

Taken from `docs/v2/PROGRESS.md` and commit history (examples):

Defect	Found by	Fix
Final shown in the same bracket column as the semifinals	User report (screenshot)	Column order inferred from round names (`bracketColumns`)
Out-of-range page number showed a raw database error	Security review	`pastLastPage` treats PostgREST 416 as an empty page
Anonymous users could reach `set_publish_access`	Live probe	Execute revoked from `anon`
Anonymous uploads to `intake` were unlimited	Security review	Signed upload links through `/api/submit`
Generated games' teams editable but overwritten	Pre-release review	Team fields locked in the game form
Build schedule could place games after the edition	Pre-release review	Warning and refusal in the modal
Production build failed on server imports without `.js`	Build check	`.js` extensions; regression test `esm-imports.test.ts`

9. Gaps and recommendations

1. Add CI (GitHub Actions) to run `npm run check` and `npm run db:test` on every push.
2. Add browser end-to-end tests (for example Playwright) for login, publish, submit form, and live score.
3. Execute and record MT-01 – MT-20 before release to `main`.
4. Add load testing for event days (many readers on `/salingkusog`).

Appendix A. Automated test suites

Unit and component tests (Vitest): **255 cases in 48 suites**, run on 2026-10-19 — 255 passed, 0 failed.

Suite	File	Cases	Passed	Requirements
TS-U-01	`api/_lib/esm-imports.test.ts`	1	1	NFR-18
TS-U-02	`api/_lib/forms.test.ts`	4	4	FR-040, FR-046
TS-U-03	`api/_lib/seo.test.ts`	5	5	FR-037, FR-038
TS-U-04	`api/_lib/traffic.test.ts`	9	9	FR-080, FR-081
TS-U-05	`src/app/router.test.tsx`	5	5	FR-039
TS-U-06	`src/components/story/Story.test.tsx`	3	3	FR-030, NFR-13
TS-U-07	`src/components/ui/LogoLoader.test.tsx`	1	1	NFR-13
TS-U-08	`src/components/ui/Modal.test.tsx`	3	3	FR-050, NFR-13
TS-U-09	`src/components/ui/Tag.test.tsx`	3	3	FR-030
TS-U-10	`src/components/ui/pageWindow.test.ts`	4	4	FR-024, NFR-04
TS-U-11	`src/features/auth/authHelpers.test.ts`	2	2	FR-001, FR-002
TS-U-12	`src/features/auth/permissions.test.ts`	12	12	FR-005, FR-014, FR-016, FR-017, FR-106
TS-U-13	`src/features/cms/articles/editor/HistoryPanels.test.tsx`	3	3	FR-013, FR-019
TS-U-14	`src/features/cms/articles/editor/WorkflowPanel.test.tsx`	3	3	FR-014, FR-018
TS-U-15	`src/features/cms/dashboard/DashboardView.test.tsx`	4	4	FR-074
TS-U-16	`src/features/cms/intake/IntakeDetailView.test.tsx`	3	3	FR-047

Suite	File	Cases	Passed	Requirements
TS-U-17	`src/features/cms/media/MediaDetails.test.tsx`	2	2	FR-051, FR-053
TS-U-18	`src/features/cms/media/uploadMedia.test.ts`	3	3	FR-050, FR-054
TS-U-19	`src/features/cms/salingkusog/TeamTab.test.tsx`	4	4	FR-106
TS-U-20	`src/features/cms/salingkusog/awards/socialCard.test.ts`	4	4	FR-123
TS-U-21	`src/features/cms/salingkusog/scheduleCsv.test.ts`	12	12	FR-107
TS-U-22	`src/features/cms/salingkusog/scores/contest.test.tsx`	7	7	FR-114, FR-115, FR-111
TS-U-23	`src/features/cms/salingkusog/scores/scores.test.tsx`	7	7	FR-116, FR-118, FR-119
TS-U-24	`src/features/cms/salingkusog/scoring.test.ts`	8	8	FR-116, FR-119, NFR-21
TS-U-25	`src/features/cms/salingkusog/setup/buildSchedule.test.tsx`	4	4	FR-108, FR-109
TS-U-26	`src/features/cms/salingkusog/setup/setup.test.tsx`	4	4	FR-102, FR-104, FR-107
TS-U-27	`src/features/cms/statistics/statistics.test.tsx`	13	13	FR-083
TS-U-28	`src/features/public/archives/archivesData.test.ts`	3	3	FR-035
TS-U-29	`src/features/public/article/Comments.test.tsx`	4	4	FR-043, FR-045
TS-U-30	`src/features/public/home/HomeView.test.tsx`	3	3	FR-030
TS-U-31	`src/features/public/intake/forms.test.tsx`	3	3	FR-040, FR-042
TS-U-32	`src/features/public/intake/validate.test.ts`	2	2	FR-040, FR-042

Suite	File	Cases	Passed	Requirements
TS-U-33	`src/features/public/layout/Header.test.tsx`	2	2	FR-031
TS-U-34	`src/features/public/salingkusog/countdown.test.tsx`	11	11	FR-126
TS-U-35	`src/features/public/salingkusog/race.test.tsx`	6	6	FR-121, FR-122
TS-U-36	`src/features/public/salingkusog/salingkusog.test.tsx`	22	22	FR-120, FR-124, FR-117
TS-U-37	`src/features/public/section/SportsLanding.test.tsx`	3	3	FR-124, FR-031
TS-U-38	`src/features/public/section/landing/landing.test.tsx`	26	26	FR-031
TS-U-39	`src/lib/edgeCache.test.ts`	7	7	NFR-02
TS-U-40	`src/lib/format.test.ts`	5	5	FR-012, NFR-21
TS-U-41	`src/lib/images.test.ts`	3	3	FR-050
TS-U-42	`src/lib/media.test.ts`	2	2	NFR-03
TS-U-43	`src/lib/sanitize.test.ts`	4	4	FR-032, NFR-08
TS-U-44	`src/lib/skContests.test.ts`	5	5	FR-115, FR-119
TS-U-45	`src/lib/skSources.test.ts`	5	5	FR-125, FR-113, FR-108
TS-U-46	`src/lib/slug.test.ts`	3	3	FR-012
TS-U-47	`src/lib/useTween.test.ts`	2	2	NFR-13
TS-U-48	`src/styles/contrast.test.ts`	1	1	NFR-12

Database tests (`npm run db:test`): **278 checks in 6 suites**, run on 2026-10-19 — all passed (the runner stops at the first failure).

Suite	File	Checks	Requirements
TS-D-01	`supabase/tests/public_access.test.sql`	39	FR-023, FR-033, FR-034, FR-035, FR-041, FR-043, FR-045, FR-046, FR-048, FR-060, FR-063, FR-073, NFR-05, NFR-11, NFR-16
TS-D-02	`supabase/tests/salingkusog.test.sql`	71	FR-100, FR-101, FR-102, FR-103, FR-104, FR-105, FR-106, FR-118, FR-120, FR-121, FR-126, FR-127
TS-D-03	`supabase/tests/sk_brackets.test.sql`	28	FR-108, FR-109, FR-110, FR-111, FR-112, FR-113
TS-D-04	`supabase/tests/sk_contests.test.sql`	17	FR-114, FR-115
TS-D-05	`supabase/tests/stats.test.sql`	48	FR-026, FR-080, FR-081, FR-082, FR-083, FR-084, NFR-10
TS-D-06	`supabase/tests/workflow.test.sql`	75	FR-003, FR-005, FR-006, FR-007, FR-010, FR-011, FR-012, FR-014, FR-015, FR-016, FR-017, FR-018, FR-019, FR-020, FR-021, FR-022, FR-023, FR-047, FR-049, FR-053, FR-004, FR-008, FR-048, FR-055, FR-061, FR-075

Appendix B. Requirements traceability matrix (RTM)

Requirement → automated suites (Appendix A) → manual cases (§7). Design references: AMR-DOC-02 §9 and AMR-DOC-03 §5.

Requirement	Summary	Automated suites	Manual case	Status
FR-001	The system shall let staff sign in with email and password.	TS-U-11	MT-19	Automated — passing
FR-002	The system shall let staff reset a forgotten password by email and set a new password from the …	TS-U-11		Automated — passing
FR-003	The system shall grant a role only to emails pre-authorised in `staff_invites`; any other new a…	TS-D-06		Automated — passing
FR-004	The system shall block inactive or uninvited users from the newsroom and show an “access pendin…	TS-D-06		Automated — passing
FR-005	Only SA and EE shall change roles and activation; only SA shall grant or remove the super_admin…	TS-U-12, TS-D-06		Automated — passing
FR-006	The system shall refuse to demote or deactivate the last active super admin.	TS-D-06		Automated — passing
FR-007	The system shall record role, activation and publishing-access changes in the audit log.	TS-D-06		Automated — passing
FR-008	The database shall let staff update their own display name, bio and avatar (but not role or acc…	TS-D-06		Automated — passing
FR-010	Any active staff member shall be able to create a draft article with title, category, kicker, d…	TS-D-06		Automated — passing

Requirement	Summary	Automated suites	Manual case	Status
FR-011	The section of an article shall always follow its category.	TS-D-06		Automated — passing
FR-012	The system shall generate a unique URL slug from the title when none is given, and compute read…	TS-U-40, TS-U-46, TS-D-06		Automated — passing
FR-013	The editor shall autosave changes 2.5 seconds after the last edit and keep the newest 50 revisi…	TS-U-13		Automated — passing
FR-014	The system shall enforce these status transitions: draft → submitted (owner or editor); submitt…	TS-U-12, TS-U-14, TS-D-06		Automated — passing
FR-015	Owners who are not editors shall not edit an article while it is submitted, in review, schedule…	TS-D-06		Automated — passing
FR-016	Only publishers (SA, EE, ME, BM, or staff granted “Can publish”) shall publish, schedule, resch…	TS-U-12, TS-D-06		Automated — passing
FR-017	Only SA, EE and ME shall grant or remove another person’s publishing access; changes shall be a…	TS-U-12, TS-D-06		Automated — passing
FR-018	Scheduling shall require a future time; a scheduled article shall go live automatically when th…	TS-U-14, TS-D-06		Automated — passing
FR-019	Every status change shall be logged with an optional note; staff shall be able to add notes wit…	TS-U-13, TS-D-06		Automated — passing
FR-020	Only editors of the article’s section shall feature an article or change its editorial priority…	TS-D-06		Automated — passing
FR-021	Categories marked not assignable shall be reserved for editors.	TS-D-06		Automated — passing

Requirement	Summary	Automated suites	Manual case	Status
FR-022	SA and EE shall delete any article; other staff shall delete only their own drafts.	TS-D-06		Automated — passing
FR-023	Staff shall see their own articles, articles they co-author, and articles in sections they edit…	TS-D-01, TS-D-06		Automated — passing
FR-024	The article list shall filter by status, section, “mine” and “needs review”, with pagination.	TS-U-10		Automated — passing
FR-025	An article may be linked to a print issue and position.		MT-01	Manual — not yet run
FR-026	Readers shall see article view counts only through aggregated “most read” lists.	TS-D-05		Automated — passing
FR-030	The home page shall show the lead story, latest ticker, most read, latest stories, literary fol…	TS-U-06, TS-U-09, TS-U-30		Automated — passing
FR-031	Each of the 12 sections shall have a landing page with its own layout and a paginated “all stor…	TS-U-33, TS-U-37, TS-U-38		Automated — passing
FR-032	Article pages shall show sanitised body HTML, authors, tags, related stories, share buttons and…	TS-U-43		Automated — passing
FR-033	The system shall count at most one view per visitor, per article, per 30 minutes.	TS-D-01		Automated — passing
FR-034	Readers shall search live articles by text, section and date range, and browse by tag.	TS-D-01		Automated — passing
FR-035	Readers shall browse archives by month and download published print issues.	TS-U-28, TS-D-01		Automated — passing

Requirement	Summary	Automated suites	Manual case	Status
FR-036	The site shall provide About, Editorial Board and Privacy pages.		MT-02	Manual — not yet run
FR-037	Link previews on social media shall show the article’s or section’s title, description and imag…	TS-U-03		Automated — passing
FR-038	The site shall publish `sitemap.xml`, `rss.xml` and `robots.txt`.	TS-U-03		Automated — passing
FR-039	Old v1 addresses shall redirect to their v2 equivalents.	TS-U-05		Automated — passing
FR-040	Readers shall submit a story with name, email, affiliation, section, title, summary, body and u…	TS-U-02, TS-U-31, TS-U-32		Automated — passing
FR-041	Readers shall report an event with organisation, contact, title, date, venue, summary and attac…	TS-D-01		Automated — passing
FR-042	Readers shall send a contact message with a topic category.	TS-U-31, TS-U-32		Automated — passing
FR-043	Readers shall comment on live articles that allow comments; comments shall stay hidden until an…	TS-U-29, TS-D-01		Automated — passing
FR-044	Every public form shall pass a Cloudflare Turnstile check on the server before anything is save…		MT-03	Manual — not yet run
FR-045	Public forms shall be rate-limited per email: comments 5 per 10 minutes, submissions 3 per hour…	TS-U-29, TS-D-01		Automated — passing
FR-046	Attachments shall upload only through one-time signed links issued after the CAPTCHA, and the s…	TS-U-02, TS-D-01		Automated — passing

Requirement	Summary	Automated suites	Manual case	Status
FR-047	Editors shall review submissions and event reports (submitted, reviewing, accepted, declined) w…	TS-U-16, TS-D-06		Automated — passing
FR-048	Editors shall manage the contact inbox (unread, in progress, resolved, spam).	TS-D-01, TS-D-06		Automated — passing
FR-049	Editors shall moderate comments (pending, approved, rejected, spam) and the site may switch com…	TS-D-06		Automated — passing
FR-050	Staff shall upload images that are compressed in the browser to WebP and stored with sizes 320,…	TS-U-08, TS-U-18, TS-U-41		Automated — passing
FR-051	Each asset shall have alt text, caption and credit.	TS-U-17		Automated — passing
FR-052	The library shall be searchable, filterable to “mine”, and paginated.		MT-04	Manual — not yet run
FR-053	An asset in use (cover, social image, logo, issue, etc.) shall not be deletable; the library sh…	TS-U-17, TS-D-06		Automated — passing
FR-054	Editors shall generate missing web sizes for older images (“Make web sizes”).	TS-U-18		Automated — passing
FR-055	Only publishers shall upload to the `archives` bucket (print-issue PDFs).	TS-D-06		Automated — passing
FR-060	SA and EE shall invite staff by email with a role and, for section editors, sections.	TS-D-01		Automated — passing
FR-061	SA and EE shall change a member’s role, active status and assigned sections; SA, EE and ME shal…	TS-D-06		Automated — passing

Requirement	Summary	Automated suites	Manual case	Status
FR-062	Editors shall manage contributor bylines (staff, student, guest, faculty, alumni); staff may ed…		MT-05	Manual — not yet run
FR-063	Publishers (SA, EE, ME) shall manage editorial board terms and members shown on the public Edit…	TS-D-01		Automated — passing
FR-070	SA, EE and ME shall choose the home page lead story and literary folio pick (with pull quote).		MT-06	Manual — not yet run
FR-071	SA and EE shall manage sections (name, description, navigation placement and order, accepts sub…		MT-07	Manual — not yet run
FR-072	SA and EE shall manage desk contacts shown on the contact page.		MT-08	Manual — not yet run
FR-073	SA and EE shall edit general settings (site details, social links, comments switch); SA, EE and…	TS-D-01		Automated — passing
FR-074	The dashboard shall show the counts the user may see: published, published this week, scheduled…	TS-U-15		Automated — passing
FR-075	SA and EE shall read the audit log.	TS-D-06		Automated — passing
FR-080	The public site shall record pageviews, reading time, scroll depth and share clicks without coo…	TS-U-04, TS-D-05		Automated — passing
FR-081	The ingest endpoint shall drop bots and other origins, never store IP addresses, and hash visit…	TS-U-04, TS-D-05		Automated — passing

Requirement	Summary	Automated suites	Manual case	Status
FR-082	Do Not Track and Global Privacy Control shall limit tracking to an anonymous pageview.	TS-D-05		Automated — passing
FR-083	SA, EE and ME shall view statistics by date range: summary with change versus the previous peri…	TS-U-27, TS-D-05		Automated — passing
FR-084	Raw events shall be kept 13 months, staff activity 24 months, daily totals indefinitely.	TS-D-05		Automated — passing
FR-100	The team shall manage editions: year, title, dates, medal points (default 5/3/1), current and p…	TS-D-02		Automated — passing
FR-101	Only one edition shall be current at a time; readers shall see an edition only once it is publi…	TS-D-02		Automated — passing
FR-102	The team shall manage factions (name, short name, motto, colour, logo, order).	TS-U-26, TS-D-02		Automated — passing
FR-103	The team shall manage venues with a description and map position.	TS-D-02		Automated — passing
FR-104	The team shall manage sports (category) and events (division, format, team or individual, resul…	TS-U-26, TS-D-02		Automated — passing
FR-105	The team shall manage athletes (name, program, photo) per faction.	TS-D-02		Automated — passing
FR-106	Only SA, EE and ME shall add or remove Salingkusog team members; changes shall be audited.	TS-U-12, TS-U-19, TS-D-02		Automated — passing

Requirement	Summary	Automated suites	Manual case	Status
FR-107	The team shall import the schedule from CSV (date, time, sport, event, round, side A, side B, v…	TS-U-21, TS-U-26		Automated — passing
FR-108	“Build schedule” shall create an event’s games for single elimination (with bronze game and bye…	TS-U-25, TS-U-45, TS-D-03		Automated — passing
FR-109	Build schedule shall refuse to rebuild an event with games played and shall warn when the last …	TS-U-25, TS-D-03		Automated — passing
FR-110	After each result, the system shall fill the next games automatically: winners and losers of kn…	TS-D-03		Automated — passing
FR-111	A game that has started shall never change teams automatically; an edit that would change it sh…	TS-U-22, TS-D-03		Automated — passing
FR-112	A knockout game shall not be finished as a tie.	TS-D-03		Automated — passing
FR-113	Round-robin standings shall rank by wins, point difference, head-to-head among tied teams, poin…	TS-U-45, TS-D-03		Automated — passing
FR-114	“Create contest” shall create one session with every chosen faction for timed or judged events,…	TS-U-22, TS-D-04		Automated — passing
FR-115	The contest scorekeeper shall take each entry’s result (time, distance or points; unit “s” mean…	TS-U-22, TS-U-44, TS-D-04		Automated — passing
FR-116	The scorekeeper shall update scores, periods, a live note, team statistics and athlete box scor…	TS-U-23, TS-U-24		Automated — passing

Requirement	Summary	Automated suites	Manual case	Status
FR-117	Score changes, results and contest entries shall reach open public pages in real time.	TS-U-36	MT-18	Automated — passing
FR-118	The team shall enter results per event (places, faction, athlete, result, day awarded); medals …	TS-U-23, TS-D-02		Automated — passing
FR-119	When an event's last game is final, the system shall suggest the podium from the games and staf…	TS-U-23, TS-U-24, TS-U-44		Automated — passing
FR-120	The medal table shall be computed: gold, silver, bronze, total, points and rank (by points, the…	TS-U-36, TS-D-02		Automated — passing
FR-121	The points race shall show cumulative points by day, or in 1, 3, 6 or 12-hour steps.	TS-U-35, TS-D-02		Automated — passing
FR-122	The tally shall show "results in: X of Y matches" and, once every match is final, read "Officia…	TS-U-35	MT-20	Automated — passing
FR-123	The team shall record awards (MVP of the day, athlete of the day, finals MVP) with stat lines a…	TS-U-20		Automated — passing
FR-124	The public hub shall provide medal tally, schedule and results (day tabs, filters by sport, fac…	TS-U-36, TS-U-37		Automated — passing
FR-125	Upcoming generated games shall say where each side comes from ("Winner of Semifinal 1", "2nd in…	TS-U-45		Automated — passing
FR-126	SA, EE and ME shall control a public countdown page that works before the edition is published.	TS-U-34, TS-D-02		Automated — passing

Requirement	Summary	Automated suites	Manual case	Status
FR-127	The system shall compute head-to-head records, event completion and stat leaders.	TS-D-02		Automated — passing
NFR-01	Performance		MT-09	Manual — not yet run
NFR-02	Performance	TS-U-39		Automated — passing
NFR-03	Performance	TS-U-42		Automated — passing
NFR-04	Performance	TS-U-10		Automated — passing
NFR-05	Security	TS-D-01		Automated — passing
NFR-06	Security		MT-10	Manual — not yet run
NFR-07	Security		MT-11	Manual — not yet run
NFR-08	Security	TS-U-43		Automated — passing
NFR-09	Security		MT-12	Manual — not yet run
NFR-10	Privacy	TS-D-05		Automated — passing
NFR-11	Privacy	TS-D-01		Automated — passing
NFR-12	Accessibility	TS-U-48		Automated — passing
NFR-13	Accessibility	TS-U-06, TS-U-07, TS-U-08, TS-U-47		Automated — passing
NFR-14	Usability		MT-13	Manual — not yet run
NFR-15	Usability		MT-14	Manual — not yet run
NFR-16	Reliability	TS-D-01		Automated — passing
NFR-17	Reliability		MT-15	Manual — not yet run
NFR-18	Maintainability	TS-U-01		Automated — passing
NFR-19	Maintainability		MT-16	Manual — not yet run
NFR-20	Compatibility		MT-17	Manual — not yet run
NFR-21	Time	TS-U-24, TS-U-40		Automated — passing

Coverage: 115 requirements — 98 with automated tests, 17 with manual cases only, 0 not covered.

Appendix C. Unit and component test cases

ID	Suite	Test (describe › it)	Result	ms
TC-U-001	TS-U-01	server code imports › uses .js extensions on every relative import	passed	40
TC-U-002	TS-U-02	form api › keeps only the form's own fields	passed	15
TC-U-003	TS-U-02	form api › checks attachments like the form and the bucket do	passed	3
TC-U-004	TS-U-02	form api › names upload paths safely inside the folder	passed	3
TC-U-005	TS-U-02	form api › accepts its own ticket only for the same content, before it expires	passed	13
TC-U-006	TS-U-03	injectMeta › replaces title/description and adds Open Graph tags	passed	11
TC-U-007	TS-U-03	crawler detection › matches Facebook/Messenger and X but not browsers	passed	2
TC-U-008	TS-U-03	feeds › builds a sitemap with absolute URLs	passed	1
TC-U-009	TS-U-03	feeds › builds escaped RSS	passed	2
TC-U-010	TS-U-03	feeds › builds public storage URLs and skips site-relative paths	passed	1
TC-U-011	TS-U-04	bots › drops facebookexternalhit/1.1 (+http://www.facebook.com/externalhit_uatext.php)	passed	46
TC-U-012	TS-U-04	bots › drops Mozilla/5.0 (compatible; Googlebot/2.1; +http://www.google.com/bot.html)	passed	2

ID	Suite	Test (describe › it)	Result	ms
TC-U-013	TS-U-04	bots › drops Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) HeadlessChrome/126.0 Safari/537.36	passed	1
TC-U-014	TS-U-04	bots › drops curl/8.4.0	passed	2
TC-U-015	TS-U-04	bots › drops	passed	1
TC-U-016	TS-U-04	bots › keeps real browsers	passed	2
TC-U-017	TS-U-04	classifyUa › reduces the user-agent to families	passed	11
TC-U-018	TS-U-04	traffic source › classifies referrers	passed	8
TC-U-019	TS-U-04	shapeEvents › keeps known events, drops the rest and never forwards a full referrer	passed	8
TC-U-020	TS-U-05	router › redirects old URL /features to /feature	passed	879
TC-U-021	TS-U-05	router › redirects old URL /blog to /blogs	passed	101
TC-U-022	TS-U-05	router › redirects old URL /s&t to /science-technology	passed	100
TC-U-023	TS-U-05	router › redirects old URL /home to /	passed	627
TC-U-024	TS-U-05	router › sends unknown deep paths to the public 404	passed	756
TC-U-025	TS-U-06	story components › StoryCard links the headline to the article and shows meta	passed	649
TC-U-026	TS-U-06	story components › StoryRow pads the number	passed	25
TC-U-027	TS-U-06	story components › SectionHeader gives the see-all link an accessible name	passed	146
TC-U-028	TS-U-07	LogoLoader › announces loading and spins the Amaranth mark	passed	635

ID	Suite	Test (describe › it)	Result	ms
TC-U-029	TS-U-08	Modal › ignores a nested dialog closing (the media picker inside a form)	passed	525
TC-U-030	TS-U-08	Modal › still closes when its own dialog closes	passed	75
TC-U-031	TS-U-08	mediaProblem › accepts the types the media bucket stores and explains the rest	passed	7
TC-U-032	TS-U-09	CategoryPill › puts the specific category first, heavy, then the section	passed	150
TC-U-033	TS-U-09	CategoryPill › shows a single label on its own	passed	32
TC-U-034	TS-U-09	article byline chip › reads “via names | Amaranth”	passed	342
TC-U-035	TS-U-10	pageWindow › shows everything for short lists	passed	20
TC-U-036	TS-U-10	pageWindow › collapses long ranges with gaps	passed	2
TC-U-037	TS-U-10	pageWindow › fills a single missing page instead of a gap	passed	2
TC-U-038	TS-U-10	parsePage › defaults to 1 for junk	passed	4
TC-U-039	TS-U-11	auth helpers › translates Supabase auth errors	passed	6
TC-U-040	TS-U-11	auth helpers › validates new passwords	passed	1
TC-U-041	TS-U-12	roles › ranks roles like the database	passed	11
TC-U-042	TS-U-12	roles › only super admins grant super admin	passed	11
TC-U-043	TS-U-12	Salingkusog › only super admin, Executive Editor and managing editor choose the team	passed	2
TC-U-044	TS-U-12	Salingkusog › team members of any role can enter data	passed	1

ID	Suite	Test (describe › it)	Result	ms
TC-U-045	TS-U-12	allowedTransitions › writer: draft → submitted only	passed	3
TC-U-046	TS-U-12	allowedTransitions › writer can withdraw a submission but not review it	passed	2
TC-U-047	TS-U-12	allowedTransitions › section editor reviews, returns and rejects but cannot publish	passed	3
TC-U-048	TS-U-12	allowedTransitions › publisher can publish, schedule, archive and unpublish	passed	2
TC-U-049	TS-U-12	allowedTransitions › locks content for owners once submitted	passed	3
TC-U-050	TS-U-12	publishing access › staff without access only save drafts and submit for review	passed	2
TC-U-051	TS-U-12	publishing access › the Business Manager and granted staff publish and schedule	passed	3
TC-U-052	TS-U-12	publishing access › only the three managing roles give access; the Business Manager does not	passed	1
TC-U-053	TS-U-13	ActivityPanel › describes workflow steps in plain words with notes	passed	180
TC-U-054	TS-U-13	ActivityPanel › adds a note	passed	777
TC-U-055	TS-U-13	RevisionsPanel › restores after confirmation	passed	93
TC-U-056	TS-U-14	WorkflowPanel › requires a note to reject	passed	963
TC-U-057	TS-U-14	WorkflowPanel › needs a time to schedule	passed	311
TC-U-058	TS-U-14	WorkflowPanel › explains when nothing is allowed	passed	46

ID	Suite	Test (describe › it)	Result	ms
TC-U-059	TS-U-15	DashboardView › greets by first name and shows the four stats	passed	1267
TC-U-060	TS-U-15	DashboardView › lists recent articles with edit links and the submission queue	passed	1303
TC-U-061	TS-U-15	DashboardView › hides editor-only blocks for writers	passed	233
TC-U-062	TS-U-15	DashboardView › greets by Manila time	passed	4
TC-U-063	TS-U-16	IntakeDetailView › converts to a draft in the chosen category	passed	1055
TC-U-064	TS-U-16	IntakeDetailView › declines with the internal note	passed	528
TC-U-065	TS-U-16	IntakeDetailView › links to the draft once converted	passed	278
TC-U-066	TS-U-17	MediaDetails › flags missing alt text and saves edits	passed	1112
TC-U-067	TS-U-17	MediaDetails › blocks deleting images that are in use	passed	199
TC-U-068	TS-U-18	missingWidths › lists the standard sizes smaller than the original that are missing	passed	6
TC-U-069	TS-U-18	missingWidths › matches by file name, so portrait variants (smaller actual width) count as present	passed	1
TC-U-070	TS-U-18	missingWidths › skips GIFs, documents and images with no known size	passed	1
TC-U-071	TS-U-19	buildTeam › splits members, always-allowed roles and people who can be added	passed	104
TC-U-072	TS-U-19	TeamView › lets managers add and remove members	passed	1344

ID	Suite	Test (describe › it)	Result	ms
TC-U-073	TS-U-19	TeamView › is read-only for team members	passed	34
TC-U-074	TS-U-19	Salingkusog nav item › shows for the managing roles and team members only	passed	3
TC-U-075	TS-U-20	social cards › MVP of the Day: portrait, stats and game footer	passed	19
TC-U-076	TS-U-20	social cards › Athlete of the Day: square, medal line and big result	passed	2
TC-U-077	TS-U-20	social cards › Finals MVP uses the portrait layout	passed	1
TC-U-078	TS-U-20	social cards › wraps long names and builds a file name	passed	10
TC-U-079	TS-U-21	parseCsv › handles quotes, commas inside quotes, CRLF and blank lines	passed	16
TC-U-080	TS-U-21	parseTime › 14:30 → 14:30	passed	2
TC-U-081	TS-U-21	parseTime › 2:30 PM → 14:30	passed	1
TC-U-082	TS-U-21	parseTime › 12 am → 00:00	passed	2
TC-U-083	TS-U-21	parseTime › 7 → 07:00	passed	1
TC-U-084	TS-U-21	parseTime › 25:00 → null	passed	1
TC-U-085	TS-U-21	parseTime › noon → null	passed	1
TC-U-086	TS-U-21	resolveSchedule › turns the template into matches in Philippine time	passed	15
TC-U-087	TS-U-21	resolveSchedule › accepts header aliases, a single-event sport and the event venue as default	passed	10
TC-U-088	TS-U-21	resolveSchedule › reports every problem on a line	passed	3

ID	Suite	Test (describe › it)	Result	ms
TC-U-089	TS-U-21	resolveSchedule › marks games already in the schedule or repeated in the file	passed	5
TC-U-090	TS-U-21	resolveSchedule › needs a header with date, time and sport	passed	1
TC-U-091	TS-U-22	contest scorekeeper › lists the line-up in order and places the results as they come in	passed	1290
TC-U-092	TS-U-22	contest scorekeeper › won’t finish while a result is missing	passed	334
TC-U-093	TS-U-22	contest scorekeeper › judged events take places, not results	passed	185
TC-U-094	TS-U-22	Create contest › is offered for timed and judged events	passed	712
TC-U-095	TS-U-22	game details form › locks the teams of a generated game	passed	482
TC-U-096	TS-U-22	game details form › has no sides for a contest	passed	298
TC-U-097	TS-U-22	game details form › sends timed and judged events to Create contest	passed	455
TC-U-098	TS-U-23	ScoresView › shows one day: live, then upcoming, then finished	passed	1379
TC-U-099	TS-U-23	ScorekeeperView › basketball gets +1/+2/+3 and the live actions	passed	719
TC-U-100	TS-U-23	ScorekeeperView › a final game is locked until reopened	passed	445
TC-U-101	TS-U-23	results › validates the form rows	passed	3
TC-U-102	TS-U-23	results › fills the podium from finished games and saves	passed	502
TC-U-103	TS-U-23	AwardsView groups awards by day	passed	199

ID	Suite	Test (describe › it)	Result	ms
TC-U-104	TS-U-23	stat lines round-trip	passed	5
TC-U-105	TS-U-24	Manila time › converts instants to the Philippine calendar day and back	passed	13
TC-U-106	TS-U-24	Manila time › lists edition days and picks today when inside	passed	6
TC-U-107	TS-U-24	orders a day: live first, then by time	passed	2
TC-U-108	TS-U-24	score steps and period labels follow the sport	passed	3
TC-U-109	TS-U-24	status actions	passed	2
TC-U-110	TS-U-24	podiumFromGames › final + bronze game	passed	3
TC-U-111	TS-U-24	podiumFromGames › no bronze game: both semifinal losers share third	passed	2
TC-U-112	TS-U-24	podiumFromGames › nothing finished: no suggestion	passed	1
TC-U-113	TS-U-25	Build schedule › is offered for team brackets and round robins only	passed	1573
TC-U-114	TS-U-25	Build schedule › is offered for individual events too (chess, arnis, taekwondo)	passed	385
TC-U-115	TS-U-25	Build schedule › orders the seeds and says how many games it creates	passed	1279
TC-U-116	TS-U-25	Build schedule › won’t rebuild once games are played	passed	281
TC-U-117	TS-U-26	FactionsView › lists factions with short names and edit/delete per row	passed	1142
TC-U-118	TS-U-26	SportsView › groups events under their sport	passed	726
TC-U-119	TS-U-26	ImportView › previews pasted rows and imports only the ready ones	passed	192

ID	Suite	Test (describe › it)	Result	ms
TC-U-120	TS-U-26	parseCsv drops a byte-order mark	passed	4
TC-U-121	TS-U-27	statsFormat › formats active time as H:MM:SS without capping hours	passed	8
TC-U-122	TS-U-27	statsFormat › formats read time as M:SS	passed	2
TC-U-123	TS-U-27	statsFormat › compares with the previous period	passed	5
TC-U-124	TS-U-27	statsFormat › reads the range from the URL, falling back to the last 30 days	passed	6
TC-U-125	TS-U-27	who can see statistics › is only Super admin, Executive Editor and Managing editor	passed	1
TC-U-126	TS-U-27	tracker helpers › tracks only on the production host	passed	2
TC-U-127	TS-U-27	tracker helpers › respects Do Not Track and Global Privacy Control	passed	1
TC-U-128	TS-U-27	tracker helpers › starts a new session after 30 minutes away	passed	2
TC-U-129	TS-U-27	tracker helpers › measures scroll depth through the article body	passed	3
TC-U-130	TS-U-27	staff heartbeat › beats only for a visible, recently used tab	passed	2
TC-U-131	TS-U-27	StatisticsView › shows the summary, top articles, shares and staff time	passed	1221
TC-U-132	TS-U-27	StatisticsView › explains an empty range	passed	177
TC-U-133	TS-U-27	StatisticsView › renders while loading	passed	368
TC-U-134	TS-U-28	archive months › builds Manila-time ranges	passed	10

ID	Suite	Test (describe › it)	Result	ms
TC-U-135	TS-U-28	archive months › rejects junk	passed	1
TC-U-136	TS-U-28	archive months › labels months	passed	61
TC-U-137	TS-U-29	Comments › threads replies under their parent	passed	818
TC-U-138	TS-U-29	Comments › submits and shows the moderation notice	passed	1128
TC-U-139	TS-U-29	Comments › shows RPC errors such as rate limits	passed	621
TC-U-140	TS-U-29	Comments › hides the form when comments are closed	passed	19
TC-U-141	TS-U-30	HomeView › renders the lead story as the page h1 and every Figma block	passed	1642
TC-U-142	TS-U-30	HomeView › shows the ticker with links	passed	789
TC-U-143	TS-U-30	HomeView › has an empty state before anything is published	passed	22
TC-U-144	TS-U-31	SubmitArticleForm › requires a body or attachment and consent	passed	1954
TC-U-145	TS-U-31	SubmitArticleForm › sends a complete submission	passed	1151
TC-U-146	TS-U-31	ContactForm › validates email and shows server errors	passed	1163
TC-U-147	TS-U-32	validate › checks required, email and length	passed	10
TC-U-148	TS-U-32	validate › passes valid input	passed	3
TC-U-149	TS-U-33	More menu › shows each section with its pitch and latest story, plus the Also links	passed	1537
TC-U-150	TS-U-33	More menu › opens from the header and closes with Escape, returning focus	passed	1160

ID	Suite	Test (describe › it)	Result	ms
TC-U-151	TS-U-34	countdown maths (Asia/Manila target, same for everyone) › splits the time left and stops at zero	passed	27
TC-U-152	TS-U-34	countdown maths (Asia/Manila target, same for everyone) › says it in words for screen readers	passed	5
TC-U-153	TS-U-34	CountdownView › shows days, hours, minutes, seconds and an accessible timer	passed	938
TC-U-154	TS-U-34	CountdownView › switches to “The games have begun” at zero	passed	266
TC-U-155	TS-U-34	Continue launch › plays the launch, then opens the hub	passed	654
TC-U-156	TS-U-34	Continue launch › a click on the launch skips straight to the hub	passed	480
TC-U-157	TS-U-34	Continue launch › ctrl/cmd-click opens normally (no animation)	passed	254
TC-U-158	TS-U-34	CountdownPage › switched off: sends visitors to the Salingkusog hub	passed	59
TC-U-159	TS-U-34	CountdownPage › switched on: shows the countdown	passed	170
TC-U-160	TS-U-34	newsroom setting › only the managing roles see “Show Countdown Page”	passed	97
TC-U-161	TS-U-34	newsroom setting › team members do not	passed	30
TC-U-162	TS-U-35	points race steps › labels step ends in Manila time	passed	13
TC-U-163	TS-U-35	points race steps › turns hour-step rows into ordered steps per faction	passed	5

ID	Suite	Test (describe › it)	Result	ms
TC-U-164	TS-U-35	points race steps › offers 1h, 3h, 6h, 12h and Day, and reports the choice	passed	1353
TC-U-165	TS-U-35	official tally › counts finished matches, leaving cancelled ones out	passed	3
TC-U-166	TS-U-35	official tally › declares the champions once every match is final	passed	427
TC-U-167	TS-U-35	official tally › shows results in while matches are left	passed	464
TC-U-168	TS-U-36	hub helpers › knows where the edition stands	passed	9
TC-U-169	TS-U-36	hub helpers › writes the podium headline	passed	2
TC-U-170	TS-U-36	hub helpers › live games first, then the next ones	passed	1
TC-U-171	TS-U-36	hub helpers › points race stops at the current day	passed	2
TC-U-172	TS-U-36	hub helpers › picks readable text on faction colours	passed	2
TC-U-173	TS-U-36	hub helpers › formats times in Philippine time	passed	4
TC-U-174	TS-U-36	Medal Tally › header shows the edition, live count and day	passed	768
TC-U-175	TS-U-36	Medal Tally › podium, table and live games	passed	693
TC-U-176	TS-U-36	Schedule & Results › opens on today during the games, else the first or last day	passed	3
TC-U-177	TS-U-36	Schedule & Results › groups a day into live, later, completed and postponed	passed	9
TC-U-178	TS-U-36	Schedule & Results › day tabs, filters and the PDF button	passed	842

ID	Suite	Test (describe › it)	Result	ms
TC-U-179	TS-U-36	Winners per game › builds one card per event with results, newest first	passed	4
TC-U-180	TS-U-36	Winners per game › filters by category and links the recap	passed	192
TC-U-181	TS-U-36	Faction profile › computes recent results, head-to-head, medals and top athletes	passed	4
TC-U-182	TS-U-36	Faction profile › renders the hero, switcher and panels	passed	686
TC-U-183	TS-U-36	Sports and bracket › splits knockout games into columns and keeps the bronze game apart	passed	11
TC-U-184	TS-U-36	Sports and bracket › round-robin table and leaders	passed	3
TC-U-185	TS-U-36	Sports and bracket › renders the bracket, champion, division toggle and top scorers	passed	884
TC-U-186	TS-U-36	Match recap › helpers: record, title, stat bars	passed	2
TC-U-187	TS-U-36	Match recap › renders the scoreboard, periods, MVP, team and player stats, and the article	passed	974
TC-U-188	TS-U-36	Venue guide › says what is on at each venue	passed	3
TC-U-189	TS-U-36	Venue guide › pins venues with a map position and links them to the list	passed	214
TC-U-190	TS-U-37	results strip helpers	passed	8
TC-U-191	TS-U-37	SportsLanding › shows results, the Salingkusog block, latest stories and the spotlight	passed	2147
TC-U-192	TS-U-37	SportsLanding › works without a Salingkusog edition	passed	370

ID	Suite	Test (describe › it)	Result	ms
TC-U-193	TS-U-38	landing helpers › hands out each story once	passed	13
TC-U-194	TS-U-38	landing helpers › formats brief times in Manila	passed	4
TC-U-195	TS-U-38	literary folio cover › shows the featured work’s photo in the green cover, or the drawn art without one	passed	249
TC-U-196	TS-U-38	bulletin calendar › lays out the month with today and posting days marked	passed	876
TC-U-197	TS-U-38	news landing › renders every story once, linked to the article, with one h1	passed	2047
TC-U-198	TS-U-38	news landing › shows an empty state with no stories	passed	39
TC-U-199	TS-U-38	feature landing › renders every story once, linked to the article, with one h1	passed	881
TC-U-200	TS-U-38	feature landing › shows an empty state with no stories	passed	18
TC-U-201	TS-U-38	literary landing › renders every story once, linked to the article, with one h1	passed	763
TC-U-202	TS-U-38	literary landing › shows an empty state with no stories	passed	50
TC-U-203	TS-U-38	blogs landing › renders every story once, linked to the article, with one h1	passed	779
TC-U-204	TS-U-38	blogs landing › shows an empty state with no stories	passed	13
TC-U-205	TS-U-38	specials landing › renders every story once, linked to the article, with one h1	passed	697
TC-U-206	TS-U-38	specials landing › shows an empty state with no stories	passed	21

ID	Suite	Test (describe › it)	Result	ms
TC-U-207	TS-U-38	views landing › renders every story once, linked to the article, with one h1	passed	853
TC-U-208	TS-U-38	views landing › shows an empty state with no stories	passed	7
TC-U-209	TS-U-38	science-technology landing › renders every story once, linked to the article, with one h1	passed	840
TC-U-210	TS-U-38	science-technology landing › shows an empty state with no stories	passed	32
TC-U-211	TS-U-38	investigative landing › renders every story once, linked to the article, with one h1	passed	610
TC-U-212	TS-U-38	investigative landing › shows an empty state with no stories	passed	21
TC-U-213	TS-U-38	explainer landing › renders every story once, linked to the article, with one h1	passed	860
TC-U-214	TS-U-38	explainer landing › shows an empty state with no stories	passed	61
TC-U-215	TS-U-38	bulletin landing › renders every story once, linked to the article, with one h1	passed	885
TC-U-216	TS-U-38	bulletin landing › shows an empty state with no stories	passed	11
TC-U-217	TS-U-38	stories landing › renders every story once, linked to the article, with one h1	passed	908
TC-U-218	TS-U-38	stories landing › shows an empty state with no stories	passed	15
TC-U-219	TS-U-39	edgeUrl › routes anonymous public reads through /api/r	passed	48
TC-U-220	TS-U-39	edgeUrl › keeps everything else direct	passed	3

ID	Suite	Test (describe › it)	Result	ms
TC-U-221	TS-U-39	edgeUrl › uses the same list as the server	passed	3
TC-U-222	TS-U-39	edgeCachedFetch › falls back to Supabase when the proxy is missing	passed	10
TC-U-223	TS-U-39	api/r › forwards whitelisted reads with the anon key and caches successes for a minute	passed	28
TC-U-224	TS-U-39	api/r › refuses other tables and never caches errors	passed	15
TC-U-225	TS-U-39	liveCutoff › rounds down to the minute so readers share cache entries	passed	2
TC-U-226	TS-U-40	format › formats dates in Manila time	passed	9
TC-U-227	TS-U-40	format › formats reading time with a minimum of one minute	passed	2
TC-U-228	TS-U-40	format › formats relative time	passed	4
TC-U-229	TS-U-40	format › formats compact numbers	passed	12
TC-U-230	TS-U-40	format › joins meta parts	passed	1
TC-U-231	TS-U-41	images › fits within a max dimension keeping the ratio	passed	9
TC-U-232	TS-U-41	images › makes storage-safe file names	passed	8
TC-U-233	TS-U-41	images › formats sizes	passed	1
TC-U-234	TS-U-42	media › builds public storage URLs	passed	6
TC-U-235	TS-U-42	media › builds srcset from variants plus the original	passed	2
TC-U-236	TS-U-43	sanitizeArticleHtml › removes scripts, handlers and inline styles	passed	125

ID	Suite	Test (describe › it)	Result	ms
TC-U-237	TS-U-43	sanitizeArticleHtml › keeps known embeds and drops unknown iframes	passed	47
TC-U-238	TS-U-43	sanitizeArticleHtml › opens external links in a new tab safely	passed	44
TC-U-239	TS-U-43	sanitizeArticleHtml › blocks javascript: URLs	passed	15
TC-U-240	TS-U-44	contest results › reads and writes times, distances and points	passed	29
TC-U-241	TS-U-44	contest results › times: fastest first, ties share a place, DNF last	passed	7
TC-U-242	TS-U-44	contest results › distances: longest first	passed	1
TC-U-243	TS-U-44	contest results › judged: the places staff enter, shared places allowed	passed	2
TC-U-244	TS-U-44	contest results › suggests medals once every entry has a result	passed	3
TC-U-245	TS-U-45	generated games › says where a side comes from	passed	13
TC-U-246	TS-U-45	generated games › puts a final saved with the semifinals' round order in its own column	passed	6
TC-U-247	TS-U-45	generated games › counts the games a build creates	passed	2
TC-U-248	TS-U-45	round-robin standings and medals › breaks a tie on wins and difference by the head-to-head game	passed	3
TC-U-249	TS-U-45	round-robin standings and medals › suggests medals only when every game is final	passed	3
TC-U-250	TS-U-46	slugify › matches the database rules	passed	9

ID	Suite	Test (describe › it)	Result	ms
TC-U-251	TS-U-46	autoExcerpt › cuts at a word boundary	passed	2
TC-U-252	TS-U-46	autoExcerpt › keeps short text	passed	1
TC-U-253	TS-U-47	useTweenedNumbers › counts up from 0 to the data, then moves to new values	passed	133
TC-U-254	TS-U-47	useTweenedNumbers › shows the data at once when the reader prefers reduced motion	passed	11
TC-U-255	TS-U-48	design tokens › meet WCAG AA for every text pair	passed	32

Appendix D. Database test checks

Kinds: **ok** = the statement must succeed; **error** = it must be refused; **count** = an exact row count (often 0 to prove a row is hidden); **eq** = an exact value.

ID	Suite	Check	Kind	Result
TC-D-001	TS-D-01	anon sees only live articles	count	passed
TC-D-002	TS-D-01	anon card view shows live articles	count	passed
TC-D-003	TS-D-01	card byline from authors	eq	passed
TC-D-004	TS-D-01	anon reads sections	count	passed
TC-D-005	TS-D-01	anon reads authors of live article only	count	passed
TC-D-006	TS-D-01	anon cannot read submissions	count	passed
TC-D-007	TS-D-01	anon cannot read contact inbox	count	passed
TC-D-008	TS-D-01	anon cannot read profiles	count	passed
TC-D-009	TS-D-01	anon cannot read staff invites	count	passed
TC-D-010	TS-D-01	anon cannot read private site settings	count	passed
TC-D-011	TS-D-01	anon cannot insert articles	error	passed
TC-D-012	TS-D-01	anon cannot insert submissions directly	error	passed
TC-D-013	TS-D-01	anon cannot call staff RPC	error	passed
TC-D-014	TS-D-01	view counted on live article	eq	passed
TC-D-015	TS-D-01	most read lists live article	count	passed
TC-D-016	TS-D-01	search finds live article	count	passed
TC-D-017	TS-D-01	search hides drafts	count	passed
TC-D-018	TS-D-01	archive months lists live months only	count	passed
TC-D-019	TS-D-01	board seed: one current term with 18 members	count	passed
TC-D-020	TS-D-01	anon cannot call the comment form directly	error	passed

ID	Suite	Check	Kind	Result
TC-D-021	TS-D-01	anon cannot call the contact form directly	error	passed
TC-D-022	TS-D-01	anon cannot upload to intake (signed links only)	error	passed
TC-D-023	TS-D-01	form api can comment	ok	passed
TC-D-024	TS-D-01	pending comment is hidden	count	passed
TC-D-025	TS-D-01	commenter email is private	count	passed
TC-D-026	TS-D-01	cannot comment on a draft	error	passed
TC-D-027	TS-D-01	bad email rejected	error	passed
TC-D-028	TS-D-01	form api can submit an article	ok	passed
TC-D-029	TS-D-01	submission file must exist	error	passed
TC-D-030	TS-D-01	form api can send event report	ok	passed
TC-D-031	TS-D-01	form api can send contact message	ok	passed
TC-D-032	TS-D-01	rate limit allows 3	ok	passed
TC-D-033	TS-D-01	rate limit allows 3 (3rd)	ok	passed
TC-D-034	TS-D-01	rate limit blocks the 4th	error	passed
TC-D-035	TS-D-01	honeypot silently accepted	ok	passed
TC-D-036	TS-D-01	anon cannot upload to media	error	passed
TC-D-037	TS-D-01	anon cannot list intake files	count	passed
TC-D-038	TS-D-01	honeypot stored nothing	count	passed
TC-D-039	TS-D-01	submission file attached	count	passed
TC-D-040	TS-D-02	readers cannot see an unpublished edition	count	passed
TC-D-041	TS-D-02	readers cannot see its factions	count	passed
TC-D-042	TS-D-02	readers cannot write	error	passed
TC-D-043	TS-D-02	staff can preview an unpublished edition	count	passed
TC-D-044	TS-D-02	staff outside the team cannot enter data	error	passed

ID	Suite	Check	Kind	Result
TC-D-045	TS-D-02	staff outside the team cannot add themselves	error	passed
TC-D-046	TS-D-02	section editor cannot pick the team	error	passed
TC-D-047	TS-D-02	managing editor adds a writer to the team	ok	passed
TC-D-048	TS-D-02	managing editor can enter data too	ok	passed
TC-D-049	TS-D-02	executive editor (editor_in_chief) adds a member	ok	passed
TC-D-050	TS-D-02	executive editor removes a member	ok	passed
TC-D-051	TS-D-02	super admin manages the team	ok	passed
TC-D-052	TS-D-02	team changes are audited	count	passed
TC-D-053	TS-D-02	team members cannot remove others from the team	count	passed
TC-D-054	TS-D-02	team member adds a sport	ok	passed
TC-D-055	TS-D-02	team member adds a venue	ok	passed
TC-D-056	TS-D-02	team member adds events (edition filled in from the sport)	ok	passed
TC-D-057	TS-D-02	event inherited its edition	eq	passed
TC-D-058	TS-D-02	team member schedules a semifinal and the final	ok	passed
TC-D-059	TS-D-02	a faction cannot play itself	error	passed
TC-D-060	TS-D-02	scorekeeper starts the game	ok	passed
TC-D-061	TS-D-02	start time recorded when it went live	count	passed
TC-D-062	TS-D-02	a final needs both scores	error	passed
TC-D-063	TS-D-02	scorekeeper finishes both games	ok	passed
TC-D-064	TS-D-02	winner computed from the score	eq	passed

ID	Suite	Check	Kind	Result
TC-D-065	TS-D-02	finish clears the live note and records the end	count	passed
TC-D-066	TS-D-02	athletes and box scores	ok	passed
TC-D-067	TS-D-02	results and medals	ok	passed
TC-D-068	TS-D-02	a gold must be first place	error	passed
TC-D-069	TS-D-02	award: athlete of the day	ok	passed
TC-D-070	TS-D-02	medal table: Tiamat 1 gold + 1 silver = 8 pts, rank 1	eq	passed
TC-D-071	TS-D-02	medal table: Dragons 6 pts, rank 2	eq	passed
TC-D-072	TS-D-02	medal table: Vanguard 3 pts, rank 3	eq	passed
TC-D-073	TS-D-02	points race is cumulative by day (Tiamat: 0,3,8,8,8,8)	eq	passed
TC-D-074	TS-D-02	points race has 6 days for each of 3 factions	count	passed
TC-D-075	TS-D-02	timeline 1 h: latest day with results, 24 steps, Tiamat 3 → 8	eq	passed
TC-D-076	TS-D-02	timeline 6 h: 4 steps	eq	passed
TC-D-077	TS-D-02	timeline 12 h: whole edition, 12 steps for Tiamat (0,0,0,3,3,8,8,8,8,8,8,8)	eq	passed
TC-D-078	TS-D-02	timeline: only 1, 3, 6 or 12 hours	error	passed
TC-D-079	TS-D-02	head-to-head: Tiamat beat Vanguard once	eq	passed
TC-D-080	TS-D-02	events completed (gold awarded)	count	passed
TC-D-081	TS-D-02	top scorer totals across games	eq	passed
TC-D-082	TS-D-02	publish the edition	ok	passed
TC-D-083	TS-D-02	readers now see the medal table	count	passed
TC-D-084	TS-D-02	readers see box scores of the edition	count	passed
TC-D-085	TS-D-02	readers cannot see who is on the team	count	passed

ID	Suite	Check	Kind	Result
TC-D-086	TS-D-02	managing editor removes the writer from the team	ok	passed
TC-D-087	TS-D-02	removed member can no longer enter scores	error	passed
TC-D-088	TS-D-02	…and score updates change nothing	count	passed
TC-D-089	TS-D-02	set_placements replaces the 100m results	eq	passed
TC-D-090	TS-D-02	medals follow the place; blank result is null	eq	passed
TC-D-091	TS-D-02	set_placements rolls back on a bad row	error	passed
TC-D-092	TS-D-02	…leaving the previous results	count	passed
TC-D-093	TS-D-02	set_placements needs Salingkusog access	error	passed
TC-D-094	TS-D-02	readers cannot call set_placements	error	passed
TC-D-095	TS-D-02	countdown is off by default: nothing public	count	passed
TC-D-096	TS-D-02	managing editor adds a team member	ok	passed
TC-D-097	TS-D-02	team member still edits the edition	ok	passed
TC-D-098	TS-D-02	team member cannot switch the countdown on	error	passed
TC-D-099	TS-D-02	team member cannot move the countdown time	error	passed
TC-D-100	TS-D-02	section editor cannot switch it either	error	passed
TC-D-101	TS-D-02	executive editor switches the countdown on	ok	passed
TC-D-102	TS-D-02	readers get the default time: first day 8:00 AM Manila	eq	passed
TC-D-103	TS-D-02	super admin sets the time	ok	passed
TC-D-104	TS-D-02	readers see the new time	eq	passed

ID	Suite	Check	Kind	Result
TC-D-105	TS-D-02	managing editor switches it off	ok	passed
TC-D-106	TS-D-02	off: nothing public	count	passed
TC-D-107	TS-D-02	combat sports, pageant and others can be used	ok	passed
TC-D-108	TS-D-02	…and are saved	count	passed
TC-D-109	TS-D-02	a new current edition replaces the old one	ok	passed
TC-D-110	TS-D-02	only one current edition	count	passed
TC-D-111	TS-D-03	only the Salingkusog team builds schedules	error	passed
TC-D-112	TS-D-03	a faction can only be picked once	error	passed
TC-D-113	TS-D-03	4 factions: 2 semifinals, a final and a bronze game	eq	passed
TC-D-114	TS-D-03	seeds 1 v 4 and 2 v 3 meet in the semifinals	eq	passed
TC-D-115	TS-D-03	final and bronze game wait for the semifinals	count	passed
TC-D-116	TS-D-03	a knockout game cannot end in a tie	error	passed
TC-D-117	TS-D-03	the semifinal winner moves into the final	eq	passed
TC-D-118	TS-D-03	the semifinal loser goes to the bronze game	eq	passed
TC-D-119	TS-D-03	final is set	eq	passed
TC-D-120	TS-D-03	a corrected score keeps the same winner	eq	passed
TC-D-121	TS-D-03	a corrected result changes the final while it hasn’t started	eq	passed
TC-D-122	TS-D-03	once the final starts, the semifinal result is locked	error	passed
TC-D-123	TS-D-03	a started event can’t be rebuilt	error	passed
TC-D-124	TS-D-03	3 factions: the top seed gets a bye	eq	passed
TC-D-125	TS-D-03	the bye seed waits in the final	eq	passed

ID	Suite	Check	Kind	Result
TC-D-126	TS-D-03	4 factions: 3 rounds of 2 games	eq	passed
TC-D-127	TS-D-03	everyone meets everyone once	count	passed
TC-D-128	TS-D-03	round 1 is filled in	count	passed
TC-D-129	TS-D-03	round 2 waits for round 1	count	passed
TC-D-130	TS-D-03	still waiting while a round 1 game is left	count	passed
TC-D-131	TS-D-03	round robin games may be drawn, and round 2 is filled in	count	passed
TC-D-132	TS-D-03	round 3 still waits	count	passed
TC-D-133	TS-D-03	round robin + final: 6 games, then bronze and final	eq	passed
TC-D-134	TS-D-03	final waits for the last round	count	passed
TC-D-135	TS-D-03	top 2 of the table play the final	eq	passed
TC-D-136	TS-D-03	3rd and 4th play the bronze game	eq	passed
TC-D-137	TS-D-03	readers see where sides come from	count	passed
TC-D-138	TS-D-03	readers cannot build schedules	error	passed
TC-D-139	TS-D-04	only the Salingkusog team creates contests	error	passed
TC-D-140	TS-D-04	contests are for timed or judged events	error	passed
TC-D-141	TS-D-04	hip-hop: one session with every faction	ok	passed
TC-D-142	TS-D-04	performance order follows the faction order	eq	passed
TC-D-143	TS-D-04	the session has no sides and is called Final	eq	passed
TC-D-144	TS-D-04	swimming: only the chosen factions	ok	passed
TC-D-145	TS-D-04	in the order given	eq	passed
TC-D-146	TS-D-04	one entry per faction	error	passed
TC-D-147	TS-D-04	the athlete must be from the entry’s faction	error	passed

ID	Suite	Check	Kind	Result
TC-D-148	TS-D-04	times are entered per athlete	ok	passed
TC-D-149	TS-D-04	a contest can finish without scores	ok	passed
TC-D-150	TS-D-04	entries only go into contest sessions	error	passed
TC-D-151	TS-D-04	a contest session has no sides	error	passed
TC-D-152	TS-D-04	judged places are entered, and two factions can share one	ok	passed
TC-D-153	TS-D-04	did not start / finish / disqualified only	error	passed
TC-D-154	TS-D-04	readers see contest entries	count	passed
TC-D-155	TS-D-04	readers cannot change them	count	passed
TC-D-156	TS-D-05	readers cannot call the ingest function	error	passed
TC-D-157	TS-D-05	readers cannot insert pageviews	error	passed
TC-D-158	TS-D-05	readers cannot read pageviews	error	passed
TC-D-159	TS-D-05	readers cannot read the overview	error	passed
TC-D-160	TS-D-05	a batch is recorded	eq	passed
TC-D-161	TS-D-05	the same visitor reading again within 30 min is not a new view	eq	passed
TC-D-162	TS-D-05	pageviews stored (bad id skipped; the draft page is a plain pageview)	count	passed
TC-D-163	TS-D-05	draft articles are not linked or counted	eq	passed
TC-D-164	TS-D-05	counted views: first read, other visitor, DNT visitor	count	passed
TC-D-165	TS-D-05	no raw ids are stored (hashed)	eq	passed
TC-D-166	TS-D-05	engagement attached to the pageview	eq	passed
TC-D-167	TS-D-05	DNT pageview has no id or engagement	eq	passed

ID	Suite	Check	Kind	Result
TC-D-168	TS-D-05	shares: known channel, unknown becomes other, DNT dropped	eq	passed
TC-D-169	TS-D-05	referrer host lower-cased	eq	passed
TC-D-170	TS-D-05	“Most read” counter still updated (3 counted views)	eq	passed
TC-D-171	TS-D-05	replayed pageview id not stored twice	count	passed
TC-D-172	TS-D-05	staff writer cannot read the overview	error	passed
TC-D-173	TS-D-05	staff writer sees no raw pageviews (RLS)	count	passed
TC-D-174	TS-D-05	section editor cannot read staff usage	error	passed
TC-D-175	TS-D-05	managing editor reads the overview	eq	passed
TC-D-176	TS-D-05	overview: counted views and shares	eq	passed
TC-D-177	TS-D-05	overview series has one row per day	eq	passed
TC-D-178	TS-D-05	top article: views, avg read, finish rate, shares	eq	passed
TC-D-179	TS-D-05	author breakdown	eq	passed
TC-D-180	TS-D-05	sources	eq	passed
TC-D-181	TS-D-05	devices	eq	passed
TC-D-182	TS-D-05	share channels	eq	passed
TC-D-183	TS-D-05	unknown breakdown rejected	error	passed
TC-D-184	TS-D-05	date range limited	error	passed
TC-D-185	TS-D-05	executive editor can read too	count	passed
TC-D-186	TS-D-05	super admin reads raw rows (RLS)	count	passed
TC-D-187	TS-D-05	readers cannot send heartbeats	error	passed
TC-D-188	TS-D-05	first beat of a session credits nothing	eq	passed
TC-D-189	TS-D-05	a beat 30 s after the last one credits 30 s	eq	passed
TC-D-190	TS-D-05	credit is capped at 45 s per beat	eq	passed

ID	Suite	Check	Kind	Result
TC-D-191	TS-D-05	after idling (no beats for minutes) the gap is not counted	eq	passed
TC-D-192	TS-D-05	staff cannot read their own usage	count	passed
TC-D-193	TS-D-05	staff cannot use the usage report	error	passed
TC-D-194	TS-D-05	staff cannot write their own time	error	passed
TC-D-195	TS-D-05	managing editor sees the writer’s active time (75 s) and last active	eq	passed
TC-D-196	TS-D-05	yesterday rolled up	eq	passed
TC-D-197	TS-D-05	article totals rolled up	eq	passed
TC-D-198	TS-D-05	dimension totals rolled up	count	passed
TC-D-199	TS-D-05	overview reads the daily totals once raw data is gone	eq	passed
TC-D-200	TS-D-05	top articles read the daily totals	eq	passed
TC-D-201	TS-D-05	breakdowns read the daily totals	eq	passed
TC-D-202	TS-D-05	staff activity older than 24 months is deleted	count	passed
TC-D-203	TS-D-05	old daily salts are deleted	count	passed
TC-D-204	TS-D-06	writer sees own + live articles	count	passed
TC-D-205	TS-D-06	writer cannot see writer2 drafts	count	passed
TC-D-206	TS-D-06	writer creates a draft	ok	passed
TC-D-207	TS-D-06	slug generated from title	eq	passed
TC-D-208	TS-D-06	section derived from category	eq	passed
TC-D-209	TS-D-06	creator credited as author	count	passed
TC-D-210	TS-D-06	writer cannot create published	error	passed
TC-D-211	TS-D-06	writer cannot file an editorial	error	passed
TC-D-212	TS-D-06	writer cannot publish own draft	error	passed
TC-D-213	TS-D-06	writer cannot feature	error	passed

ID	Suite	Check	Kind	Result
TC-D-214	TS-D-06	writer submits for review	ok	passed
TC-D-215	TS-D-06	submitted article is locked for writer	error	passed
TC-D-216	TS-D-06	writer can withdraw to draft	ok	passed
TC-D-217	TS-D-06	writer resubmits	ok	passed
TC-D-218	TS-D-06	activity logged with note	count	passed
TC-D-219	TS-D-06	writer saves an autosave revision	ok	passed
TC-D-220	TS-D-06	writer cannot change own role	error	passed
TC-D-221	TS-D-06	writer edits own bio	ok	passed
TC-D-222	TS-D-06	writer delete of published is a no-op	count	passed
TC-D-223	TS-D-06	writer cannot read inbox	count	passed
TC-D-224	TS-D-06	writer uploads media	ok	passed
TC-D-225	TS-D-06	writer cannot write archives	error	passed
TC-D-226	TS-D-06	writer dashboard stats limited to own drafts	eq	passed
TC-D-227	TS-D-06	news editor sees news submissions	count	passed
TC-D-228	TS-D-06	news editor cannot see sports submissions	count	passed
TC-D-229	TS-D-06	section editor moves to in_review	ok	passed
TC-D-230	TS-D-06	section editor edits in review	ok	passed
TC-D-231	TS-D-06	section editor cannot publish	error	passed
TC-D-232	TS-D-06	section editor returns with note	ok	passed
TC-D-233	TS-D-06	section editor rejects	ok	passed
TC-D-234	TS-D-06	section editor cannot touch sports	error	passed
TC-D-235	TS-D-06	section editor cannot change the ticker	error	passed
TC-D-236	TS-D-06	section editor reads submissions intake	count	passed

ID	Suite	Check	Kind	Result
TC-D-237	TS-D-06	eic publishes directly from draft	ok	passed
TC-D-238	TS-D-06	eic schedules sports piece	ok	passed
TC-D-239	TS-D-06	eic cannot schedule in the past	error	passed
TC-D-240	TS-D-06	eic archives	ok	passed
TC-D-241	TS-D-06	eic restores	ok	passed
TC-D-242	TS-D-06	eic features an article	ok	passed
TC-D-243	TS-D-06	publisher can change the ticker	ok	passed
TC-D-244	TS-D-06	eic sets lead slot	ok	passed
TC-D-245	TS-D-06	eic cannot grant super_admin	error	passed
TC-D-246	TS-D-06	eic promotes writer to section editor	ok	passed
TC-D-247	TS-D-06	role change audited	count	passed
TC-D-248	TS-D-06	scheduled article hidden before time	count	passed
TC-D-249	TS-D-06	published + restored articles visible	count	passed
TC-D-250	TS-D-06	scheduled article visible after time	count	passed
TC-D-251	TS-D-06	cron flips due scheduled rows	eq	passed
TC-D-252	TS-D-06	eic sees pending comments	count	passed
TC-D-253	TS-D-06	eic approves comment	ok	passed
TC-D-254	TS-D-06	approved comment is public	count	passed
TC-D-255	TS-D-06	eic converts submission to draft	ok	passed
TC-D-256	TS-D-06	converted submission marked	count	passed
TC-D-257	TS-D-06	eic reads stats	ok	passed
TC-D-258	TS-D-06	super admin grants super admin	ok	passed
TC-D-259	TS-D-06	super admin demotes the other super admin	ok	passed
TC-D-260	TS-D-06	last super admin cannot step down	error	passed
TC-D-261	TS-D-06	uninvited user sees only live articles	count	passed

ID	Suite	Check	Kind	Result
TC-D-262	TS-D-06	uninvited user has no contributor byline	count	passed
TC-D-263	TS-D-06	uninvited user cannot write	error	passed
TC-D-264	TS-D-06	uninvited user cannot upload media	error	passed
TC-D-265	TS-D-06	uninvited user cannot read stats	error	passed
TC-D-266	TS-D-06	writer drafts another story	ok	passed
TC-D-267	TS-D-06	writer without access cannot publish	error	passed
TC-D-268	TS-D-06	writer cannot give self publishing access	error	passed
TC-D-269	TS-D-06	writer cannot call the grant	error	passed
TC-D-270	TS-D-06	business manager cannot give publishing access	error	passed
TC-D-271	TS-D-06	business manager drafts a story	ok	passed
TC-D-272	TS-D-06	business manager publishes	ok	passed
TC-D-273	TS-D-06	managing editor gives the writer publishing access	ok	passed
TC-D-274	TS-D-06	writer with access publishes own story	ok	passed
TC-D-275	TS-D-06	managing editor takes the access back	ok	passed
TC-D-276	TS-D-06	writer without access cannot unpublish	error	passed
TC-D-277	TS-D-06	publishing access changes are audited	count	passed
TC-D-278	TS-D-06	used cover cannot be deleted	error	passed

Deployment and Operations Runbook

Amaranth v2 — environments, setup, database changes, backups, monitoring, incidents and event week · 1.0 · 2026-10-19

Document control

Field	Value
Title	Deployment and Operations Runbook
Document ID	AMR-DOC-09
Version	1.0
Date	2026-10-19
Author	Amaranth development team [TO CONFIRM: names]
Status	Final draft for defense
Sources	`docs/v2/DB.md`, `docs/v2/STATS.md`, `vercel.json`, `.env.example`, `scripts/`

Contents

1. Environments
2. Environment variables
3. Setting up from scratch
4. Releasing a new version
5. Database changes (“migrations”)
6. Backup and restore
7. Monitoring
8. Routine operations
9. Incident response
10. Salingkusog event-week checklist
11. Contacts and access [TO CONFIRM]

1. Environments

Environment	Front end	Database	Who uses it
Local	`npm run dev` (Vite, port 3000; the dev server also runs `/api/submit` with Cloudflare test keys)	The Supabase project in `.env.local`; SQL tests use a throwaway local PostgreSQL	Developers
Preview	Vercel preview deployment of any pushed branch (e.g. `develop`), protected by Vercel sign-in	Same Supabase project as production unless configured otherwise [TO CONFIRM]	Developers, reviewers
Production	Vercel production deployment of `main` (`amaranth-iota.vercel.app` [TO CONFIRM domain])	Production Supabase project	Public and staff

Branch flow: `feature/*` → merge into `develop` (after `npm run check` and, for SQL, `npm run db:test`) → pull request `develop` → `main` → Vercel deploys production.

2. Environment variables

Set in Vercel → Project → Settings → Environment Variables (Production and Preview), and in `.env.local` for local development. Never commit real values; `.env.example` lists the names.

Name	Scope	Purpose
`VITE_SUPABASE_URL`	browser + server	Supabase project URL
`VITE_SUPABASE_ANON_KEY`	browser + server	Public anonymous key
`VITE_SITE_URL`	browser + server	Canonical URL; statistics run only on this host
`VITE_TURNSTILE_SITE_KEY`	browser	Turnstile site key (public)
`SUPABASE_URL`, `SUPABASE_ANON_KEY`, `SITE_URL`	server	Optional server copies (fall back to the `VITE_` values)
`SUPABASE_SERVICE_ROLE_KEY`	server only	Statistics ingest (`/api/e`) and public forms (`/api/submit`)
`TURNSTILE_SECRET_KEY`	server only	Turnstile verification
`VITE_STATS_DEBUG`	browser (optional)	Log statistics events to the console
`VITE_ENABLE_FIXTURES`	build (never on Vercel)	Keep `/dev` screens in a build for performance testing

After changing a variable, **redeploy** so the new value is used.

3. Setting up from scratch

3.1 Supabase

1. Create a Supabase project (region close to the Philippines, e.g. Singapore) [TO CONFIRM region].
2. SQL editor → paste and run, in order: `supabase/v2/schema.sql`, `supabase/v2/salingkusog.sql`, `supabase/v2/stats.sql`, `supabase/v2/seed.sql`; optionally `supabase/v2/seed_board.sql` (edit the term label first).
3. Database → Extensions → enable **pg_cron** (needed by the nightly statistics job; optional `publish-due-articles` job): `select cron.schedule('publish-due-articles', '*/5 * * * *', $$select app.publish_due_articles()$$);`
4. Authentication → Providers → Email: **disable "Allow new users to sign up"**. URL configuration: Site URL = production URL; add redirect URLs for `/admin/set-password` on production and `http://localhost:3000/**`.
5. Invite the first administrator: `insert into public.staff_invites (email, role) values ('<admin email>', 'super_admin');` then Authentication → Users → Invite user with that email.
6. Recommended: enable leaked-password protection and MFA for administrators (AMR-DOC-06 R-03, R-04).

3.2 Cloudflare Turnstile

Cloudflare dashboard → Turnstile → Add widget: name "Amaranth forms", hostnames = production domain, the `*.vercel.app` domain and `localhost`; mode **Managed**; pre-clearance **No**. Copy the site key (public) and the secret key (server only).

3.3 Vercel

1. Import the GitHub repository; framework preset Vite; build command `npm run build`; output `dist` (already in `vercel.json`).
2. Add the environment variables in §2 for Production and Preview.
3. Deploy. Check: home page loads, `/sitemap.xml`, `/rss.xml`, `/robots.txt` respond, `/admin/login` works.

3.4 Local development

```
npm install
cp .env.example .env.local   # then fill in the values
npm run dev
```

Run the gates before merging:

```
npm run check
npm run db:test
```

`npm run db:test` needs PostgreSQL 16+ binaries (set `PG_BIN` if they are not in `C:/Program Files/PostgreSQL/<v>/bin`). If a previous run left a cluster on port 54329, stop it first.

4. Releasing a new version

1. Make sure `develop` passes `npm run check` and `npm run db:test`.
2. Open a pull request `develop` → `main` with the description from `docs/v2/PR.md`.
3. If the release includes SQL changes: **deploy the code first when the SQL removes access the old code relies on** (for example, closing the form RPCs required the new `/api/submit` to be live first), otherwise paste SQL first. The release notes say which.
4. Run the manual checklist on the preview (AMR-DOC-08 §7).
5. Merge; Vercel deploys production automatically.
6. Paste the changed SQL files in the Supabase SQL editor (they are re-runnable).
7. Smoke test production: home, an article, search, `/salingkusog`, sign in, open an article in the editor.

Rollback (front end): Vercel → Deployments → choose the previous production deployment → “Promote to production”. **Rollback (database):** re-run the previous version of the SQL file from git (functions and policies are replaced); data changes cannot be undone this way, so take a backup first (§6).

5. Database changes (“migrations”)

The project uses **re-runnable SQL files** instead of a migration tool.

Rules for writing changes (AMR-DOC-10 §6):

- `create table if not exists`, `alter table … add column if not exists`, `create or replace function`, `drop policy if exists` + `create policy`, `create index if not exists`, enum values with `alter type … add value if not exists`.
- Never drop data in these files.
- Keep `security definer` functions with `set search_path = ''`.
- Add or update SQL tests; run `npm run db:test` (it applies every file twice).

Applying in production: SQL editor → paste the whole file → Run. Order when several change: `schema.sql` → `salingkusog.sql` → `stats.sql` → `seed.sql`. Re-running `schema.sql` does not remove the Salingkusog or statistics policies.

6. Backup and restore

What	How	Frequency
Database	Supabase automatic daily backups (plan-dependent) [TO CONFIRM plan and retention]; manual export: Dashboard → Database → Backups, or `pg_dump` with the connection string	Before every SQL change and before event week
Storage files	Supabase Storage (buckets `media`, `archives`, `intake`); download important files or copy with the Storage API [TO CONFIRM procedure]	Before event week; monthly
Code	GitHub repository (`main`, `develop`)	Every push

What	How	Frequency
Configuration	Environment variable names in this runbook; values in a password manager held by the Executive Editor and the technical lead [TO CONFIRM]	When changed

Restore drill (recommended once per semester): restore the latest backup into a new Supabase project, point a preview deployment at it, and verify sign-in and a few pages.

7. Monitoring

Signal	Where
Front-end errors, function logs (`/api/*`)	Vercel → Project → Logs / Functions
Traffic	Vercel Web Analytics; newsroom Statistics (`/admin/statistics`)
Database health, slow queries, API errors	Supabase → Reports and Logs
Auth problems	Supabase → Authentication → Logs
Form spam or CAPTCHA failures	Cloudflare → Turnstile analytics; 403 responses in `/api/submit` logs
Scheduled jobs	`select * from cron.job;` and `cron.job_run_details`
CSP violations (report-only)	Browser console on the preview

8. Routine operations

Task	Who	How
Invite staff / change roles	SA, EE	Newsroom → Staff → Invites / Accounts
Give publishing access	SA, EE, ME	Staff → Accounts → “Can publish and schedule articles”
Deactivate a leaving member	SA, EE	Staff → Accounts → untick Active
Moderate comments, inbox, submissions	Editors	Settings → Comments / Inbox; Submissions; Events
Front page and ticker	SA, EE, ME	Settings → Front page / General
Review statistics	SA, EE, ME	Statistics
Generate missing image sizes	Editors	Media → “Make web sizes”
Rotate secrets	Technical lead	Regenerate in Supabase / Cloudflare, update Vercel, redeploy

9. Incident response

Severity	Examples	Response time
S1 Critical	Site down; data breach; wrong official results published	Immediately
S2 High	Newsroom unusable for a role; forms failing; live scores not updating	Within hours
S3 Normal	Layout bug; one page failing	Next working day

Steps:

1. **Detect and record** time, symptoms, affected users.
2. **Contain:** for a compromised account, deactivate it (Staff → Accounts) and reset its password; for a leaked key, rotate it and redeploy; for wrong results, correct the placement (the medal table recomputes) and post a correction.
3. **Diagnose** with Vercel and Supabase logs; reproduce on a preview.
4. **Fix** on a branch; run the gates; deploy (or roll back, §4).
5. **Communicate:** inform the Executive Editor; for personal-data breaches, follow the VSU Data Protection Officer's procedure and the National Privacy Commission's 72-hour notification rule where it applies [TO CONFIRM contacts].
6. **Review:** write a short post-incident note and add a test that would have caught it.

Common problems:

Symptom	Likely cause	Fix
Forms say "Forms aren't set up yet"	`SUPABASE_SERVICE_ROLE_KEY` or `TURNSTILE_SECRET_KEY` missing	Add in Vercel, redeploy
Forms say "Please complete the security check"	Turnstile hostname not listed	Add the domain to the widget
Statistics empty	Service key missing or `VITE_SITE_URL` not the production host	Set variables, redeploy
Live scores not updating	Realtime publication missing a table	Re-run `salingkusog.sql`
"permission denied for function …" after an SQL change	Grant missing	Re-run the full SQL file (grants are at the end)
Staff member sees "access pending"	No invite, or inactive	Add invite / activate in Staff

10. Salingkusog event-week checklist

Two weeks before

- ☐ Edition created with correct dates, medal points and venue map; `is_current` on, `is_published` off until ready.
- ☐ Factions (names, colours, logos), venues, sports and events (division, format, team/individual, unit) entered.
- ☐ Salingkusog team members added (Salingkusog → Team) and trained (AMR-DOC-11c).

- ☐ Schedule imported (CSV) or built (Build schedule / Create contest); check the Scores tab day by day.
- ☐ Athletes entered (name, program, photo with consent) [TO CONFIRM consent process].
- ☐ Countdown page switched on if wanted.
- ☐ Backup taken; manual test MT-18 (live updates on two devices) passed.

Day before

- ☐ Publish the edition (`is_published`); check `/salingkusog`, schedule, sports pages.
- ☐ Scorekeepers signed in on their phones; test a score on a test event, then reopen and reset it.
- ☐ Contact list of scorekeepers and the duty editor [TO CONFIRM].

Each competition day

- ☐ Start games on time (Scores → Start game); finish with both scores.
- ☐ For contests, enter every result or place before Finish.
- ☐ After each event's last game, confirm medals in Results ("Ready to confirm").
- ☐ Record MVP / Athlete of the Day; download social cards for posting.
- ☐ Watch Vercel and Supabase logs for errors; check the medal tally after each confirmation.

After the games

- ☐ All games final → tally reads "Official and final medal tally"; verify champion and runners-up with the organisers.
- ☐ Finals MVP recorded; recaps linked to games.
- ☐ Take a backup; switch off the countdown; remove temporary team members.

11. Contacts and access [TO CONFIRM]

Role	Name	Access
Technical lead	[TO CONFIRM]	Vercel, Supabase, GitHub, Cloudflare
Executive Editor	[TO CONFIRM]	Newsroom (EE), secrets vault
Adviser	[TO CONFIRM]	—
VSU Data Protection Officer	[TO CONFIRM]	—

Maintenance and Developer Guide

Amaranth v2 — set up, conventions, extending the system and common fixes · 1.0 · 2026-10-19

Document control

Field	Value
Title	Maintenance and Developer Guide
Document ID	AMR-DOC-10
Version	1.0
Date	2026-10-19
Author	Amaranth development team [TO CONFIRM: names]
Status	Final draft for defense
Audience	Future student developers and maintainers

Contents

1. Quick start
2. Scripts
3. Folder map
4. Coding conventions
5. How data flows in a feature
6. Writing a database change
7. How-to recipes
8. Documentation and diagrams
9. Common errors and fixes
10. Where to start reading

1. Quick start

Requirements: Node.js ≥ 20.19, npm, Git; PostgreSQL 16+ binaries for the SQL tests.

```
git clone <repository>
cd <repository>
npm install
cp .env.example .env.local   # fill in Supabase URL and anon key; optional service key for forms
npm run dev                  # http://localhost:3000
```

- `/dev` (development builds only) lists fixture screens with fake data, so most UI can be worked on without a database.
- Without `VITE_TURNSTILE_SITE_KEY`, the dev server uses Cloudflare's test keys and serves `/api/submit` itself (`devApi` plugin in `vite.config.ts`); forms then need `SUPABASE_SERVICE_ROLE_KEY` in `.env.local`.

Branching: create `feature/<name>` from `develop`; one feature per branch; Conventional Commits (`feat(scope): …`, `fix: …`, `docs: …`); merge with `--no-ff` into `develop` after the gates pass; never commit to `main` directly; never commit `.env*` files (except `.env.example`).

2. Scripts

Command	What it does
`npm run dev`	Vite dev server (port 3000)
`npm run build`	Type-check and production build to `dist/`
`npm run preview`	Serve the production build
`npm run lint`	ESLint
`npm run typecheck`	`tsc -b` (strict)
`npm run test` / `test:watch`	Vitest
`npm run check`	lint + typecheck + test + build (**the merge gate**)
`npm run db:test`	SQL tests in a throwaway PostgreSQL (**gate for database changes**)
`npm run check:contrast`	WCAG contrast of design-token pairs

3. Folder map

Path	Contents
`src/app/`	`router.tsx` (all routes, lazy), `RootLayout.tsx`, `RouteError.tsx`
`src/components/ui/`	Buttons, fields, modal, image (`Img`), pagination, pills, Turnstile widget, loader
`src/components/story/`	Story cards, rows, section header

Path	Contents
`src/features/auth/`	`AuthProvider`, guards, login/reset pages, `permissions.ts` (UI mirror of DB rules)
`src/features/public/`	Public site: `home`, `section` (+ `landing/` per section), `article`, `search`, `archives`, `about`, `intake`, `salingkusog`, `analytics`
`src/features/cms/`	Newsroom: `dashboard`, `articles` (+ `editor/`), `intake`, `media`, `people`, `settings`, `statistics`, `salingkusog` (`setup/`, `scores/`, `results/`, `awards/`)
`src/lib/`	`supabase.ts`, `database.types.ts` (hand-written), `sanitize.ts`, `images.ts`, `media.ts`, `format.ts`, `manila.ts`, `edgeCache.ts`, `sendForm.ts`, `turnstile.ts`, `skSources.ts`, `skContests.ts`, `skLabels.ts`
`src/styles/`	`tokens.css` (design tokens), `app.css`
`src/dev/`	Fixture screens and the DOM checker (never in production)
`api/`	Vercel functions; `_lib/` shared server code (Node ESM — relative imports need `.js`)
`middleware.ts`	Crawler link previews
`supabase/v2/`	`schema.sql`, `salingkusog.sql`, `stats.sql`, `seed.sql`, `seed_board.sql`
`supabase/tests/`	Stub, helpers, fixtures and `*.test.sql`
`scripts/`	`db-test.mjs`, `check-contrast.mjs`, `countdown-og.py`
`docs/`	This documentation; `docs/v2/` holds the build plans and progress log

4. Coding conventions

- **TypeScript strict;** no `any` without a reason.
- **Design tokens only:** no hex colours in components (ESLint rule `no-restricted-syntax`); use token utilities (`bg-paper`, `text-ink`, `text-lime-dark-text`) from `src/styles/tokens.css`.
- **Server state through TanStack Query hooks** in the feature's `*Api.ts` / `*Data.ts`; components do not call `supabase` directly.
- **Permissions:** add the rule in SQL first (helper + policy or trigger), then mirror it in `permissions.ts` for display. Never rely on the UI check.
- **Accessibility:** every control has a label; one `h1` per page; headings in order; images need `alt`; respect `prefers-reduced-motion`.
- **Copy:** plain, short English; error messages say what to do next.
- **Server code (`api/`, `middleware.ts`):** relative imports end in `.js` (a test enforces this); never return secrets; validate all input.
- **Philippine time:** use `src/lib/manila.ts` helpers for Salingkusog dates.
- **Tests:** add a unit or component test with each feature; SQL tests for every database rule.

5. How data flows in a feature

Example: the Salingkusog results editor.

1. Route in the feature router (`cms/salingkusog/SalingkusogPage.tsx` → `results/:eventId`).
2. The page uses hooks from `cms/salingkusog/skApi.ts` (`useSkPlacements`, `useSkMatches`, `useSetPlacements`).
3. Hooks call Supabase (`from('sk_placements')`, `rpc('sk_set_placements')`).
4. The database enforces `app.can_edit_sk()` (RLS and inside the RPC) and the medal/place check constraint.
5. On success the mutation invalidates the `['sk', …]` query keys; Realtime notifies public pages.

6. Writing a database change

1. Edit the right file: newsroom → `schema.sql`; Salingkusog → `salingkusog.sql`; statistics → `stats.sql`.
2. Keep it **re-runnable**: `if not exists`, `create or replace`, `drop … if exists` before `create policy`/`create trigger`, `add value if not exists` for enums (compare new enum values as text inside functions created in the same file).
3. Private helpers go in schema `app` with `security definer` and `set search_path = ''`; public RPCs need explicit `revoke … from public, anon` / `grant … to authenticated` (or `service_role`).
4. Enable RLS and add policies for any new table; add it to the touch/edition trigger loops if relevant.
5. Update `src/lib/database.types.ts` by hand.
6. Add checks to the right `supabase/tests/*.test.sql` file (use `tests.login('<fixture email>')`).
7. Run `npm run db:test` (applies each file twice) and `npm run check`.
8. In the release notes, say which file must be re-pasted and in which order relative to the code deploy.

7. How-to recipes

7.1 Add a sport or event (no code)

Sports, events, factions and venues are **data**. In the newsroom: Salingkusog → Sports & events → Add sport (choose a category) → Add event (division, format, team or individual, result unit “s” for times or “m”/“pts” for highest-wins, venue).

7.2 Add a new sport category (code)

1. `salingkusog.sql`: `alter type public.sk_category add value if not exists '<value>';`
2. `src/lib/database.types.ts`: add to `SkCategory`.
3. `src/lib/skLabels.ts`: add the label to `categoryLabels`.
4. Run the gates; re-paste `salingkusog.sql`.

7.3 Add a role

1. `schema.sql`: `alter type public.app_role add value if not exists '<role>' …;`

2. Decide its powers and update the helpers (`app.can_publish_articles`, `app.is_editor`, …) — compare as text (`app.current_role()::text = '<role>'`) in the same file.
3. `src/lib/database.types.ts` (`AppRole`), `src/features/auth/permissions.ts` (`roleLabels`, `roleOrder`, helper mirrors, `assignableRoles`).
4. Add a fixture user in `supabase/tests/02_fixtures.sql` and SQL tests for what the role may and may not do; unit tests in `permissions.test.ts`.
5. Example to follow: the Business Manager role (`feature/publish-permissions`).

7.4 Add a newsroom screen

1. Create `src/features/cms/<area>/<Screen>.tsx` and hooks in `<area>Api.ts`.
2. Add a lazy route in `src/app/router.tsx` (or the area's nested router) and a menu entry in `src/features/cms/nav.ts`, limited by role.
3. Wrap role-limited screens with `RequireRole`.
4. Add a fixture screen in `src/dev/` and a component test.

7.5 Add a public form field

1. Add the parameter to the RPC in `schema.sql` (validation inside).
2. Add the parameter name to `FORMS[kind].params` in `api/_lib/forms.ts`.
3. Send it from the form via `sendForm`.
4. Update tests (`forms.test.ts`, SQL tests).

7.6 Add a new Salingkusog format

Follow ADR-012 and ADR-014 (AMR-DOC-03): add the enum value, generate games with sources in `sk_build_schedule` (or a contest RPC), teach `app.sk_resolve`/`app.sk_source_side` how to fill sides, extend `suggestedPodium`, the sport page and tests in `sk_brackets.test.sql`.

8. Documentation and diagrams

- Markdown sources: `docs/*.md`; diagrams: `docs/diagrams/*.mmd` (Mermaid) with rendered `.png`/`.svg`.
- Exports (`docs/export/`): `.docx` per document, one combined `.pdf`, and the `.pptx` decks, produced with Pandoc (DOCX), Chrome print-to-PDF (PDF), python-pptx (decks) and Mermaid (diagrams). The build scripts are in `docs/tools/`; see "Rebuilding" in `docs/README.md`.
- Update `docs/v2/PROGRESS.md` with every merge.

9. Common errors and fixes

Error	Cause	Fix
Production 500 / `ERR_MODULE_NOT_FOUND` from `/api` or middleware	Relative import without `.js` in server code	Add `.js`; `esm-imports.test.ts` catches it
"permission denied for function …"	Missing grant after editing a function	Re-run the whole SQL file (grants at the end)

Error	Cause	Fix
“unsafe use of new value … of enum type”	Using a new enum value in the same transaction	Compare as text (`::text`)
`npm run db:test` fails to start on port 54329	A previous cluster still running	Stop the stray PostgreSQL process, rerun
Dev page “Something went wrong” after moving files	Vite’s module graph stale	Restart the dev server
`showModal is not a function` in tests	jsdom lacks `<dialog>` methods	Stub `HTMLDialogElement.prototype.showModal` in `beforeAll`
Types don’t match the database	Hand-written types not updated	Update `database.types.ts` with the schema change
Lint: hex colour not allowed	Colour literal in a component	Use a token
Forms answer 501 locally	No `SUPABASE_SERVICE_ROLE_KEY` in `.env.local`	Add it (never commit it)
Live scores don’t update	Table not in the Realtime publication	Re-run `salingkusog.sql`

10. Where to start reading

1. `docs/00-audit-report.md` — the whole system on a few pages.
2. `src/app/router.tsx` — every screen.
3. `supabase/v2/schema.sql` sections “Helpers” and “Article workflow”.
4. `src/features/cms/articles/editor/ArticleEditorPage.tsx` — a full feature end to end.
5. `supabase/v2/salingkusog.sql` — “Auto brackets and round robins” and “Contests”.

User Manual — Executive Editor, Managing Editor and Super Admin

Amaranth v2 newsroom: staff, publishing, settings, statistics and Salingkusog oversight · 1.0 · 2026-10-19

Document control

Field	Value
Title	User Manual — Executive Editor, Managing Editor and Super Admin
Document ID	AMR-DOC-11a
Version	1.0
Date	2026-10-19
Author	Amaranth development team [TO CONFIRM: names]
Status	Final draft
Screens	Captured from the development fixture screens with fictional data

Contents

1. Who this manual is for
2. Signing in
3. The newsroom at a glance
4. Managing staff
5. Publishing: review, publish, schedule, archive
6. Giving publishing access to staff
7. The front page and the ticker
8. Settings
9. Moderation: comments, inbox, submissions, events
10. Statistics
11. Salingkusog oversight
12. Good practice

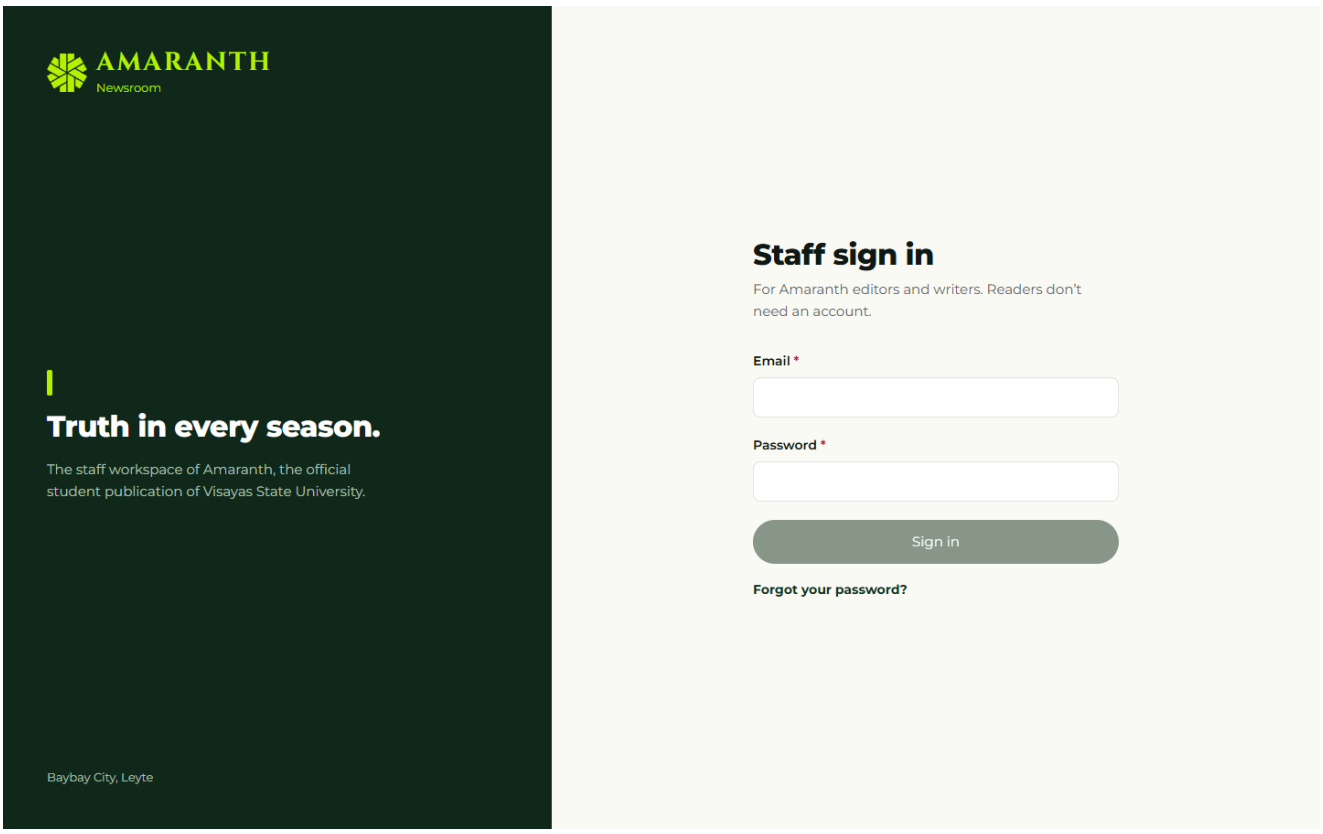
1. Who this manual is for

You are	Role in the system	What only you can do
Executive Editor (Editor-in-Chief)	`editor_in_chief`	Everything below except granting the Super admin role
Super admin	`super_admin`	Everything, including granting or removing Super admin
Managing Editor	`managing_editor`	Publish, front page, ticker, editorial board, print issues, statistics, publishing access, Salingkusog team — but not roles, invites or general settings

Writing and editing articles is covered in AMR-DOC-11b; Salingkusog data entry in AMR-DOC-11c.

2. Signing in

1. Go to `/admin/login` (or “Staff login” in the site footer).
2. Enter your email and password, then **Sign in**.
3. Forgot your password? Choose **Forgot password**, enter your email, and follow the link in the email to set a new one.

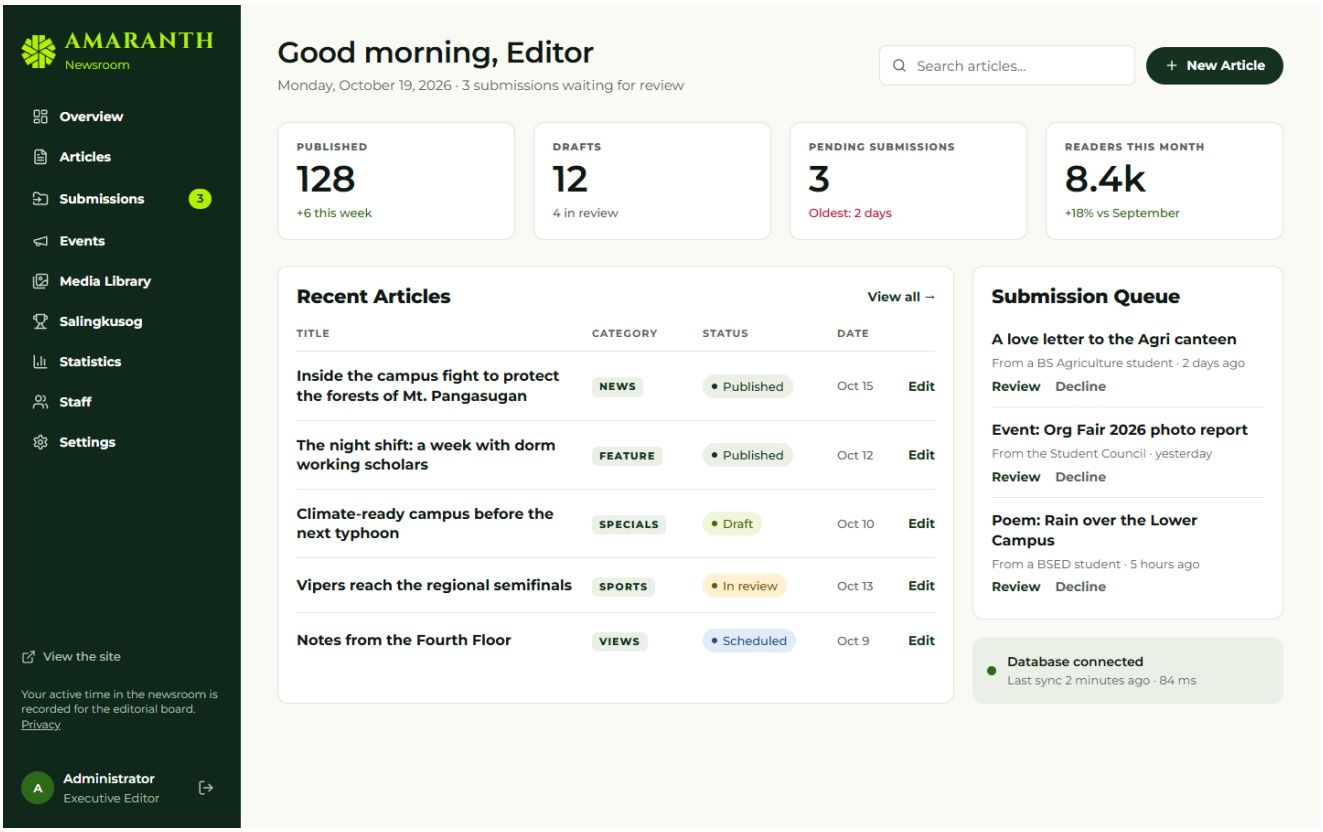


The staff sign-in screen.

If you see **Access pending**, your account has not been invited or has been deactivated; ask the Super admin.

3. The newsroom at a glance

The left menu shows **Overview, Articles, Submissions, Events, Media Library, Salingkusog, Statistics, Staff, Settings**. Items you cannot use are hidden. The **Overview** dashboard shows published, scheduled, drafts and in-review counts, and article views this month compared with last month.



The newsroom dashboard for an editor.

4. Managing staff (Executive Editor, Super admin)

Invite someone

1. **Staff** → **Invites**.
2. Enter the **Email**, choose the **Role**, and for a Section editor tick the **Sections** they will edit.
3. Save. Then, in Supabase, send the invitation email (Authentication → Invite user) [TO CONFIRM who does this step]; when they accept, their account is active with that role.

Change a role or deactivate

1. **Staff** → **Accounts** → **Edit** next to the person.
2. Change the **Role**, the sections, or untick **Active — can sign in to the newsroom**.
3. **Save**. The change is recorded in the audit log.

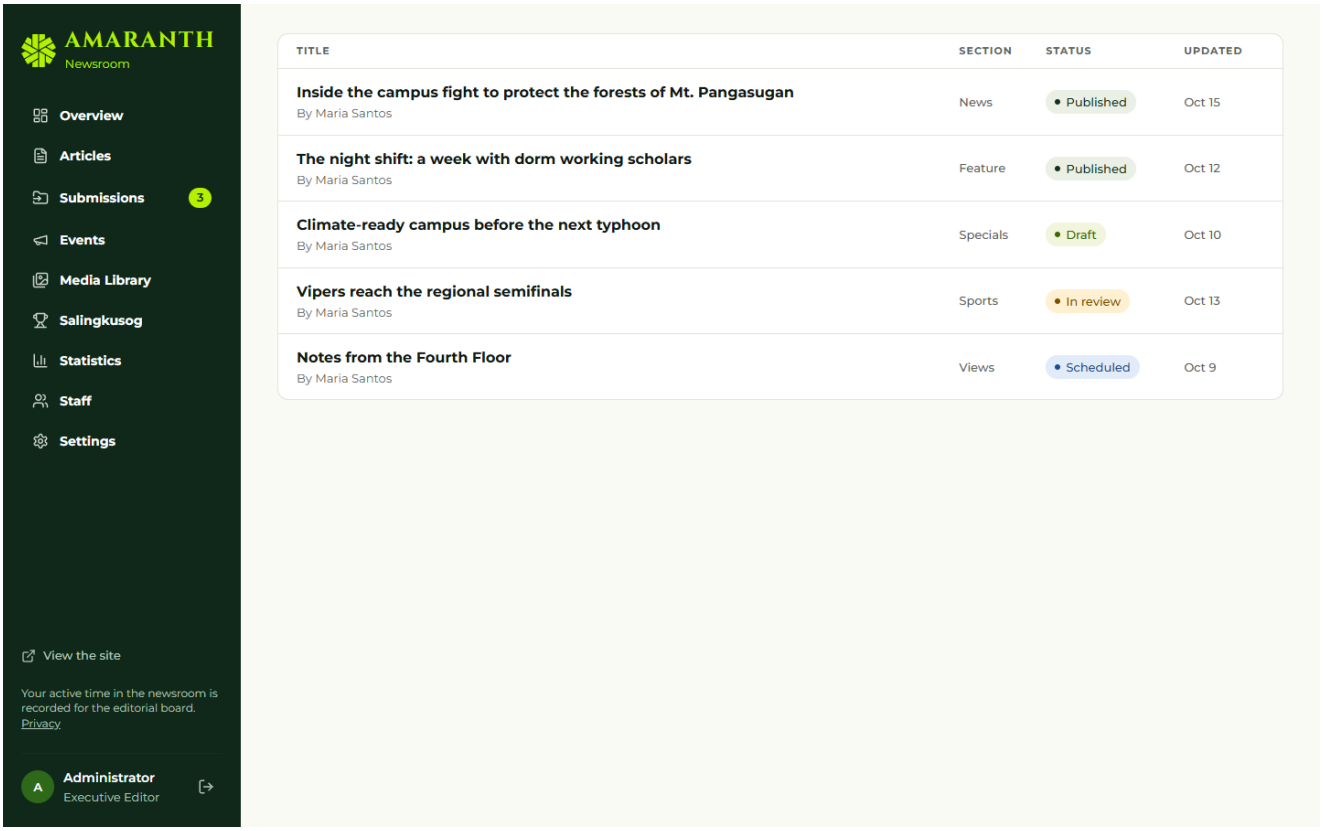
Rules the system enforces: only a Super admin can make or remove another Super admin; you cannot deactivate yourself; at least one active Super admin must remain.

Contributors and editorial board

- **Staff → Contributors**: public bylines (staff, student, guest, faculty, alumni). Editors can add and edit them.
- **Staff → Editorial board** (Executive Editor, Managing Editor, Super admin): terms and members shown on the public Editorial Board page.

5. Publishing: review, publish, schedule, archive

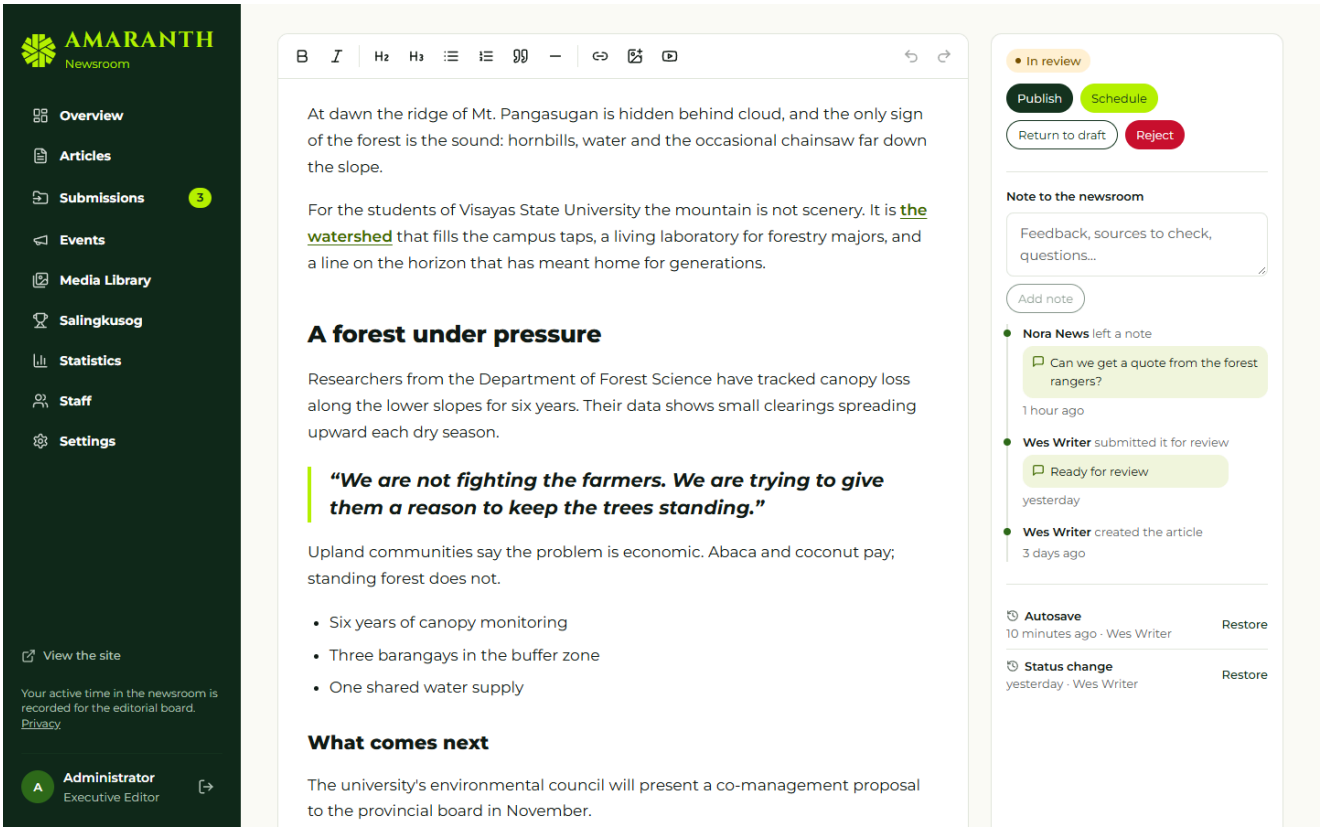
Open **Articles**. Use the tabs **All, Mine, Drafts, Needs review, Scheduled, Published**, the section and status filters, or search by title.



The articles list with filters.

Open an article. The **Status** panel on the right shows only the actions you are allowed to take:

Button	Effect
Start review	Submitted → In review (tells the writer you are on it)
Return to draft	Back to the writer, with your note
Reject	Rejected (a note is required)
Publish	Live now
Schedule	Pick a future date and time; it goes live by itself
Archive	Removes a published article from the site (it can be restored)



The article editor with the workflow panel.

Use **Note to the newsroom** → **Add note** to leave feedback without changing the status. **Notes & activity** shows who did what; **Revisions** lets you restore an earlier version. **Article settings** lets editors tick **Feature on the front page**.

6. Giving publishing access to staff (Executive Editor, Managing Editor, Super admin)

By default only the Super admin, Executive Editor, Managing Editor and Business Manager can publish or schedule. To trust a writer or section editor with publishing:

1. **Staff** → **Accounts** → **Edit** next to the person.
2. Under **Publishing**, tick **Can publish and schedule articles**.
3. **Save**. The change is audited. Untick it to take the access back.

The Business Manager can publish but cannot give access to others.

7. The front page and the ticker (Executive Editor, Managing Editor, Super admin)

Settings → **Front page**:

- choose the **lead story**;
- choose the **literary folio** piece, with a **Pull quote** and **Attribution**;

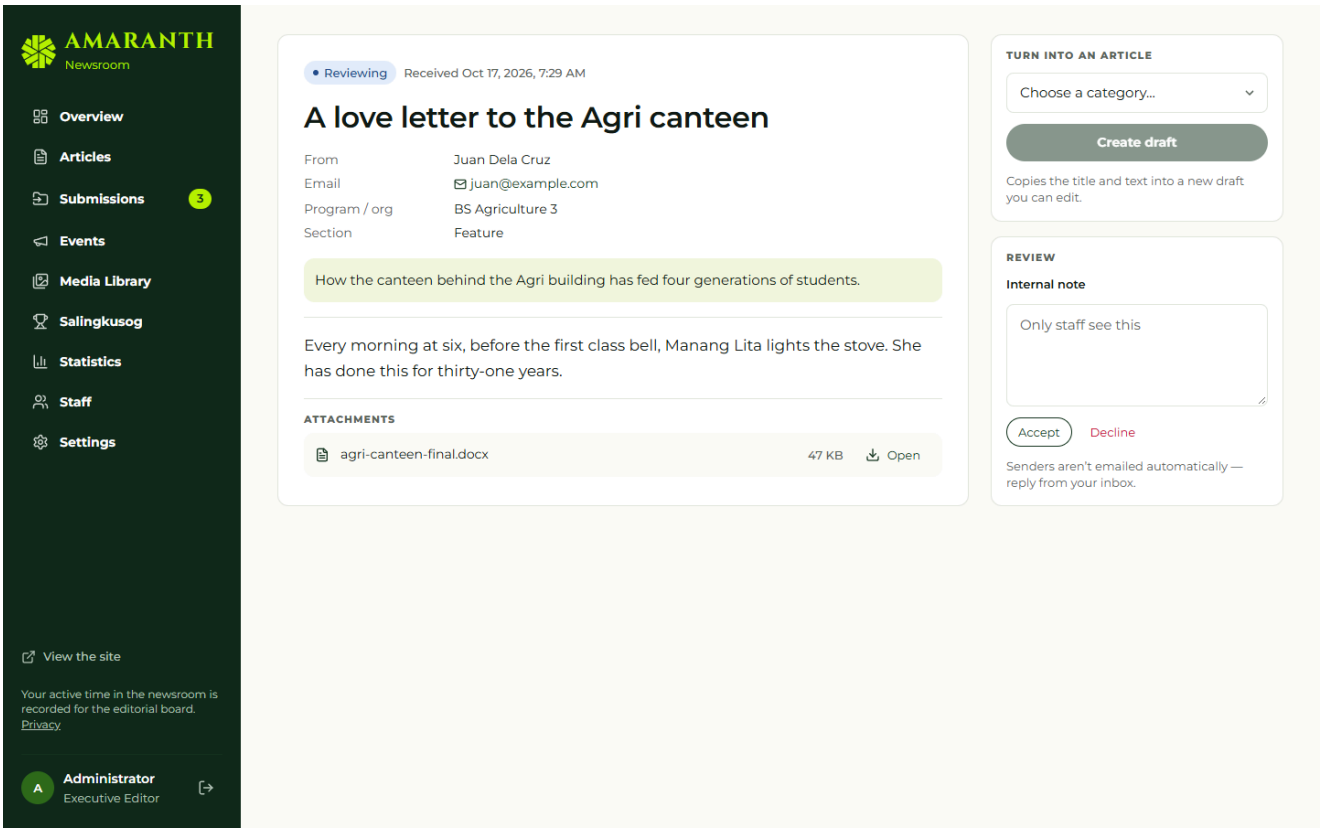
- the latest-news ticker: **Show** “The newest headlines” or “My own announcements”, and **How many**.

8. Settings (Executive Editor, Super admin)

Tab	What you manage
Front page	Lead story, folio pick, ticker (also Managing Editor)
Comments	Moderation queue
Inbox	Contact form messages
Sections	Section names, descriptions, menu placement (main menu or “More”), order, whether they accept submissions; categories, including categories reserved for editors
Desks	Desk contacts shown on the Contact page
General	Site details, social links (web addresses only), comments on/off

9. Moderation: comments, inbox, submissions, events

- **Settings** → **Comments**: approve, reject or mark as spam. Comments are hidden until approved.
- **Settings** → **Inbox**: contact messages; mark in progress, resolved or spam.
- **Submissions** and **Events**: open an item, read it and its **Attachments**, write a **Review** note, then **Decline**, accept, or **Turn into an article** by choosing a **Category for the new article**. The draft opens in the editor and the submission is marked converted.



Reviewing a story submission.

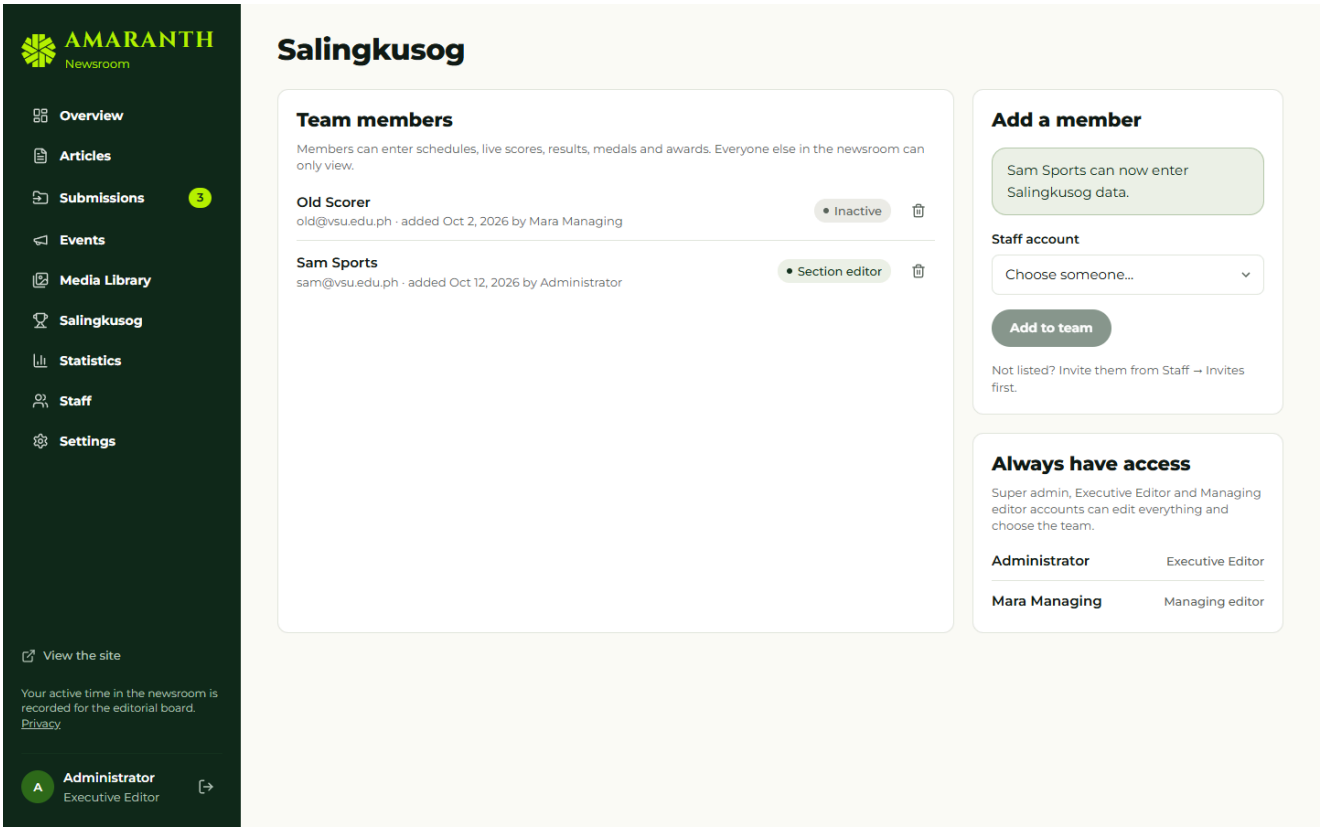
10. Statistics (Executive Editor, Managing Editor, Super admin)

Statistics shows, for the date range you choose: pageviews, article views, readers, average read time, shares, a daily trend, top articles, authors, sections, traffic sources and referrers, devices and browsers, share channels, and staff active time (H:MM:SS). Each figure compares with the previous period.

Privacy: no cookies are set on readers, IP addresses are not stored, readers who ask browsers not to track are counted only as an anonymous pageview, and individual visits are deleted after 13 months (see `/privacy`).

11. Salingkusog oversight (Executive Editor, Managing Editor, Super admin)

- **Salingkusog** → **Team**: add or remove the staff who may enter Salingkusog data. You three always have access.
- **Salingkusog** → **Edition**: dates, medal points, publish the edition, and switch the countdown page on or off.
- **Results & medals**: confirm medals when an event shows **Ready to confirm**. Nothing counts in the medal tally until someone saves the results.



Choosing the Salingkusog team.

Full Salingkusog procedures: AMR-DOC-11c.

12. Good practice

- Give publishing access sparingly and review it each semester.
- Deactivate accounts of members who leave.
- Keep at least two people who can publish during events.
- Before the games, check the event-week checklist (AMR-DOC-09 §10).

User Manual — Section Editors, Staff Writers, Contributors and Business Manager

Amaranth v2 newsroom: writing, submitting, reviewing and media · 1.0 · 2026-10-19

Document control

Field	Value
Title	User Manual — Editors and Staff
Document ID	AMR-DOC-11b
Version	1.0
Date	2026-10-19
Author	Amaranth development team [TO CONFIRM: names]
Status	Final draft

Contents

1. What you can do
2. Signing in
3. Writing an article
4. Submitting for review
5. Reviewing (Section editors)
6. Publishing (if you have access)
7. Media library
8. Your byline
9. Tips and messages you may see

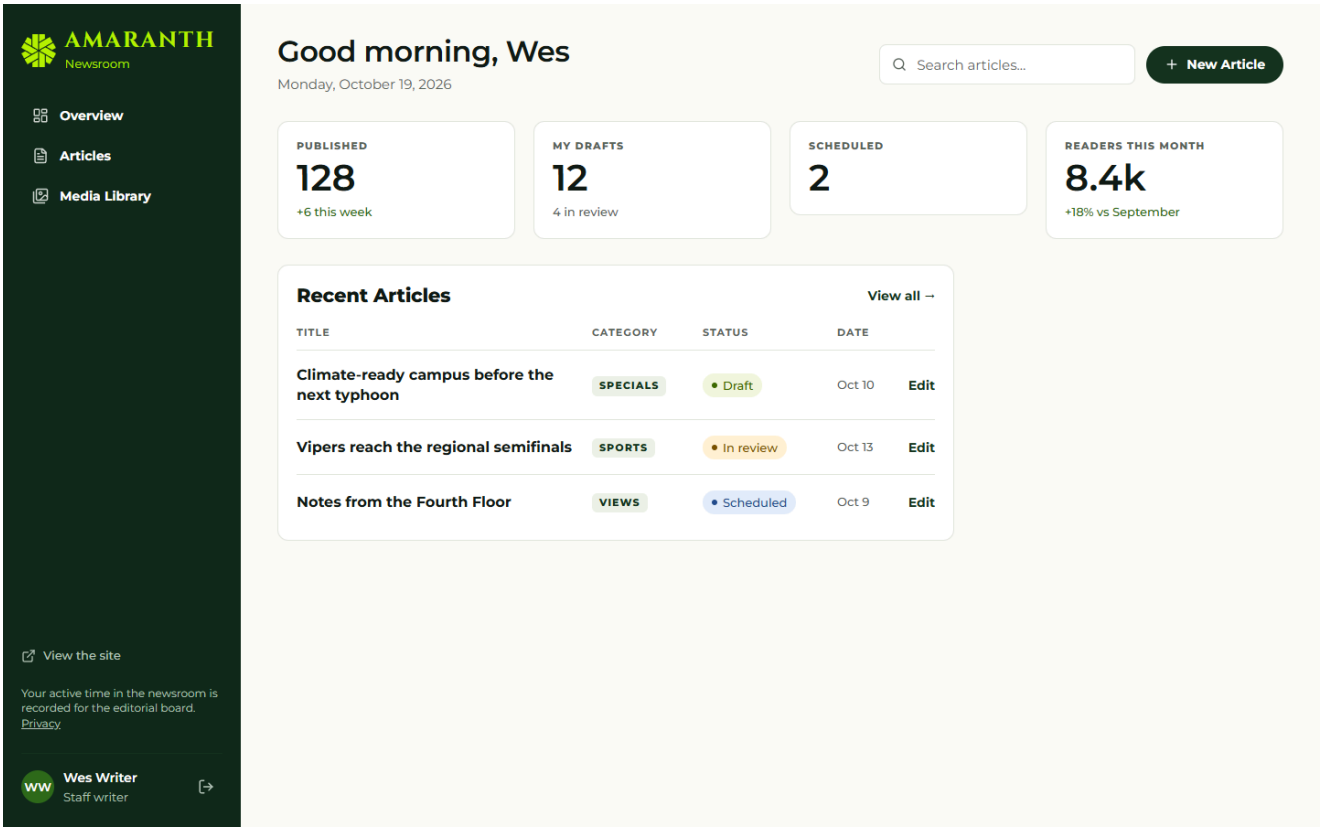
1. What you can do

Role	Write drafts	Submit for review	Review and edit your sections	Publish / schedule	Moderate and review submissions
Staff writer, Contributor	✓	✓	—	only if given access	—
Section editor	✓	✓	✓ (assigned sections)	only if given access	✓
Business Manager	✓	✓	—	✓	—

You see your own articles, articles you co-author, and (Section editors) every article in your sections.

2. Signing in

Go to `/admin/login`, enter your email and password and choose **Sign in**. Use **Forgot password** if needed. Your dashboard shows your drafts and recent articles.

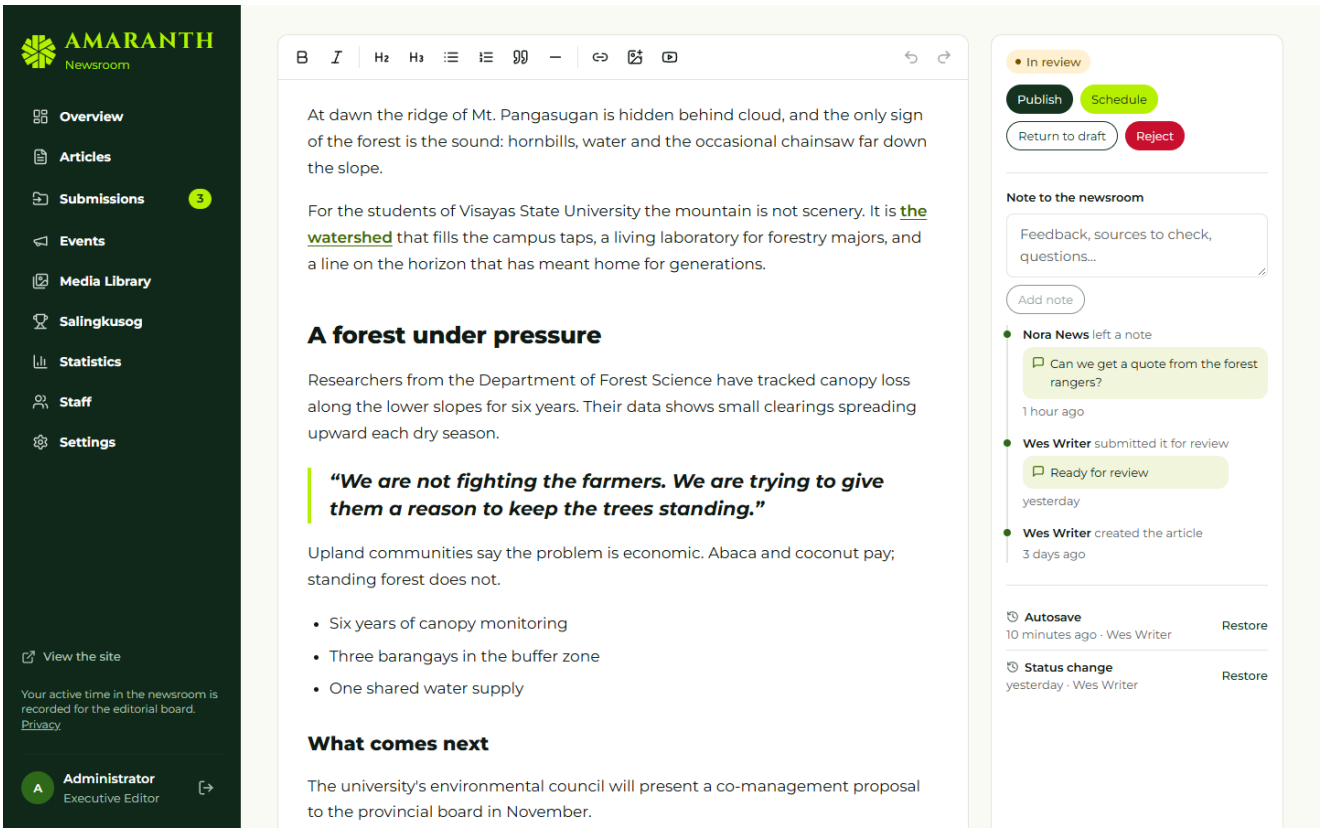


The dashboard as a writer sees it.

3. Writing an article

1. **Articles** → **New article** (also on the dashboard).
2. Choose a **category** (this sets the section). Some categories are reserved for editors.

3. Write the title and the body. The toolbar has bold, italic, headings, lists, quotes, a divider, links, images from the media library and video embeds.
4. Fill in the side panels:
    - **Tags & summary**: kicker, dek (subtitle) and tags;
    - **Cover photo**: choose from the media library and write a caption;
    - **Byline**: authors (you are added automatically) or a byline override such as “By Sports Desk”;
    - **Search & sharing**: SEO title, description and social image, with a **Search result preview**;
    - **Article settings**: **Allow comments**.
5. Your work **saves automatically** 2.5 seconds after you stop typing. **Revisions** keeps the last 50 versions; choose **Restore** to go back.



The editor: body on the left, status and history on the right.

4. Submitting for review

When the draft is ready, choose **Submit for review** (add a note if useful). The article is then locked for you; you can **Return to draft** (withdraw) while it is still waiting. Your editor’s notes appear under **Notes & activity**. If it is returned or rejected, revise it and submit again.

5. Reviewing (Section editors)

1. **Articles** → **Needs review** shows submitted articles in your sections.
2. Open one and choose **Start review**.
3. Edit as needed, then either **Return to draft** with a note, or **Reject** (a note is required), or — if you have publishing access — publish or schedule it.
4. You can tick **Feature on the front page** for articles in your sections.

6. Publishing (if you have access)

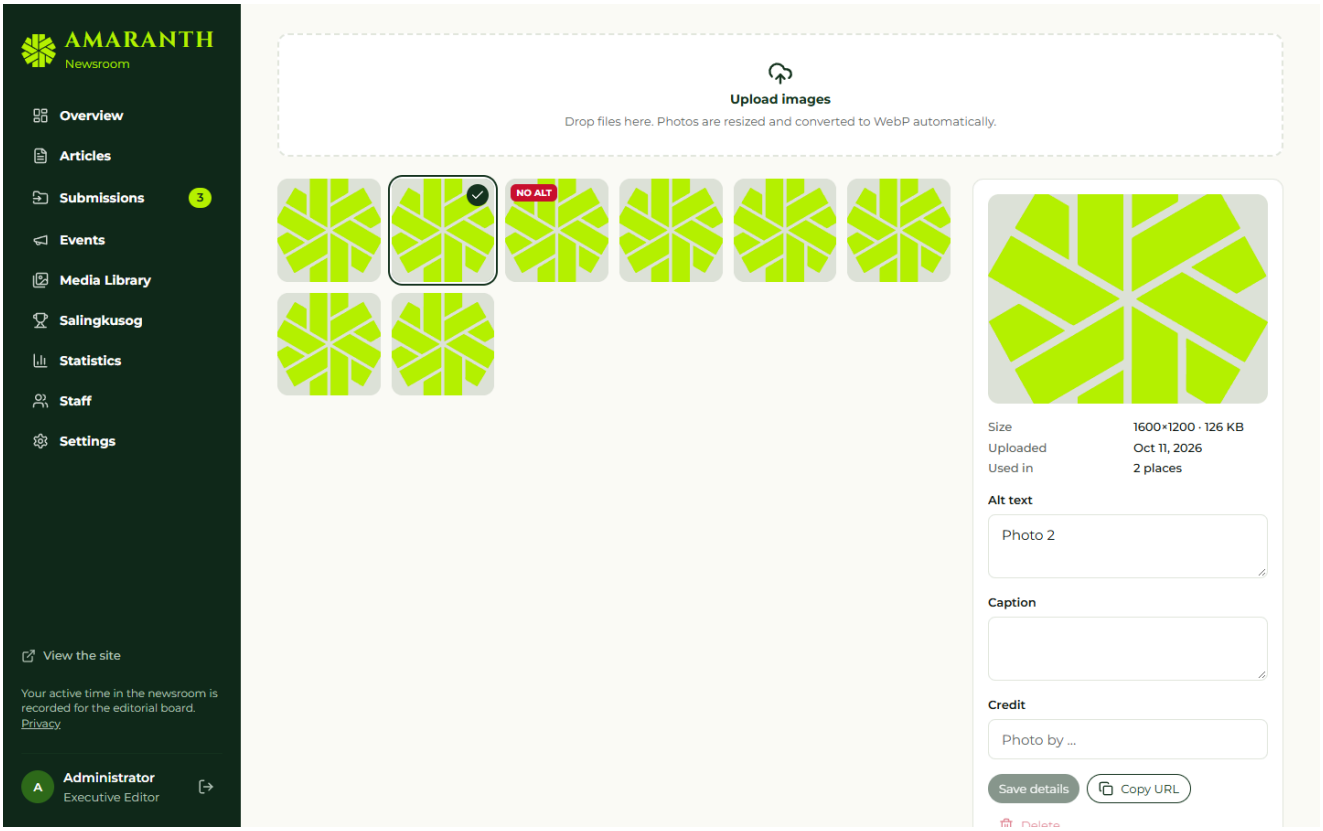
The Business Manager always has it; others only if the Executive Editor, Managing Editor or Super admin turned on **Can publish and schedule articles** for them.

- **Publish** makes the article live now.
- **Schedule** asks for a future date and time; the article goes live on its own at that time.
- **Archive** takes a published article off the site; it can be published again later.

If you try something you are not allowed to do, the newsroom explains it (for example “You cannot move this article from draft to published”).

7. Media library

1. **Media Library** → **Upload images**. Images are resized and compressed automatically.
2. Open an image to add **alt text** (required for accessibility; images without it are flagged “No alt”), a caption and a credit.
3. Search by text, or show **My uploads** only.
4. An image that is **Used in** an article, logo or issue cannot be deleted until it is replaced.



The media library.

8. Your byline

Your public byline (name, position, program, bio, photo) is a contributor profile. Editors manage contributors in **Staff** → **Contributors**; ask your section editor to update yours.

9. Tips and messages you may see

Message	Meaning
“This article is locked while it is submitted”	Wait for the editor, or withdraw it with Return to draft
“This category is reserved for editors”	Choose another category or ask an editor
“Pick a future time to schedule”	The chosen time is in the past
“Only editors can feature or prioritise articles”	Featuring is for section editors and above
“Access pending”	Your account is not active yet; contact the Executive Editor

User Manual — Salingkusog Team

Setting up the games, schedules, live scores, contests, results, medals and awards · 1.0 · 2026-10-19

Document control

Field	Value
Title	User Manual — Salingkusog Team
Document ID	AMR-DOC-11c
Version	1.0
Date	2026-10-19
Author	Amaranth development team [TO CONFIRM: names]
Status	Final draft
Screens	Development fixture screens with fictional factions and athletes

Contents

1. Who can do what
2. The Salingkusog newsroom
3. Before the games: setup
4. Creating the schedule (three ways)
5. Keeping live scores
6. Contests (dance, swimming, athletics, painting…)
7. Results and medals
8. Awards and social cards
9. What readers see
10. Troubleshooting

1. Who can do what

- **Super admin, Executive Editor, Managing Editor:** everything, plus choosing the team (**Team** tab) and the countdown.
- **Team members** (any staff added in **Team**): all setup, schedules, scores, results and awards.
- **Other staff:** can view but not change.

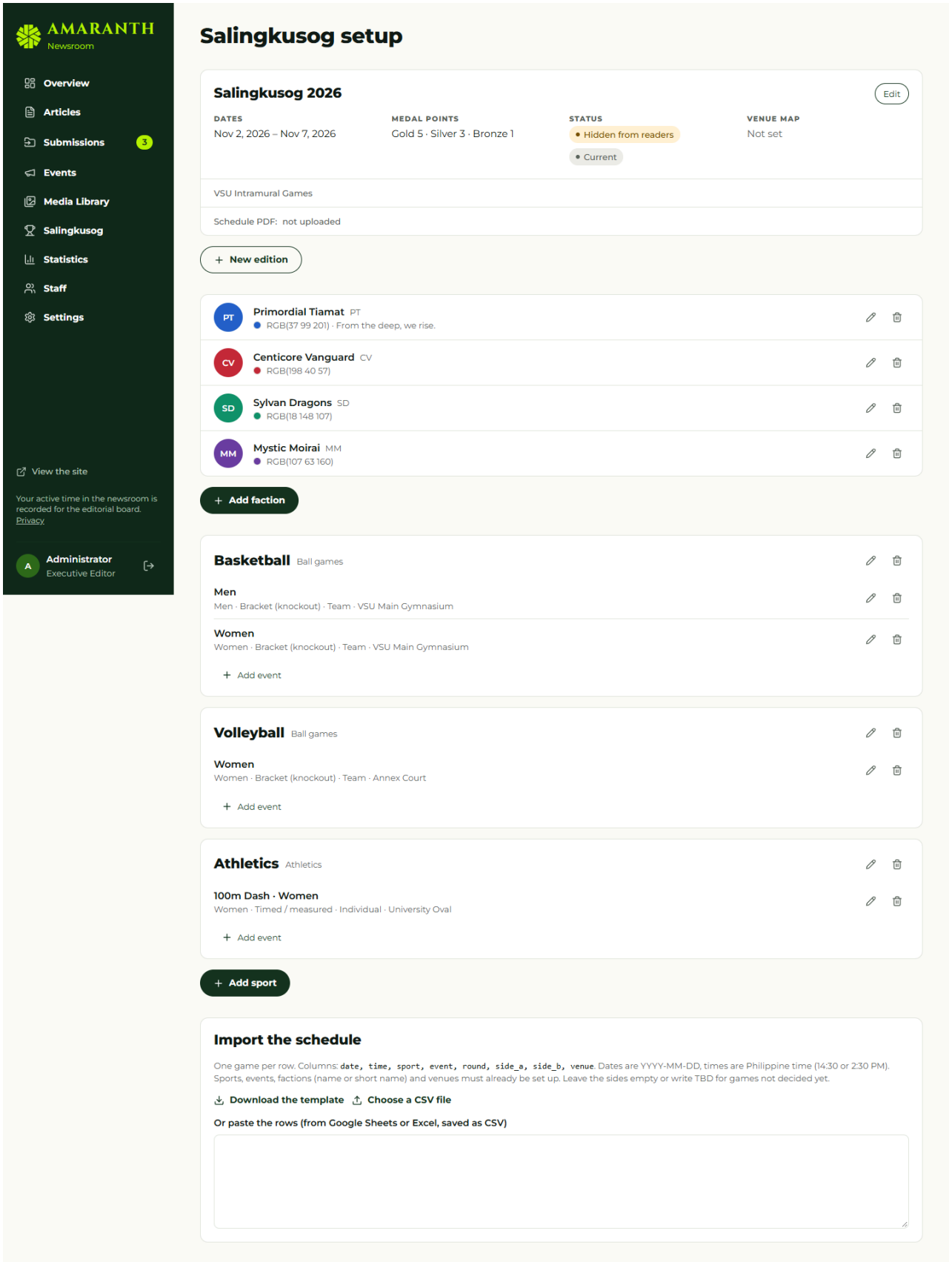
2. The Salingkusog newsroom

Salingkusog in the newsroom menu has these tabs: **Scores, Results & medals, Awards, Athletes, Edition, Factions, Sports & events, Venues, Schedule import, Team**.

3. Before the games: setup

Do these in order (about two weeks before):

1. **Edition** — year, title, start and end dates, medal points (default Gold 5, Silver 3, Bronze 1), schedule PDF, venue map. Keep **Published** off until you are ready; only one edition can be current.
2. **Factions** — name, short name (e.g. “PT”), motto, colour and logo for each team.
3. **Venues** — name, description, and the number and position on the venue map.
4. **Sports & events** — add each sport with its category, then its events:
    - **Division:** men, women, mixed or open;
    - **Format:** Bracket (knockout), Round robin, Round robin + final, Timed / measured, or Judged;
    - **Team event** or individual;
    - **Result unit** for timed or measured events: **s** = fastest wins; **m** or **pts** = highest wins;
    - main venue.
5. **Athletes** — name, program (course and year) and an optional photo, per faction. Only publish photos with the athlete’s consent [TO CONFIRM consent process].



Setup screens: edition, factions, sports and events, import.

4. Creating the schedule (three ways)

Use	For	How
Build schedule	Brackets and round robins (team or individual, e.g. basketball, chess, arnis, kyorugi)	Sports & events → **Build schedule** on the event
Create contest	Events where every faction competes at once (dance, Pinoy pop, folkdance, hip-hop, painting, swimming, athletics)	Sports & events → **Create contest** on the event
Schedule import (CSV)	Any games you already have in a spreadsheet	Schedule import tab

4.1 Build schedule

1. Choose the **format**: Single elimination (with optional bronze game; top seeds get a bye if the field isn't 2, 4, 8 or 16), Round robin (everyone plays everyone once; each round fills in once the previous round is finished), or Round robin + final (1st v 2nd in the final, 3rd v 4th in the bronze game).
2. Put the factions in **seed order** (seed 1 strongest) with the arrows; untick factions not taking part.
3. Set the **First game** time, **Hours between rounds** (24 = one round a day), **Minutes between games** within a round, and the **Venue**.
4. Check the line "Creates N games over R rounds", then **Create games**. A list of the created games appears.

You only enter scores from now on: winners (and semifinal losers for the bronze game) move on by themselves. The system refuses to build if a game has been played, or if the last round would fall after the games end.

4.2 Create contest

Choose the start time, a name (usually "Final"), the venue, the performance order (faction order or **Random draw**), and tick the factions taking part. **Create contest** opens the contest scorekeeper (section 6).

4.3 CSV import

The file must have a header row. Columns (other accepted names in brackets):

Column	Example	Rules
`date` (`day`)	2026-10-26	YYYY-MM-DD, within the edition's dates
`time` (`start`)	08:00 or 2:30 PM	24-hour or AM/PM
`sport`	Basketball	Must match a sport name
`event` (`division`)	Men	Needed when the sport has more than one event
`round` (`stage`)	Semifinal 1	Free text
`side_a` (`team_a`, `home`)	Faction A or its short name	Must match a faction, or leave empty

Column	Example	Rules
`side_b` (`team_b`, `away`)	Faction B	Must differ from side A
`venue` (`location`)	Main Gym	Must match a venue name

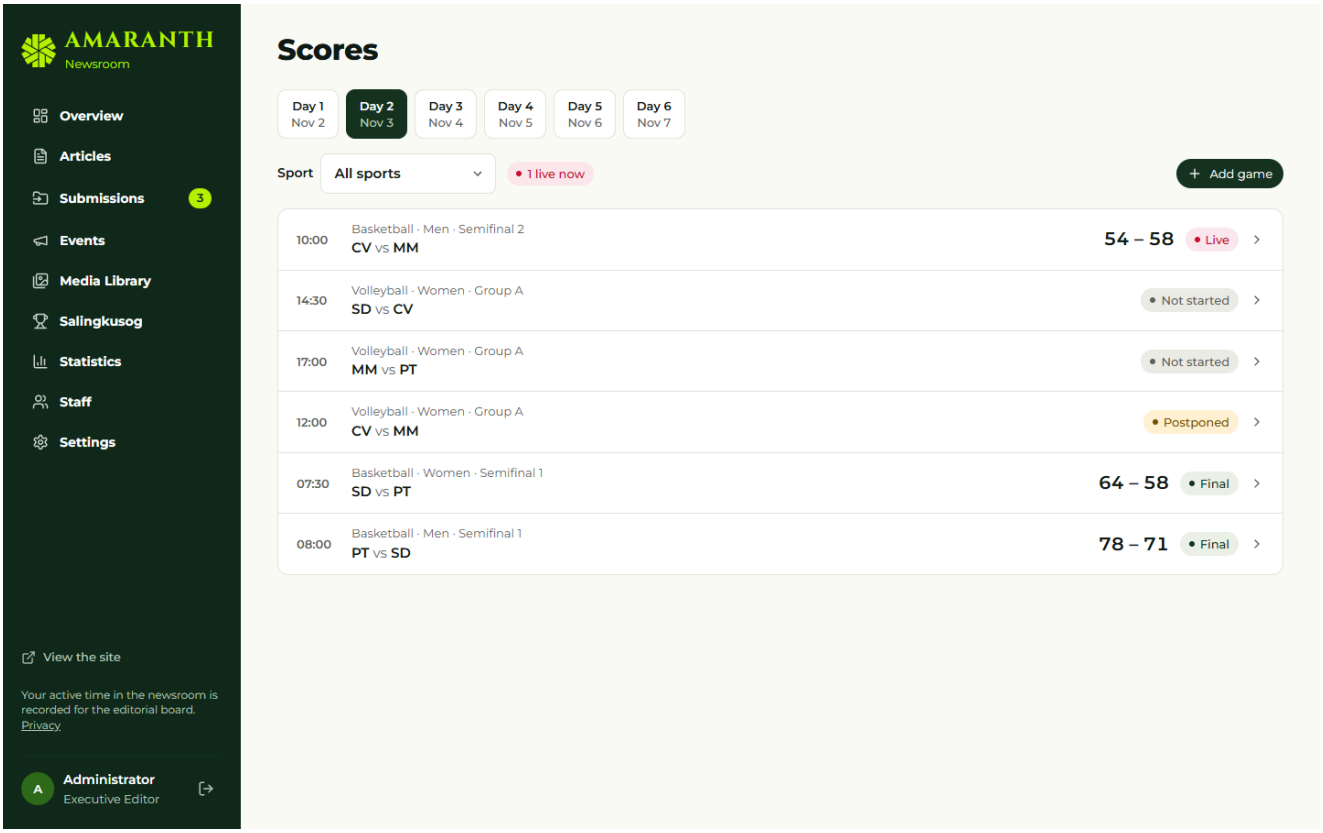
Example:

```
date,time,sport,event,round,side_a,side_b,venue
2026-10-26,08:00,Basketball,Men,Semifinal 1,Faction A,Faction D,Main Gym
2026-10-26,2:30 PM,Volleyball,Women,Group A,FA,FB,Annex Court
```

Steps: paste or choose the file → the preview marks each row **ready**, **duplicate** (same event, time and sides already exist) or with an **error** message → **Import** brings in only the ready rows. A template can be downloaded from the tab.

5. Keeping live scores

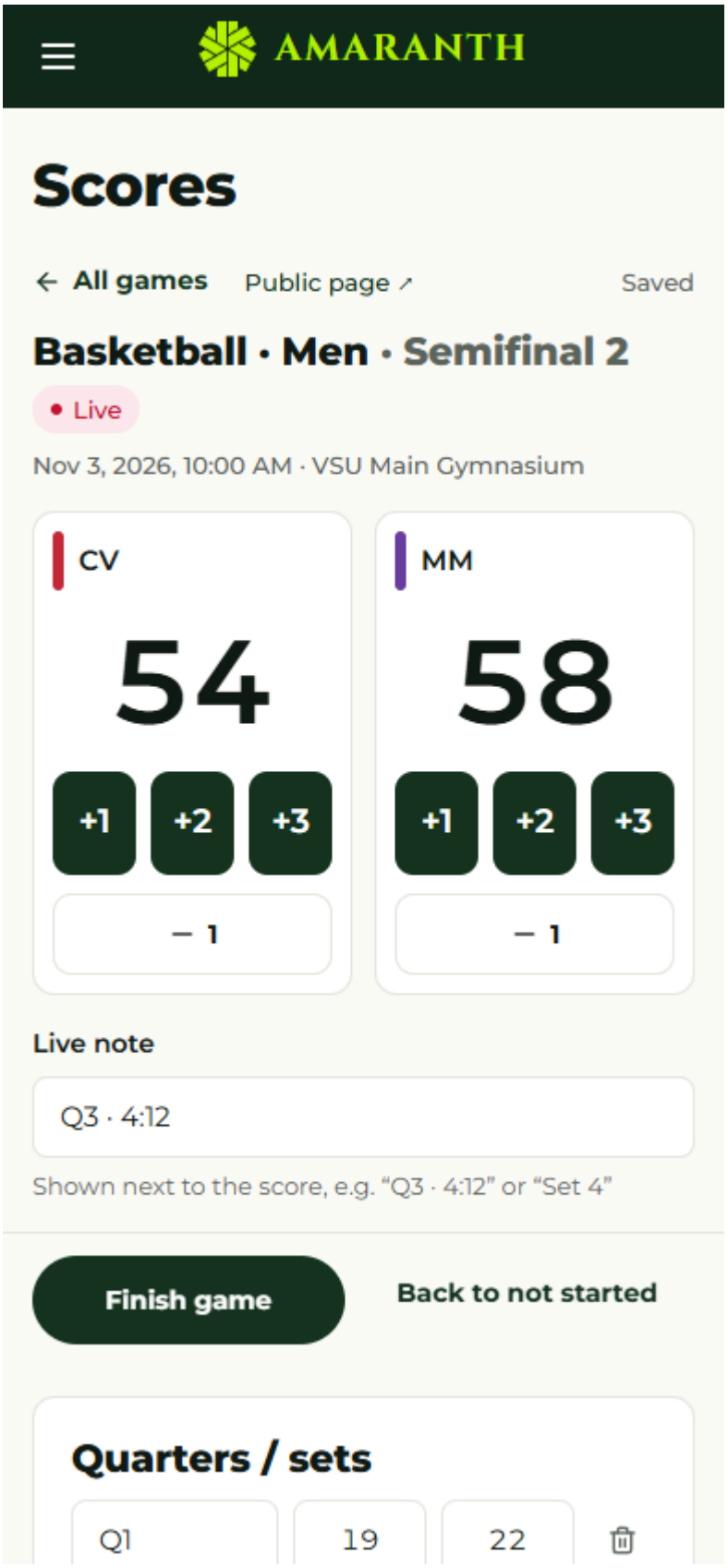
1. **Scores** — pick the day (and a sport if you like). Games are listed live first, then by time.



Games by day.

2. Open a game. Press **Start game** at tip-off.
3. Use the **+ / −** buttons for each side (basketball has +1, +2, +3). The screen updates at once and saves after a short pause; readers see the change within seconds.
4. Add periods (Q1, Q2 … or Set 1, Set 2 …), and a **Live note** such as “Q3 · 4:12”.
5. Optional: team statistics and the athletes’ box score.
6. Press **Finish game** when it ends. Both scores are needed; a **knockout game cannot finish tied** — add the overtime or tiebreak to the score first.

7. Made a mistake? **Reopen (fix the score)**. If the next game has already started, the system will not let the result change its teams; fix that game first.
8. Other actions: **Postpone**, **Cancel game**, **Back on the schedule**, **Edit game details** (time, venue, recap article), **Delete game**.



The scorekeeper on a phone.

For games created by Build schedule, the teams are filled in automatically and cannot be changed in **Edit game details**.

6. Contests

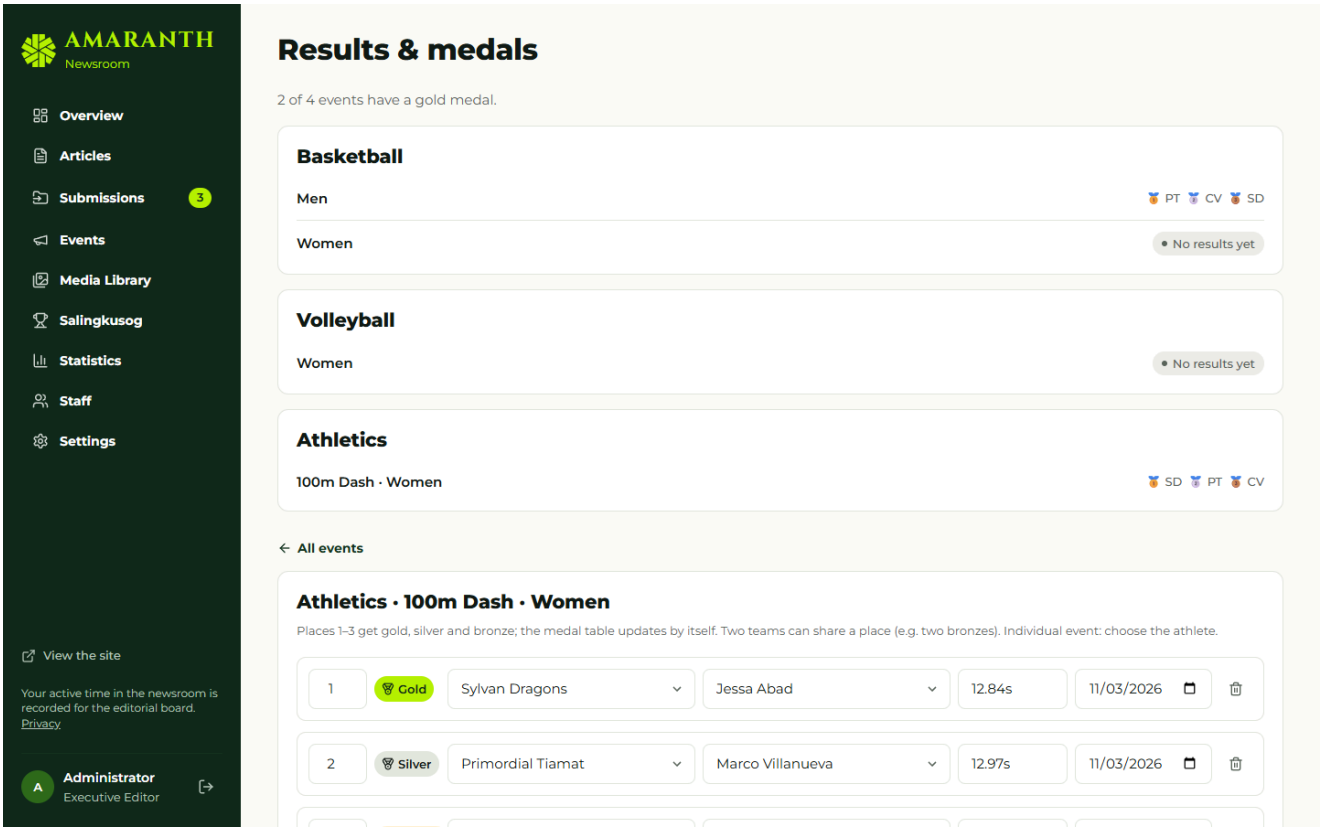
The contest scorekeeper lists every entry in performance order:

- **Timed or measured:** for individual events choose the faction's **athlete**, then type the result: `12.84` or `1:02.45` for times, `5.42` for metres. Places update as you type (fastest or highest first); exact ties share a place.
- **Judged:** type the **place** each faction gets as the judges announce it; two factions can share a place.
- Mark entries **Did not start**, **Did not finish** or **Disqualified**; they get no place.
- **Start contest** at the beginning; **Finish contest** only works when every competing entry has a result or a place.
- **Edit contest details** changes time, venue or name.

7. Results and medals

When an event's last game or contest is final:

1. The scorekeeper shows "That was the last game of … **Check and confirm the medals**". **Results & medals** marks the event **Ready to confirm**.
2. Open the event. Places 1–3 are filled in from the games: for brackets, from the final and bronze game; for round robins, from the standings; for contests, from the places.
3. Check them. For individual events choose the **athlete** for each place. You can add places, share a place (e.g. two bronzes), set the result text and the day awarded.
4. **Save results**. Only now do the medals count; the medal tally, points race and podium update for everyone.

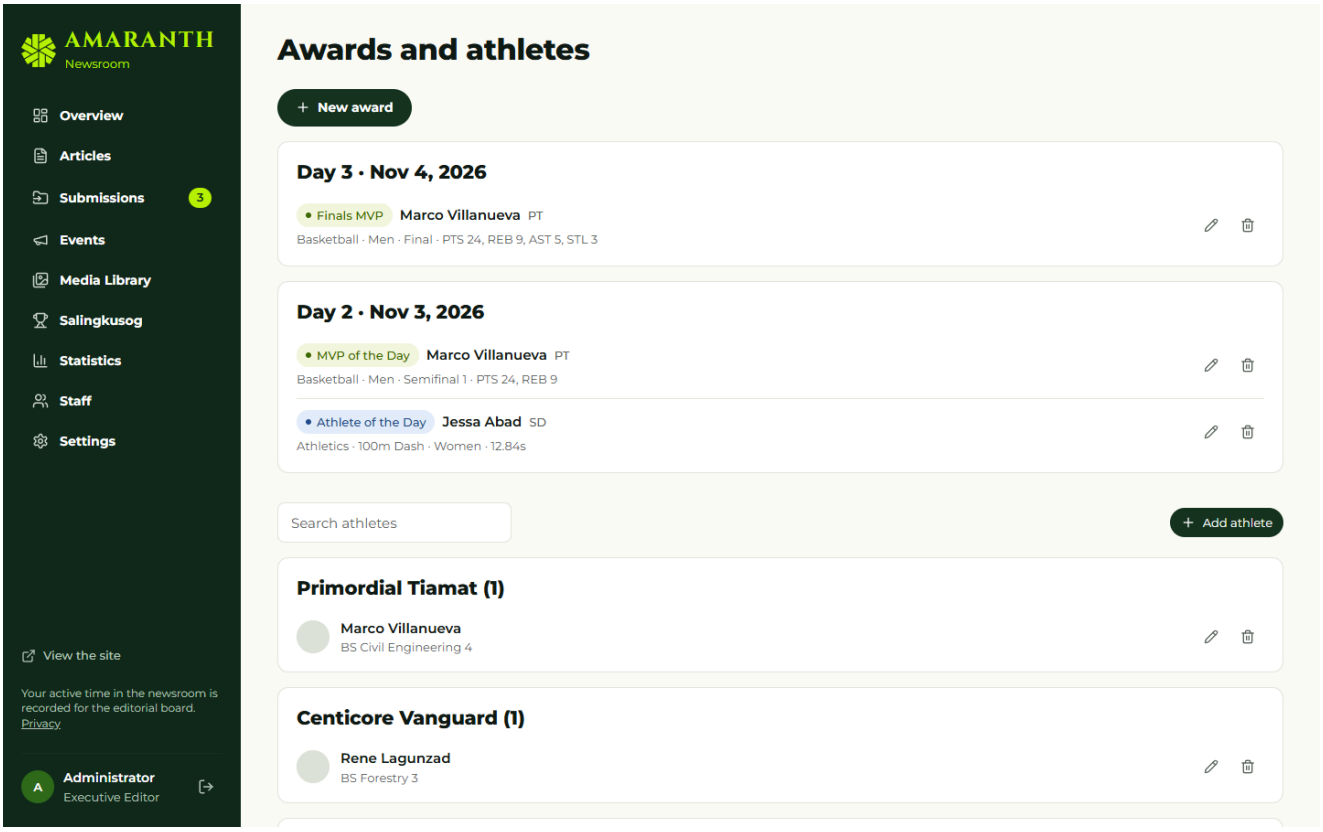


Results and medals for an event.

Medals follow the place automatically: 1 gold, 2 silver, 3 bronze.

8. Awards and social cards

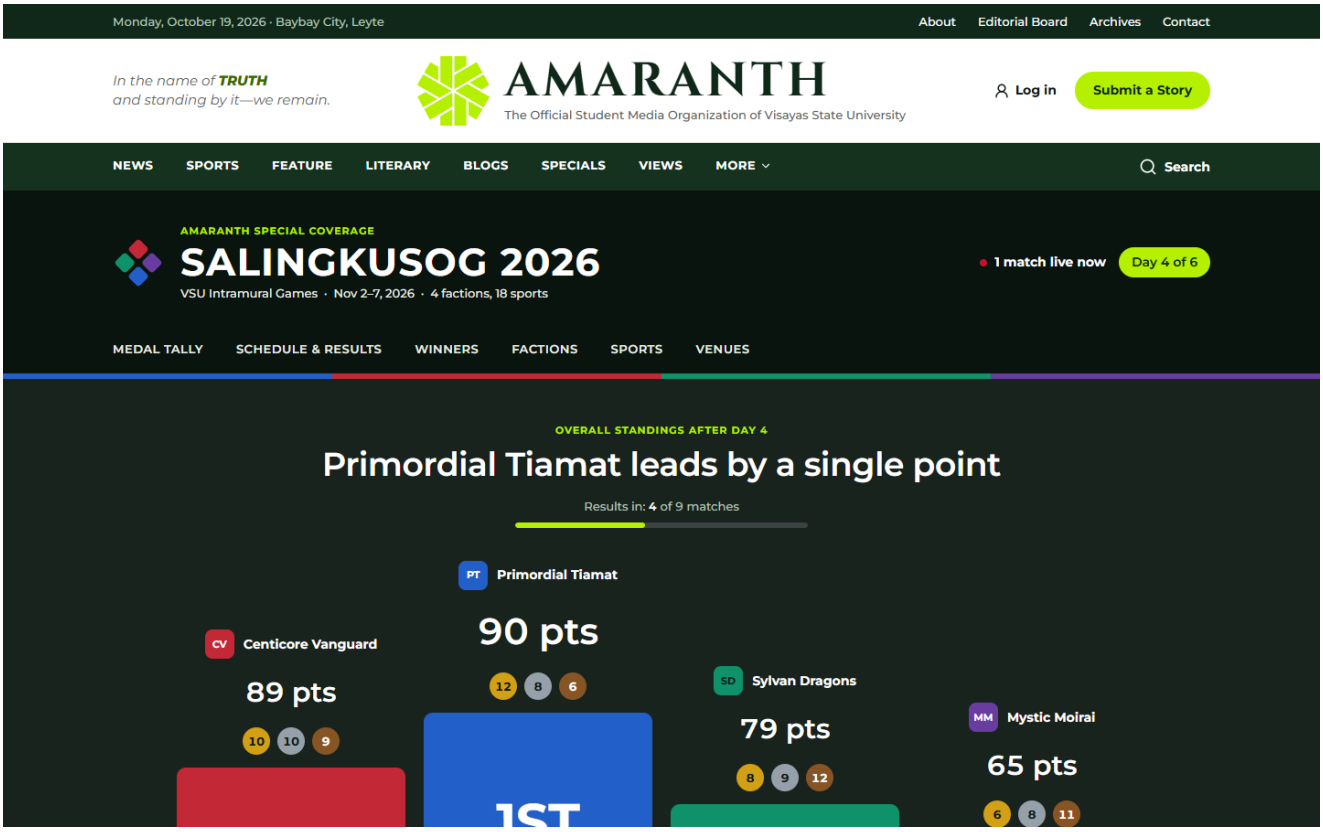
Awards — choose the award (MVP of the Day, Athlete of the Day, Finals MVP), the day, the athlete, the event and game, a headline, a stat line (e.g. “PTS 24, REB 9”) and a quote; **Save**. Download the social cards (MVP 1080×1350, Athlete of the Day 1080×1080) as PNG for posting.



Awards and athletes.

9. What readers see

Page	Content
`/salingkusog`	Podium, “Results in: X of Y matches”, medal table (gold, silver, bronze tints for 1–3), points race (1h, 3h, 6h, 12h, Day), live and up next
`/salingkusog/schedule`	Day tabs; filters by sport, faction, venue; live, later, completed
`/salingkusog/winners`	Medallists per event
`/salingkusog/sports/<sport>`	Bracket with champion, round-robin standings, or contest results
`/salingkusog/factions/<faction>`	Faction profile, medals, head-to-head
`/salingkusog/venues`	Venue guide and map



The public medal tally.

When every match is final, the page reads **Official and final medal tally** and names the champion and runners-up.

10. Troubleshooting

Message	What to do
“A knockout game needs a winner…”	Add the tiebreak to the score, then Finish
“… has already started, so its teams can’t change”	Correct the later game first, or ask an editor
“This event already has games played, so its schedule can’t be rebuilt”	Edit games one by one instead
“Enter every result first…”	Give every entry a result or mark it DNS/DNF/DQ
“Only the Salingkusog team can …”	Ask the Managing Editor to add you to the team
A game is not on the Scores tab	Check its date is within the edition and the day tab is correct

User Manual — Public Readers

Reading Amaranth, searching, commenting, submitting stories and following Salingkusog · 1.0 · 2026-10-19

Document control

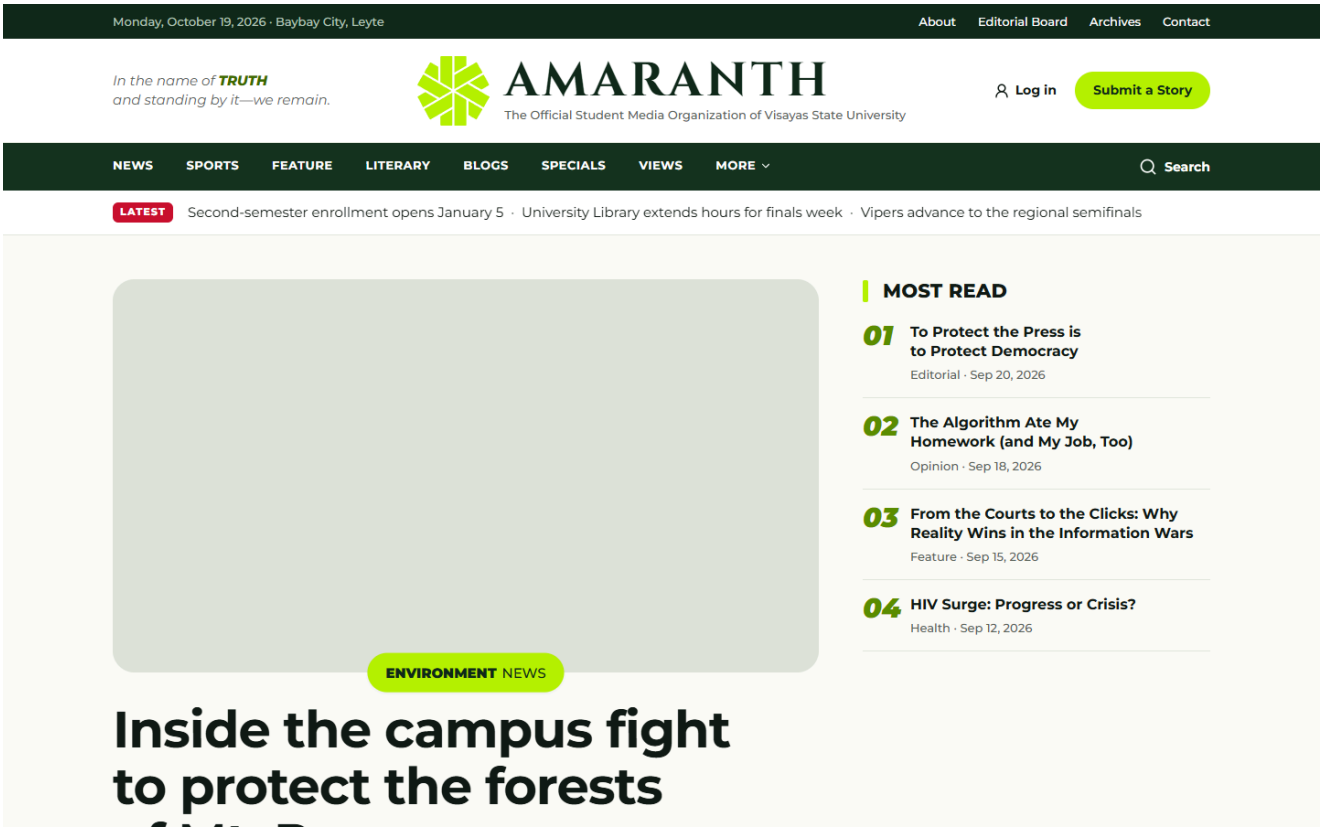
Field	Value
Title	User Manual — Public Readers
Document ID	AMR-DOC-11d
Version	1.0
Date	2026-10-19
Author	Amaranth development team [TO CONFIRM: names]
Status	Final draft

Contents

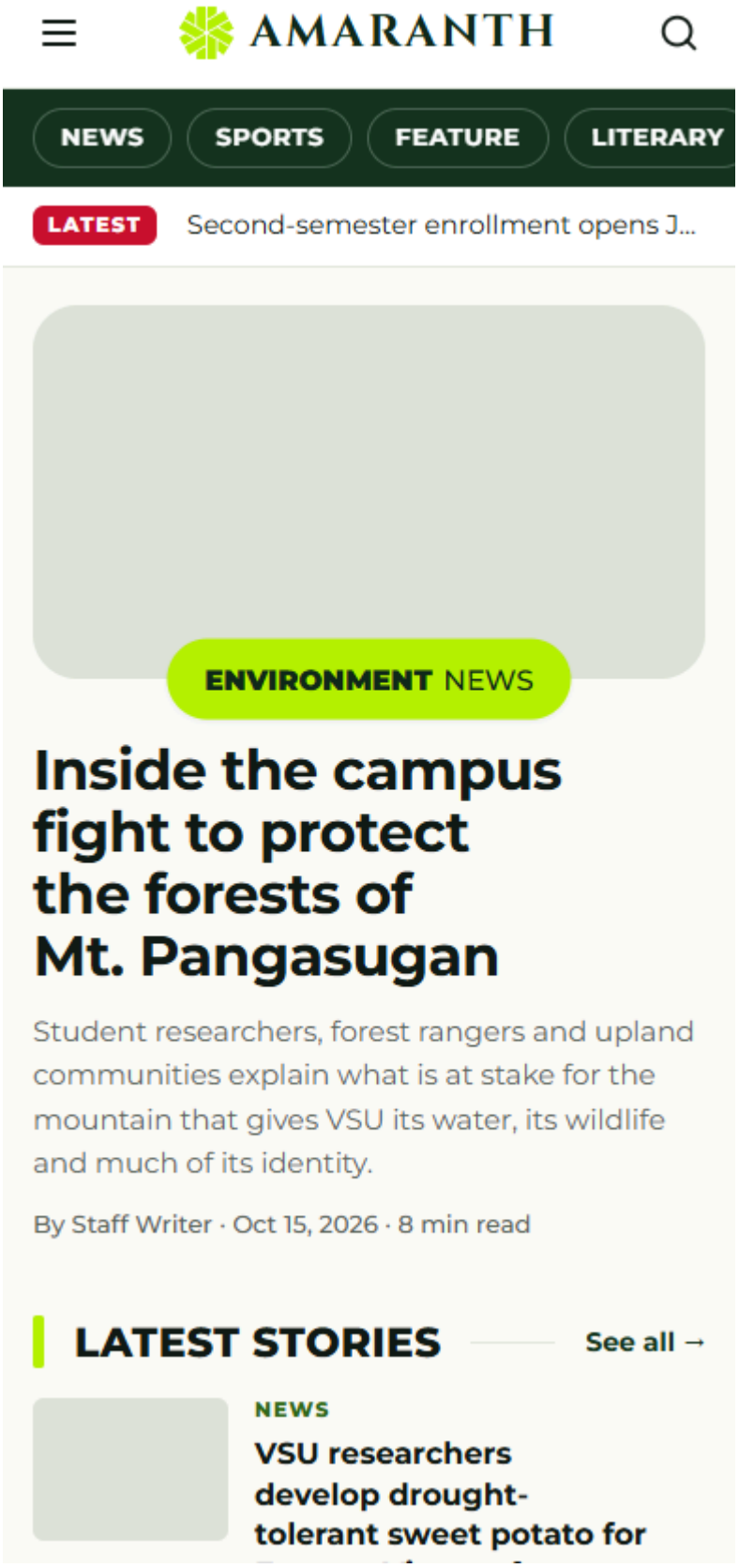
1. Finding your way
2. Reading an article
3. Search and archives
4. Commenting
5. Submitting a story or reporting an event
6. Contacting the newsroom
7. Following Salingkusog
8. Privacy

1. Finding your way

The home page shows the lead story, the latest headlines ticker, **Most read**, **Latest stories**, the **Literary Folio**, Sports and Views. The green menu lists the main sections (News, Sports, Feature, Literary, Blogs, Specials, Views); **More** opens Science & Technology, Investigative, Explainer, Bulletin and Stories. On phones, swipe the section chips under the header.



The home page on a desktop.



The home page on a phone.

Each section has its own landing page and an “all stories” list you can filter by category.

2. Reading an article

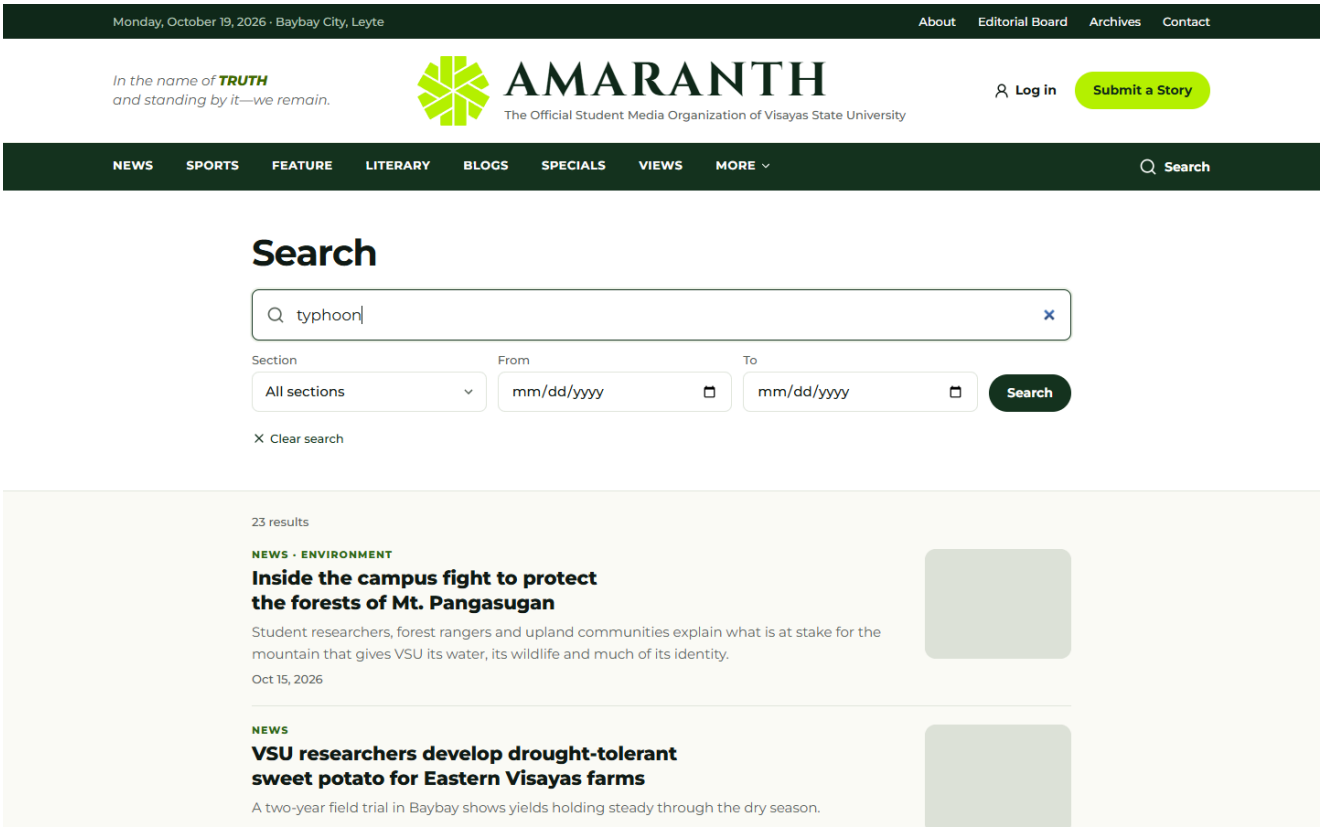
An article shows its section, headline, summary, byline, date and reading time, the cover photo, the story, tags, related stories and share buttons (Facebook, Messenger, X, and more sharing options on phones). You do not need an account.



An article page.

3. Search and archives

- **Search** (magnifier in the menu): type words; filter by section and dates. Tag links show all stories with that tag.
- **Archives**: browse stories by month and download print issues as PDF.



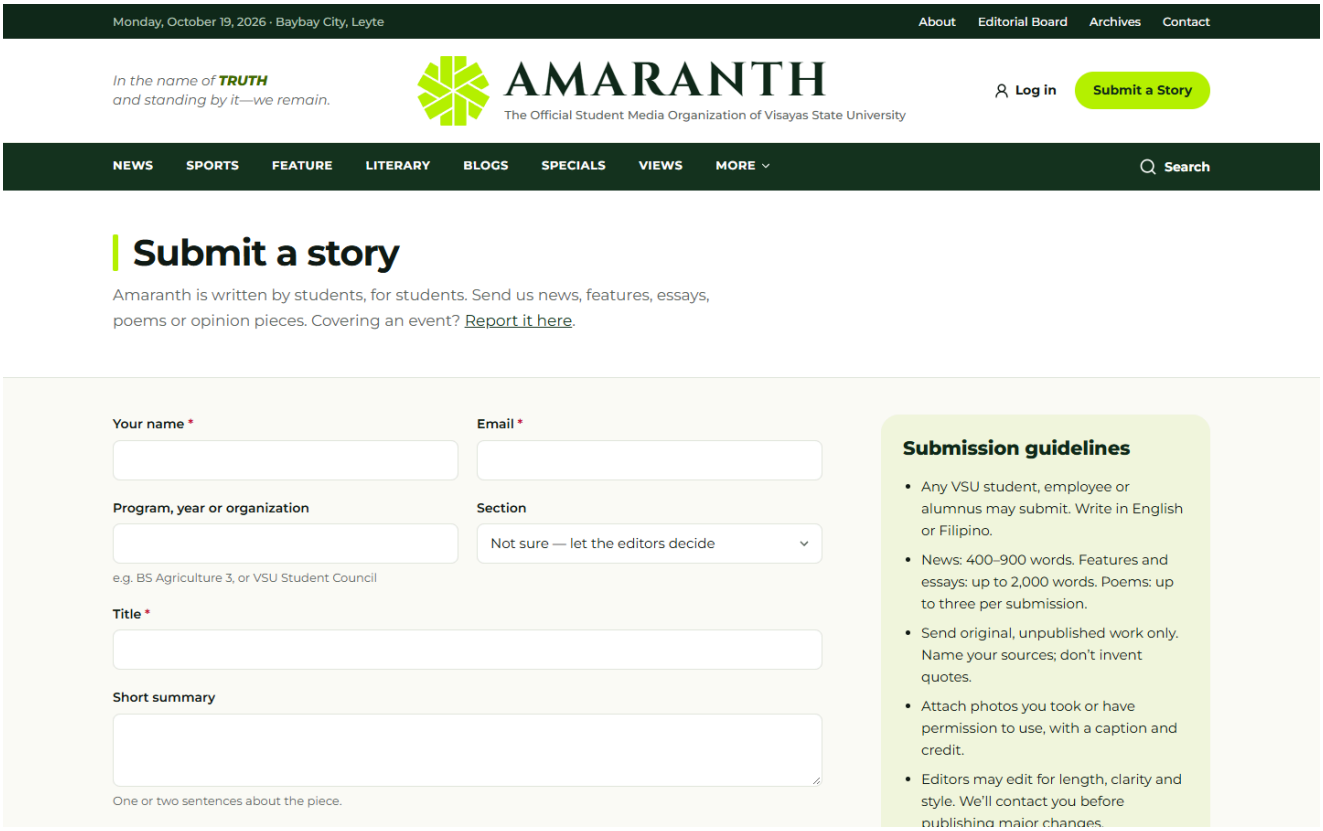
Search results.

4. Commenting

1. Under an article, write your name, email and comment, then **Post comment**.
2. A quick security check runs in the background (rarely a box to tick).
3. Comments appear **after an editor approves them**. Your email is never shown publicly.
4. You can reply to approved comments. Up to 5 comments per email every 10 minutes.

5. Submitting a story or reporting an event

- **Submit a Story** (header button or `/submit`): your name, email, affiliation, section, title, summary and text, plus up to **5 files of up to 20 MB** each (JPG, PNG, WebP, PDF, Word or text). Tick the consent box and send. Editors review every submission.
- **Report an Event** (`/submit/event`): organisation, contact, event title, date, venue and summary, with optional files.



The story submission form.

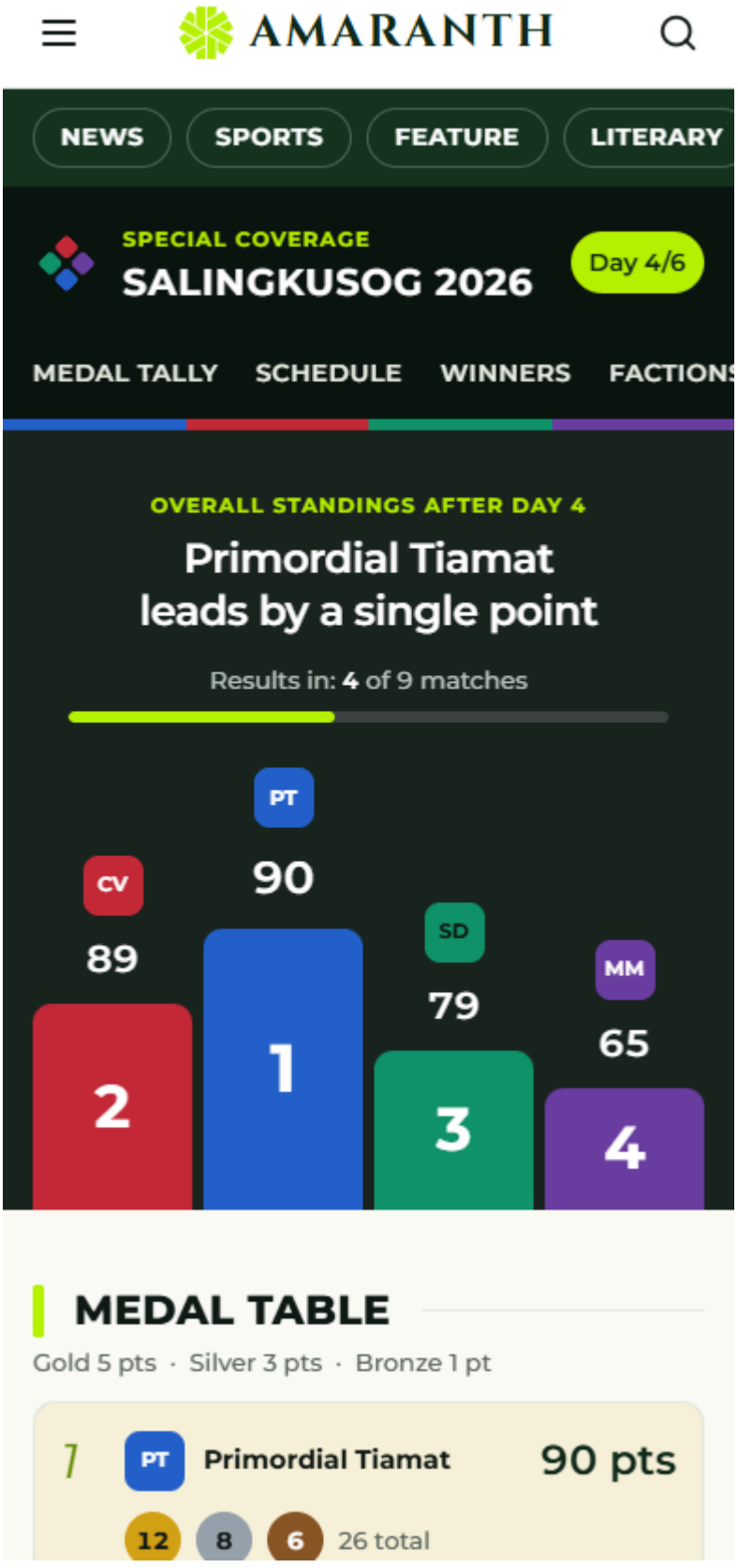
Up to 3 submissions and 3 event reports per email per hour.

6. Contacting the newsroom

`/contact` lists the desks. Choose a topic (general, editorial, submission, technical, advertising, feedback), write your message and send. Up to 3 messages per email every 10 minutes.

7. Following Salingkusog

- `/salingkusog` — the medal tally: podium, “Results in: X of Y matches”, the medal table, the points race (switch between 1h, 3h, 6h, 12h and Day), and live games. It updates by itself while open.
- **Schedule & Results** — day by day, filter by sport, faction or venue.
- **Winners**, **Factions**, **Sports** (brackets, standings, contest results), **Venues** (with map).
- When every match is final, the page says **Official and final medal tally**.



The Salingkusog medal tally on a phone.

8. Privacy

Amaranth counts visits without cookies and without storing your IP address; if your browser sends “Do Not Track”, only an anonymous pageview is counted. See `/privacy` for details and your rights under the Data Privacy Act of 2012.

Project Management Documents

Amaranth v2 — WBS, timeline, risks, resources and change log · 1.0 · 2026-10-19

Document control

Field	Value
Title	Project Management Documents
Document ID	AMR-DOC-12
Version	1.0
Date	2026-10-19
Author	Amaranth development team [TO CONFIRM: names]
Status	Final draft for defense
Sources	git history (183 commits, 82 merged branches), `docs/v2/PLAN.md`, `docs/v2/PLAN-PHASE2.md`, `docs/v2/PROGRESS.md`

Contents

1. Approach
2. Work breakdown structure (WBS)
3. Timeline
4. Milestones
5. Risk register
6. Cost and resources
7. Change log
8. Lessons learned

1. Approach

Incremental delivery, **one feature per branch**: each feature branch starts from `develop`, passes the gates (`npm run check`, and `npm run db:test` for database work), is merged with a merge commit into `develop`, and is recorded in `docs/v2/PROGRESS.md`. Plans were approved before large changes (`docs/v2/PLAN.md`, `PLAN-PHASE2.md`). Releases go to `main` through a pull request.

2. Work breakdown structure (WBS)

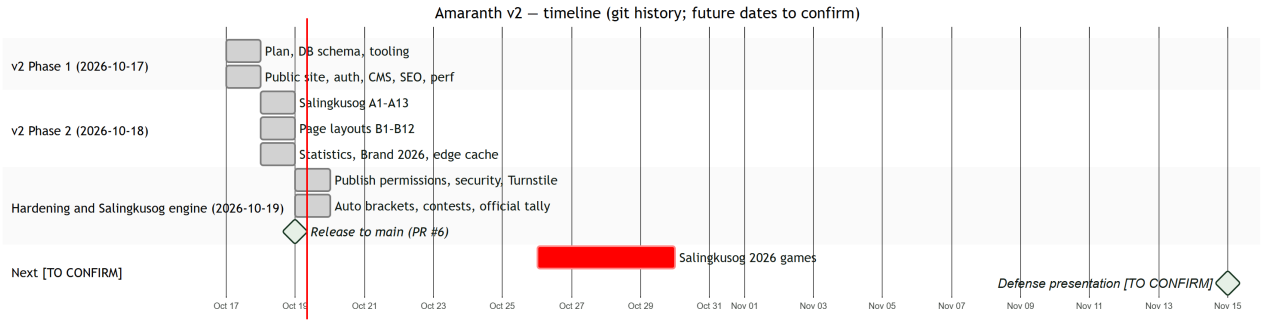
WBS	Work package	Branches (examples)	Status
1	Project setup	`docs/phase2-plan`, `feature/tooling-design-system`	Done
2	Database	`feature/db-v2-schema`, `feature/salingkusog-db`, `feature/statistics`	Done
2.1	Newsroom schema, RLS, workflow	`feature/db-v2-schema`	Done
2.2	Salingkusog schema and views	`feature/salingkusog-db`	Done
2.3	Statistics schema	`feature/statistics`	Done
3	Authentication and access	`feature/auth-staff`, `feature/publish-permissions`, `feature/salingkusog-team-access`	Done
4	Public site	`feature/public-shell-home`, `feature/section-pages`, `feature/article-page`, `feature/search-archives`, `feature/submissions-contact`, `feature/nav-more-menu`, `feature/section-*` (12)	Done
5	Newsroom (CMS)	`feature/cms-shell-dashboard`, `feature/cms-article-editor`, `feature/cms-editorial-workflow`, `feature/cms-media-library`, `feature/cms-people-board`, `feature/cms-site-settings`	Done
6	Salingkusog module	`feature/salingkusog-*` (13), `feature/sports-landing`, `feature/sk-*` (5), `feature/countdown-*`	Done

WBS	Work package	Branches (examples)	Status
6.1	Setup, import, scorekeeper	`feature/salingkusog-cms-setup`, `feature/salingkusog-scorekeeper`	Done
6.2	Public hub	`feature/salingkusog-medal-tally`, `-schedule`, `-winners`, `-factions`, `-sports-bracket`, `-match-recap`, `-venues`	Done
6.3	Automation	`feature/sk-auto-brackets`, `feature/sk-contests`, `feature/sk-official-tally`, `feature/sk-points-race-ranges`	Done
7	Branding	`feature/brand-*` (6)	Done
8	SEO, performance	`feature/seo-sharing`, `feature/perf-pass`, `perf/edge-cache-images`	Done
9	Security	`fix/security-hardening`, `feature/turnstile-captcha`	Done
10	Quality and polish	`feature/v2-polish`, `fix/*`	Done
11	Documentation	`docs/*`, this package	In progress
12	Acceptance and release	PR #6 `develop` → `main` (merged); manual tests MT-01 – MT-20	Release done; manual tests [TO CONFIRM]

3. Timeline

Period	Work	Evidence
2025-09-23 – 2025-10-12	v1 partial front end (16 commits)	git history
2026-04-12 – 2026-04-13	Interim admin and article pages (6 commits)	`feature/admin`, `feature/article-card`, `feature/article-details-page`
2026-10-17	v2 Phase 1: plan, database, tooling, public site, newsroom, SEO, performance (54 commits)	`docs/v2/PROGRESS.md`
2026-10-18	Phase 2: Salingkusog A1–A13, section layouts B1–B12, statistics, Brand 2026, edge cache (81 commits)	`docs/v2/PROGRESS.md`
2026-10-19	Publishing permissions, security, Turnstile, auto brackets, contests, official tally (25 commits so far)	`docs/v2/PROGRESS.md`
2026-10-26 – 2026-10-30	Salingkusog 2026 (from the live edition record)	`sk_editions`

Period	Work	Evidence
[TO CONFIRM]	Defense presentation	—



PM-1 — v2 timeline. Future dates are placeholders.

Note: the dense v2 dates reflect the commit history. Planning, design (Figma) and review time before and between commits are not visible in git [TO CONFIRM actual effort].

4. Milestones

ID	Milestone	Date	Status
M1	v2 plan approved	2026-10-17	Done
M2	Newsroom and public site complete (Phase 1)	2026-10-17	Done
M3	Salingkusog module and section layouts (Phase 2)	2026-10-18	Done
M4	Brand 2026 approved by the Executive Editor and applied	2026-10-18	Done
M5	Security hardening and CAPTCHA	2026-10-19	Done
M6	Automatic brackets, round robins and contests	2026-10-19	Done
M7	Release to `main` (pull request #6)	2026-10-19	Done
M8	Salingkusog 2026 run on the system	2026-10-26 – 30	Upcoming
M9	Defense	[TO CONFIRM]	Upcoming

5. Risk register

Likelihood (L) and impact (I): 1 low – 3 high; score = L × I.

ID	Risk	L	I	Score	Mitigation	Owner
RK-01	Database rule change breaks a role's access	2	3	6	278 SQL checks; re-runnable files; backup before paste	Technical lead
RK-02	Internet or Supabase outage during the games	2	3	6	Paper score sheets as fallback; enter results afterwards; status page monitoring	Salingkusog team
RK-03	Wrong official result published	2	3	6	Medals need staff confirmation; corrections recompute the tally; audit of team	Managing Editor
RK-04	Admin account compromised	1	3	3	Invite-only; deactivate quickly; MFA recommended (R-03)	Super admin
RK-05	Free-tier limits exceeded during peak traffic	2	2	4	Edge cache; CDN; monitor usage [TO CONFIRM plan limits]	Technical lead
RK-06	Knowledge loss when developers graduate	3	2	6	This documentation; developer guide; simple stack	Adviser
RK-07	No CI lets a regression reach production	2	2	4	Gates in workflow; add CI (R-05)	Technical lead
RK-08	Privacy complaint about published athlete data	1	2	2	Minimal data; consent for photos; update `/privacy` (R-10, R-12)	Executive Editor

ID	Risk	L	I	Score	Mitigation	Owner
RK-09	Spam on public forms	2	1	2	Turnstile, rate limits, moderation	Editors
RK-10	Vendor lock-in (Supabase/Vercel)	1	2	2	Plain PostgreSQL schema; standard React app	Technical lead

6. Cost and resources

Item	Plan	Cost
Supabase	Database, Auth, Storage, Realtime	[TO CONFIRM plan and monthly cost]
Vercel	Hosting, functions	[TO CONFIRM plan]
Cloudflare Turnstile	CAPTCHA	Free tier
Domain	[TO CONFIRM]	[TO CONFIRM]
Software	Open-source libraries (MIT/Apache licences)	none
People	Developer(s), editorial reviewers, adviser	[TO CONFIRM]
Hardware	Personal laptops and phones	none

7. Change log

Merged branches by date (from `git log --merges`):

Date	Merged into `develop`
2026-04-12/13	`feature/admin`, `feature/article-card`, `feature/article-details-page`
2026-10-17	`feature/db-v2-schema`, `feature/tooling-design-system`, `feature/auth-staff`, `feature/public-shell-home`, `feature/section-pages`, `feature/article-page`, `feature/search-archives`, `feature/submissions-contact`, `feature/cms-shell-dashboard`, `feature/cms-article-editor`, `feature/cms-editorial-workflow`, `feature/cms-media-library`, `feature/cms-people-board`, `feature/cms-site-settings`, `feature/seo-sharing`, `feature/v2-polish`, `feature/upgrade-tiptap-3`, `feature/upgrade-vite-7`, `feature/perf-pass`, `feature/salingkusog-db`, `feature/salingkusog-team-access`, `feature/salingkusog-cms-setup`, `fix/router-test-timeout`, `content/media-organization-copy`, `docs/phase2-plan`, `docs/release-pr`

Date	Merged into `develop`
2026-10-18	`feature/salingkusog-scorekeeper`, `-medal-tally`, `-schedule`, `-winners`, `-factions`, `-sports-bracket`, `-match-recap`, `-venues`, `-countdown`, `feature/sports-landing`, `feature/countdown-launch`, `feature/countdown-redesign`, `feature/nav-more-menu`, `feature/section-news`, `-feature`, `-literary`, `-blogs`, `-specials`, `-views`, `-science-technology`, `-investigative`, `-explainer`, `-bulletin`, `-stories`, `feature/statistics`, `feature/brand-tokens-fonts`, `-header-footer`, `-header-tagline`, `-components`, `-pages`, `-lime-hover`, `feature/literary-folio-photo`, `feature/logo-loader`, `perf/edge-cache-images`, `fix/nested-dialog-close`, `fix/server-esm-imports`, `fix/sk-more-categories`, `docs/pr-track-a`, `docs/stats-vercel-analytics`, `docs/track-b-wrapup`
2026-10-19	`feature/sk-chart-animations`, `feature/sk-points-race-ranges`, `feature/publish-permissions`, `fix/security-hardening`, `feature/sk-official-tally`, `feature/turnstile-captcha`, `feature/sk-auto-brackets`, `feature/sk-contests`, `fix/build-schedule-individual`, `fix/pre-release-polish`, `docs/pr-brand-2026`, `docs/pr-auto-brackets`

Releases to `main`: pull requests #1, #2, #4, #5 and #6 (merge commits on `main`); #6 released everything up to commit `d7f6f34`.

8. Lessons learned

- Putting rules in the database made late changes (Business Manager role, publishing access) safe: one helper, tests, done.
- Re-runnable SQL removed deployment fear but requires discipline about order when access is narrowed.
- Fixture screens (`/dev`) let design and accessibility be checked without real data.
- A short pre-release review found real gaps (game form, schedule dates) that tests had not.

Limitations, Known Issues and Future Work

Amaranth v2 — honest list of what is missing and a prioritised roadmap · 1.0 · 2026-10-19

Document control

Field	Value
Title	Limitations, Known Issues and Future Work
Document ID	AMR-DOC-13
Version	1.0
Date	2026-10-19
Author	Amaranth development team [TO CONFIRM: names]
Status	Final draft for defense

Contents

1. Limitations by design
2. Known issues
3. Security and privacy open items
4. Prioritised roadmap

1. Limitations by design

ID	Limitation	Why	Effect
LIM-01	Readers have no accounts	Simpler privacy and support (ADR-007)	No saved articles, newsletters or personalisation
LIM-02	English interface only	Scope	Filipino/Waray readers read English UI
LIM-03	Straight finals only for timed events; no heats	Chosen for Salingkusog 2026	Large fields need games entered by hand
LIM-04	One entry per faction in contests	Chosen for Salingkusog 2026	A faction cannot win two medals in one event
LIM-05	Judged events record places, not judges' score sheets	Chosen by the user	No breakdown of scores per judge
LIM-06	Medals need a person to confirm	Accuracy of official results (ADR-013)	One extra step per event
LIM-07	Needs an internet connection	Thick client on managed services	Scorekeeping fails without signal
LIM-08	No migration tool; SQL pasted by hand	Simplicity (ADR-002)	Manual deploy steps
LIM-09	Edge-cached public data can be up to a minute old	Performance (ADR-010)	Live scores are not cached, other lists may lag
LIM-10	v1 content not migrated	Fresh schema chosen	Old articles live only on the v1 site [TO CONFIRM]

2. Known issues

ID	Issue	Severity	Workaround	Evidence
KI-01	No continuous integration	Medium	Run `npm run check` and `npm run db:test` before every merge	`.github/` has no workflows
KI-02	`docs/v2/DB.md` partly out of date (intake upload policy, test counts, Realtime tables)	Low	Use AMR-DOC-04 and AMR-DOC-09	`docs/v2/DB.md`
KI-03	Content-Security-Policy in report-only mode	Medium	Check console; switch to enforcing	`vercel.json`
KI-04	Database types written by hand	Low	Update with each schema change	`src/lib/database.types.ts`

ID	Issue	Severity	Workaround	Evidence
KI-05	Games entered before “Build schedule” do not advance automatically	Medium	Fill later rounds by hand or rebuild events with no games played	ADR-012
KI-06	A contest session opened at `/salingkusog/match/:id` shows an empty two-sided recap	Low	Cards do not link there; use the sport page	`RecapView.tsx`
KI-07	Faction profiles list only two-sided games, not contest results	Low	Medals still counted	`factions.ts`
KI-08	No email notifications for workflow steps	Medium	Check the dashboard and “Needs review”	no mail service
KI-09	No self-service profile screen for staff	Low	Editors update contributors	FR-008
KI-10	Development fixture for the editorial board contains real names and photos	Low	Replace with fictional data	`src/dev/DevRoutes.tsx`
KI-11	Old Katalon project in ignored `Amaranth/` folder	Low	Delete or archive	`.gitignore`
KI-12	No browser end-to-end tests	Medium	Manual cases MT-01 – MT-20	AMR-DOC-08
KI-13	No history of score edits (only last editor)	Medium	Ask the team; check `updated_by`	`sk_matches.updated_by`

3. Security and privacy open items

From AMR-DOC-06 §12: R-01 enforce CSP; R-02 confirm sign-ups disabled; R-03 MFA for admins; R-04 leaked-password protection; R-05 CI; R-06 generated types; R-07 score history; R-08 orphan intake files; R-09 retention for forms and comments; R-10 privacy page section on athletes; R-11 privacy contact/DPO; R-12 consent for athlete photos.

4. Prioritised roadmap

Priority	Item	Addresses	Effort
P1 (before/at the games)	Update `/privacy` for athletes; consent process for photos	R-10, R-12	S
P1	Confirm Supabase Auth settings; enable MFA for SA and EE	R-02, R-03, R-04	S
P1	Run manual tests MT-01 – MT-20 and record results	KI-12	S

Priority	Item	Addresses	Effort
P2	GitHub Actions running `npm run check` and `npm run db:test`	KI-01, R-05	S
P2	Enforce CSP after a clean preview	KI-03, R-01	S
P2	Score and result change history table with a viewer	KI-13, R-07	M
P2	Retention jobs for comments, submissions, events, contact messages	R-09	S
P3	Playwright end-to-end tests for the main flows	KI-12	M
P3	Generated database types (Supabase CLI)	KI-04, R-06	S
P3	Email or in-app notifications (submitted, returned, published)	KI-08	M
P3	Self-service profile page	KI-09	S
P3	Contest results on faction profiles; contest-aware recap page	KI-06, KI-07	S
P4	Heats → finals and multiple entries per faction	LIM-03, LIM-04	L
P4	Judges’ score sheets with averaging	LIM-05	M
P4	Offline scorekeeping with later sync	LIM-07	L
P4	Filipino/Waray interface	LIM-02	M
P4	Import of v1 articles	LIM-10	M

Effort: S ≈ under a day, M ≈ a few days, L ≈ a week or more for one developer [estimates].

Glossary and Appendices

Amaranth v2 — acronyms, newsroom terms and Salingkusog terms · 1.0 · 2026-10-19

Document control

Field	Value
Title	Glossary and Appendices
Document ID	AMR-DOC-14
Version	1.0
Date	2026-10-19
Author	Amaranth development team [TO CONFIRM: names]
Status	Final draft for defense

Contents

1. Acronyms
2. Newsroom terms
3. Salingkusog terms
4. Roles
5. Document index

1. Acronyms

Acronym	Meaning
ADR	Architecture decision record
API	Application programming interface
CDN	Content delivery network
CI	Continuous integration
CMS	Content management system (the newsroom)
CSP	Content-Security-Policy (browser security header)
CSV	Comma-separated values (spreadsheet file)
DFD	Data flow diagram
DNS / DNF / DQ	Did not start / did not finish / disqualified
DNT / GPC	Do Not Track / Global Privacy Control (browser privacy signals)
DPO	Data Protection Officer
ERD	Entity–relationship diagram
FR / NFR	Functional / non-functional requirement
HMAC	Hash-based message authentication code (used to sign form tickets)
HSTS	HTTP Strict Transport Security
JWT	JSON Web Token (the signed-in session)
MFA	Multi-factor authentication
NPC	National Privacy Commission (Philippines)
OG	Open Graph (link-preview tags)
RA 10173	Republic Act No. 10173, the Data Privacy Act of 2012
RBAC	Role-based access control
RLS	Row Level Security (PostgreSQL)
RPC	Remote procedure call (a database function called by the app)
RTM	Requirements traceability matrix
SDD / SRS	Software design document / software requirements specification
SPA	Single-page application
SQL	Structured Query Language
STRIDE	Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
UC	Use case
VSU	Visayas State University
WBS	Work breakdown structure

Acronym	Meaning
WCAG	Web Content Accessibility Guidelines

2. Newsroom terms

Term	Meaning
Article status	draft, submitted, in review, scheduled, published, archived, rejected
Live article	Published or scheduled with a publish time in the past
Publisher	Anyone allowed to publish or schedule (SA, EE, ME, BM, or granted)
Section / category	Site area (e.g. News) / subdivision of a section (e.g. Campus News)
Kicker	Short label above a headline
Dek	Subtitle or standfirst below the headline
Byline	Author credit, built from contributors or overridden
Contributor	A public byline record (staff, student, guest, faculty, alumni)
Revision	A saved snapshot of an article (autosave, manual, status)
Intake	Story submissions and event reports from the public
Front page slots	The lead story and literary folio pick chosen by managers
Ticker	The latest-headlines strip at the top of the site
Folio	The Literary Folio block on the home page
Print issue	An uploaded PDF of a printed edition
Turnstile	Cloudflare’s CAPTCHA used on public forms
Edge cache	Short-lived copies of public data kept by Vercel’s CDN

3. Salingkusog terms

Term	Meaning
Salingkusog	VSU’s annual intramural games
Edition	One year of the games (dates, medal points, published flag)
Faction	A competing team in an edition (name, colour, logo)
Sport	A sport in an edition, with a category (ball games, athletics, aquatics, dance & cheer…)
Event	A competition inside a sport: division (men, women, mixed, open) and format
Format	Bracket (knockout), round robin, round robin + final, timed / measured, judged
Game (match)	One scheduled competition between two factions

Term	Meaning
Contest	One session in which every faction competes at once (dance, swimming, athletics…)
Entry	One faction's (or its athlete's) line-up slot and result in a contest
Seed	A faction's ranking used to build a bracket or round robin (seed 1 strongest)
Bye	A free pass to the next round when the field is not 2, 4, 8 or 16
Bracket	Single-elimination draw from the first round to the final
Bronze game	Match between the semifinal losers for third place
Round robin	Every faction plays every other once
Source	Where a generated game's side comes from: a seed, the winner or loser of a game, or a standings rank
Placement	An official place (1st, 2nd, 3rd …) in an event; 1–3 carry gold, silver and bronze
Medal points	Points per medal for the overall standing (default 5 / 3 / 1)
Medal table	Computed table of medals, points and rank per faction
Points race	Cumulative points over time (by day or 1/3/6/12-hour steps)
Head-to-head	Wins and losses between two factions
Official and final medal tally	The tally once every match is final
Salingkusog team	Staff allowed to enter Salingkusog data
Scorekeeper	The live-scoring screen (or the person using it)
Awards	MVP of the Day, Athlete of the Day, Finals MVP

4. Roles

Role value	Display name	Short
`super_admin`	Super admin	SA
`editor_in_chief`	Executive Editor	EE
`managing_editor`	Managing editor	ME
`business_manager`	Business Manager	BM
`section_editor`	Section editor	SE
`staff_writer`	Staff writer	SW
`contributor`	Contributor	CO
(`sk_team` member)	Salingkusog team member	SKT

5. Document index

ID	Document	File
AMR-DOC-00	Repository Audit Report	`00-audit-report.md`
AMR-DOC-01	Project Overview and Vision	`01-project-overview.md`
AMR-DOC-02	Software Requirements Specification	`02-srs.md`
AMR-DOC-03	Software Design Document and Architecture	`03-architecture-sdd.md`
AMR-DOC-04	Database Design	`04-database-design.md`
AMR-DOC-05	UML and Process Diagrams	`05-diagrams.md`
AMR-DOC-06	Security and Privacy	`06-security-privacy.md`
AMR-DOC-07	API and Integration Reference	`07-api-reference.md`
AMR-DOC-08	Test Documentation	`08-test-documentation.md`
AMR-DOC-09	Deployment and Operations Runbook	`09-deployment-runbook.md`
AMR-DOC-10	Maintenance and Developer Guide	`10-developer-guide.md`
AMR-DOC-11a–d	User Manuals	`11a` … `11d`
AMR-DOC-12	Project Management	`12-project-management.md`
AMR-DOC-13	Limitations and Future Work	`13-limitations-future-work.md`
AMR-DOC-14	Glossary and Appendices	`14-glossary-appendices.md`
AMR-DOC-20 – 23	Defense support	`20-demo-script.md` … `23-screenshot-list.md`

Defense Demo Script

Minute-by-minute live demonstration with test accounts, seed data and a fallback plan · 1.0 · 2026-10-19

Document control

Field	Value
Title	Defense Demo Script
Document ID	AMR-DOC-20
Version	1.0
Date	2026-10-19
Author	Amaranth development team [TO CONFIRM: names]
Status	Final draft

Contents

1. Goal and setup
2. Test accounts (fictional)
3. Demo seed data
4. Minute-by-minute script (15 minutes)
5. Fallback plan
6. Pre-demo checklist

1. Goal and setup

Show, in 15 minutes, that (1) the newsroom enforces editorial roles in the database, (2) readers get a fast, accessible site with safe public forms, and (3) Salingkusog results flow from a score tap to the official medal table automatically.

Use a separate demo environment, never production with real people's data: a second Supabase project (or the production project with a **test edition** that stays unpublished) and a Vercel preview, or `npm run dev` locally [TO CONFIRM which]. Two screens: laptop (projector) and a phone (scorekeeper).

2. Test accounts (fictional)

Create these in the demo project before the defense (Staff → Invites, then accept the invitations). The passwords below are **placeholders**: choose your own and keep them off the slides.

Account	Role	Placeholder password
`demo.ee@example.test`	Executive Editor	`<set before demo>`
`demo.me@example.test`	Managing Editor	`<set before demo>`
`demo.editor@example.test`	Section editor (News)	`<set before demo>`
`demo.writer@example.test`	Staff writer	`<set before demo>`
`demo.scorer@example.test`	Staff writer + Salingkusog team	`<set before demo>`

Use separate browser profiles (or one normal and one private window) to switch roles quickly.

3. Demo seed data

All fictional:

1. Articles: two published News articles with cover images (any campus photos you own), one draft by the writer.
2. Salingkusog test edition "Demo Games" (unpublished until the demo, dates covering the demo day), factions "Faction A–D" with distinct colours, sports Basketball (Men, format Bracket) and Swimming (50m Free, Timed, individual, unit "s"), one athlete per faction for swimming.
3. Build schedule for Basketball Men: 4 factions, single elimination + bronze. Create contest for Swimming.
4. Pre-finish Semifinal 1 so the final already shows "Faction A vs Winner of Semifinal 2".

4. Minute-by-minute script (15 minutes)

Time	Who / screen	Action	Point to make
0:00–1:00	Laptop, public home	Scroll the home page; open a section; open an article	Brand 2026, mobile-first layout, accessibility 100

Time	Who / screen	Action	Point to make
1:00–2:00	Phone, public home	Same page on the phone; search for a word	Responsive; search is full-text in the database
2:00–3:30	Laptop, writer	Sign in as writer; create a draft; show autosave and **Submit for review**; show there is no Publish button	Writers cannot publish
3:30–4:30	Laptop, writer (DevTools optional)	Try to publish anyway via the API console snippet [optional]	“You cannot move this article…” comes from the **database**
4:30–6:00	Laptop, section editor	**Needs review** → **Start review** → **Return to draft** with a note; show Notes & activity	Workflow history and notes
6:00–7:30	Laptop, Managing Editor	Publish an article; Staff → Accounts → turn on **Can publish** for the writer	Granular publishing access, audited
7:30–8:30	Laptop, public	Submit the contact form (Turnstile runs invisibly); show it in Settings → Inbox	Safe public forms
8:30–11:00	Phone, scorer + laptop public tally	Open Semifinal 2 on the phone: **Start game**, tap scores, **Finish game**; laptop shows the final filling in and the bronze game	Live updates and automatic brackets
11:00–12:30	Phone + laptop	Swimming contest: enter four times, mark one DNF, **Finish contest**	Contest ranking, ties, DNF
12:30–13:30	Laptop, Managing Editor	Results & medals → **Ready to confirm** → **Save results**; switch to `/salingkusog`	Staff-confirmed medals; computed medal table
13:30–14:30	Laptop	Statistics screen (or screenshot); `/privacy`	First-party, cookie-less statistics
14:30–15:00	—	Summary slide	Database-enforced roles, live results, privacy

Optional API snippet for 3:30 (run in the browser console of the writer’s newsroom tab; it only works for signed-in staff):

```
const { getSupabase } = await import('/src/lib/supabase.ts'); // dev server only
await getSupabase().rpc('transition_article', { p_article_id: '<draft id>', p_to: 'published' });
```

5. Fallback plan

Failure	Fallback
No internet / Supabase down	Run `npm run dev` and use the `/dev` fixture screens (no database needed): `/dev/admin/editor`, `/dev/admin/salingkusog-scorekeeper` (buttons work), `/dev/salingkusog/sports/auto-bracket`, `/dev/salingkusog/final`
Laptop fails	Present the screenshots in `docs/screenshots/` and the slide deck (PDF copy on a USB drive)
Sign-in problem	Second browser profile already signed in for each role
Realtime slow	Refresh the tally; explain the 60-second polling fallback
Turnstile blocks the form	Show the 403 handling and the Inbox with a pre-sent message

6. Pre-demo checklist

- ☐ Demo accounts created and tested; passwords not on slides.
- ☐ Demo edition and articles seeded; semifinal 1 finished; medals not yet confirmed.
- ☐ Phone charged, signed in as scorer, brightness up, notifications off.
- ☐ Laptop: two browser profiles, projector resolution checked, zoom 110%.
- ☐ Offline fallback tested (`npm run dev`, `/dev` screens).
- ☐ After the demo: unpublish/delete the demo edition and deactivate demo accounts.

Defense Q&A Preparation

55 likely panel questions with honest, code-grounded answers · 1.0 · 2026-10-19

Document control

Field	Value
Title	Defense Q&A Preparation
Document ID	AMR-DOC-21
Version	1.0
Date	2026-10-19
Author	Amaranth development team [TO CONFIRM: names]
Status	Final draft

Contents

1. Problem, scope and value
2. Architecture and stack
3. Database and data model
4. Security
5. Privacy
6. Salingkusog module
7. Testing and quality
8. Performance and scalability
9. Operations and failure
10. Process, limitations and future

1. Problem, scope and value

Q1. What problem does Amaranth v2 solve? The v1 site was a partial front end with no newsroom workflow, no tests, slow mobile pages (Lighthouse performance 56, accessibility 91) and security gaps (exposed helper functions, open storage deletes). v2 runs the whole editorial process and Salingkusog results inside one system, with permissions enforced by the database (AMR-DOC-01 §2).

Q2. Who are the users? Seven staff roles (Super admin, Executive Editor, Managing Editor, Business Manager, Section editor, Staff writer, Contributor), a Salingkusog team group, and anonymous readers (AMR-DOC-02 §3).

Q3. What is out of scope? Reader accounts, native apps, email notifications, ads, athlete eligibility, heats and multiple entries per faction, and v1 content migration (AMR-DOC-01 §7).

Q4. How do you measure success? RLS on all 50 tables; all 255 unit tests and 278 SQL checks passing; accessibility 100 on key pages; WCAG AA contrast; no secrets in the browser; standings computed automatically (AMR-DOC-01 §8). The live-event criterion is confirmed only after the 2026 games.

Q5. Why not use WordPress or another existing CMS? We needed the Salingkusog engine (brackets, contests, live scores, computed medal table) tightly integrated with the newsroom, database-enforced roles specific to the publication (e.g. the Business Manager and per-person publishing access), and a free managed stack. A generic CMS would need extensive plugins and still leave the sports logic custom.

2. Architecture and stack

Q6. Describe the architecture in one sentence. A React single-page app that talks directly to Supabase (PostgreSQL with RLS, Auth, Storage, Realtime), plus a thin Vercel layer for secrets, caching and SEO (AMR-DOC-03 §3).

Q7. Why React + Vite instead of Next.js? The app is client-rendered; we did not need server rendering. Vite gives fast builds. Link previews for social media are handled by a small middleware that injects Open Graph tags for crawlers only (ADR-003).

Q8. Why Supabase? Relational data fits a newsroom and sports results; RLS lets the database enforce permissions; Auth, Storage and Realtime come with it; it has a free tier and plain PostgreSQL avoids lock-in to a proprietary data model.

Q9. Where is the business logic? In the database: RLS policies, triggers (workflow, advancement, guards) and RPCs. The UI mirrors rules only to hide buttons (`permissions.ts`). This means the API cannot be used to bypass rules (ADR-001).

Q10. Isn’t putting logic in SQL hard to maintain? It needs SQL skills, but rules sit in one place and are covered by 278 SQL checks run against a throwaway database, applying every file twice. Helpers keep policies short; changing who may publish means editing one function.

Q11. What runs on the server? Six Vercel functions — `/api/submit` (forms with CAPTCHA), `/api/e` (statistics ingest), `/api/r` (edge cache), sitemap, RSS, robots — and the crawler middleware (AMR-DOC-07 §2).

Q12. How does real-time work? Supabase Realtime streams changes of `sk_matches`, `sk_placements` and `sk_entries`; public pages subscribe per edition and refetch affected queries; a 60-second poll is the fallback (SEQ-4).

Q13. How do you handle state in the front end? TanStack Query caches server data per feature; mutations invalidate query keys; Realtime events also invalidate keys.

3. Database and data model

Q14. How many tables and why so many? 50 tables: 29 newsroom, 12 Salingkusog, 9 statistics. Many are link tables (article authors, tags, media), private split tables (commenter emails) or daily rollups (AMR-DOC-04 §1).

Q15. Is the schema normalised? Transactional tables are in 3NF. Deliberate denormalisations (article section, edition id on Salingkusog rows, reading time, search vector, view counts, rollups) are maintained by triggers or generated columns, never by the app (AMR-DOC-04 §4).

Q16. How is the medal table computed? The view `sk_medal_table` counts gold, silver and bronze placements per faction, multiplies by the edition's points (default 5/3/1), and ranks by points, then golds, then silvers. Nothing is typed by hand (AMR-DOC-04 §9).

Q17. What if two factions tie on points? Rank uses gold count, then silver; if still equal, `rank()` gives them the same rank.

Q18. How do you prevent a wrong medal for a place? A check constraint ties medal to place (1 gold, 2 silver, 3 bronze), and `sk_set_placements` sets the medal from the place.

Q19. How are database changes deployed without a migration tool? Re-runnable SQL files (`if not exists`, `create or replace`, `drop … if exists`) pasted in order; the test runner applies them twice to prove idempotency (AMR-DOC-09 §5).

Q20. Why hand-written TypeScript types? No Supabase CLI/Docker at the start; it is a known debt with a plan to generate them (ADR-015, KI-04).

4. Security

Q21. How do you stop a writer from publishing by calling the API directly? The `app.article_before_write` trigger checks `app.can_publish_articles()` on every status change; the API returns "You cannot move this article…" (SQL test in `workflow.test.sql`).

Q22. Can someone make themselves an admin? No. Only SA/EE change roles; only SA grants SA; the last active SA cannot be removed (`profiles_guard`); new accounts get roles only from `staff_invites`.

Q23. What stops spam on the forms? Cloudflare Turnstile verified on the server, per-email rate limits, a honeypot field, moderation of comments, and the form RPCs are callable only by the server (ADR-011).

Q24. Could someone fill your storage with uploads? Uploads to the private intake bucket happen only through one-time signed links issued after the CAPTCHA, at most 5 files of 20 MB, with bucket type/size limits.

Q25. How do you prevent XSS? Rich text is sanitised with DOMPurify, iframes allowed only from YouTube, Vimeo, Facebook and Spotify; comments and form text are rendered as plain text; meta tags are escaped.

Q26. Where are secrets kept? Only the anon key and Turnstile site key are public. The service-role key and Turnstile secret live in Vercel environment variables and are used only by `/api` functions.

Q27. Do you use HTTPS and security headers? Yes: HSTS, X-Frame-Options, nosniff, Referrer-Policy, Permissions-Policy and a CSP (currently report-only; enforcing is on the roadmap).

Q28. What is your biggest remaining security risk? Admin account takeover, because MFA is not yet required. Mitigations: invite-only accounts, quick deactivation; recommendation R-03 is to enforce MFA for SA and EE.

Q29. Is there an audit trail? Role, activation, publishing-access and Salingkusog team changes go to `audit_log`; article status changes and notes to `article_activity`; the last scorer of each game is `updated_by`. A full score-edit history is future work.

5. Privacy

Q30. What personal data do you collect about athletes? Only name, program (course/year) and an optional photo. No birth dates, student numbers or eligibility documents are stored (`sk_athletes`).

Q31. How does the system comply with RA 10173? Transparency (`/privacy`), proportionality (minimal fields), security (RLS, private tables and bucket, HTTPS), and data-subject rights by request. Open items: a privacy section on athletes, photo consent, retention for forms, a named DPO contact (AMR-DOC-06 §11).

Q32. Do you track readers? Only first-party, cookie-less statistics: a visitor id hashed with a daily salt that is deleted after two days, no IP stored, DNT/GPC respected; raw events deleted after 13 months.

Q33. Who can see commenter emails? Editors only; emails are in a separate table (`article_comment_private`) with editor-only RLS.

Q34. Why do you also use Vercel Web Analytics? It was in v1 and kept by the team's choice; the privacy page mentions it (`docs/v2/STATS.md`).

6. Salingkusog module

Q35. How does the bracket fill itself in? Build schedule creates games whose sides have sources (seed, winner of, loser of, rank). After each game change a trigger calls `app.sk_resolve`, which fills the next games; it never changes a game that has started (ADR-012).

Q36. What if a score is corrected after the next game started? The database refuses the correction with "… has already started, so its teams can't change", so the bracket cannot become inconsistent.

Q37. Can a knockout game end in a tie? No; the trigger `app.sk_knockout_needs_winner` and the scorekeeper both refuse it.

Q38. How are round-robin standings ranked? Wins, then point difference, then head-to-head among tied teams, then points scored, then seed — identical in SQL (`app.sk_rr_table`) and TypeScript.

Q39. How do you handle dance or swimming where all factions compete together? A contest: one session (`sk_matches.is_contest`) with entries per faction (`sk_entries`); times rank automatically (unit "s" lowest wins), judged events take places; DNS/DNF/DQ get no place (ADR-014).

Q40. Why do medals need manual confirmation? Official results should be checked by a person. The system suggests the podium and shows "Ready to confirm", but nothing counts until staff save it (ADR-013).

Q41. How many sports and events does the 2026 edition have? From the live data on 2026-10-19: 4 factions, 27 sports, 74 events, 21 venues, 169 games and sessions.

Q42. Can the scoring be done on phones? Yes; the scorekeeper is designed for one-handed use with large buttons and a sticky action bar; changes save after a short pause and reach readers in seconds.

7. Testing and quality

Q43. How did you test the system? 255 unit/component tests (Vitest), 278 SQL checks against a real PostgreSQL, lint, strict type-check, build, a contrast script, and an accessibility/overflow sweep of 53 screens; results in AMR-DOC-08.

Q44. Do you have end-to-end tests? Not yet; 20 manual cases are written and scheduled. Playwright E2E is on the roadmap. We say this openly.

Q45. How do requirements trace to tests? The RTM maps all 115 requirements: 98 to automated suites, 17 to manual cases; none uncovered (AMR-DOC-08 Appendix B).

Q46. Did you find bugs through testing? Yes — e.g. a final placed in the semifinal column, raw errors on out-of-range pages, an RPC callable by anonymous users, unlimited anonymous uploads; all fixed (AMR-DOC-08 §8).

8. Performance and scalability

Q47. How fast is the site? Production Lighthouse on mobile (2026-10-18): performance 67–72, accessibility, best practices and SEO 100. Image-size and caching fixes were deployed after that run.

Q48. What happens when thousands view the medal tally at once? Static assets come from Vercel's CDN; slow-changing public data is edge-cached for 60 s; live data goes to Supabase, which scales on its managed infrastructure; queries are indexed. A load test is future work.

Q49. What are your scaling limits? Mostly the Supabase and Vercel plan quotas [TO CONFIRM plan]; the architecture itself needs no change to scale up a tier.

9. Operations and failure

Q50. What if Supabase goes down during the games? Scorekeepers keep paper sheets and enter results afterwards; the computed tally updates once data is in. Recovery from backups is described in the runbook.

Q51. What if Vercel goes down? The site is unavailable until Vercel recovers or a previous deployment is promoted; data is safe in Supabase.

Q52. How do you back up data? Supabase backups (plan-dependent [TO CONFIRM]) plus manual exports before SQL changes and before event week; a restore drill is recommended each semester.

Q53. How do you roll back a bad release? Promote the previous Vercel deployment; re-run the previous SQL file from git for functions and policies; restore a backup for data.

10. Process, limitations and future

Q54. How was the project managed? One feature per branch, gates before every merge, plans approved before large changes, 82 merged branches and a progress log (AMR-DOC-12).

Q55. What would you do next? Privacy updates for athletes and MFA (P1), CI and CSP enforcement (P2), score history, E2E tests, notifications (P3), then heats, judges' score sheets and offline scoring (P4) (AMR-DOC-13 §4).

One-Page Cheat Sheets

Architecture, roles, database and features at a glance · 1.0 · 2026-10-19

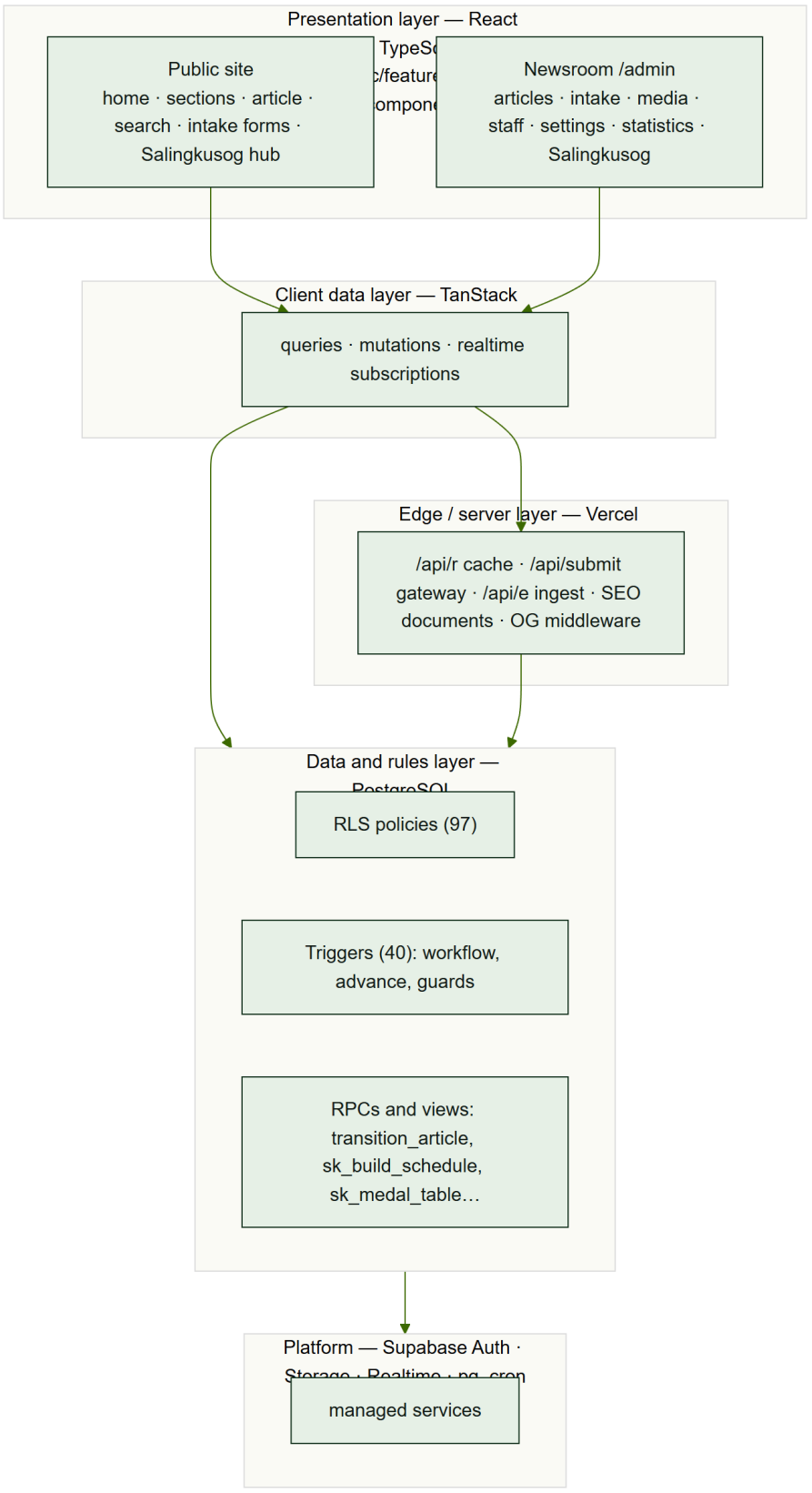
Document control

Field	Value
Title	One-Page Cheat Sheets
Document ID	AMR-DOC-22
Version	1.0
Date	2026-10-19
Status	Final draft

Cheat sheet 1 — Architecture

Layer	What	Key files
Browser	React 18 + TypeScript SPA (Vite 7), TanStack Query, Tailwind v4 tokens	`src/app/router.tsx`, `src/features/*`
Edge	Vercel CDN; `/api/r` edge cache (60 s)	`api/r.ts`, `src/lib/edgeCache.ts`
Server	`/api/submit` (Turnstile + signed uploads + service RPCs), `/api/e` (stats), sitemap/RSS/robots, crawler middleware	`api/*.ts`, `middleware.ts`
Data and rules	PostgreSQL: 50 tables, 97 RLS policies, 40 triggers, 76 functions, 6 views	`supabase/v2/*.sql`
Platform	Supabase Auth, Storage (media, archives, intake), Realtime, pg_cron	Supabase

Principles: the database is the security boundary · secrets only in `/api` · computed standings · re-runnable SQL · one feature per branch.


Presentation layer — React
Public site
home · sections · article ·
search · intake forms ·
Salingkusog hub
Newsroom /admin
articles · intake · media ·
staff · settings · statistics ·
Salingkusog
Client data layer — TanStack
queries · mutations · realtime
subscriptions
Edge / server layer — Vercel
/api/r cache · /api/submit
gateway · /api/e ingest · SEO
documents · OG middleware
Data and rules layer —
RLS policies (97)
Triggers (40): workflow,
advance, guards
RPCs and views:
transition_article,
sk_build_schedule,
sk_medal_table…
Platform — Supabase Auth ·
managed services

Layers

Cheat sheet 2 — Roles

Role	Write	Review	Publish	Staff & settings	Front page	Stats	Salingkusog
Super admin	✓	✓	✓	✓ (+ grant SA)	✓	✓	✓ + team
Executive Editor	✓	✓	✓	✓	✓	✓	✓ + team
Managing Editor	✓	✓	✓	publishing access only	✓	✓	✓ + team
Business Manager	✓	—	✓	—	—	—	if in team
Section editor	✓	own sections	if granted	—	—	—	if in team
Staff writer / Contributor	✓	—	if granted	—	—	—	if in team
Reader	—	—	—	—	—	—	view published

Helpers: `is_staff` · `is_admin` (SA, EE) · `can_publish` (SA, EE, ME) · `can_publish_articles` (+ BM, granted) · `is_editor` (+ SE) · `can_view_stats` · `can_edit_sk` (+ team).

Cheat sheet 3 — Database

Group	Tables
People	profiles, staff_invites, profile_sections, contributors, team_terms, team_members, audit_log
Content	sections, categories, tags, articles, article_authors, article_tags, article_media, article_revisions, article_activity, article_daily_views, publication_issues, media_assets, home_slots
Public input	article_comments, article_comment_private, article_submissions, event_reports, intake_files, contact_messages
Settings	site_settings, department_contacts
Salingkusog	sk_editions, sk_factions, sk_venues, sk_sports, sk_events, sk_athletes, sk_matches, sk_entries, sk_athlete_stats, sk_placements, sk_awards, sk_team
Statistics	stats_pageviews, stats_shares, staff_activity_daily, stats_daily_site, stats_daily_articles, stats_daily_dims, stats_daily_shares, stats_rollup_log (+ app.stats_salts)

Article workflow: draft → submitted → in_review → scheduled/published → archived (rejected → draft).
Medal table: points = 5·gold + 3·silver + 1·bronze; rank by points, gold, silver.

Cheat sheet 4 — Features

Area	Features
Public site	Home, 12 section landings, article, search, archives, about, board, privacy, submit story, report event, contact, comments
Newsroom	Dashboard, article editor (autosave, revisions, notes, workflow, scheduling), submissions and events, media library (WebP sizes, in-use guard), staff and publishing access, settings, statistics
Salingkusog	Edition, factions, venues, sports and events, athletes, team; CSV import, Build schedule (bracket, round robin, RR + final), contests; live scorekeeper; results and medals (suggest + confirm); awards and social cards; medal tally, points race (1/3/6/12 h, day), official final tally, schedule, winners, factions, brackets, recaps, venues, countdown
Security	Invite-only accounts, RLS everywhere, workflow trigger, Turnstile gateway, signed uploads, rate limits, sanitising, security headers
Quality	255 unit tests, 278 SQL checks, 53-screen accessibility sweep, WCAG AA contrast, Lighthouse a11y 100

Screenshot List

Every screen to capture, with its route and the captured file · 1.0 · 2026-10-19

Document control

Field	Value
Title	Screenshot List
Document ID	AMR-DOC-23
Version	1.0
Date	2026-10-19
Status	Final draft

Contents

1. How the screenshots were made
2. Captured screens
3. Screens still to capture on a seeded instance
4. Excluded screens

1. How the screenshots were made

Captured on 2026-10-19 with headless Chrome against the local development server, using the `/dev` fixture screens, which show **fictional data** (fictional factions, athletes and staff such as “Wes Writer”). Desktop 1440×900, phone 390×844. Files are in `docs/screenshots/`.

2. Captured screens

File	Screen	Fixture route	Real route
`pub-home.png`, `pub-home-mobile.png`	Home	`/dev/home`	`/`
`pub-section.png`	Section page	`/dev/section`	`/:section`
`pub-article.png`, `pub-article-mobile.png`	Article	`/dev/article`	`/article/:slug`
`pub-search.png`	Search	`/dev/search`	`/search`
`pub-archives.png`	Archives	`/dev/archives`	`/archives`
`pub-submit.png`	Submit a story	`/dev/submit`	`/submit`
`pub-submit-event.png`	Report an event	`/dev/submit/event`	`/submit/event`
`pub-contact.png`	Contact	`/dev/contact`	`/contact`
`pub-sports-landing.png`	Sports landing	`/dev/sports`	`/sports`
`sk-tally.png`, `sk-tally-mobile.png`	Medal tally	`/dev/salingkusog`	`/salingkusog`
`sk-tally-final.png`	Official and final tally	`/dev/salingkusog/final`	`/salingkusog`
`sk-schedule.png`	Schedule and results	`/dev/salingkusog/schedule`	`/salingkusog/schedule`
`sk-winners.png`	Winners	`/dev/salingkusog/winners`	`/salingkusog/winners`
`sk-faction.png`	Faction profile	`/dev/salingkusog/factions`	`/salingkusog/factions/:slug`
`sk-bracket.png`	Auto bracket	`/dev/salingkusog/sports/auto-bracket`	`/salingkusog/sports/:slug`
`sk-round-robin.png`	Round robin + final	`/dev/salingkusog/sports/auto-round-robin`	`/salingkusog/sports/:slug`
`sk-contest-results.png`	Contest results	`/dev/salingkusog/sports/contest-race`	`/salingkusog/sports/:slug`
`sk-contest-lineup.png`	Contest line-up	`/dev/salingkusog/sports/contest-dance`	`/salingkusog/sports/:slug`
`sk-match-recap.png`	Match recap	`/dev/salingkusog/match`	`/salingkusog/match/:id`
`sk-venues.png`	Venue guide	`/dev/salingkusog/venues`	`/salingkusog/venues`
`sk-countdown.png`	Countdown	`/dev/countdown`	`/salingkusog-2026/countdown`

File	Screen	Fixture route	Real route
`adm-login.png`	Staff sign in	`/admin/login`	`/admin/login`
`adm-access-pending.png`	Access pending	`/dev/auth/pending`	`/admin` (inactive user)
`adm-dashboard.png`	Dashboard (editor)	`/dev/admin`	`/admin`
`adm-dashboard-writer.png`	Dashboard (writer)	`/dev/admin/writer`	`/admin`
`adm-articles.png`	Articles list	`/dev/admin/articles`	`/admin/articles`
`adm-editor.png`	Article editor + workflow	`/dev/admin/editor`	`/admin/articles/:id`
`adm-media.png`	Media library	`/dev/admin/media`	`/admin/media`
`adm-intake.png`	Submission review	`/dev/admin/intake`	`/admin/submissions/:id`
`adm-sk-team.png`	Salingkusog team	`/dev/admin/salingkusog-team`	`/admin/salingkusog/team`
`adm-sk-setup.png`	Salingkusog setup (full page)	`/dev/admin/salingkusog-setup`	`/admin/salingkusog/edition` … `/import`
`adm-sk-scores.png`	Games by day	`/dev/admin/salingkusog-scores`	`/admin/salingkusog/scores`
`adm-sk-scorekeeper.png`, `-mobile.png`	Scorekeeper	`/dev/admin/salingkusog-scorekeeper`	`/admin/salingkusog/scores/:id`
`adm-sk-results.png`	Results and medals	`/dev/admin/salingkusog-results`	`/admin/salingkusog/results/:eventId`
`adm-sk-awards.png`	Awards and athletes	`/dev/admin/salingkusog-awards`	`/admin/salingkusog/awards`
`adm-sk-cards.png`	Social cards	`/dev/admin/salingkusog-cards`	`/admin/salingkusog/awards`

3. Screens still to capture on a seeded instance

These have no fixture screen and need a demo database with fictional data (AMR-DOC-20 §3):

Screen	Route
Staff accounts and publishing access	`/admin/staff`
Invites	`/admin/staff/invites`
Settings tabs (Front page, Comments, Inbox, Sections, Desks, General)	`/admin/settings/*`
Statistics	`/admin/statistics`
Build schedule modal	`/admin/salingkusog/sports`
Create contest modal and contest scorekeeper	`/admin/salingkusog/sports`, `/admin/salingkusog/scores/:id`
Schedule import with preview	`/admin/salingkusog/import`
Privacy page	`/privacy`

4. Excluded screens

`/dev/about` and `/dev/editorial-board` were **not captured**: their fixtures contain real names and photos of editorial-board members (KI-10). Use a seeded demo board with fictional names instead.